



社区维护中文译本

掌握 FreeRTOS 实时内核

实践教程指南

Richard Barry 与 FreeRTOS 团队

英文原书 V1.1.0

FreeRTOS Kernel Chinese PDF

版权与译本说明

除非正文另有说明，原书中的文字、源代码和图表均为 Amazon Web Services 的专有财产。

Copyright 2023 Amazon.com, Inc. or its affiliates. All Rights Reserved.

原书由 Richard Barry 和 FreeRTOS 团队编写。本中文译本以 FreeRTOS Kernel Book **V1.1.0** 为英文基线，对应上游提交 `f3e284bbdd1e1489c9d59c3cea477ae58855a0c9`。

原书及本中文翻译依照 [Creative Commons Attribution-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-sa/4.0/) 许可协议分发。书中的示例代码适用上游仓库中单独列明的示例代码许可条款。

本项目不是 Amazon Web Services 或 FreeRTOS 官方中文版本。涉及 API 行为、内核配置、安全性或实时性要求时，请以英文原版、FreeRTOS 官方文档和实际源码为准。

- FreeRTOS 官方网站: <https://www.FreeRTOS.org>
- FreeRTOS 社区论坛: <https://forums.FreeRTOS.org>
- FreeRTOS GitHub 组织: <https://github.com/FreeRTOS>

目录

1 前言

- 1.1 小型嵌入式系统中的多任务处理
 - 1.1.1 关于 FreeRTOS 内核
 - 1.1.2 价值主张
 - 1.1.3 关于术语的说明
 - 1.1.4 为什么使用 RTOS?
 - 1.1.5 FreeRTOS 内核的功能
 - 1.1.6 许可，以及 FreeRTOS、OpenRTOS 和 SafeRTOS 产品系列
- 1.2 随书提供的源文件和项目
 - 1.2.1 获取本书配套示例

2 FreeRTOS 内核发行版

- 2.1 简介
- 2.2 了解 FreeRTOS 发行版
 - 2.2.1 定义：FreeRTOS 移植版本
 - 2.2.2 构建 FreeRTOS
 - 2.2.3 FreeRTOSConfig.h
 - 2.2.4 官方发行版
 - 2.2.5 所有移植版本通用的 FreeRTOS 源文件
 - 2.2.6 特定于移植版本的 FreeRTOS 源文件
 - 2.2.7 头文件搜索路径
 - 2.2.8 头文件
- 2.3 演示应用
- 2.4 创建 FreeRTOS 项目
 - 2.4.1 修改随附的演示项目
 - 2.4.2 从零创建新项目
- 2.5 数据类型和编码风格指南
 - 2.5.1 数据类型

- 2.5.2 变量命名
- 2.5.3 函数命名
- 2.5.4 格式
- 2.5.5 宏命名
- 2.5.6 大量类型转换的原因

3 堆内存管理

- 3.1 引言
 - 3.1.1 先决条件
 - 3.1.2 范围
 - 3.1.3 静态与动态内存分配的切换
 - 3.1.4 使用动态内存分配
 - 3.1.5 动态内存分配的选项
- 3.2 示例内存分配方案
 - 3.2.1 Heap_1
 - 3.2.2 Heap_2
 - 3.2.3 Heap_3
 - 3.2.4 Heap_4
 - 3.2.5 Heap_5
 - 3.2.6 初始化 heap_5: vPortDefineHeapRegions() API 函数
- 3.3 堆相关的实用函数和宏
 - 3.3.1 定义堆起始地址
 - 3.3.2 xPortGetFreeHeapSize() API 函数
 - 3.3.3 xPortGetMinimumEverFreeHeapSize() API 函数
 - 3.3.4 vPortGetHeapStats() API 函数
 - 3.3.5 收集每个任务的堆使用统计信息
 - 3.3.6 Malloc 失败钩子函数
 - 3.3.7 将任务栈放置在快速内存中
- 3.4 使用静态内存分配
 - 3.4.1 启用静态内存分配
 - 3.4.2 静态内部内核内存
 - 3.4.2.1 vApplicationGetTimerTaskMemory
 - 3.4.2.2 vApplicationGetIdleTaskMemory

4 任务管理

- 4.1 引言
 - 4.1.1 范围
- 4.2 任务函数
- 4.3 顶层任务状态
- 4.4 任务创建
 - 4.4.1 xTaskCreate() API 函数
- 4.5 任务优先级
 - 4.5.1 通用调度器
 - 4.5.2 架构优化调度器
- 4.6 时间测量和 Tick 中断
- 4.7 扩展“未运行”状态
 - 4.7.1 “阻塞”状态
 - 4.7.2 “挂起”状态
 - 4.7.3 就绪状态
 - 4.7.4 完善状态转换图
 - 4.7.5 vTaskDelayUntil() API 函数
- 4.8 空闲任务和空闲任务钩子
 - 4.8.1 空闲任务钩子函数
 - 4.8.2 空闲任务钩子函数实现的限制
- 4.9 更改任务优先级
 - 4.9.1 vTaskPrioritySet() API 函数
 - 4.9.2 uxTaskPriorityGet() API 函数
- 4.10 删除任务
 - 4.10.1 vTaskDelete() API 函数
- 4.11 线程局部存储和可重入性
 - 4.11.1 C 运行时线程局部存储实现
 - 4.11.2 自定义 C 运行时线程局部存储
 - 4.11.3 应用程序线程局部存储
- 4.12 调度算法
 - 4.12.1 任务状态和事件回顾

- 4.12.2 选择调度算法
- 4.12.3 带时间片的优先级抢占式调度
- 4.12.4 不带时间片的优先级抢占式调度
- 4.12.5 协作式调度

5 队列管理

- 5.1 引言
 - 5.1.1 范围
- 5.2 队列的特性
 - 5.2.1 数据存储
 - 5.2.2 多个任务访问
 - 5.2.3 在队列读取时阻塞
 - 5.2.4 在队列写入时阻塞
 - 5.2.5 在多个队列上阻塞
 - 5.2.6 创建队列：静态分配与动态分配
- 5.3 使用队列
 - 5.3.1 xQueueCreate() API 函数
 - 5.3.2 xQueueSendToBack() 和 xQueueSendToFront() API 函数
 - 5.3.3 xQueueReceive() API 函数
 - 5.3.4 uxQueueMessagesWaiting() API 函数
- 5.4 从多个数据源接收数据
- 5.5 处理大型或长度可变的数据
 - 5.5.1 将指针入队
 - 5.5.2 使用队列发送类型和长度不同的数据
- 5.6 从多个队列接收数据
 - 5.6.1 队列集
 - 5.6.2 xQueueCreateSet() API 函数
 - 5.6.3 xQueueAddToSet() API 函数
 - 5.6.4 xQueueSelectFromSet() API 函数
 - 5.6.5 更贴近实际的队列集用例
- 5.7 使用队列创建邮箱
 - 5.7.1 xQueueOverwrite() API 函数

- 5.7.2 xQueuePeek() API 函数

6 软件定时器管理

- 6.1 本章简介与范围
 - 6.1.1 范围
- 6.2 软件定时器回调函数
- 6.3 软件定时器的属性和状态
 - 6.3.1 软件定时器的周期
 - 6.3.2 单次定时器与自动重载定时器
 - 6.3.3 软件定时器状态
- 6.4 软件定时器的上下文
 - 6.4.1 RTOS 守护任务（定时器服务任务）
 - 6.4.2 定时器命令队列
 - 6.4.3 守护任务调度
- 6.5 创建和启动软件定时器
 - 6.5.1 xTimerCreate() API 函数
 - 6.5.2 xTimerStart() API 函数
- 6.6 定时器 ID
 - 6.6.1 vTimerSetTimerID() API 函数
 - 6.6.2 pvTimerGetTimerID() API 函数
- 6.7 更改定时器的周期
 - 6.7.1 xTimerChangePeriod() API 函数
- 6.8 复位软件定时器
 - 6.8.1 xTimerReset() API 函数

7 中断管理

- 7.1 引言
 - 7.1.1 事件
 - 7.1.2 范围
- 7.2 从 ISR 中使用 FreeRTOS API
 - 7.2.1 中断安全 API
 - 7.2.2 使用单独的中断安全 API 的好处

- 7.2.3 使用单独的中断安全 API 的缺点
 - 7.2.4 xHigherPriorityTaskWoken 参数
 - 7.2.5 portYIELD_FROM_ISR() 和 portEND_SWITCHING_ISR() 宏
- 7.3 延迟中断处理
- 7.4 用于同步的二值信号量
 - 7.4.1 xSemaphoreCreateBinary() API 函数
 - 7.4.2 xSemaphoreTake() API 函数
 - 7.4.3 xSemaphoreGiveFromISR() API 函数
 - 7.4.4 改进示例 7.1 中的任务实现
- 7.5 计数信号量
 - 7.5.1 xSemaphoreCreateCounting() API 函数
- 7.6 将工作延迟到 RTOS 守护任务
 - 7.6.1 xTimerPendFunctionCallFromISR() API 函数
- 7.7 在中断服务程序中使用队列
 - 7.7.1 xQueueSendToFrontFromISR() 和 xQueueSendToBackFromISR() API 函数
 - 7.7.2 从 ISR 使用队列时的注意事项
- 7.8 中断嵌套
 - 7.8.1 ARM Cortex-M 和 ARM GIC 用户注意事项

8 资源管理

- 8.1 本章简介与范围
 - 8.1.1 互斥
 - 8.1.2 范围
- 8.2 临界区与挂起调度器
 - 8.2.1 基本临界区
 - 8.2.2 挂起（或锁定）调度器
 - 8.2.3 vTaskSuspendAll() API 函数
 - 8.2.4 xTaskResumeAll() API 函数
- 8.3 互斥量（与二值信号量）
 - 8.3.1 xSemaphoreCreateMutex() API 函数
 - 8.3.2 优先级反转
 - 8.3.3 优先级继承

- 8.3.4 死锁（或称致命拥抱）
 - 8.3.5 递归互斥量
 - 8.3.6 互斥量与任务调度
- 8.4 门卫任务
 - 8.4.1 重写 `vPrintString()` 以使用门卫任务

9 事件组

- 9.1 本章简介与范围
 - 9.1.1 范围
- 9.2 事件组的特性
 - 9.2.1 事件组、事件标志和事件位
 - 9.2.2 深入了解 `EventBits_t` 数据类型
 - 9.2.3 多任务访问
 - 9.2.4 使用事件组的实际示例
- 9.3 使用事件组管理事件
 - 9.3.1 `xEventGroupCreate()` API 函数
 - 9.3.2 `xEventGroupSetBits()` API 函数
 - 9.3.3 `xEventGroupSetBitsFromISR()` API 函数
 - 9.3.4 `xEventGroupWaitBits()` API 函数
 - 9.3.5 `xEventGroupGetStaticBuffer()` API 函数
- 9.4 使用事件组同步任务
 - 9.4.1 `xEventGroupSync()` API 函数

10 任务通知

- 10.1 简介
 - 10.1.1 通过中介对象通信
 - 10.1.2 任务通知：直接面向任务的通信
 - 10.1.3 范围
- 10.2 任务通知的优点与局限
 - 10.2.1 任务通知的性能优势
 - 10.2.2 任务通知的 RAM 占用优势
 - 10.2.3 任务通知的局限
- 10.3 使用任务通知

- 10.3.1 任务通知 API 选项
 - 10.3.1.1 API 函数完整列表
- 10.3.2 xTaskNotifyGive() API 函数
- 10.3.3 vTaskNotifyGiveFromISR() API 函数
- 10.3.4 ulTaskNotifyTake() API 函数
- 10.3.5 xTaskNotify() 和 xTaskNotifyFromISR() API 函数
- 10.3.6 xTaskNotifyWait() API 函数
- 10.3.7 在外设驱动程序中使用任务通知：UART 示例
- 10.3.8 在外设驱动程序中使用任务通知：ADC 示例
- 10.3.9 在应用程序中直接使用任务通知

11 低功耗支持

- 11.1 节能简介
- 11.2 FreeRTOS 睡眠模式
- 11.3 启用内置无 Tick 空闲功能及相关函数
 - 11.3.1 portSUPPRESS_TICKS_AND_SLEEP() 宏
 - 11.3.2 vPortSuppressTicksAndSleep 函数
 - 11.3.3 eTaskConfirmSleepModeStatus 函数
 - 11.3.4 configPRE_SLEEP_PROCESSING 配置项
 - 11.3.5 configPOST_SLEEP_PROCESSING 配置项
- 11.4 实现 portSUPPRESS_TICKS_AND_SLEEP() 宏
- 11.5 空闲任务钩子函数

12 开发支持

- 12.1 简介
- 12.2 configASSERT()
 - 12.2.1 configASSERT() 定义示例
- 12.3 Tracealyzer for FreeRTOS
- 12.4 调试相关的钩子（回调）函数
 - 12.4.1 内存分配失败钩子
 - 12.4.2 栈溢出钩子
- 12.5 查看运行时间与任务状态信息
 - 12.5.1 任务运行时间统计

- 12.5.2 运行时间统计时钟
- 12.5.3 配置应用程序以收集运行时间统计
- 12.5.4 uxTaskGetSystemState() API 函数
- 12.5.5 vTaskListTasks() 辅助函数
- 12.5.6 vTaskGetRunTimeStatistics() 辅助函数
- 12.5.7 生成并显示运行时间统计：完整示例
- 12.6 跟踪钩子宏
 - 12.6.1 可用的跟踪钩子宏
 - 12.6.2 定义跟踪钩子宏
 - 12.6.3 支持 FreeRTOS 感知的调试器插件

13 故障排查

- 13.1 本章简介与范围
- 13.2 中断优先级
- 13.3 栈溢出
 - 13.3.1 uxTaskGetStackHighWaterMark() API 函数
 - 13.3.2 运行时栈检查：概述
 - 13.3.3 运行时栈检查：方法 1
 - 13.3.4 运行时栈检查：方法 2
 - 13.3.4 运行时栈检查：方法 3
- 13.4 printf() 和 sprintf() 的使用
 - 13.4.1 printf-stdarg.c
- 13.5 其他常见错误来源
 - 13.5.1 症状：向演示程序添加一个简单任务后，演示程序崩溃
 - 13.5.2 症状：在中断中使用 API 函数导致应用程序崩溃
 - 13.5.3 症状：应用程序有时会在中断服务程序中崩溃
 - 13.5.4 症状：调度器尝试启动第一个任务时崩溃
 - 13.5.5 症状：中断意外保持禁用，或临界区嵌套不正确
 - 13.5.6 症状：应用程序在调度器启动前就已经崩溃
 - 13.5.7 症状：调度器挂起或处于临界区时调用 API 函数导致应用程序崩溃
- 13.6 其他调试步骤

1 前言

1.1 小型嵌入式系统中的多任务处理

1.1.1 关于 FreeRTOS 内核

FreeRTOS 是一组 C 语言库，由一个实时内核以及一组用于实现补充功能的模块化库组成。

Richard Barry 于 2003 年前后开始开发 FreeRTOS。此后，他创办的 Real-Time Engineers Ltd 与世界领先的芯片公司密切合作，持续推进 FreeRTOS 的开发，直到 Amazon Web Services (AWS) 于 2016 年接管 FreeRTOS 的维护工作。如今，Richard 仍在 AWS IoT 团队担任高级首席工程师，继续从事 FreeRTOS 相关工作。FreeRTOS 是采用 MIT 许可证的开源软件，可用于任何用途。即使不是 AWS 客户，也能受益于 AWS 对 FreeRTOS 的维护。

FreeRTOS 内核非常适合运行在微控制器或小型微处理器上的深度嵌入式实时应用。这类应用通常同时包含硬实时要求和软实时要求。

软实时要求规定了一个时间期限，但即使超过期限，系统也不会因此完全失效。例如，对按键操作的响应过慢，可能会让人觉得系统反应迟钝，却不至于使系统无法使用。

硬实时要求同样规定了一个时间期限，但一旦超过期限，系统便会彻底失败。例如，如果驾驶员安全气囊对碰撞传感器输入的响应过慢，它造成的伤害可能反而大于所能提供的保护。

FreeRTOS 内核是一种实时内核（也称实时调度器），它帮助基于 FreeRTOS 构建的应用满足硬实时要求。应用可以被组织为一组相互独立的执行线程。例如，在只有一个内核的处理器上，任意时刻只能有一个执行线程运行。内核通过检查应用设计者为各线程分配的优先级，决定当前应运行哪个线程。最简单的做法是，为实现硬实时要求的线程分配较高优先级，为实现软实时要求的线程分配较低优先级。这样可以确保硬实时线程总是先于软实时线程运行，但实际的优先级分配决策并不总是如此简单。

即使你现在还没有完全理解上一段中的概念，也不必担心。后续章节将通过详细说明和大量示例，帮助你理解如何使用实时内核，尤其是如何使用 FreeRTOS。

1.1.2 价值主张

FreeRTOS 内核在全球获得前所未有的成功，源于它极具吸引力的价值主张：FreeRTOS 由专业团队开发，经过严格的质量控制，稳健且有完善支持，不存在知识产权归属不清的问题；同时，它可以真正免费用于商业应用，也不要求公开专有源代码。此外，AWS 的维护还带来了全球化支持、专业的安全事件

响应流程、规模庞大且背景多元的开发团队，以及形式化验证、渗透测试、内存安全证明和长期支持等专业能力。在此过程中，FreeRTOS 始终保持为一个不依赖特定硬件、开发工具或云服务的开源项目。FreeRTOS 的开发过程公开透明，由社区通过 GitHub 共同推动，不要求使用任何特殊工具或开发实践。

你完全可以使用 FreeRTOS 将产品推向市场，甚至无需告知我们，更不需要支付任何费用；事实上，已有数千家公司这样做。如果你在任何时候希望获得额外支持，或者公司的法务团队要求更多书面保证或赔偿承诺，我们的战略合作伙伴也可以提供简单、低成本的商业许可方案。你可以随时选择商业许可，这一点本身便能带来安心。

1.1.3 关于术语的说明

在 FreeRTOS 中，每个执行线程都称为“任务”（task）。嵌入式社区对相关术语并没有统一约定，但我更倾向于使用“任务”而不是“线程”，因为“线程”在某些应用领域中可能具有更加特定的含义。

1.1.4 为什么使用 RTOS?

即使不使用多线程内核，也有许多成熟的方法可以编写出优秀的嵌入式软件。如果正在开发的系统比较简单，这些方法可能才是最合适的选择。对于更复杂的系统，使用内核通常更为合适，但复杂到什么程度才需要跨过这条分界线，始终是一个主观判断。

如前所述，任务优先级有助于确保应用满足处理时限，但内核还会带来一些不那么显而易见的好处。下面对其中部分好处作简要说明。

- **抽象时间信息**

RTOS 负责控制执行时序，并向应用提供与时间相关的 API。这样可以让应用代码的结构更加直接，同时缩小整体代码规模。

- **可维护性和可扩展性**

将时序细节抽象出来，可以减少模块之间的相互依赖，使软件能够以可控且可预测的方式演进。此外，由内核负责时序后，底层硬件的变化也不容易影响应用性能。

- **模块化**

各个任务都是相互独立的模块，并且每个任务都应具有明确的用途。

- **团队开发**

各个任务还应具有定义清晰的接口，从而降低团队协作开发的难度。

- **更易测试**

定义明确、相互独立且接口清晰的任务模块，更容易进行隔离测试。

- 代码复用

模块化程度越高、相互依赖越少，代码就越容易复用。

- 提高效率

使用 RTOS 的应用代码可以完全由事件驱动，因此不必把处理时间浪费在轮询尚未发生的事件上。

与事件驱动带来的效率提升相对应，系统需要处理 RTOS 时钟节拍中断，并在不同任务之间切换执行。不过，不使用 RTOS 的应用通常也会包含某种形式的时钟节拍中断。

- 空闲时间

当没有应用任务需要处理时，系统自动创建的空闲任务会开始运行。空闲任务可以用来测量剩余处理能力、执行后台检查，或者让处理器进入低功耗模式。

- 电源管理

使用 RTOS 带来的效率提升，使处理器能够在低功耗模式中停留更长时间。

每当空闲任务运行时，都可以让处理器进入低功耗状态，从而显著降低功耗。FreeRTOS 还提供一种特殊的无时钟节拍模式。使用该模式后，处理器可以进入比通常情况下更低的功耗状态，并在低功耗状态中停留更长时间。

- 灵活的中断处理

可以将具体处理工作延后，交给应用开发者创建的任务，或者交给系统自动创建的 RTOS 守护任务（也称定时器服务任务），从而让中断处理程序保持非常简短。

- 混合处理要求

通过简单的设计模式，可以在同一个应用中实现周期处理、连续处理和事件驱动处理。通过选择合适的任务优先级和中断优先级，还可以同时满足硬实时要求和软实时要求。

1.1.5 FreeRTOS 内核的功能

FreeRTOS 内核具备以下标准功能：

- 支持抢占式或协作式运行
- 可选的时间片调度
- 非常灵活的任务优先级分配

- 灵活、快速且轻量的任务通知机制
- 队列
- 二值信号量
- 计数信号量
- 互斥量
- 递归互斥量
- 软件定时器
- 事件组
- 流缓冲区
- 消息缓冲区
- 协程（已弃用）
- 时钟节拍钩子函数
- 空闲任务钩子函数
- 栈溢出检查
- 跟踪宏
- 任务运行时间统计
- 可选的商业许可与支持
- 完整的中断嵌套模型（适用于部分架构）
- 面向超低功耗应用的无时钟节拍功能（适用于部分架构）
- 使用内存保护单元隔离任务并提高应用安全性（适用于部分架构）
- 在适当情况下使用软件管理的中断栈，这有助于节省 RAM
- 可以使用静态分配或动态分配的内存创建 RTOS 对象

1.1.6 许可，以及 FreeRTOS、OpenRTOS 和 SafeRTOS 产品系列

FreeRTOS 采用 MIT 开源许可证，其设计目标是确保：

- FreeRTOS 可以用于商业应用。
- FreeRTOS 本身始终可以由所有人免费获得。
- FreeRTOS 用户保留其知识产权的所有权。

最新的开源许可信息请参阅 <https://www.FreeRTOS.org/license>。

OpenRTOS 是 FreeRTOS 的商业许可版本，由第三方获得 Amazon Web Services 授权后提供。

SafeRTOS 与 FreeRTOS 采用相同的使用模型，但其开发遵循了声明符合多项国际公认安全相关标准所必需的实践、规程和流程。

1.2 随书提供的源文件和项目

1.2.1 获取本书配套示例

可以从 <https://www.FreeRTOS.org/Documentation/code> 下载一个 zip 文件，其中包含构建和运行本书示例所需的全部源代码、预配置项目文件和说明。请注意，该 zip 文件不一定包含 FreeRTOS 的最新版本。

本书中的屏幕截图展示了示例在 Microsoft Windows 环境中通过 FreeRTOS Windows 移植层运行时的情况。使用 FreeRTOS Windows 移植层的项目已预先配置为使用免费的 Visual Studio Community 版进行构建，该版本可从 <https://www.visualstudio.com/> 获取。需要注意的是，FreeRTOS Windows 移植层虽然提供了便利的评估、测试和开发平台，但并不能提供真正的实时行为。

2 FreeRTOS 内核发行版

2.1 简介

为了帮助读者熟悉 FreeRTOS 内核的文件和目录，本章将：

- 概览 FreeRTOS 的目录结构。
- 说明任一 FreeRTOS 项目所需的源文件。
- 介绍演示应用。
- 说明如何创建新的 FreeRTOS 项目。

这里的说明仅适用于官方 FreeRTOS 发行版。本书配套示例采用的组织方式略有不同。

2.2 了解 FreeRTOS 发行版

2.2.1 定义：FreeRTOS 移植版本

FreeRTOS 可以使用大约 20 种不同的编译器构建，并可运行在 40 多种不同的处理器架构上。每一种受支持的编译器与处理器组合，都称为一个 FreeRTOS 移植版本（FreeRTOS port）。

2.2.2 构建 FreeRTOS

FreeRTOS 是一个库，它为原本只能单线程运行的裸机应用提供多任务处理能力。

FreeRTOS 以一组 C 源文件的形式提供。有些源文件对所有移植版本通用，另一些则只适用于特定移植版本。将这些源文件作为项目的一部分进行构建后，应用便可使用 FreeRTOS API。每个官方 FreeRTOS 移植版本都提供一个可供参考的演示应用。演示应用已经预先配置，会构建正确的源文件并包含正确的头文件。

每个演示应用在创建时都可以“开箱即用”地完成构建，不会产生编译器错误或警告。如果后续构建工具发生变化，导致情况不再如此，请通过 FreeRTOS 支持论坛（<https://forums.FreeRTOS.org>）告知我们。第 2.3 节将介绍这些演示应用。

2.2.3 FreeRTOSConfig.h

内核由名为 `FreeRTOSConfig.h` 的头文件中定义的常量进行配置。不要在源文件中直接包含 `FreeRTOSConfig.h`！应当包含 `FreeRTOS.h`，后者会在适当的时候包含 `FreeRTOSConfig.h`。

`FreeRTOSConfig.h` 用于针对具体应用裁剪 FreeRTOS 内核。例如，`FreeRTOSConfig.h` 中的 `configUSE_PREEMPTION` 常量用于指定 FreeRTOS 采用协作式调度还是抢占式调度^[1]。

由于 `FreeRTOSConfig.h` 用于针对具体应用定制 FreeRTOS，因此它应位于应用自身的目录中，而不应位于包含 FreeRTOS 源代码的目录中。

FreeRTOS 主发行版为每个 FreeRTOS 移植版本都提供一个演示应用，并且每个演示应用都有自己的 `FreeRTOSConfig.h` 文件。建议以所用移植版本对应演示应用中的 `FreeRTOSConfig.h` 为起点，在此基础上进行修改，而不是从头创建该文件。

FreeRTOS 参考手册和 <https://www.freertos.org/a00110.html> 介绍了 `FreeRTOSConfig.h` 中可以使用的常量。无需在 `FreeRTOSConfig.h` 中包含所有常量，许多未写出的常量会采用默认值。

2.2.4 官方发行版

包括内核在内的各个 FreeRTOS 库，既可以从各自的 GitHub 仓库获取，也可以通过 zip 归档文件获取。在生产代码中使用 FreeRTOS 时，能够单独获取某个库会很方便。不过，对于刚开始使用 FreeRTOS 的读者，最好下载同时包含各个库和示例项目的 FreeRTOS 主发行版。

主发行版包含所有 FreeRTOS 库的源代码、所有 FreeRTOS 内核移植版本，以及所有 FreeRTOS 演示应用的项目文件。不要被文件数量吓到！实际应用只需要其中很小的一部分。

可以通过 <https://github.com/FreeRTOS/FreeRTOS/releases/latest> 下载包含最新发行版的 zip 文件。也可以使用下面任一 Git 命令从 GitHub 克隆主发行版，并同时克隆从各自 Git 仓库以子模块形式引入的各个库：

```
git clone https://github.com/FreeRTOS/FreeRTOS.git --recurse-submodules  
  
git clone git@github.com:FreeRTOS/FreeRTOS.git --recurse-submodules
```

图 2.1 展示了 FreeRTOS 发行版的一级和二级目录：

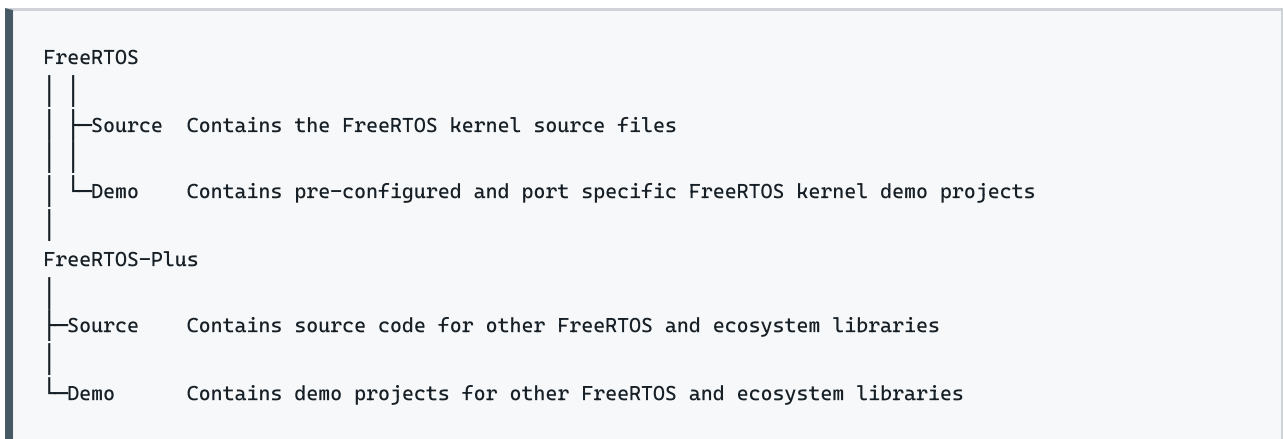


图 2.1 FreeRTOS 发行版中的顶层目录

发行版只包含一份 FreeRTOS 内核源文件。所有演示项目都预期在 `FreeRTOS/Source` 目录中找到内核源文件；如果改变目录结构，演示项目可能无法构建。

2.2.5 所有移植版本通用的 FreeRTOS 源文件

`tasks.c` 和 `list.c` 实现 FreeRTOS 内核的核心功能，始终是必需的。它们直接位于 `FreeRTOS/Source` 目录中，如图 2.2 所示。同一目录还包含以下可选源文件：

- `queue.c`

`queue.c` 提供队列和信号量服务，本书后续章节将对此进行介绍。几乎所有项目都需要 `queue.c`。

- `timers.c`

`timers.c` 提供软件定时器功能，本书后续章节将对此进行介绍。只有应用使用软件定时器时，才需要构建该文件。

- `event_groups.c`

`event_groups.c` 提供事件组功能，本书后续章节将对此进行介绍。只有应用使用事件组时，才需要构建该文件。

- `stream_buffer.c`

`stream_buffer.c` 同时提供流缓冲区和消息缓冲区功能，本书后续章节将对此进行介绍。只有应用使用流缓冲区或消息缓冲区时，才需要构建该文件。

- `croutine.c`

`croutine.c` 实现 FreeRTOS 协程功能。只有应用使用协程时，才需要构建该文件。协程面向非常小的微控制器设计，如今已很少使用。因此，协程不再维护，也不建议在新设计中使用。本书不介绍协程。

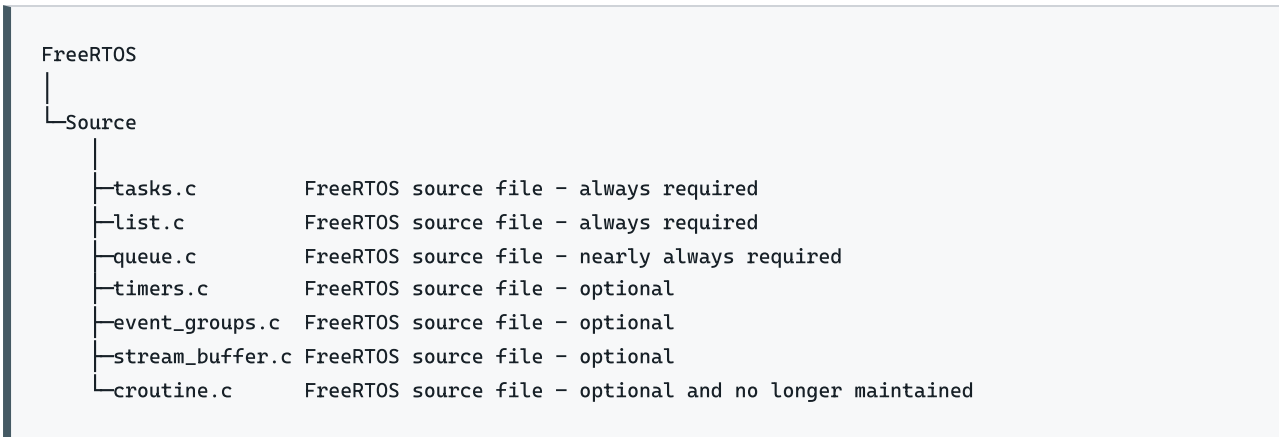


图 2.2 FreeRTOS 目录树中的核心 FreeRTOS 源文件

可以理解，zip 发行版中使用的文件名可能导致命名空间冲突，因为许多项目已经使用了相同的文件名。用户可以在必要时更改这些文件名，但发行版中的名称不能改变，否则会破坏与现有用户项目以及支持 FreeRTOS 的开发工具之间的兼容性。

2.2.6 特定于移植版本的 FreeRTOS 源文件

`FreeRTOS/Source/portable` 目录包含特定于 FreeRTOS 移植版本的源文件。`portable` 目录采用层级结构组织，第一层按编译器划分，第二层按处理器架构划分。图 2.3 展示了这一层级结构。

如果要在架构为“*architecture*”的处理器上使用“*compiler*”编译器运行 FreeRTOS，除了核心 FreeRTOS 源文件外，还必须构建 `FreeRTOS/Source/portable/[compiler]/[architecture]` 目录中的文件。

如第 3 章“堆内存管理”所述，FreeRTOS 也将堆内存分配视为可移植层的一部分。如果 `configSUPPORT_DYNAMIC_ALLOCATION` 设置为 0，则不要在项目中包含堆内存分配方案。

FreeRTOS 在 `FreeRTOS/Source/portable/MemMang` 目录中提供了示例堆分配方案。如果 FreeRTOS 配置为使用动态内存分配，则必须在项目中包含该目录中的一个堆实现源文件，或者提供自己的实现。

！ 不要在同一项目中包含多个示例堆分配实现。

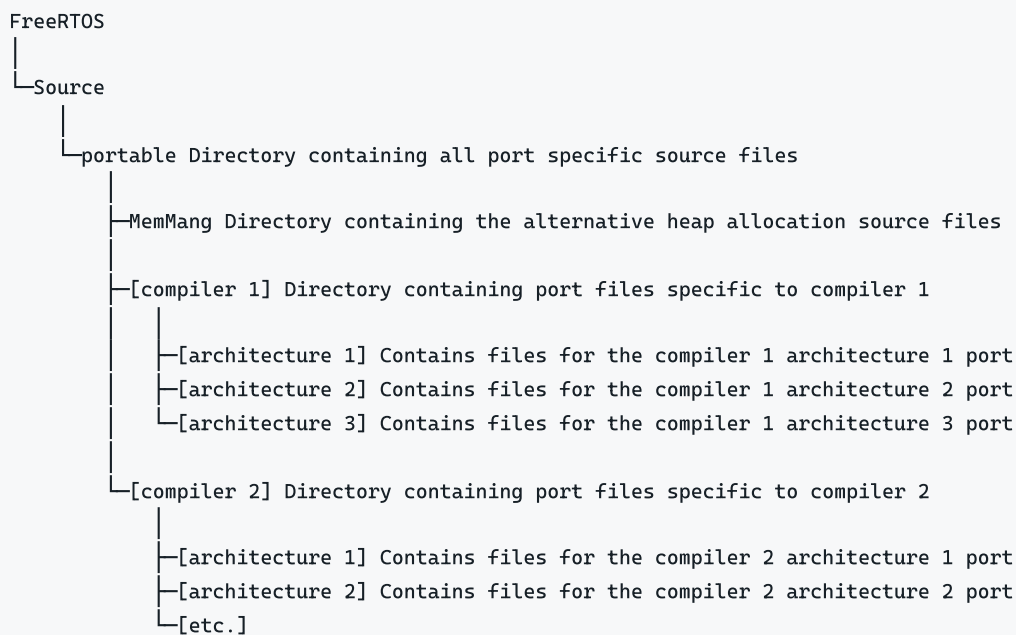


图 2.3 FreeRTOS 目录树中特定于移植版本的源文件

2.2.7 头文件搜索路径

FreeRTOS 要求将三个目录加入编译器的头文件搜索路径:

1. FreeRTOS 内核核心头文件所在的路径, 即 `FreeRTOS/Source/include`。
2. 当前所用 FreeRTOS 移植版本的专用源文件所在路径, 即 `FreeRTOS/Source/portable/[compiler]/[architecture]`。
3. 正确的 `FreeRTOSConfig.h` 头文件所在路径。

2.2.8 头文件

使用 FreeRTOS API 的源文件必须先包含 `FreeRTOS.h`, 再包含声明所用 API 函数原型的头文件, 即 `task.h`、`queue.h`、`semphr.h`、`timers.h`、`event_groups.h`、`stream_buffer.h`、`message_buffer.h` 或 `croutine.h`。不要显式包含其他 FreeRTOS 头文件; `FreeRTOS.h` 会自动包含 `FreeRTOSConfig.h`。

2.3 演示应用

每个 FreeRTOS 移植版本都至少带有一个演示应用。在创建时，这些演示应用都可以“开箱即用”地完成构建，不会产生编译器错误或警告。如果后续构建工具发生变化，导致情况不再如此，请通过 FreeRTOS 支持论坛 (<https://forums.FreeRTOS.org>) 告知我们。

跨平台支持：FreeRTOS 在 Windows、Linux 和 macOS 系统上进行开发和测试，使用的工具链既有嵌入式工具链，也有传统工具链。不过，版本差异或遗漏的测试偶尔也会导致构建错误。如果遇到此类错误，请通过 FreeRTOS 支持论坛 (<https://forums.FreeRTOS.org>) 提醒我们。

演示应用有以下几个用途：

- 提供一个能够正常工作且预先配置好的项目示例，其中包含正确的文件，并设置了正确的编译器选项。
- 让用户只需极少的设置和预备知识，即可进行“开箱即用”的实验。
- 演示如何使用 FreeRTOS API。
- 作为创建实际应用的基础。
- 对内核实现进行压力测试。

每个演示项目都位于 **FreeRTOS/Demo** 目录下的独立子目录中。子目录的名称表明该演示项目对应的移植版本。

FreeRTOS.org 网站为每个演示应用都提供了一个页面，其中包含以下信息：

- 如何在 FreeRTOS 目录结构中找到该演示的项目文件。
- 项目配置使用的硬件或仿真器。
- 如何配置硬件以运行演示。
- 如何构建演示。
- 演示的预期行为。

所有演示项目都会创建“公共演示任务”（common demo tasks）的一个子集，这些任务的实现在 **FreeRTOS/Demo/Common/Minimal** 目录中。公共演示任务用于展示 FreeRTOS API 的用法并测试 FreeRTOS 内核移植版本，它们不实现任何特定的实用功能。

许多演示项目还可以配置为创建一个简单的“闪烁灯”（blinky）风格入门项目。这类项目通常会创建两个 RTOS 任务和一个队列。

每个演示项目都包含一个名为 `main.c` 的文件，其中包含 `main()` 函数。该函数在启动 FreeRTOS 内核之前创建演示应用的任务。有关某个具体演示的信息，请参阅相应 `main.c` 文件中的注释。

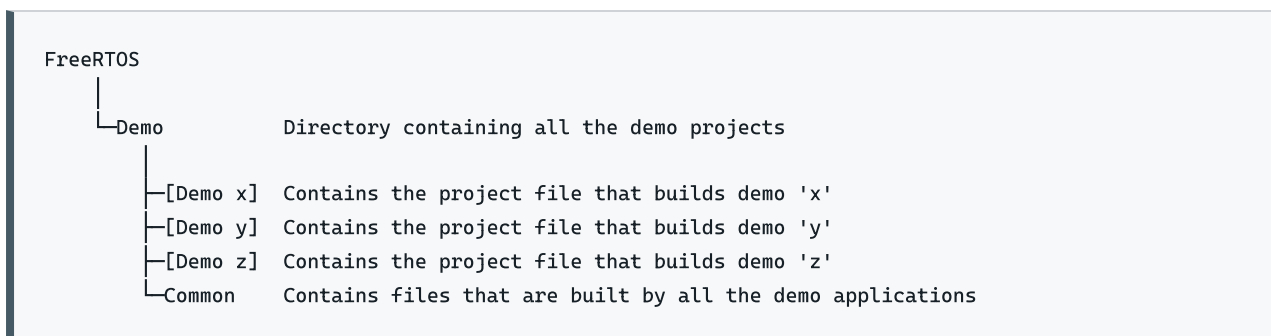


图 2.4 演示目录的层级结构

2.4 创建 FreeRTOS 项目

2.4.1 修改随附的演示项目

每个 FreeRTOS 移植版本都至少带有一个预先配置好的演示应用。建议通过修改其中一个现有项目来创建新项目，以确保新项目包含正确的文件、安装了正确的中断处理程序，并设置了正确的编译器选项。

要从现有演示项目创建新的应用，请执行以下步骤：

1. 打开随附的演示项目，确认它能够按预期构建和运行。
2. 删除实现演示任务的源文件，也就是位于 `Demo/Common` 目录中的文件。
3. 删除 `main()` 中除 `prvSetupHardware()` 和 `vTaskStartScheduler()` 之外的所有函数调用，如代码清单 2.1 所示。
4. 确认项目仍能正常构建。

完成这些步骤后，将得到一个包含正确 FreeRTOS 源文件、但尚未定义任何功能的项目。

```

int main( void )
{
    /* 执行必要的硬件初始化。 */
    prvSetupHardware();

    /* --- 可在此处创建应用任务。 --- */

    /* 启动已创建的任务。 */
    vTaskStartScheduler();

    /* 只有堆空间不足，无法启动调度器时，程序才会执行到这里。 */
    for( ;; );
    return 0;
}

```

代码清单 2.1 新 `main()` 函数的模板

2.4.2 从零创建新项目

如前所述，建议从现有演示项目创建新项目。如果不希望采用这种方式，则可以按照以下步骤从零创建新项目：

1. 使用所选工具链创建一个尚未包含任何 FreeRTOS 源文件的新项目。
2. 确认新项目能够构建、下载到目标硬件并正常运行。
3. 只有在确认已经拥有一个可以正常工作的项目后，才将表 1 所列的 FreeRTOS 源文件添加到项目中。
4. 将所用移植版本对应演示项目中的 `FreeRTOSConfig.h` 头文件复制到新项目目录。
5. 将以下目录加入项目搜索头文件时使用的路径：
 - `FreeRTOS/Source/include`
 - `FreeRTOS/Source/portable/[compiler]/[architecture]`（其中 `compiler` 和 `architecture` 应与所选移植版本匹配）
 - 包含 `FreeRTOSConfig.h` 头文件的目录
6. 从相应的演示项目复制编译器设置。
7. 安装可能需要的 FreeRTOS 中断处理程序。可以参考所用移植版本的说明网页以及该移植版本随附的演示项目。

文件	位置
tasks.c	FreeRTOS/Source
queue.c	FreeRTOS/Source
list.c	FreeRTOS/Source
timers.c	FreeRTOS/Source
event_groups.c	FreeRTOS/Source
stream_buffer.c	FreeRTOS/Source
所有 C 和汇编源文件	FreeRTOS/Source/portable/[compiler]/[architecture]
heap_n.c	FreeRTOS/Source/portable/MemMang，其中 n 为 1、2、3、4 或 5

表 1 项目中需要包含的 FreeRTOS 源文件

堆内存说明：如果 `configSUPPORT_DYNAMIC_ALLOCATION` 为 0，则不要在项目中包含堆内存分配方案。否则，请在项目中加入一种堆内存分配方案，可以选择某个 `heap_n.c` 文件，也可以使用自己提供的实现。更多信息请参阅第 3 章“堆内存管理”。

2.5 数据类型和编码风格指南

2.5.1 数据类型

FreeRTOS 的每个移植版本都有一个独立的 `portmacro.h` 头文件，其中定义了两个特定于移植版本的数据类型：`TickType_t` 和 `BaseType_t`。下面分别介绍这些宏或 typedef 及其实际类型：

- `TickType_t`

FreeRTOS 会配置一个称为时钟节拍中断（tick interrupt）的周期性中断。

FreeRTOS 应用启动后累计发生的时钟节拍中断次数称为**时钟节拍计数**（tick count）。时钟节拍计数用于度量时间。

两次时钟节拍中断之间的时间称为**时钟节拍周期**（tick period）。时间以时钟节拍周期的倍数表示。

`TickType_t` 是用于保存时钟节拍计数值和指定时间的数据类型。

根据 `FreeRTOSConfig.h` 中 `configTICK_TYPE_WIDTH_IN_BITS` 的设置，`TickType_t` 可以是无符号 16 位、无符号 32 位或无符号 64 位类型。`configTICK_TYPE_WIDTH_IN_BITS` 的设置取决于处理器架构，FreeRTOS 移植版本也会检查该设置是否有效。

在 8 位和 16 位架构上使用 16 位类型可以显著提高效率，但会严重限制 FreeRTOS API 调用所能指定的最大阻塞时间。在 32 位或 64 位架构上，没有理由使用 16 位的 `TickType_t`。

过去使用的 `configUSE_16_BIT_TICKS` 已由 `configTICK_TYPE_WIDTH_IN_BITS` 取代，以支持超过 32 位的时钟节拍计数。新设计应使用 `configTICK_TYPE_WIDTH_IN_BITS`，而不是 `configUSE_16_BIT_TICKS`。

<code>configTICK_TYPE_WIDTH_IN_BITS</code>	8 位架构	16 位架构	32 位架构	64 位架构
<code>TICK_TYPE_WIDTH_16_BITS</code>	<code>uint16_t</code>	<code>uint16_t</code>	<code>uint16_t</code>	N/A
<code>TICK_TYPE_WIDTH_32_BITS</code>	<code>uint32_t</code>	<code>uint32_t</code>	<code>uint32_t</code>	N/A
<code>TICK_TYPE_WIDTH_64_BITS</code>	N/A	N/A	<code>uint64_t</code>	<code>uint64_t</code>

表 2 `TickType_t` 数据类型与 `configTICK_TYPE_WIDTH_IN_BITS` 配置

- `BaseType_t`

`BaseType_t` 始终定义为对当前架构而言效率最高的数据类型。通常，它在 64 位架构上是 64 位类型，在 32 位架构上是 32 位类型，在 16 位架构上是 16 位类型，在 8 位架构上是 8 位类型。

`BaseType_t` 通常用于取值范围非常有限的返回类型，以及 `pdTRUE` / `pdFALSE` 这类布尔值。

FreeRTOS 使用的特定于移植版本的数据类型列表

2.5.2 变量命名

变量名称以其类型作为前缀：`c` 表示 `char`，`s` 表示 `int16_t`（short），`l` 表示 `int32_t`（long），`x` 表示 `BaseType_t` 以及其他非标准类型，例如结构体、任务句柄和队列句柄等。

如果变量为无符号类型，其名称还会带有 `u` 前缀；如果变量是指针，还会带有 `p` 前缀。例如，`uint8_t` 类型变量使用 `uc` 前缀，指向 `char` 的指针变量（`char *`）使用 `pc` 前缀。

2.5.3 函数命名

函数名称的前缀同时表示返回值类型和函数定义所在的文件。例如：

- `vTaskPrioritySet()` 返回 `void`，定义在 `tasks.c` 中。
- `xQueueReceive()` 返回 `BaseType_t` 类型的变量，定义在 `queue.c` 中。
- `pvTimerGetTimerID()` 返回指向 `void` 的指针，定义在 `timers.c` 中。

文件作用域内的私有函数使用 `prv` 前缀。

2.5.4 格式

某些演示应用使用制表符，并始终将一个制表符设置为四个空格。内核已不再使用制表符。

2.5.5 宏命名

大多数宏均使用大写字母书写，并以小写字母作为前缀，用于表明宏的定义位置。表 3 列出了这些前缀。

前缀	宏定义位置
port（例如 <code>portMAX_DELAY</code> ）	<code>portable.h</code> 或 <code>portmacro.h</code>
task（例如 <code>taskENTER_CRITICAL()</code> ）	<code>task.h</code>
pd（例如 <code>pdTRUE</code> ）	<code>projdefs.h</code>
config（例如 <code>configUSE_PREEMPTION</code> ）	<code>FreeRTOSConfig.h</code>
err（例如 <code>errQUEUE_FULL</code> ）	<code>projdefs.h</code>

表 3 宏前缀

请注意，信号量 API 几乎完全由一组宏构成，但它遵循函数命名约定，而不是宏命名约定。

表 4 中定义的宏在 FreeRTOS 源代码中被广泛使用。

宏	值
pdTRUE	1
pdFALSE	0
pdPASS	1
pdFAIL	0

表 4 常用宏定义

2.5.6 大量类型转换的原因

FreeRTOS 源代码需要使用许多不同的编译器进行编译，而这些编译器在生成警告的方式和时机上存在差异。尤其是，不同编译器对类型转换的使用方式有不同要求。因此，FreeRTOS 源代码中的类型转换比通常情况下需要的更多。

1. 当前版本的调度算法见第 4.12 节；英文原文此处写作第 4.13 节。↩

3 堆内存管理

3.1 引言

3.1.1 先决条件

成为熟练的 C 语言程序员是使用 FreeRTOS 的先决条件，因此本章假设读者熟悉以下概念：

- 构建 C 项目的不同编译和链接阶段。
- 栈和堆是什么。
- 标准 C 库的 `malloc()` 和 `free()` 函数。

3.1.2 范围

本章涵盖：

- FreeRTOS 何时分配 RAM。
- FreeRTOS 提供的五种示例内存分配方案。
- 选择哪种内存分配方案。

3.1.3 静态与动态内存分配的切换

以下章节将介绍内核对象，如任务、队列、信号量和事件组。保存这些对象所需的 RAM 可以在编译时静态分配，也可以在运行时动态分配。动态分配减少了设计和规划工作，简化了 API，并最小化了 RAM 占用。静态分配更具确定性，无需处理内存分配失败，并消除了堆碎片化的风险（即堆中有足够的空闲内存，但不在一个可用的连续块中）。

使用静态分配内存创建内核对象的 FreeRTOS API 函数仅在 `configSUPPORT_STATIC_ALLOCATION` 在 FreeRTOSConfig.h 中设置为 1 时可用。使用动态分配内存创建内核对象的 FreeRTOS API 函数仅在 `configSUPPORT_DYNAMIC_ALLOCATION` 在 FreeRTOSConfig.h 中设置为 1 或未定义时可用。两个常量可以同时设置为 1。

有关 `configSUPPORT_STATIC_ALLOCATION` 的更多信息，请参见第 3.4 节“使用静态内存分配”。

3.1.4 使用动态内存分配

动态内存分配是一个 C 编程概念，不是 FreeRTOS 或多任务处理特有的概念。它与 FreeRTOS 相关，因为内核对象可以选择使用动态分配的内存创建，而通用的 C 库函数 `malloc()` 和 `free()` 可能因以下一个或多个原因不适用：

- 它们并不总是适用于小型嵌入式系统。
- 它们的实现可能相对较大，占用宝贵的代码空间。
- 它们很少是线程安全的。
- 它们不是确定性的：执行这些函数所需的时间每次调用都会不同。
- 它们可能遭受碎片化问题（堆中有足够的空闲内存，但不在一个可用的连续块中）。
- 它们可能使链接器配置复杂化。
- 如果堆空间允许增长到被其他变量使用的内存中，它们可能成为难以调试的错误来源。

3.1.5 动态内存分配的选项

早期版本的 FreeRTOS 使用内存池分配方案，其中不同大小的内存块池在编译时预分配，然后由内存分配函数返回。尽管块分配在实时系统中很常见，但它已从 FreeRTOS 中移除，因为在非常小的嵌入式系统中，它对 RAM 的低效使用导致了大量的支持请求。

FreeRTOS 现在将内存分配视为可移植层的一部分（而不是核心代码库的一部分）。这是因为不同的嵌入式系统有不同的动态内存分配和时序要求，因此单一的动态内存分配算法只适用于一部分应用程序。此外，从核心代码库中移除动态内存分配使应用程序编写者能够在适当的时候提供自己的特定实现。

当 FreeRTOS 需要 RAM 时，它调用 `pvPortMalloc()` 而不是 `malloc()`。同样，当 FreeRTOS 释放之前分配的 RAM 时，它调用 `vPortFree()` 而不是 `free()`。`pvPortMalloc()` 具有与标准 C 库 `malloc()` 函数相同的原型，`vPortFree()` 具有与标准 C 库 `free()` 函数相同的原型。

`pvPortMalloc()` 和 `vPortFree()` 是公开函数，因此也可以从应用程序代码中调用。

FreeRTOS 附带了五个 `pvPortMalloc()` 和 `vPortFree()` 的示例实现，所有这些都在本章中记录。FreeRTOS 应用程序可以使用其中一个示例实现，或提供自己的实现。

五个示例分别在 `heap_1.c`、`heap_2.c`、`heap_3.c`、`heap_4.c` 和 `heap_5.c` 源文件中定义，所有这些文件都位于 `FreeRTOS/Source/portable/MemMang` 目录中。

3.2 示例内存分配方案

3.2.1 Heap_1

小型专用嵌入式系统通常只在启动 FreeRTOS 调度器之前创建任务和其他内核对象。在这种情况下，内存存在应用程序开始执行任何实时功能之前仅由内核（动态）分配，并且该内存存在应用程序的整个生命周期中保持分配状态。这意味着所选分配方案不必考虑更复杂的内存分配问题，如确定性和碎片化，而是可以优先考虑代码大小和简洁性等属性。

Heap_1.c 实现了一个非常基本的 `pvPortMalloc()` 版本，并且不实现 `vPortFree()`。从不删除任务或其他内核对象的应用程序有可能使用 heap_1。一些商业关键和安全关键系统通常会禁止使用动态内存分配，但也可以使用 heap_1。关键系统通常禁止动态内存分配，因为存在与非确定性、内存碎片化和分配失败相关的不确定性。Heap_1 始终是确定性的，并且不会导致内存碎片化。

Heap_1 的 `pvPortMalloc()` 实现每次被调用时，只是简单地将一个名为 FreeRTOS 堆的 `uint8_t` 数组细分为更小的块。FreeRTOSConfig.h 中的常量 `configTOTAL_HEAP_SIZE` 设置数组的大小（以字节为单位）。将堆实现为静态分配的数组使得 FreeRTOS 看起来消耗了大量 RAM，因为堆成为了 FreeRTOS 数据的一部分。

每个动态分配的任务会导致两次调用 `pvPortMalloc()`。第一次分配任务控制块（TCB），第二次分配任务的栈。图 3.1 演示了 heap_1 如何在创建任务时细分简单数组。

参考图 3.1:

- A 显示了创建任何任务之前的数组——整个数组都是空闲的。
- B 显示了创建一个任务后的数组。
- C 显示了创建三个任务后的数组。

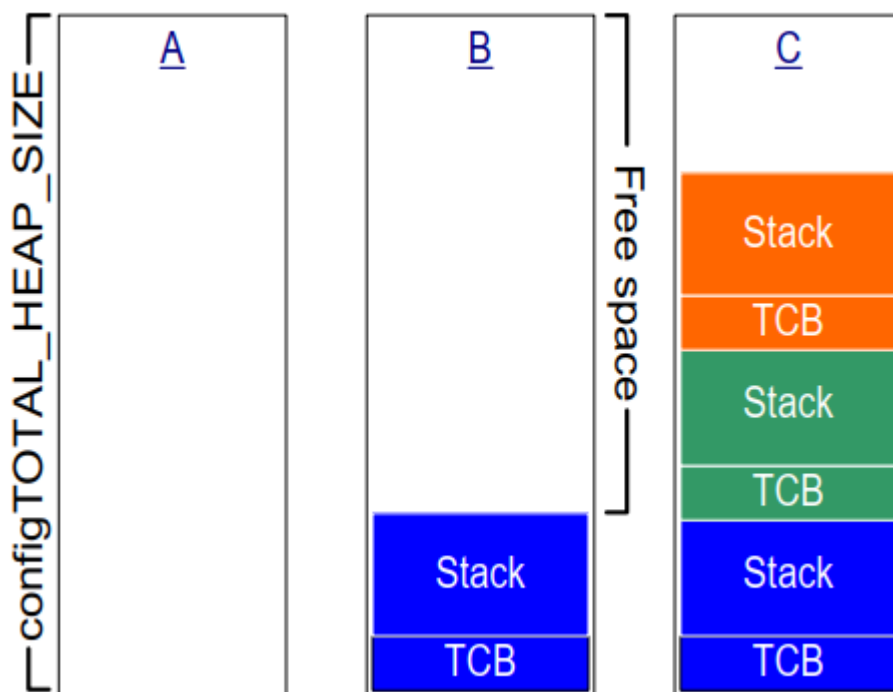


图 3.1 每次创建任务时从 `heap_1` 数组分配 RAM

3.2.2 Heap_2

Heap_2 已被 heap_4 取代，后者包含增强的功能。Heap_2 保留在 FreeRTOS 发行版中是为了向后兼容，不建议用于新设计。

Heap_2.c 也通过细分由 `configTOTAL_HEAP_SIZE` 常量确定大小的数组来工作。它使用最佳匹配算法分配内存，并且与 heap_1 不同，它确实实现了 `vPortFree()`。同样，将堆实现为静态分配的数组使得 FreeRTOS 看起来消耗了大量 RAM，因为堆成为了 FreeRTOS 数据的一部分。

最佳匹配算法确保 `pvPortMalloc()` 使用与请求字节数大小最接近的空闲内存块。例如，考虑以下场景：

- 堆包含三个空闲内存块，分别为 5 字节、25 字节和 100 字节。
- `pvPortMalloc()` 请求 20 字节的 RAM。

能够容纳所请求字节数的最小空闲 RAM 块是 25 字节的块，因此 `pvPortMalloc()` 将 25 字节的块分割成一个 20 字节的块和一个 5 字节的块，然后返回指向 20 字节块的指针^[1]。新的 5 字节块仍可用于将来的 `pvPortMalloc()` 调用。

与 heap_4 不同，heap_2 不会将相邻的空闲块合并成一个更大的块，因此它比 heap_4 更容易受到碎片化的影响。然而，如果分配和随后释放的块始终是相同的大小，则碎片化不是问题。

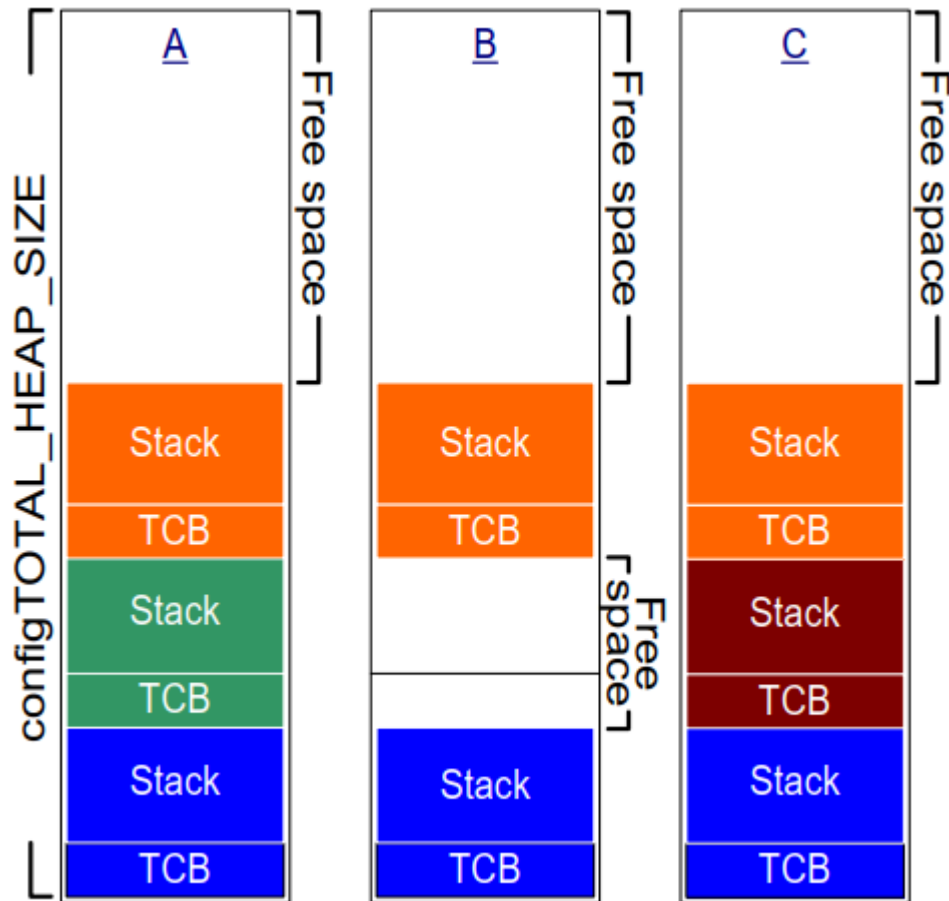


图 3.2 创建和删除任务时从 heap_2 数组分配和释放 RAM

图 3.2 演示了在创建、删除和再次创建任务时最佳匹配算法的工作方式。参考图 3.2:

- A 显示了分配三个任务后的数组。数组顶部保留了一个大的空闲块。
- B 显示了删除其中一个任务后的数组。数组顶部的大空闲块仍然存在。现在还有两个较小的空闲块，它们之前保存了被删除任务的 TCB 和栈。
- C 显示了创建另一个任务后的情况。创建任务导致了从 `xTaskCreate()` API 函数内部两次调用 `pvPortMalloc()`，一次分配新的 TCB，另一次分配任务栈。本书第 3.4 节描述了 `xTaskCreate()`。

每个 TCB 的大小相同，因此最佳匹配算法重用之前保存被删除任务 TCB 的 RAM 块来保存创建任务的 TCB。

如果分配给新创建任务的栈大小与分配给之前删除任务的栈大小相同，则最佳匹配算法重用之前保存被删除任务栈的 RAM 块来保存创建任务的栈。

数组顶部较大的未分配块保持不变。

Heap_2 不是确定性的，但比大多数标准库实现的 `malloc()` 和 `free()` 更快。

3.2.3 Heap_3

Heap_3.c 使用标准库的 `malloc()` 和 `free()` 函数，因此链接器配置定义堆大小，而不使用 `configTOTAL_HEAP_SIZE` 常量。

Heap_3 通过在执行期间临时挂起 FreeRTOS 调度器来使 `malloc()` 和 `free()` 成为线程安全的。第 8 章“资源管理”涵盖了线程安全和调度器挂起。

3.2.4 Heap_4

与 heap_1 和 heap_2 一样，heap_4 通过将数组细分为更小的块来工作。和之前一样，该数组是静态分配的，并由 `configTOTAL_HEAP_SIZE` 确定大小，这使得 FreeRTOS 看起来使用了大量 RAM，因为堆成为了 FreeRTOS 数据的一部分。

Heap_4 使用首次适应算法分配内存。与 heap_2 不同，heap_4 将相邻的空闲内存块合并（结合）成一个更大的块，从而最小化内存碎片化的风险。

首次适应算法确保 `pvPortMalloc()` 使用第一个足够大以容纳所请求字节数的空闲内存块。例如，考虑以下场景：

- 堆包含三个空闲内存块，按照它们在数组中出现的顺序，分别为 5 字节、200 字节和 100 字节。
- `pvPortMalloc()` 请求 20 字节的 RAM。

能够容纳所请求字节数的第一个空闲 RAM 块是 200 字节的块，因此 `pvPortMalloc()` 将 200 字节的块分割成一个 20 字节的块和一个 180 字节的块^[2]，然后返回指向 20 字节块的指针。新的 180 字节块仍可用于将来的 `pvPortMalloc()` 调用。

Heap_4 将相邻的空闲块合并（结合）成一个更大的块，最小化碎片化的风险，使其适用于重复分配和释放不同大小 RAM 块的应用程序。

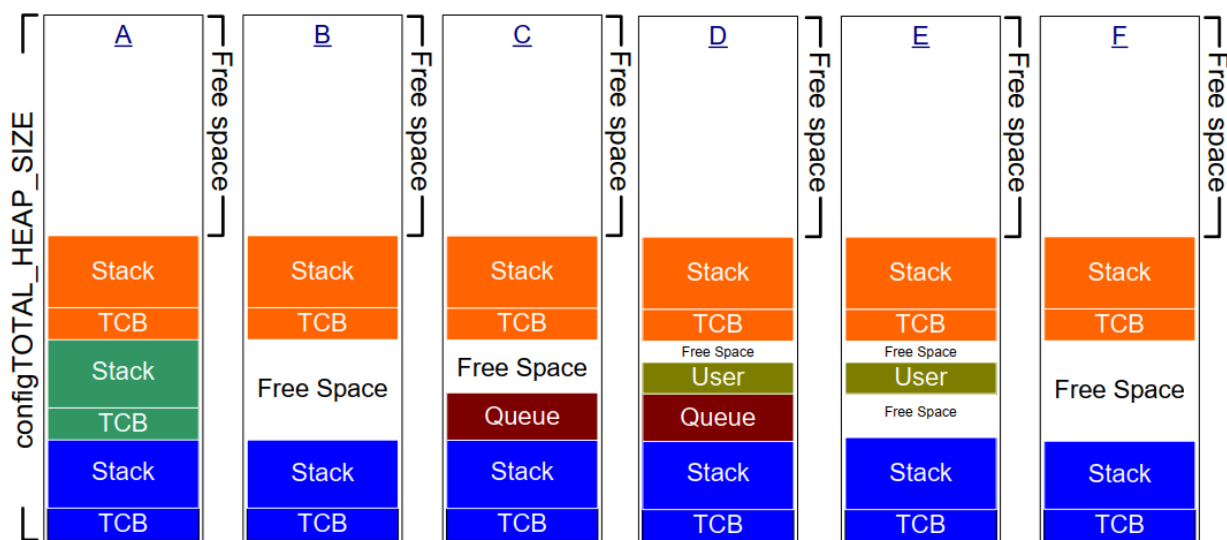


图 3.3 从 heap_4 数组分配和释放 RAM

图 3.3 演示了带有内存合并功能的 heap_4 首次适应算法的工作方式。参考图 3.3：

- A 显示了创建三个任务后的数组。数组顶部保留了一个大的空闲块。
- B 显示了删除其中一个任务后的数组。数组顶部的大空闲块仍然存在。现在在之前保存被删除任务的 TCB 和栈的位置有一个空闲块。与 heap_2 示例不同，heap_4 将之前分别保存被删除任务的 TCB 和栈的两个内存块合并成一个更大的单独空闲块。
- C 显示了创建 FreeRTOS 队列后的情况。本书第 5.3 节描述了用于动态分配队列的 `xQueueCreate()` API 函数。`xQueueCreate()` 调用 `pvPortMalloc()` 来分配队列使用的 RAM。由于 heap_4 使用首次适应算法，`pvPortMalloc()` 从第一个足够大以容纳队列的空闲 RAM 块中分配 RAM，在图 3.3 中，这是通过删除任务释放的 RAM。队列并没有消耗空闲块中的所有 RAM，因此该块被分割成两块，未使用的部分仍可用于将来的 `pvPortMalloc()` 调用。
- D 显示了从应用程序代码直接调用 `pvPortMalloc()` 后的情况，而不是通过调用 FreeRTOS API 函数间接调用。用户分配的块足够小，可以放入第一个空闲块中，即队列分配的内存和其后跟随的 TCB 分配的内存之间的块。

通过删除任务释放的内存现在已分割成三个单独的块；第一个块保存队列，第二个块保存用户分配的内存，第三个块保持空闲。

- E 显示了删除队列后的情况，这会自动释放分配给已删除队列的内存。现在用户分配块的两侧都有空闲内存。

- F 显示了释放用户分配内存后的情况。之前由用户分配块使用的内存已与两侧的空闲内存合并，创建了一个更大的单独空闲块。

Heap_4 不是确定性的，但比大多数标准库实现的 `malloc()` 和 `free()` 更快。

3.2.5 Heap_5

Heap_5 使用与 heap_4 相同的分配算法。与 heap_4 不同，heap_4 仅限于从单个数组分配内存，而 heap_5 可以将来自多个分离内存空间的内存组合成单个堆。当运行 FreeRTOS 的系统提供的 RAM 在系统内存映射中不作为单个连续（无间隔）块出现时，Heap_5 非常有用。

3.2.6 初始化 heap_5: vPortDefineHeapRegions() API 函数

`vPortDefineHeapRegions()` 通过指定构成 heap_5 管理的堆的每个独立内存区域的起始地址和大小来初始化 heap_5。Heap_5 是唯一需要显式初始化且在调用 `vPortDefineHeapRegions()` 之后才能使用的堆分配方案。这意味着内核对象，如任务、队列和信号量，不能在调用 `vPortDefineHeapRegions()` 之前动态创建。

```
void vPortDefineHeapRegions( const HeapRegion_t * const pxHeapRegions );
```

代码清单 3.1 `vPortDefineHeapRegions()` API 函数原型

`vPortDefineHeapRegions()` 将 `HeapRegion_t` 结构数组作为其唯一参数。每个结构定义将成为堆一部分的内存块的起始地址和大小——整个结构数组定义了整个堆空间。

```
typedef struct HeapRegion
{
    /* 将成为堆一部分的内存块的起始地址。*/
    uint8_t *pucStartAddress;

    /* 内存块的大小（以字节为单位）。*/
    size_t xSizeInBytes;
} HeapRegion_t;
```

代码清单 3.2 `HeapRegion_t` 结构

参数：

- `pxHeapRegions`

指向 `HeapRegion_t` 结构数组起始位置的指针。每个结构定义将成为堆一部分的内存块的起始地址和大小。

数组中的 `HeapRegion_t` 结构必须按起始地址排序；描述具有最低起始地址的内存区域的

`HeapRegion_t` 结构必须是数组中的第一个结构，描述具有最高起始地址的内存区域的

`HeapRegion_t` 结构必须是数组中的最后一个结构。

使用 `pucStartAddress` 成员设置为 `NULL` 的 `HeapRegion_t` 结构标记数组的结束。

举例说明，考虑图 3.4 A 中所示的假设内存映射，其中包含三个独立的 RAM 块：RAM1、RAM2 和 RAM3。假设可执行代码放在只读存储器中，图中未显示。

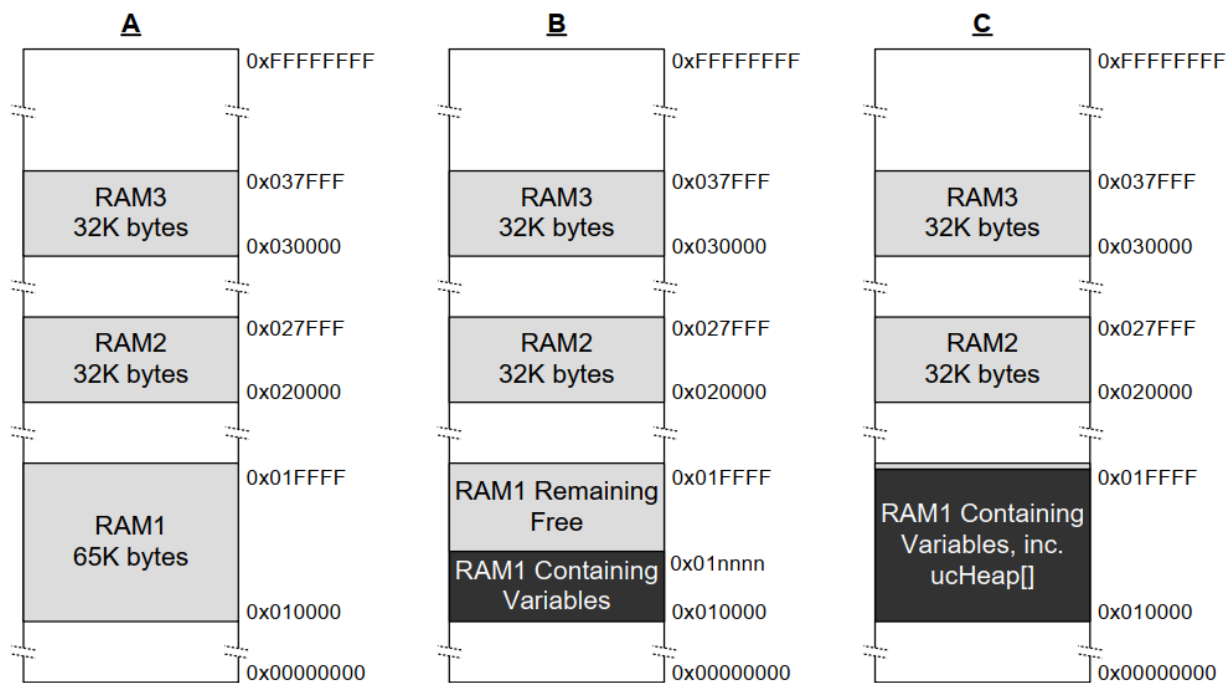


图 3.4 内存映射

代码清单 3.3 显示了一个 `HeapRegion_t` 结构数组，它们共同完整地描述了这三个 RAM 块。

```

/* 定义三个 RAM 区域的起始地址和大小。 */
#define RAM1_START_ADDRESS ( ( uint8_t * ) 0x00010000 )
#define RAM1_SIZE ( 64 * 1024 )

#define RAM2_START_ADDRESS ( ( uint8_t * ) 0x00020000 )
#define RAM2_SIZE ( 32 * 1024 )

#define RAM3_START_ADDRESS ( ( uint8_t * ) 0x00030000 )
#define RAM3_SIZE ( 32 * 1024 )

/* 创建一个 HeapRegion_t 定义数组, 为三个 RAM 区域各有一个索引,
   并以包含 NULL 地址的 HeapRegion_t 结构终止数组。HeapRegion_t 结构必须
   按起始地址顺序出现, 包含最低起始地址的结构最先出现。 */
const HeapRegion_t xHeapRegions[] =
{
    { RAM1_START_ADDRESS, RAM1_SIZE },
    { RAM2_START_ADDRESS, RAM2_SIZE },
    { RAM3_START_ADDRESS, RAM3_SIZE },
    { NULL, 0 } /* 标记数组的结束。 */
};

int main( void )
{
    /* 初始化 heap_5。 */
    vPortDefineHeapRegions( xHeapRegions );

    /* 在此处添加应用程序代码。 */
}

```

代码清单 3.3 一个 HeapRegion_t 结构数组, 共同完整描述 3 个 RAM 区域

尽管代码清单 3.3 正确地描述了 RAM, 但它并没有演示一个可用的示例, 因为它将所有 RAM 分配给了堆, 没有留下任何 RAM 供其他变量使用。

构建过程的链接阶段为每个变量分配一个 RAM 地址。链接器可用的 RAM 通常由链接器配置文件 (如链接器脚本) 描述。在图 3.4 B 中, 假设链接器脚本包含了 RAM1 的信息, 但没有包含 RAM2 或 RAM3 的信息。因此, 链接器将变量放置在 RAM1 中, 只留下 RAM1 中地址 0x0001nnnn 以上的部分可供 heap_5 使用。0x0001nnnn 的实际值取决于应用程序中包含的所有变量的总大小。链接器没有使用 RAM2 和 RAM3 的全部空间, 因此整个 RAM2 和整个 RAM3 都可供 heap_5 使用。

代码清单 3.3 中显示的代码会导致 heap_5 分配在地址 0x0001nnnn 以下的 RAM 与用于保存变量的 RAM 重叠。如果您将 `xHeapRegions[]` 数组中第一个 `HeapRegion_t` 结构的起始地址设置为 0x0001nnnn, 而不是起始地址 0x00010000, 则堆不会与链接器使用的 RAM 重叠。但是, 这不是推荐的解决方案, 因为:

- 起始地址可能不容易确定。

- 链接器使用的 RAM 量可能在将来的构建中发生变化，这将需要更新 `HeapRegion_t` 结构中使用的起始地址。
- 构建工具将不知道（因此无法警告）链接器使用的 RAM 和 `heap_5` 使用的 RAM 是否重叠。

代码清单 3.4 演示了一个更方便和可维护的示例。它声明了一个名为 `ucHeap` 的数组。`ucHeap` 是一个普通变量，因此它成为链接器分配给 RAM1 的数据的一部分。`xHeapRegions` 数组中的第一个 `HeapRegion_t` 结构描述了 `ucHeap` 的起始地址和大小，因此 `ucHeap` 成为 `heap_5` 管理的内存的一部分。`ucHeap` 的大小可以增加，直到链接器使用的 RAM 消耗完所有 RAM1，如图 3.4 C 所示。

```
/* 定义链接器未使用的两个 RAM 区域的起始地址和大小。 */
#define RAM2_START_ADDRESS ( ( uint8_t * ) 0x00020000 )
#define RAM2_SIZE ( 32 * 1024 )

#define RAM3_START_ADDRESS ( ( uint8_t * ) 0x00030000 )
#define RAM3_SIZE ( 32 * 1024 )

/* 声明一个将成为 heap_5 使用的堆的一部分的数组。该数组将由链接器放置在 RAM1 中。 */
#define RAM1_HEAP_SIZE ( 30 * 1024 )
static uint8_t ucHeap[ RAM1_HEAP_SIZE ];

/* 创建一个 HeapRegion_t 定义数组。在代码清单 3.3 中，第一个条目描述了整个 RAM1，
   因此 heap_5 将使用整个 RAM1，而这次第一个条目只描述了 ucHeap 数组，因此
   heap_5 只会使用 RAM1 中包含 ucHeap 数组的部分。HeapRegion_t 结构仍然必须按
   起始地址顺序出现，包含最低起始地址的结构最先出现。 */

const HeapRegion_t xHeapRegions[] =
{
    { ucHeap,          RAM1_HEAP_SIZE },
    { RAM2_START_ADDRESS, RAM2_SIZE },
    { RAM3_START_ADDRESS, RAM3_SIZE },
    { NULL,            0 }             /* 标记数组的结束。 */
};
```

代码清单 3.4 一个 `HeapRegion_t` 结构数组，描述所有 RAM2、所有 RAM3，但仅部分 RAM1

代码清单 3.4 中演示的技术的优点包括：

- 不需要使用硬编码的起始地址。
- `HeapRegion_t` 结构中使用的地址将由链接器自动设置，因此即使在将来的构建中链接器使用的 RAM 量发生变化，它也始终保持正确。
- `heap_5` 分配的 RAM 不可能与链接器放入 RAM1 的数据重叠。
- 如果 `ucHeap` 太大，应用程序将无法链接。

3.3 堆相关的实用函数和宏

3.3.1 定义堆起始地址

Heap_1、heap_2 和 heap_4 从由 `configTOTAL_HEAP_SIZE` 确定大小的静态分配数组中分配内存。本节将这些分配方案统称为 heap_n。

有时需要将堆放置在特定的内存地址。例如，动态创建的任务的栈来自堆，因此可能需要将堆定位在快速的内部存储器中，而不是慢速的外部存储器中。（请参见下面的子章节 将任务栈放置在快速内存中，了解在快速内存中分配任务栈的另一种方法）。`configAPPLICATION_ALLOCATED_HEAP` 编译时常量使应用程序能够声明数组，取代原本在 heap_n.c 源文件中的声明。在应用程序代码中声明数组使应用程序编写者能够指定其起始地址。

如果在 FreeRTOSConfig.h 中将 `configAPPLICATION_ALLOCATED_HEAP` 设置为 1，则使用 FreeRTOS 的应用程序必须分配一个名为 `ucHeap` 且由 `configTOTAL_HEAP_SIZE` 常量确定大小的 `uint8_t` 数组。

将变量放置在特定内存地址所需的语法取决于所使用的编译器，因此请参考编译器的文档。以下是两个编译器的示例：

- 代码清单 3.5 显示了 GCC 编译器声明数组并将数组放置在名为 `.my_heap` 的内存段中所需的语法。
- 代码清单 3.6 显示了 IAR 编译器声明数组并将数组放置在绝对内存地址 0x20000000 所需的语法。

```
uint8_t ucHeap[ configTOTAL_HEAP_SIZE ] __attribute__ ( ( section( ".my_heap" ) ) );
```

代码清单 3.5 使用 GCC 语法声明将由 heap_4 使用的数组，并将数组放置在名为 .my_heap 的内存段中

```
uint8_t ucHeap[ configTOTAL_HEAP_SIZE ] @ 0x20000000;
```

代码清单 3.6 使用 IAR 语法声明将由 heap_4 使用的数组，并将数组放置在绝对地址 0x20000000

3.3.2 xPortGetFreeHeapSize() API 函数

`xPortGetFreeHeapSize()` API 函数返回调用该函数时堆中可用的空闲字节数。它不提供有关堆碎片化的信息。

`xPortGetFreeHeapSize()` 未为 heap_3 实现。

```
size_t xPortGetFreeHeapSize( void );
```

代码清单 3.7 `xPortGetFreeHeapSize()` API 函数原型

返回值：

- `xPortGetFreeHeapSize()` 返回调用时堆中仍未分配的字节数。

3.3.3 `xPortGetMinimumEverFreeHeapSize()` API 函数

`xPortGetMinimumEverFreeHeapSize()` API 函数返回自 FreeRTOS 应用程序开始执行以来，堆中曾存在过的未分配字节的最小数量。

`xPortGetMinimumEverFreeHeapSize()` 返回的值指示应用程序曾经有多接近耗尽堆空间。例如，如果 `xPortGetMinimumEverFreeHeapSize()` 返回 200，则自应用程序开始执行以来的某个时间，它离耗尽堆空间只差 200 字节。

`xPortGetMinimumEverFreeHeapSize()` 也可用于优化堆大小。例如，如果在执行您知道具有最高堆使用率的代码后，`xPortGetMinimumEverFreeHeapSize()` 返回 2000，则 `configTOTAL_HEAP_SIZE` 可以减少最多 2000 字节。

`xPortGetMinimumEverFreeHeapSize()` 仅在 heap_4 和 heap_5 中实现。

```
size_t xPortGetMinimumEverFreeHeapSize( void );
```

代码清单 3.8 `xPortGetMinimumEverFreeHeapSize()` API 函数原型

返回值：

- `xPortGetMinimumEverFreeHeapSize()` 返回自 FreeRTOS 应用程序开始执行以来堆中存在过的未分配字节的最小数量。

3.3.4 `vPortGetHeapStats()` API 函数

Heap_4 和 heap_5 实现了 `vPortGetHeapStats()`，该函数填充通过引用传递的 `HeapStats_t` 结构作为其唯一参数。

代码清单 3.9 显示了 `vPortGetHeapStats()` 函数原型。代码清单 3.10 显示了 `HeapStats_t` 结构成员。

```
void vPortGetHeapStats( HeapStats_t *xHeapStats );
```

代码清单 3.9 `vPortGetHeapStats()` API 函数原型

```
/* vPortGetHeapStats() 函数的原型。 */
void vPortGetHeapStats( HeapStats_t *xHeapStats );

/* HeapStats_t 结构的定义。所有大小以字节为单位。 */
typedef struct xHeapStats
{
    /* 当前可用的总堆大小 - 这是所有空闲块的总和，不是最大的可用块。 */
    size_t xAvailableHeapSpaceInBytes;

    /* 调用 vPortGetHeapStats() 时堆内最大空闲块的大小。 */
    size_t xSizeOfLargestFreeBlockInBytes;

    /* 调用 vPortGetHeapStats() 时堆内最小空闲块的大小。 */
    size_t xSizeOfSmallestFreeBlockInBytes;

    /* 调用 vPortGetHeapStats() 时堆内的空闲内存块数量。 */
    size_t xNumberOfFreeBlocks;

    /* 自系统启动以来，堆中曾出现过的总空闲内存（所有空闲块的总和）的最小值。 */
    size_t xMinimumEverFreeBytesRemaining;

    /* 返回了有效内存块的 pvPortMalloc() 调用次数。 */
    size_t xNumberOfSuccessfulAllocations;

    /* 成功释放了内存块的 vPortFree() 调用次数。 */
    size_t xNumberOfSuccessfulFrees;
} HeapStats_t;
```

代码清单 3.10 `HeapStats_t()` 结构

3.3.5 收集每个任务的堆使用统计信息

`vTaskGetInfo()` API 函数（在本书的 TBD-RB 节中记录）使用有关任务的信息填充 `TaskStatus_t` 结构。如果在 `FreeRTOSConfig.h` 中将 `configTRACK_TASK_MEMORY_ALLOCATIONS` 编译时常量设置为 1，则该结构包含以下附加信息：

- 任务调用 `pvPortMalloc()` 的次数。
- 任务调用 `vPortFree()` 的次数。
- 在调用 `vTaskGetInfo()` 时，任务已分配但尚未被任何任务释放的堆字节数。

- 自任务启动以来，任务在任何给定时间分配的最大堆内存量。

3.3.6 Malloc 失败钩子函数

与标准库的 `malloc()` 函数一样，如果 `pvPortMalloc()` 无法分配所请求的 RAM 量，则返回 NULL。malloc 失败钩子（或回调）是一个应用程序提供的函数，在 `pvPortMalloc()` 返回 NULL 时被调用。您必须在 `FreeRTOSConfig.h` 中将 `configUSE_MALLOC_FAILED_HOOK` 设置为 1 才能使回调发生。如果在使用动态内存分配创建内核对象的 FreeRTOS API 函数内部调用了 malloc 失败钩子，则该对象不会被创建。

如果 `configUSE_MALLOC_FAILED_HOOK` 在 `FreeRTOSConfig.h` 中设置为 1，则应用程序必须提供一个具有代码清单 3.11 所示名称和原型的 malloc 失败钩子函数。应用程序可以以适合应用程序的任何方式实现该函数。许多提供的 FreeRTOS 演示应用程序将分配失败视为致命错误，但对于生产系统来说这不是最佳实践，生产系统应从分配失败中优雅地恢复。

```
void vApplicationMallocFailedHook( void );
```

代码清单 3.11 malloc 失败钩子函数名称和原型

3.3.7 将任务栈放置在快速内存中

由于栈的写入和读取频率很高，它们应放置在快速内存中，但这可能不是您希望堆所在的位置。FreeRTOS 使用 `pvPortMallocStack()` 和 `vPortFreeStack()` 宏来可选地使 FreeRTOS API 代码内部分配的栈拥有自己的内存分配器。如果您希望栈来自由 `pvPortMalloc()` 管理的堆，则保持 `pvPortMallocStack()` 和 `vPortFreeStack()` 未定义，因为它们默认分别调用 `pvPortMalloc()` 和 `vPortFree()`。否则，定义宏以调用应用程序提供的函数，如代码清单 3.12 所示。

```
/* 应用程序编写者提供的用于从快速 RAM 区域分配和释放内存的函数。 */

void *pvMallocFastMemory( size_t xWantedSize );

void vPortFreeFastMemory( void *pvBlockToFree );

/* 将以下内容添加到 FreeRTOSConfig.h 中，以将 pvPortMallocStack()
   和 vPortFreeStack() 宏映射到使用快速内存的函数。 */

#define pvPortMallocStack( x ) pvMallocFastMemory( x )

#define vPortFreeStack( x ) vPortFreeFastMemory( x )
```

代码清单 3.12 将 `pvPortMallocStack()` 和 `vPortFreeStack()` 宏映射到应用程序定义的内存分配器

3.4 使用静态内存分配

第 3.1.4 节列出了动态内存分配带来的一些缺点。为了避免这些问题，静态内存分配允许开发人员显式创建应用程序所需的每个内存块。这具有以下优点：

- 所有所需的内存在编译时已知。
- 所有内存都是确定性的。

还有其他优点，但这些优点也带来了一些复杂性。主要的复杂性是增加了一些额外的用户函数来管理一些内核内存，第二个复杂性是需要确保所有静态内存在合适的作用域中声明。

3.4.1 启用静态内存分配

静态内存分配通过在 `FreeRTOSConfig.h` 中将 `configSUPPORT_STATIC_ALLOCATION` 设置为 1 来启用。启用此配置后，内核会启用所有内核函数的 `static` 版本。这些函数包括：

- `xTaskCreateStatic`
- `xEventGroupCreateStatic`
- `xEventGroupGetStaticBuffer`
- `xQueueGenericCreateStatic`
- `xQueueGenericGetStaticBuffers`
- `xQueueCreateMutexStatic`
 - 如果 `configUSE_MUTEXES` 为 1
- `xQueueCreateCountingSemaphoreStatic`
 - 如果 `configUSE_COUNTING_SEMAPHORES` 为 1
- `xStreamBufferGenericCreateStatic`
- `xStreamBufferGetStaticBuffers`
- `xTimerCreateStatic`
 - 如果 `configUSE_TIMERS` 为 1
- `xTimerGetStaticBuffer`
 - 如果 `configUSE_TIMERS` 为 1

这些函数将在本书的相应章节中解释。

3.4.2 静态内部内核内存

当启用静态内存分配器时，空闲任务和定时器任务（如果启用）将使用用户函数提供的静态内存。这些用户函数是：

- `vApplicationGetTimerTaskMemory`
 - 如果 `configUSE_TIMERS` 为 1
- `vApplicationGetIdleTaskMemory`

3.4.2.1 vApplicationGetTimerTaskMemory

如果同时启用了 `configSUPPORT_STATIC_ALLOCATION` 和 `configUSE_TIMERS`，内核将调用 `vApplicationGetTimerTaskMemory()` 以允许应用程序创建并返回用于定时器任务 TCB 和定时器任务栈的内存缓冲区。该函数还将返回定时器任务栈的大小。代码清单 3.13 显示了定时器任务内存函数的一个建议实现。

```
void vApplicationGetTimerTaskMemory( StaticTask_t **ppxTimerTaskTCBBuffer,
                                     StackType_t **ppxTimerTaskStackBuffer,
                                     uint32_t *pulTimerTaskStackSize )
{
    /* 如果要提供给定时器任务的缓冲区在此函数内部声明，则必须声明为 static -
    否则它们将在栈上分配，因此在退出此函数后将不存在。 */
    static StaticTask_t xTimerTaskTCB;
    static StackType_t uxTimerTaskStack[ configMINIMAL_STACK_SIZE ];

    /* 传递指向 StaticTask_t 结构的指针，定时器任务的状态将存储在该结构中。 */
    *ppxTimerTaskTCBBuffer = &xTimerTaskTCB;

    /* 传递将用作定时器任务栈的数组。 */
    *ppxTimerTaskStackBuffer = uxTimerTaskStack;

    /* 传递 *ppxTimerTaskStackBuffer 所指向数组的栈大小。
    注意栈大小是 StackType_t 的计数。 */
    *pulTimerTaskStackSize = sizeof(uxTimerTaskStack) / sizeof(*uxTimerTaskStack);
}
```

代码清单 3.13 `vApplicationGetTimerTaskMemory` 的典型实现

由于在任何系统中（包括 SMP）只有一个定时器任务，解决定时器任务内存问题的一个有效方法是在 `vApplicationGetTimeTaskMemory()` 函数中分配静态缓冲区，并将缓冲区指针返回给内核。

3.4.2.2 vApplicationGetIdleTaskMemory

空闲任务在某个核没有可调度的任务时运行。空闲任务执行一些内务处理，如果启用了用户的 `vTaskIdleHook()`，也可以触发它。在对称多处理系统（SMP）中，还有其他核的非内务处理空闲任务，但这些任务在内部静态分配了 `configMINIMUM_STACK_SIZE` 字节。

调用 `vApplicationGetIdleTaskMemory` 函数以允许应用程序为“主”空闲任务创建所需的缓冲区。代码清单 3.14 显示了使用静态局部变量创建所需缓冲区的 `vApplicationIdleTaskMemory()` 函数的典型实现。

```
void vApplicationGetIdleTaskMemory( StaticTask_t **ppxIdleTaskTCBBuffer,
                                   StackType_t **ppxIdleTaskStackBuffer,
                                   uint32_t *pulIdleTaskStackSize )
{
    static StaticTask_t xIdleTaskTCB;
    static StackType_t uxIdleTaskStack[ configMINIMAL_STACK_SIZE ];

    *ppxIdleTaskTCBBuffer = &xIdleTaskTCB;
    *ppxIdleTaskStackBuffer = uxIdleTaskStack;
    *pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
}
```

代码清单 3.14 `vApplicationGetIdleTaskMemory` 的典型实现

-
1. 这是一个过度简化，因为 `heap_2` 在堆区域内存储块大小的信息，因此两个分割块的总和实际上将小于 25。↩
 2. 这是一个过度简化，因为 `heap_4` 在堆区域内存储块大小的信息，因此两个分割块的总和实际上将小于 200 字节。↩

4 任务管理

4.1 引言

4.1.1 范围

本章涵盖：

- FreeRTOS 如何为应用程序中的每个任务分配处理时间。
- FreeRTOS 如何在任何给定时间选择应执行哪个任务。
- 每个任务的相对优先级如何影响系统行为。
- 任务可以存在的状态。

本章还讨论：

- 如何实现任务。
- 如何创建一个或多个任务实例。
- 如何使用任务参数。
- 如何更改已创建任务的优先级。
- 如何删除任务。
- 如何使用任务实现周期性处理；后面的章节将介绍如何使用软件定时器实现相同功能。
- 空闲任务何时执行以及如何使用它。

本章介绍的概念对于理解如何使用 FreeRTOS 以及 FreeRTOS 应用程序的行为方式至关重要。因此，这是本书中最详细的章节。

4.2 任务函数

任务由 C 函数实现，并且必须符合代码清单 4.1 所示的函数原型：接收一个 `void` 指针参数，不返回任何值。

```
void vATaskFunction( void * pvParameters );
```

代码清单 4.1 任务函数原型

每个任务本身就是一个小程序。它有一个入口点，通常会永远在无限循环中运行，并且不会退出。代码清单 4.2 显示了一个典型任务的结构。

实现 FreeRTOS 任务的函数绝不能以任何方式返回：既不能包含 `return` 语句，也不能执行到函数末尾。如果不再需要某个任务，应如代码清单 4.2 所示显式删除它。

一个任务函数可以用来创建任意数量的任务；创建出的每个任务都是独立的执行实例。每个实例都有自己的栈，因此也有一份任务函数内所定义的自动变量（栈变量）副本。

```
void vATaskFunction( void * pvParameters )
{
    /*
     * 栈分配的变量可以在函数内部正常声明。
     * 使用此示例函数创建的每个任务实例都将在其栈上拥有自己的
     * lStackVariable 的独立实例。
     */
    long lStackVariable = 0;

    /*
     * 与栈分配的变量相反，使用 'static' 关键字声明的变量由链接器
     * 分配到内存中的特定位置。这意味着所有调用 vATaskFunction 的任务
     * 将共享 lStaticVariable 的同一个实例。
     */
    static long lStaticVariable = 0;

    for( ;; )
    {
        /* 实现任务功能的代码将写在这里。 */
    }

    /*
     * 如果任务实现曾经退出上述循环，则任务必须在到达其实现函数
     * 末尾之前被删除。当 NULL 作为参数传递给 vTaskDelete() API 函数时，
     * 表示要删除的任务是调用（当前）任务。
     */
    vTaskDelete( NULL );
}
```

代码清单 4.2 典型任务函数的结构

4.3 顶层任务状态

一个应用程序可能包含许多任务。如果运行应用程序的处理器包含单个核，则在任何给定时间只能执行一个任务。这意味着任务可以存在于两种状态之一：*运行中* 和 *未运行*。首先考虑这种简化的模型。本章后面将描述 *未运行* 状态的几个子状态。

当处理器正在执行该任务的代码时，任务处于 *运行中* 状态。当任务处于 *未运行* 状态时，任务暂停，其状态已保存，以便下次调度器决定它应进入 *运行中* 状态时可以恢复执行。当任务恢复执行时，它从离开 *运行中* 状态之前即将执行的指令处开始执行。

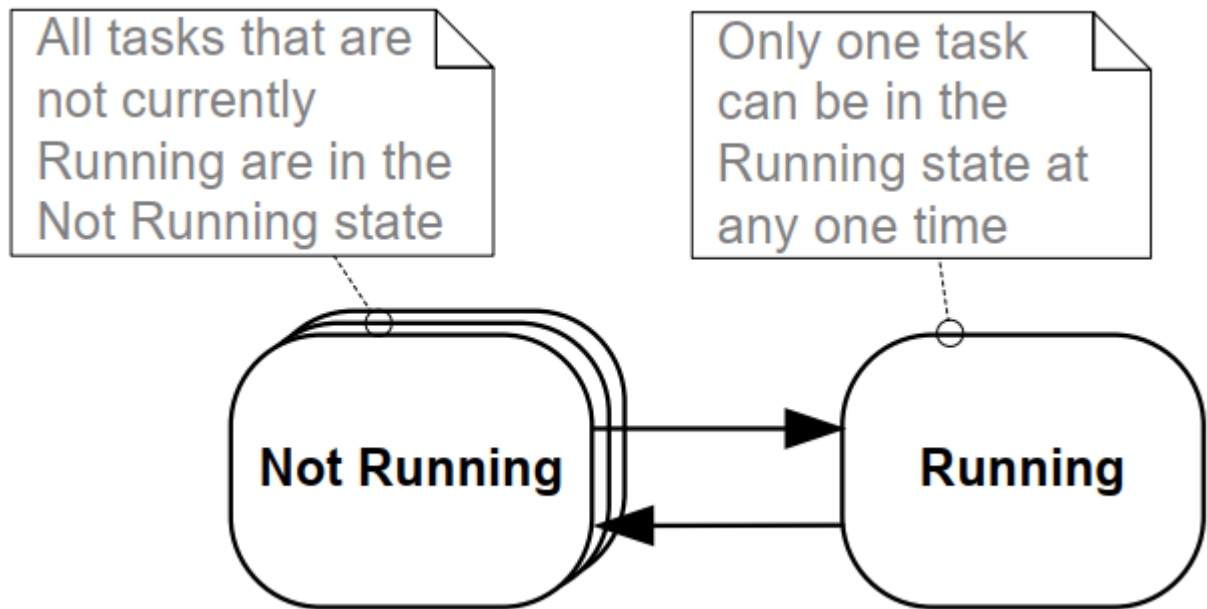


图 4.1 顶层任务状态和转换

任务从 *未运行* 状态转换到 *运行中* 状态称为“切入”或“换入”；反之，从 *运行中* 状态转换到 *未运行* 状态称为“切出”或“换出”。只有 FreeRTOS 调度器能够将任务切入或切出 *运行中* 状态。

4.4 任务创建

有六个 API 函数可用于创建任务：`xTaskCreate()`、`xTaskCreateStatic()`、`xTaskCreateRestricted()`、`xTaskCreateRestrictedStatic()`、`xTaskCreateAffinitySet()` 和 `xTaskCreateStaticAffinitySet()`

每个任务需要两个 RAM 块：一个用于保存其任务控制块（TCB），另一个用于存储其栈。名称中包含“Static”的 FreeRTOS API 函数使用预先分配的 RAM 块作为参数传入函数。相反，名称中不包含“Static”的 API 函数在运行时从系统堆动态分配所需的 RAM。

某些 FreeRTOS 移植版本支持任务以“受限”或“非特权”模式运行。名称中包含 `Restricted` 的 FreeRTOS API 函数所创建的任务只能访问有限的系统内存；名称中不包含 `Restricted` 的 API 函数所创建的任务则在“特权模式”下运行，可以访问完整的系统内存映射。

支持对称多处理（SMP）的 FreeRTOS 移植版本允许不同任务在同一 CPU 的多个核上同时运行。对于这些移植版本，可以使用名称中包含 `Affinity` 的函数指定任务在哪个核上运行。

FreeRTOS 任务创建 API 函数相当复杂。本文档中的大多数示例使用 `xTaskCreate()`，因为它是这些函数中最简单的。

4.4.1 xTaskCreate() API 函数

代码清单 4.3 显示了 `xTaskCreate()` API 函数原型。`xTaskCreateStatic()` 有两个额外的参数，分别指向预先分配用于保存任务数据结构和栈的内存。[第 2.5 节：数据类型和编码风格指南](#) 描述了使用的数据类型和命名约定。

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        configSTACK_DEPTH_TYPE usStackDepth,
                        void * pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t * pxCreatedTask );
```

代码清单 4.3 `xTaskCreate()` API 函数原型

`xTaskCreate()` 参数和返回值：

- `pvTaskCode`

任务就是永远不会退出的 C 函数，因此通常实现为无限循环。`pvTaskCode` 参数只是一个指向实现任务的函数的指针（实际上就是函数名）。

- `pcName`

任务的描述性名称。FreeRTOS 不以任何方式使用它，它纯粹作为调试辅助工具包含在内。通过人类可读的名称来识别任务比通过其句柄要简单得多。

应用程序定义的常量 `configMAX_TASK_NAME_LEN` 定义了任务名称的最大长度，包括 NULL 终止符。提供更长的字符串会导致字符串被截断。

- `usStackDepth`

指定为任务分配栈的大小。使用 `xTaskCreateStatic()` 而不是 `xTaskCreate()` 来使用预先分配的内存而不是动态分配的内存。

注意该值指定栈可以容纳的字数，而不是字节数。例如，如果栈是 32 位宽且 `usStackDepth` 为 128，则 `xTaskCreate()` 分配 512 字节的栈空间（128 * 4 字节）。

`configSTACK_DEPTH_TYPE` 是一个宏，允许应用程序编写者指定用于保存栈大小的数据类型。如果未定义，`configSTACK_DEPTH_TYPE` 默认为 `uint16_t`，因此如果栈深度乘以栈宽度大于 65535（最大可能的 16 位数字），则在 `FreeRTOSConfig.h` 中将 `configSTACK_DEPTH_TYPE` 定义为 `unsigned long` 或 `size_t`。

[第 13.3 节 栈溢出](#)，描述了一种选择最佳栈大小的实用方法。

- `pvParameters`

实现任务的函数接受一个 void 指针（`void *`）参数。`pvParameters` 是通过该参数传递给任务的值。

- `uxPriority`

定义任务的优先级。0 是最低优先级，`(configMAX_PRIORITIES - 1)` 是最高优先级。[第 4.5 节](#)介绍了由用户定义的 `configMAX_PRIORITIES` 常量。

如果指定的 `uxPriority` 大于 `(configMAX_PRIORITIES - 1)`，它将被限制为 `(configMAX_PRIORITIES - 1)`。

- `pxCreatedTask`

指向存储所创建任务句柄位置的指针。该句柄可在将来的 API 调用中使用，例如更改任务的优先级或删除任务。

`pxCreatedTask` 是一个可选参数，如果不需要任务句柄，可以设置为 NULL。

- 返回值

有两个可能的返回值：

- `pdPASS`

表示任务创建成功。

- `pdFAIL`

表示没有足够的堆内存来创建任务。[第 3 章](#)提供了更多关于堆内存管理的信息。

示例 4.1 创建任务

以下示例演示创建两个简单任务并启动它们所需的步骤。任务使用粗略的忙等待循环产生周期性延迟，从而周期性地打印字符串。两个任务以相同优先级创建，除打印的字符串不同外完全相同；其实现分别见代码清单 4.4 和代码清单 4.5。关于在任务中使用 `printf()` 的注意事项，请参见第 8 章。

```
void vTask1( void * pvParameters )
{
    /* ulCount 声明为 volatile 以确保它不会被优化掉。 */
    volatile unsigned long ulCount;

    for( ;; )
    {
        /* 打印当前任务的名称。 */
        vPrintLine( "Task 1 is running" );

        /* 延迟一段时间。 */
        for( ulCount = 0; ulCount < mainDELAY_LOOP_COUNT; ulCount++ )
        {
            /*
             * 这个循环只是一个非常粗糙的延迟实现。这里什么都不做。
             * 后面的示例将用适当的延迟/睡眠函数替换这个粗糙的循环。
             */
        }
    }
}
```

代码清单 4.4 示例 4.1 中使用的第一个任务的实现


```

void vTask2( void * pvParameters )
{
    /* ulCount 声明为 volatile 以确保它不会被优化掉。 */
    volatile unsigned long ulCount;

    /* 与大多数任务一样，此任务在无限循环中实现。 */
    for( ;; )
    {
        /* 打印此任务的名称。 */
        vPrintLine( "Task 2 is running" );

        /* 延迟一段时间。 */
        for( ulCount = 0; ulCount < mainDELAY_LOOP_COUNT; ulCount++ )
        {
            /*
             * 这个循环只是一个非常粗糙的延迟实现。这里什么都不做。
             * 后面的示例将用适当的延迟/睡眠函数替换这个粗糙的循环。
             */
        }
    }
}

```

代码清单 4.5 示例 4.1 中使用的第二个任务的实现

`main()` 函数在启动调度器之前创建任务，具体实现见代码清单 4.6。

```

int main( void )
{
    /*
     * 在此处声明的变量在启动 FreeRTOS 调度器后可能不再存在。
     * 不要试图从任务中访问在 main() 使用的栈上声明的变量。
     */

    /*
     * 创建两个任务之一。注意，实际应用程序应检查 xTaskCreate() 调用
     * 的返回值，以确保任务创建成功。
     */
    xTaskCreate( vTask1, /* 指向实现任务的函数的指针。 */
                "Task 1", /* 任务的文本名称。 */
                1000, /* 栈深度（以字为单位）。 */
                NULL, /* 此示例不使用任务参数。 */
                1, /* 此任务将以优先级 1 运行。 */
                NULL ); /* 此示例不使用任务句柄。 */

    /* 以完全相同的方式和相同的优先级创建另一个任务。 */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    /* 启动调度器，以便任务开始执行。 */
    vTaskStartScheduler();

    /*
     * 如果一切正常，main() 不会到达这里，因为调度器现在将运行
     * 创建的任务。如果 main() 到达这里，则没有足够的堆内存来创建
     * 空闲任务或定时器任务（本书后面描述）。第 3 章提供了更多关于
     * 堆内存管理的信息。
     */
    for( ;; );
}

```

代码清单 4.6 启动示例 4.1 的任务

执行该示例会产生图 4.2 所示的输出。

```
C:\Temp>rtosdemo
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
```

图 4.2 执行示例 4.1 时产生的输出^[1]

图 4.2 显示两个任务似乎同时执行；然而，两个任务都在同一个处理器核上执行，因此不可能是同时的。实际上，两个任务都在快速进入和退出 *运行中* 状态。两个任务以相同的优先级运行，因此在同一个处理器核上共享时间。图 4.3 显示了它们的实际执行模式。

图 4.3 底部的箭头表示时间从 t_1 开始向前推移。彩色线条表示各时刻正在执行的任务；例如，任务 1 在 t_1 到 t_2 之间执行。

一次只能有一个任务处于 *运行中* 状态。因此，当一个任务进入 *运行中* 状态（任务被换入）时，另一个任务进入 *未运行* 状态（任务被换出）。

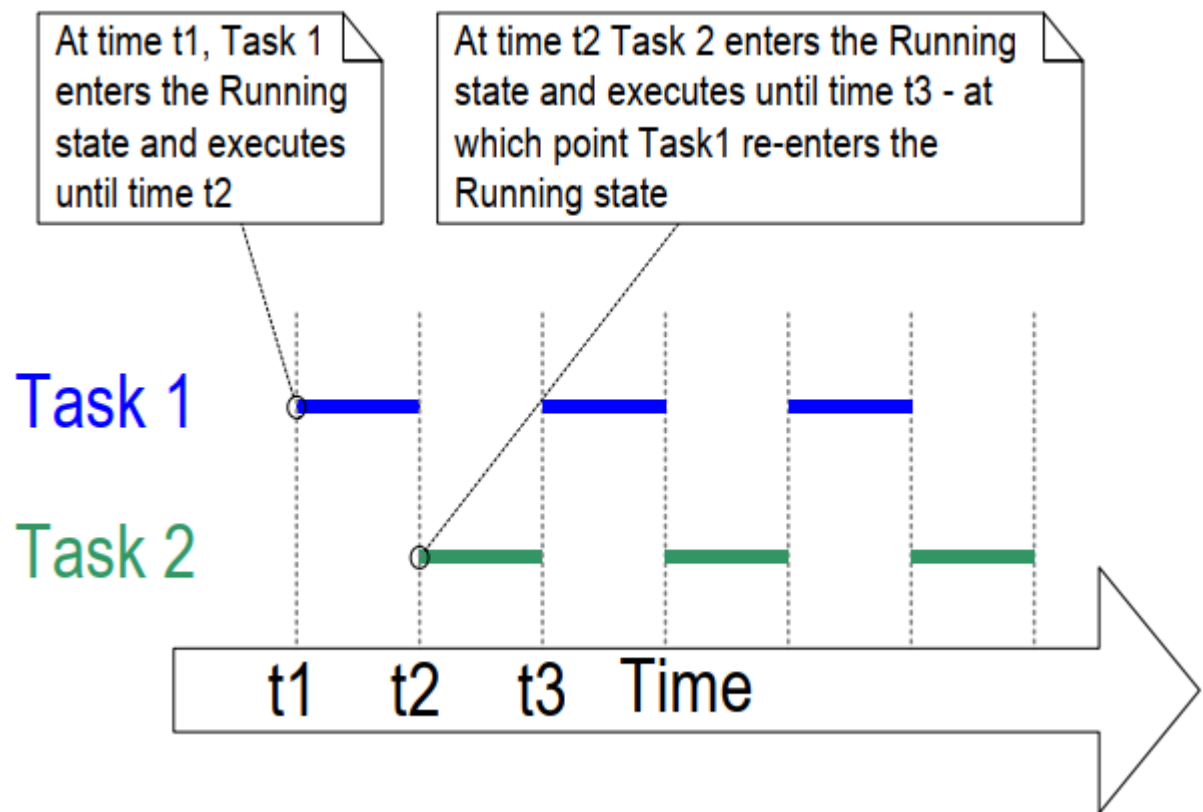


图 4.3 示例 4.1 中两个任务的实际执行模式

示例 4.1 在启动调度器之前从 `main()` 内部创建了两个任务。也可以从另一个任务内部创建一个任务。例如，任务 2 可以从任务 1 内部创建，如代码清单 4.7 所示。

```

void vTask1( void * pvParameters )
{
    const char *pcTaskName = "Task 1 is running\r\n";
    volatile unsigned long ul; /* volatile 确保 ul 不会被优化掉。 */

    /*
     * 如果此任务代码正在执行，则调度器肯定已经启动。
     * 在进入无限循环之前创建另一个任务。
     */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    for( ;; )
    {
        /* 打印此任务的名称。 */
        vPrintLine( pcTaskName );

        /* 延迟一段时间。 */
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ )
        {
            /*
             * 这个循环只是一个非常粗糙的延迟实现。这里什么都不做。
             * 后面的示例将用适当的延迟/睡眠函数替换这个粗糙的循环。
             */
        }
    }
}

```

代码清单 4.7 在调度器启动后从另一个任务内部创建任务

示例 4.2 使用任务参数

在示例 4.1 中创建的两个任务几乎相同，它们之间的唯一区别是打印的文本字符串。如果您创建单个任务实现的两个实例，并使用任务参数将字符串传递给每个实例，这将消除重复。

示例 4.2 将示例 4.1 中使用的两个任务函数替换为一个名为 `vTaskFunction()` 的单一任务函数，如代码清单 4.8 所示。注意任务参数如何被转换为 `char *` 以获取任务应打印的字符串。

```

void vTaskFunction( void * pvParameters )
{

    char *pcTaskName;
    volatile unsigned long ul; /* volatile 确保 ul 不会被优化掉。 */

    /*
     * 要打印的字符串通过参数传入。将其转换为字符指针。
     */
    pcTaskName = ( char * ) pvParameters;

    /* 与大多数任务一样，此任务在无限循环中实现。 */
    for( ;; )
    {
        /* 打印此任务的名称。 */
        vPrintLine( pcTaskName );

        /* 延迟一段时间。 */
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ )
        {
            /*
             * 这个循环只是一个非常粗糙的延迟实现。这里什么都不做。
             * 后面的练习将用适当的延迟/睡眠函数替换这个粗糙的循环。
             */
        }
    }
}

```

代码清单 4.8 用于在示例 4.2 中创建两个任务的单一任务函数

代码清单 4.9 创建了由 `vTaskFunction()` 实现的两个任务实例，使用任务参数将不同的字符串传递给每个实例。两个任务在 FreeRTOS 调度器的控制下独立执行，各自拥有自己的栈，因此拥有自己的 `pcTaskName` 和 `ul` 变量副本。

```

/*
 * 定义将作为任务参数传入的字符串。这些字符串定义为 const 且不在 main()
 * 使用的栈上，以确保它们在任务执行时保持有效。
 */
static const char * pcTextForTask1 = "Task 1 is running";
static const char * pcTextForTask2 = "Task 2 is running";

int main( void )
{
    /*
     * 在此处声明的变量在启动 FreeRTOS 调度器后可能不再存在。
     * 不要试图从任务中访问在 main() 使用的栈上声明的变量。
     */

    /* 创建两个任务之一。 */
    xTaskCreate( vTaskFunction,          /* 指向实现任务的函数的指针。 */
                "Task 1",                /* 任务的文本名称。这仅用于简化调试。 */
                1000,                    /* 栈深度 - 小型微控制器使用的栈远少于这个值。 */
                ( void * ) pcTextForTask1, /* 通过任务参数将要打印的文本传递给任务。 */
                1,                        /* 此任务将以优先级 1 运行。 */
                NULL );                  /* 此示例中不使用任务句柄。 */

    /*
     * 以完全相同的方式创建另一个任务。注意这次是从同一任务实现 (vTaskFunction)
     * 创建多个任务。只有参数中传递的值不同。正在创建同一任务定义的两个实例。
     */
    xTaskCreate( vTaskFunction,
                "Task 2",
                1000,
                ( void * ) pcTextForTask2,
                1,
                NULL );

    /* 启动调度器，以便任务开始执行。 */
    vTaskStartScheduler();

    /*
     * 如果一切正常，main() 不会到达这里，因为调度器现在将运行
     * 创建的任务。如果 main() 到达这里，则没有足够的堆内存来创建
     * 空闲任务或定时器任务（本书后面描述）。第 3 章提供了更多关于
     * 堆内存管理的信息。
     */
    for( ;; )
    {
    }
}

```

代码清单 4.9 示例 2 的 main() 函数

示例 4.2 的输出完全如图 4.2 中示例 1 所示。

4.5 任务优先级

FreeRTOS 调度器始终选择当前能够运行的最高优先级任务进入 *运行中* 状态。优先级相同的任务则轮流进入和退出 *运行中* 状态。

用于创建任务的 API 函数的 `uxPriority` 参数为任务提供初始优先级。`vTaskPrioritySet()` API 函数在任务创建后更改其优先级。

应用程序定义的编译时配置常量 `configMAX_PRIORITIES` 设置可用优先级的数量。数值越小，任务优先级越低，其中优先级 0 为最低优先级；因此，有效优先级范围是 0 到 `(configMAX_PRIORITIES - 1)`。任意数量的任务都可以共享同一优先级。

FreeRTOS 调度器有两种用于选择 *运行中* 状态任务的算法实现，`configMAX_PRIORITIES` 的最大允许值取决于所使用的实现：

4.5.1 通用调度器

通用调度器使用 C 语言编写，可用于所有 FreeRTOS 架构移植版本。它不限制 `configMAX_PRIORITIES` 的上限。通常应尽量减小 `configMAX_PRIORITIES`，因为更多的优先级会占用更多 RAM，并延长最坏情况下的执行时间。

4.5.2 架构优化调度器

架构优化的实现用特定于架构的汇编代码编写，比通用 C 实现性能更高，并且对于所有 `configMAX_PRIORITIES` 值，最坏情况执行时间相同。

架构优化的实现在 32 位架构上将 `configMAX_PRIORITIES` 的最大值限制为 32，在 64 位架构上限制为 64。与通用方法一样，建议将 `configMAX_PRIORITIES` 保持在尽可能低的值，因为更高的值需要更多的 RAM。

在 `FreeRTOSConfig.h` 中将 `configUSE_PORT_optimized_TASK_SELECTION` 设置为 1 可使用架构优化实现，设置为 0 则使用通用实现。并非所有 FreeRTOS 移植版本都提供架构优化实现。对于提供该实现的移植版本，如果未定义此宏，其默认值为 1；否则默认值为 0。

4.6 时间测量和 Tick 中断

[第 4.12 节“调度算法”](#)介绍了一项称为“时间片”的可选功能。前面的示例都使用了时间片，输出中也体现了这种行为。示例中的两个任务具有相同优先级，并且始终能够运行。因此，每个任务各执行一个“时间片”：在时间片开始时进入 *运行中* 状态，在时间片结束时退出该状态。图 4.3 中，t1 到 t2

之间的时长就是一个时间片。

调度器在每个时间片结束时运行，以选择下一个要执行的任务^[2]。为此，系统使用一个称为“Tick 中断”的周期性中断。编译时配置常量 `configTICK_RATE_HZ` 设置 Tick 中断频率，也就决定了每个时间片的长度。例如，将 `configTICK_RATE_HZ` 设置为 100 Hz 时，每个时间片持续 10 毫秒。两次 Tick 中断之间的时间称为“Tick 周期”，因此一个时间片等于一个 Tick 周期。

图 4.4 在图 4.3 的基础上补充了调度器的执行过程。顶行表示调度器何时运行，细箭头表示执行流程如何从一个任务进入 Tick 中断，再从 Tick 中断返回另一个任务。

`configTICK_RATE_HZ` 的最佳值取决于应用程序，尽管值 100 是典型的。

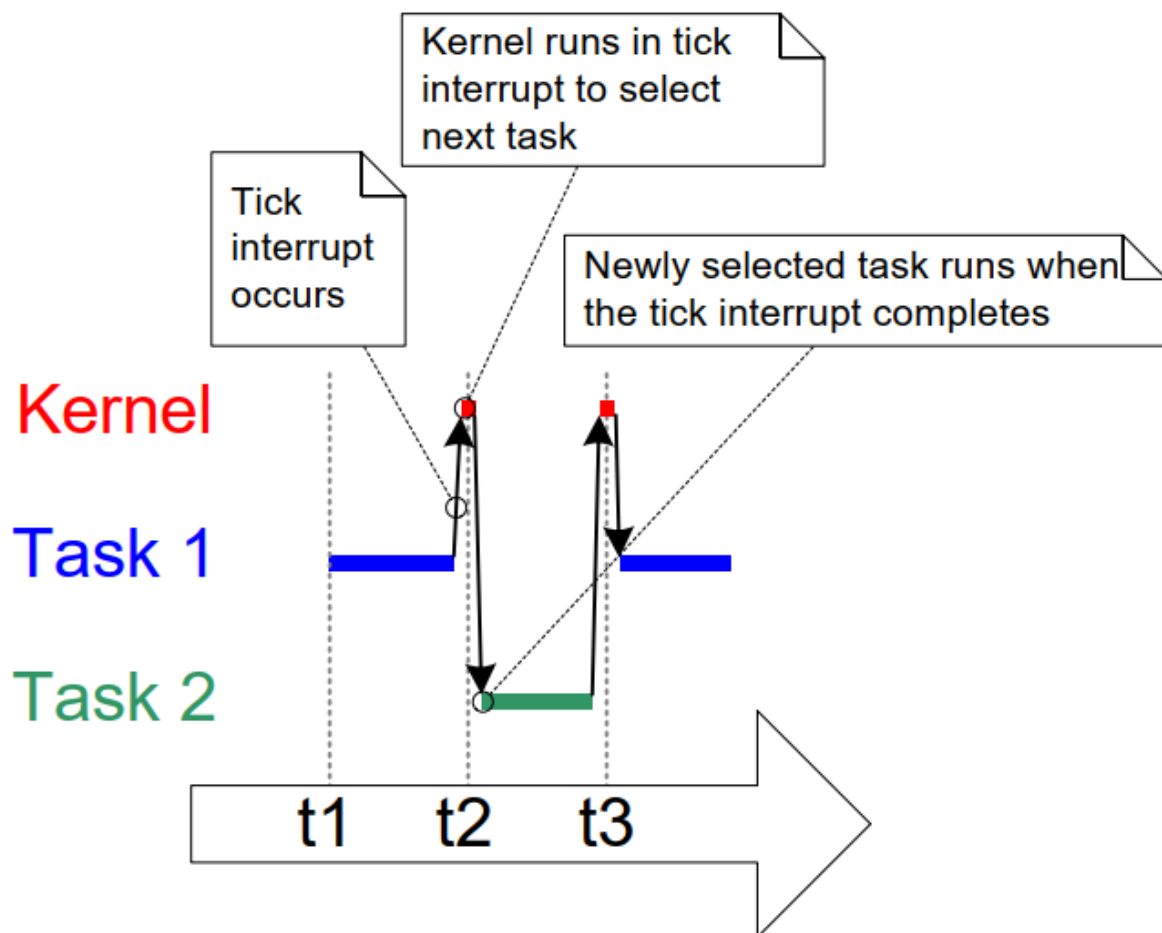


图 4.4 展开后的执行序列，显示 Tick 中断的执行过程

FreeRTOS API 调用以 Tick 周期的倍数（通常简称为 Tick）指定时间。`pdMS_TO_TICKS()` 宏可以把毫秒转换为 Tick 数。时间分辨率取决于所配置的 Tick 频率；当 Tick 频率高于 1 kHz（即 `configTICK_RATE_HZ` 大于 1000）时，不能使用 `pdMS_TO_TICKS()`。代码清单 4.10 演示了如何将 200 毫秒转换为等效的 Tick 数。

```
/*
 * pdMS_TO_TICKS() 以毫秒为单位的时间作为其唯一参数,
 * 并计算等效的 tick 周期数。此示例显示 xTimeInTicks 被设置为
 * 等效于 200 毫秒的 tick 周期数。
 */
TickType_t xTimeInTicks = pdMS_TO_TICKS( 200 );
```

代码清单 4.10 使用 `pdMS_TO_TICKS()` 宏将 200 毫秒转换为等效的 Tick 数

使用 `pdMS_TO_TICKS()` 按毫秒指定时间，而不是直接写入 Tick 数，可以保证更改 Tick 频率后，应用程序所指定的实际时长保持不变。

“Tick 计数”是调度器启动以来发生的 Tick 中断总数（暂不考虑计数值溢出）。应用程序在指定延迟时间时无需处理溢出，FreeRTOS 会在内部保证时间计算的一致性。

[第 4.12 节“调度算法”](#)介绍了影响调度器何时选择新任务运行以及 Tick 中断何时执行的配置常量。

示例 4.3 实验优先级

调度器始终选择当前能够运行的最高优先级任务进入 *运行中* 状态。前面的示例创建了两个相同优先级的任务，因此它们轮流进入和退出 *运行中* 状态。本示例研究任务优先级不同时会发生什么。代码清单 4.11 以优先级 1 创建第一个任务，以优先级 2 创建第二个任务。用于实现这两个任务的函数没有变化，仍然通过空循环产生延迟并周期性打印字符串。

```

/*
 * 定义将作为任务参数传入的字符串。
 * 这些字符串定义为 const 且不在栈上，以确保它们在任务执行时保持有效。
 */
static const char * pcTextForTask1 = "Task 1 is running";
static const char * pcTextForTask2 = "Task 2 is running";

int main( void )
{
    /* 以优先级 1 创建第一个任务。 */
    xTaskCreate( vTaskFunction,          /* 任务函数 */
                "Task 1",                /* 任务名称 */
                1000,                    /* 任务栈深度 */
                ( void * ) pcTextForTask1, /* 任务参数 */
                1,                        /* 任务优先级 */
                NULL );

    /* 以更高的优先级 2 创建第二个任务。 */
    xTaskCreate( vTaskFunction,          /* 任务函数 */
                "Task 2",                /* 任务名称 */
                1000,                    /* 任务栈深度 */
                ( void * ) pcTextForTask2, /* 任务参数 */
                2,                        /* 任务优先级 */
                NULL );

    /* 启动调度器，以便任务开始执行。 */
    vTaskStartScheduler();

    /* 不会到达这里。 */
    return 0;
}

```

代码清单 4.11 以不同优先级创建两个任务

图 4.5 显示了示例 4.3 产生的输出。

调度器始终选择当前能够运行的最高优先级任务。任务 2 的优先级高于任务 1，并且始终能够运行，因此调度器总会选择任务 2，任务 1 永远得不到执行。也就是说，任务 1 因任务 2 而“饥饿”，无法获得处理器时间；它从未进入 *运行中* 状态，自然也无法打印字符串。

```
C:\Temp>rtosdemo
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
```

图 4.5 以不同优先级运行两个任务

任务 2 始终能够运行，因为它无需等待任何事件：它不是在执行空循环，就是在向终端打印内容。

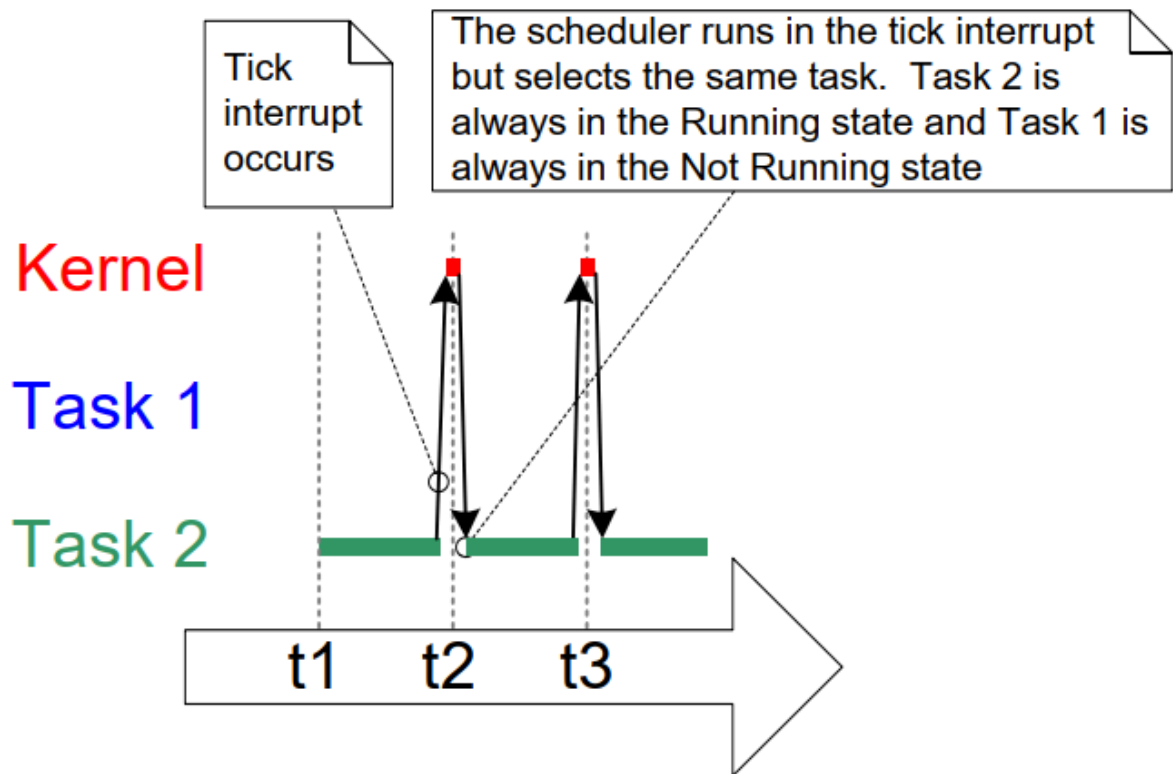


图 4.6 示例 4.3 中当一个任务优先级高于另一个任务时的执行模式

4.7 扩展 未运行 状态

到目前为止创建的任务总有工作要做，无需等待任何事件，因此始终能够进入 *运行中* 状态。这类“连续处理”任务的实用性有限，因为它们只能按最低优先级创建；如果使用其他优先级，它们就会阻止优先级更低的任务运行。

为了使这些任务有用，必须将它们重写为事件驱动。事件驱动任务只有在事件触发后才有工作（处理）要执行，并且在该时间之前不能进入 *运行中* 状态。调度器始终选择可以运行的优先级最高的任务。如果无法选择高优先级任务，因为它正在等待事件，则调度器必须改为选择可以运行的较低优先级任务。因此，编写事件驱动任务意味着可以以不同的优先级创建任务，而最高优先级的任务不会饿死所有较低优先级任务的处理时间。

4.7.1 阻塞 状态

等待事件的任务被称为处于“阻塞”状态，这是 *未运行* 状态的一个子状态。

任务可以进入 *阻塞* 状态以等待两种不同类型的事件：

1. 时间相关事件：在延迟时间到期或到达某个绝对时刻时发生。例如，任务可以进入 *阻塞* 状态，等待 10 毫秒过去。
2. 同步事件：由另一个任务或中断触发。例如，任务可以进入 *阻塞* 状态，等待数据到达队列。同步事件涵盖多种不同的事件类型。

FreeRTOS 队列、二进制信号量、计数信号量、互斥量、递归互斥量、事件组、流缓冲区、消息缓冲区和直接任务通知都可以创建同步事件。后面的章节涵盖了大多数这些功能。

任务可以在具有超时的情况下阻塞在同步事件上，从而有效地同时阻塞在两种类型的事件上。例如，任务可以选择最多等待 10 毫秒让数据到达队列。如果数据在 10 毫秒内到达，或者 10 毫秒过去而没有数据到达，任务将离开 *阻塞* 状态。

4.7.2 挂起 状态

挂起 也是 *未运行* 的一个子状态。处于挂起状态的任务对调度器不可用。进入挂起状态的唯一方法是通过调用 `vTaskSuspend()` API 函数，退出挂起状态的唯一方法是通过调用 `vTaskResume()` 或 `xTaskResumeFromISR()` API 函数。大多数应用程序不使用挂起状态。

4.7.3 就绪状态

处于 *未运行* 状态且未 *阻塞* 或 *挂起* 的任务被称为处于 *就绪* 状态。它们可以运行，因此“准备”运行，但当前不在 *运行中* 状态。

4.7.4 完善状态转换图

图 4.7 扩展了简化的状态图，以包含本节描述的所有 *未运行* 子状态。到目前为止，示例中创建的任务没有使用 *阻塞* 或 *挂起* 状态。它们只在 *就绪* 状态和 *运行中* 状态之间转换，如图 4.7 中的粗线所示。

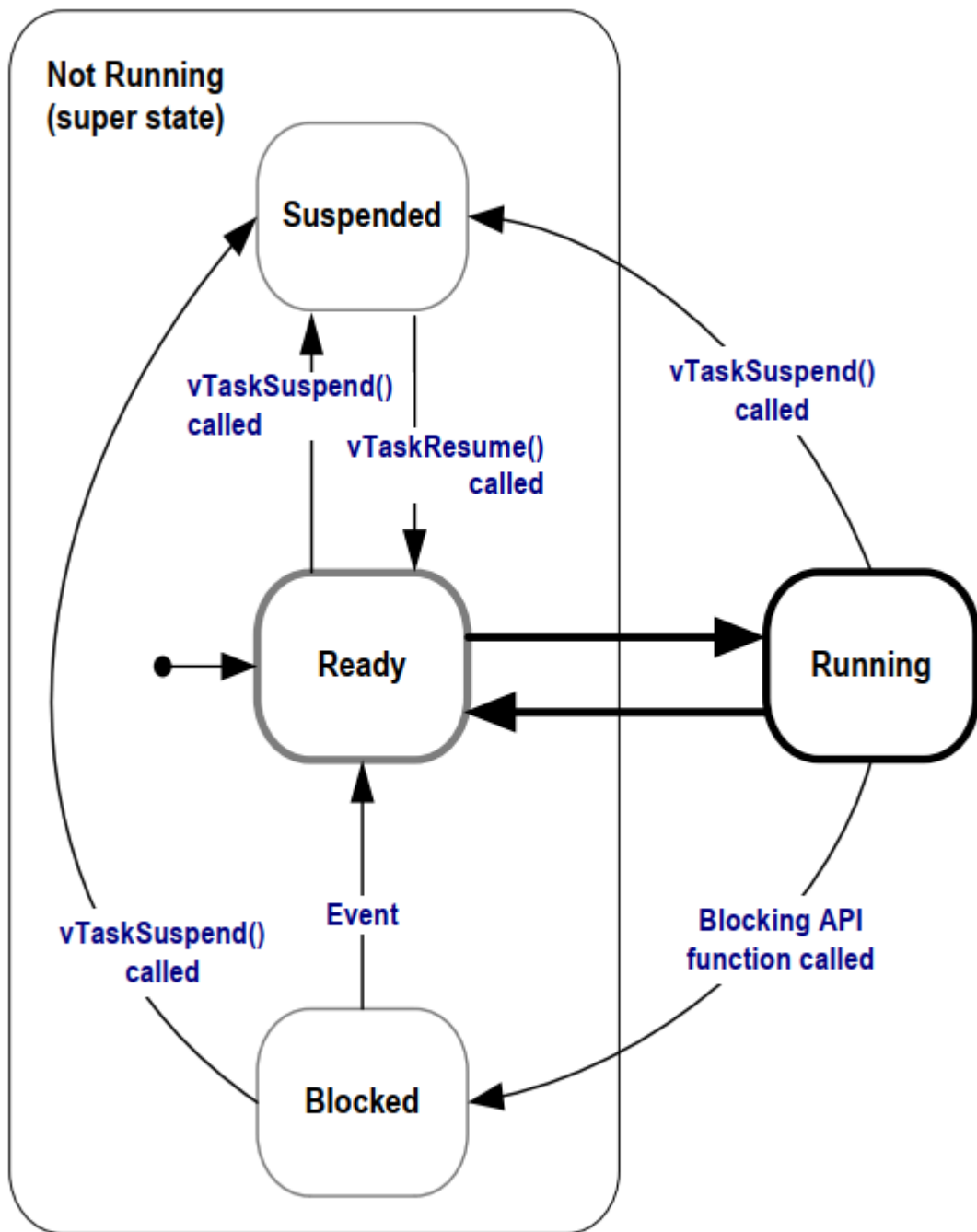


图 4.7 完整任务状态机

示例 4.4 使用 阻塞 状态创建延迟

到目前为止，示例中创建的任务都是“周期性”的：先延迟一段时间，再打印字符串，然后再次延迟，如此反复。延迟由空循环粗略产生，任务不断轮询并递增循环计数器，直到达到固定值。示例 4.3 清楚地展示了这种方法的缺点：较高优先级的任务在执行空循环期间始终处于 *运行中* 状态，使较低优先级的任务因“饥饿”而无法获得处理器时间。

任何形式的轮询还有其他缺点，其中之一就是效率低下。轮询期间，任务实际上没有工作要做，却仍占用尽可能多的处理器时间，因而浪费处理器周期。示例 4.4 使用 `vTaskDelay()` API 函数替换轮询空循环来修正这一行为，其原型见代码清单 4.12，修改后的任务定义见代码清单 4.13。注意，只有在 `FreeRTOSConfig.h` 中将 `INCLUDE_vTaskDelay` 设置为 1，`vTaskDelay()` API 函数才可用。

`vTaskDelay()` 将调用它的任务置于 *阻塞* 状态，并保持指定的 Tick 数。任务处于 *阻塞* 状态时不占用处理器时间，因此只有真正有工作要做时才会使用处理器。

```
void vTaskDelay( TickType_t xTicksToDelay );
```

代码清单 4.12 `vTaskDelay()` API 函数原型

`vTaskDelay` 参数：

- `xTicksToDelay`

调用任务返回 *就绪* 状态之前，保持 *阻塞* 状态的 Tick 数。

例如，任务在 Tick 计数为 10,000 时调用 `vTaskDelay( 100 )`，便会立即进入 *阻塞* 状态，直到 Tick 计数达到 10,100。

宏 `pdMS_TO_TICKS()` 可将毫秒转换为 Tick 数。例如，调用 `vTaskDelay( pdMS_TO_TICKS( 100 ) )` 会使调用任务保持 *阻塞* 状态 100 毫秒。


```

void vTaskFunction( void * pvParameters )
{
    char * pcTaskName;
    const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );

    /*
     * 要打印的字符串通过参数传入。将其转换为字符指针。
     */
    pcTaskName = ( char * ) pvParameters;

    /* 与大多数任务一样，此任务在无限循环中实现。 */
    for( ;; )
    {
        /* 打印此任务的名称。 */
        vPrintLine( pcTaskName );

        /*
         * 延迟一段时间。这次使用 vTaskDelay() 调用，将任务置于
         * 阻塞状态，直到延迟周期到期。参数接受以 'ticks' 为单位的时间，
         * 并使用 pdMS_TO_TICKS() 宏 (在 xDelay250ms 常量声明处)
         * 将 250 毫秒转换为等效的 tick 时间。
         */
        vTaskDelay( xDelay250ms );
    }
}

```

代码清单 4.13 用 `vTaskDelay()` 调用替换空循环延迟后的示例任务源代码

尽管两个任务仍然以不同的优先级创建，但现在两者都会运行。示例 4.4 的输出如图 4.8 所示，确认了预期行为。

```

C:\Temp>rtosdemo
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running

```

图 4.8 执行示例 4.4 时产生的输出

图 4.9 所示的执行序列解释了为什么两个任务都能运行，即使它们以不同的优先级创建。为简化起见，省略了调度器本身的执行。

空闲任务在调度器启动时自动创建，以确保始终至少有一个任务可以运行（至少有一个任务处于就绪状态）。[第 4.8 节：空闲任务和空闲任务钩子](#)更详细地描述了空闲任务。

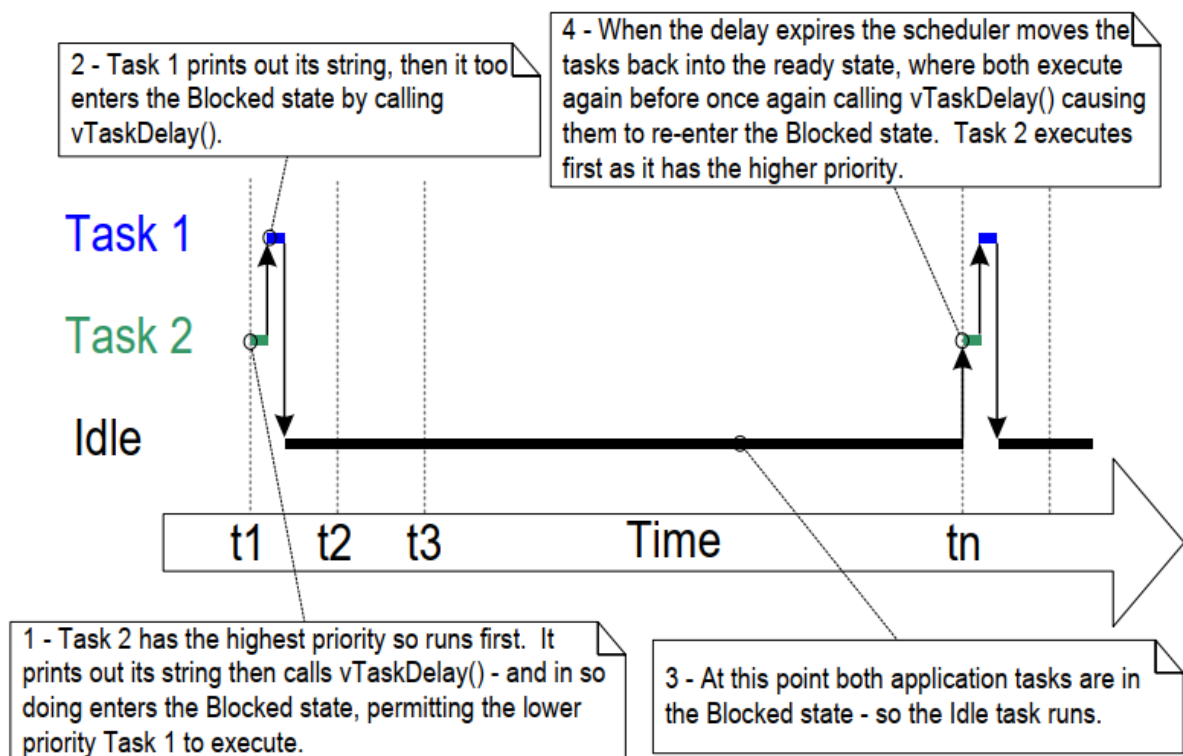


图 4.9 任务使用 `vTaskDelay()` 代替空循环时的执行序列

只有两个任务的实现发生了变化，其功能没有改变。将图 4.9 与图 4.4 进行比较，清楚地表明这种功能是以更高效的方式实现的。

图 4.4 显示了任务使用空循环创建延迟时的执行模式，因此它们始终能够运行。结果，它们共同使用了百分之百的可用处理器时间。图 4.9 显示了任务在其整个延迟期间进入阻塞状态时的执行模式。它们仅在确实有需要执行的工作（在本例中仅为要打印的消息）时才使用处理器时间，因此只使用了可用处理时间的一小部分。

在图 4.9 所示的场景中，任务每次离开 *阻塞* 状态后只执行不到一个 Tick 周期，随后便再次进入 *阻塞* 状态。大部分时间里，没有应用程序任务处于 *就绪* 状态，也就没有应用程序任务可以进入 *运行中* 状态，此时由空闲任务运行。空闲任务获得的处理器时间反映了系统的空闲处理能力。RTOS 允许应用程序完全由事件驱动，因而可以显著增加空闲处理能力。

图 4.10 中的粗线显示了示例 4.4 中任务执行的状态转换，每个任务现在在返回到 *就绪* 状态之前都经过 *阻塞* 状态。

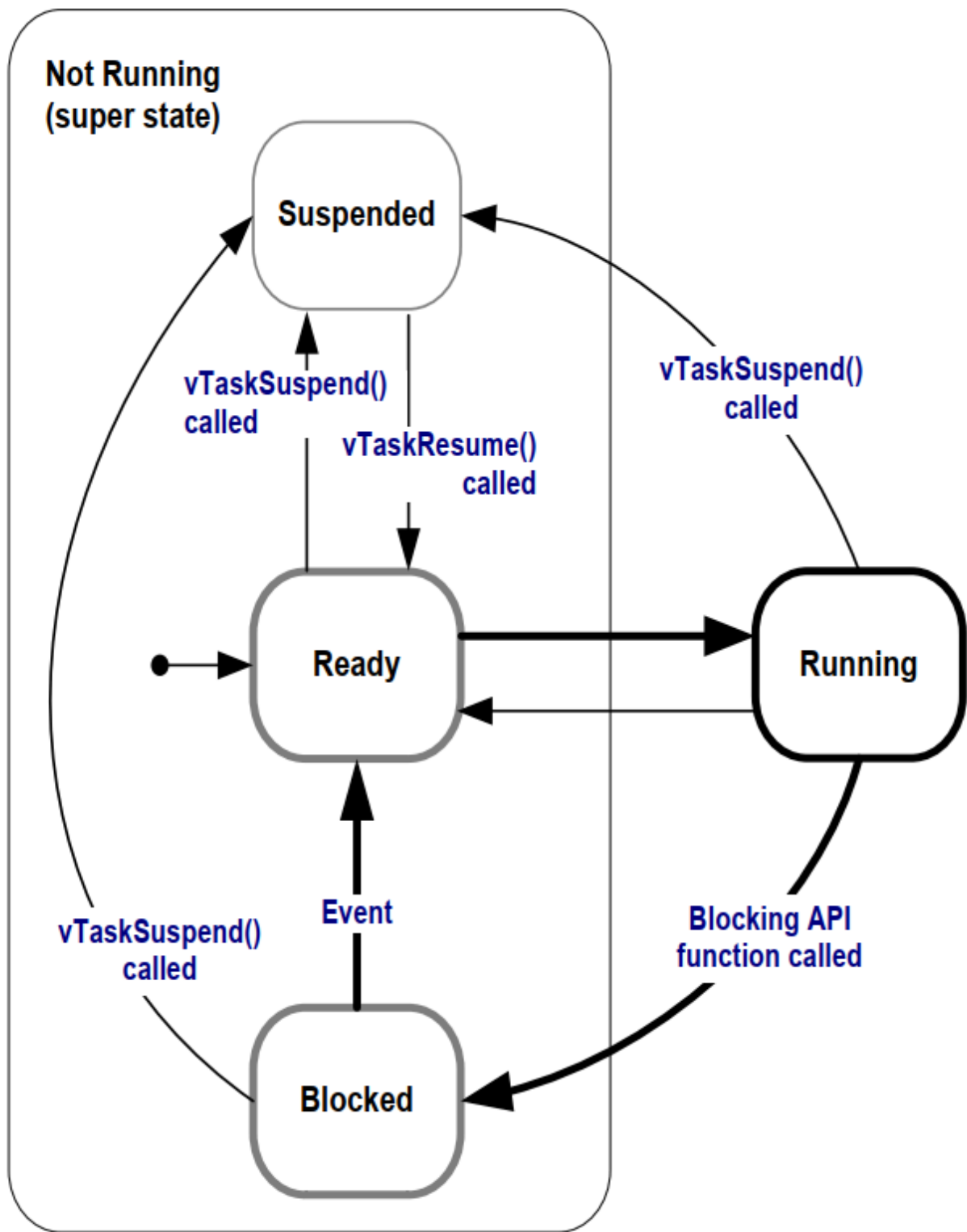


图 4.10 粗线表示示例 4.4 中任务执行的状态转换

4.7.5 vTaskDelayUntil() API 函数

`vTaskDelayUntil()` 与 `vTaskDelay()` 类似。前面已经说明，`vTaskDelay()` 的参数指定从任务调用该函数到再次离开 阻塞 状态之间经过的 Tick 数。因此，任务保持阻塞的时长由参数决定，而离开 阻塞 状态的时刻则相对于调用 `vTaskDelay()` 的时刻计算。

相比之下，`vTaskDelayUntil()` 的参数指定调用任务从 阻塞 状态转入 就绪 状态时的准确 Tick 计数。当任务需要以固定频率周期执行时，应使用 `vTaskDelayUntil()`，因为任务解除阻塞的时刻是绝对时刻，而不是像 `vTaskDelay()` 那样相对于函数调用时刻计算。

```
void vTaskDelayUntil( TickType_t * pxPreviousWakeTime,
                    TickType_t xTimeIncrement );
```

代码清单 4.14 `vTaskDelayUntil()` API 函数原型

`vTaskDelayUntil()` 参数

- `pxPreviousWakeTime`

该参数名源于这样一种用法：使用 `vTaskDelayUntil()` 实现按固定频率周期执行的任务。此时，`pxPreviousWakeTime` 保存任务上次离开 阻塞 状态（被“唤醒”）的时刻，并以此为基准计算任务下次应离开 阻塞 状态的时刻。

`vTaskDelayUntil()` 会自动更新 `pxPreviousWakeTime` 指向的变量。应用程序通常不应修改该变量，但必须在首次使用前将其初始化为当前 Tick 计数。代码清单 4.15 演示了初始化方法。

- `xTimeIncrement`

该参数名同样源于上述用法，任务的执行频率由 `xTimeIncrement` 的值决定。

`xTimeIncrement` 以 Tick 为单位。宏 `pdMS_TO_TICKS()` 可将毫秒转换为 Tick 数。

示例 4.5 将示例任务转换为使用 `vTaskDelayUntil()`

在示例 4.4 中创建的两个任务是周期性任务，但使用 `vTaskDelay()` 不能保证它们运行的频率是固定的，因为任务离开 阻塞 状态的时间相对于它们调用 `vTaskDelay()` 的时间。将任务转换为使用 `vTaskDelayUntil()` 而不是 `vTaskDelay()` 解决了这个潜在问题。

```

void vTaskFunction( void * pvParameters )
{
    char * pcTaskName;
    TickType_t xLastWakeTime;

    /*
     * 要打印的字符串通过参数传入。将其转换为字符指针。
     */
    pcTaskName = ( char * ) pvParameters;

    /*
     * xLastWakeTime 变量需要初始化为当前 tick 计数。
     * 注意这是唯一一次显式写入该变量。此后 xLastWakeTime 在
     * vTaskDelayUntil() 中自动更新。
     */
    xLastWakeTime = xTaskGetTickCount();

    /* 与大多数任务一样，此任务在无限循环中实现。 */
    for( ;; )
    {
        /* 打印此任务的名称。 */
        vPrintLine( pcTaskName );

        /*
         * 此任务应每 250 毫秒精确执行一次。与 vTaskDelay() 函数一样，
         * 时间以 tick 为单位测量，pdMS_TO_TICKS() 宏用于将毫秒转换为 tick。
         * xLastWakeTime 在 vTaskDelayUntil() 中自动更新，因此任务不显式更新它。
         */
        vTaskDelayUntil( &xLastWakeTime, pdMS_TO_TICKS( 250 ) );
    }
}

```

代码清单 4.15 使用 vTaskDelayUntil() 的示例任务实现

示例 4.5 产生的输出与示例 4.4 在图 4.8 中显示的完全相同。

示例 4.6 组合阻塞和非阻塞任务

前面的示例分别研究了轮询和阻塞任务的行为。此示例强化了我们关于预期系统行为的说法，并演示了两种方案组合时的执行序列，如下所示：

1. 创建两个优先级为 1 的任务。这些任务除了连续打印字符串外什么都不做。

这些任务从不调用可能导致它们进入 *阻塞* 状态的 API 函数，因此总是处于就绪或 *运行中* 状态。这种性质的任务称为“连续处理”任务，因为它们总是有工作要做（尽管在此例中相当简单）。

代码清单 4.16 显示了连续处理任务的源代码。

2. 然后创建第三个任务，优先级为 2，高于其他两个任务。第三个任务也只打印一个字符串，但是这次是周期性的，因此它使用 `vTaskDelayUntil()` API 函数在每次打印迭代之间将自身置于 *阻塞* 状态。

代码清单 4.17 显示了周期性任务的源代码。

```
void vContinuousProcessingTask( void * pvParameters )
{
    char * pcTaskName;

    /*
     * 要打印的字符串通过参数传入。将其转换为字符指针。
     */
    pcTaskName = ( char * ) pvParameters;

    /* 与大多数任务一样，此任务在无限循环中实现。 */
    for( ;; )
    {
        /*
         * 打印此任务的名称。此任务只是重复执行此操作，从不阻塞或延迟。
         */
        vPrintLine( pcTaskName );
    }
}
```

代码清单 4.16 示例 4.6 中使用的连续处理任务

```

void vPeriodicTask( void * pvParameters )
{
    TickType_t xLastWakeTime;

    const TickType_t xDelay3ms = pdMS_TO_TICKS( 3 );

    /*
     * xLastWakeTime 变量需要初始化为当前 tick 计数。
     * 注意这是唯一一次显式写入该变量。此后 xLastWakeTime 由
     * vTaskDelayUntil() API 函数自动管理。
     */
    xLastWakeTime = xTaskGetTickCount();

    /* 与大多数任务一样，此任务在无限循环中实现。 */
    for( ;; )
    {
        /* 打印此任务的名称。 */
        vPrintLine( "Periodic task is running" );

        /*
         * 此任务应每 3 毫秒精确执行一次，请参见此函数中的 xDelay3ms 声明。
         */
        vTaskDelayUntil( &xLastWakeTime, xDelay3ms );
    }
}

```

代码清单 4.17 示例 4.6 中使用的周期性任务

图 4.11 显示了示例 4.6 产生的输出，图 4.12 所示的执行序列解释了观察到的行为。


```
Continuous task 2 running
Continuous task 2 running
Periodic task is running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 2 running
Continuous task 2 running
Continuous task 2 running
Continuous task 2 running
Continuous task 2 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Continuous task 1 running
Periodic task is running
Continuous task 2 running
Continuous task 2 running
```

图 4.11 执行示例 4.6 时产生的输出

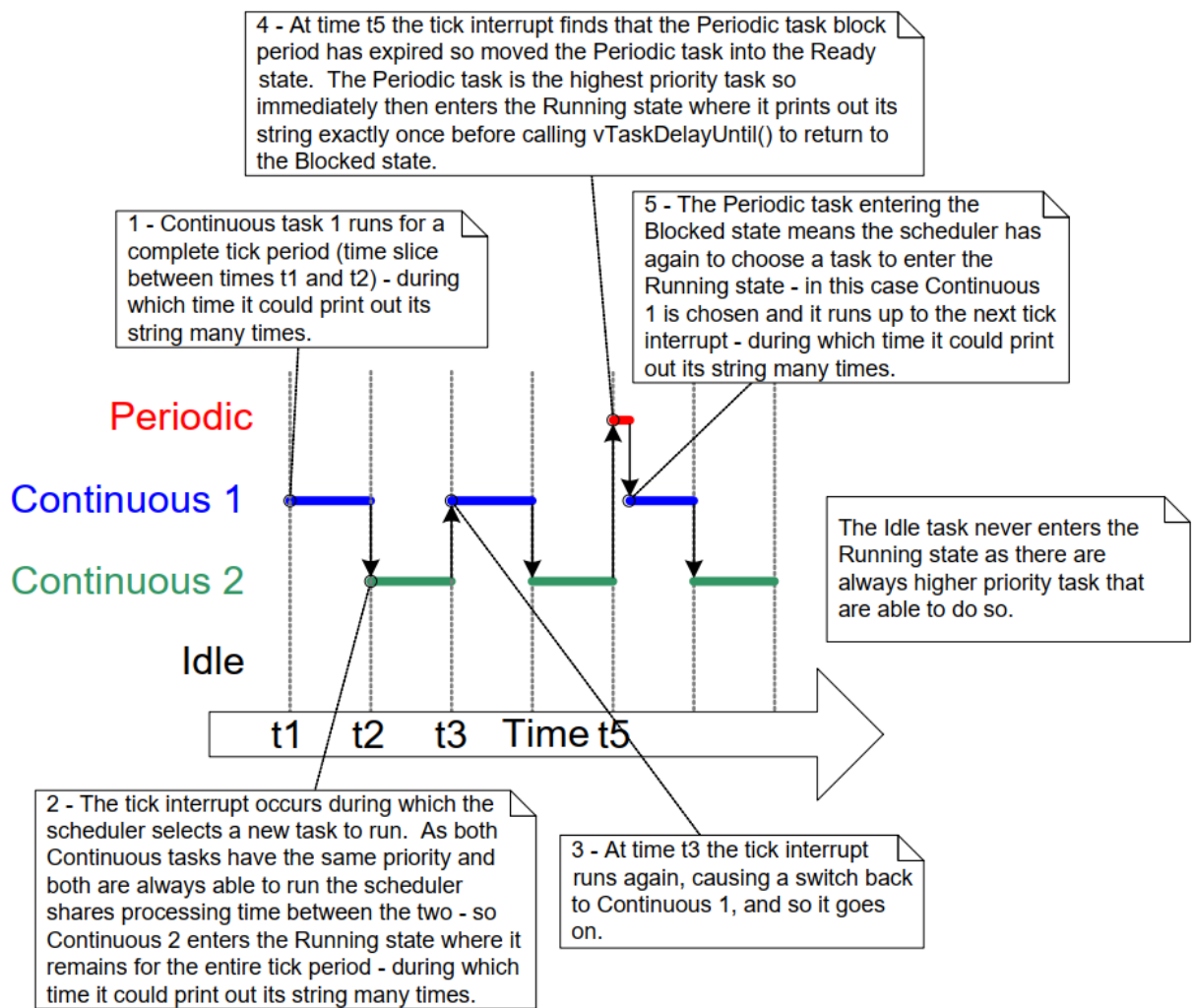


图 4.12 示例 4.6 的执行模式

4.8 空闲任务和空闲任务钩子

示例 4.4 中创建的任务大部分时间处于阻塞状态。在这种状态下，它们无法运行，因此不能被调度器选中。

必须始终至少有一个任务可以进入 *运行中* 状态^[3]。为了确保这一点，调度器在调用 `vTaskStartScheduler()` 时自动创建一个空闲任务。空闲任务几乎什么都不做，只是在一个循环中运行，因此与第一个示例中的任务一样，它始终能够运行。

空闲任务具有最低优先级（优先级 0），因此不会阻止更高优先级的应用程序任务进入 *运行中* 状态。应用程序也可以创建同为优先级 0、与空闲任务共享该优先级的任务。 `FreeRTOSConfig.h` 中的编译时配置常量 `configIDLE_SHOULD_YIELD` 可防止空闲任务占用本可分配给这些应用程序任务的处理器

时间。第 4.12 节“调度算法”将进一步介绍 `configIDLE_SHOULD_YIELD`。

空闲任务以最低优先级运行，因此一旦更高优先级的任务进入 *就绪* 状态，它就会立即退出 *运行中* 状态。图 4.9 的 *tn* 时刻展示了这一点：任务 2 一离开 *阻塞* 状态，空闲任务就立即被换出，让任务 2 开始执行。此时称任务 2 抢占（*preempt*）了空闲任务。抢占自动发生，被抢占的任务对此并不知情。

注意：如果任务使用 `vTaskDelete()` API 函数删除自身，则确保空闲任务不被饿死处理时间至关重要。这是因为空闲任务负责清理已删除自身任务所使用的内核资源。

4.8.1 空闲任务钩子函数

可以通过使用空闲钩子（或空闲回调）函数将应用程序特定的功能直接添加到空闲任务中，该函数由空闲任务在空闲任务循环的每次迭代中自动调用。

空闲任务钩子的常见用途包括：

- 执行低优先级、后台或连续处理功能，而无需为此创建应用程序任务而增加 RAM 开销。
- 测量空闲处理能力。（空闲任务只有在所有更高优先级的应用程序任务没有工作要执行时才会运行；因此，测量分配给空闲任务的处理时间量可以清楚地指示空闲处理时间。）
- 将处理器置于低功耗模式，以便在没有应用程序工作要处理时简单、自动地降低功耗（不过，其节能效果不如无 Tick 空闲模式）。

4.8.2 空闲任务钩子函数实现的限制

空闲任务钩子函数必须遵守以下规则。

- 空闲任务钩子函数绝不能尝试阻塞或挂起自身。

*注意：以任何方式阻塞空闲任务都可能导致没有任务可进入 *运行中* 状态的情况。*

- 如果应用程序任务使用 `vTaskDelete()` API 函数删除自身，则空闲任务钩子必须在合理的时间始终返回其调用者。这是因为空闲任务负责清理删除自身任务分配的内核资源。如果空闲任务永久停留在空闲钩子函数中，则此清理无法发生。

空闲任务钩子函数必须具有代码清单 4.18 所示的名称和原型。

```
void vApplicationIdleHook( void );
```

代码清单 4.18 空闲任务钩子函数名称和原型

示例 4.7 定义空闲任务钩子函数

在示例 4.4 中使用阻塞 `vTaskDelay()` API 调用产生了大量空闲时间，即两个应用程序任务都处于阻塞状态时空闲任务执行的时间。示例 4.7 通过添加空闲钩子函数来利用这些空闲时间，其源代码如代码清单 4.19 所示。

```
/* 声明一个将由钩子函数递增的变量。 */
volatile unsigned long ulIdleCycleCount = 0UL;

/*
 * 空闲钩子函数必须命名为 vApplicationIdleHook(), 不带参数, 并返回 void。
 */
void vApplicationIdleHook( void )
{
    /* 此钩子函数除了递增计数器外什么都不做。 */
    ulIdleCycleCount++;
}
```

代码清单 4.19 一个非常简单的空闲钩子函数

`configUSE_IDLE_HOOK` 必须在 `FreeRTOSConfig.h` 中设置为 1，才能使空闲钩子函数被调用。

实现所创建任务的函数稍作修改，以打印 `ulIdleCycleCount` 值，如代码清单 4.20 所示。

```

void vTaskFunction( void * pvParameters )
{
    char * pcTaskName;
    const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );

    /*
     * 要打印的字符串通过参数传入。将其转换为字符指针。
     */
    pcTaskName = ( char * ) pvParameters;

    /* 与大多数任务一样，此任务在无限循环中实现。 */
    for( ;; )
    {
        /*
         * 打印此任务的名称以及 ulIdleCycleCount 被递增的次数。
         */
        vPrintLineAndNumber( pcTaskName, ulIdleCycleCount );

        /* 延迟 250 毫秒。 */
        vTaskDelay( xDelay250ms );
    }
}

```

代码清单 4.20 示例任务源代码现在打印 `ulIdleCycleCount` 值

图 4.13 显示了示例 4.7 产生的输出。可以看到空闲任务钩子函数在应用程序任务的每次迭代之间执行了大约 400 万次（迭代次数取决于硬件速度）。

```

C:\Temp>rtosdemo
Task 2 is running
ulIdleCycleCount = 0
Task 1 is running
ulIdleCycleCount = 0
Task 2 is running
ulIdleCycleCount = 3869504
Task 1 is running
ulIdleCycleCount = 3869504
Task 2 is running
ulIdleCycleCount = 8564623
Task 1 is running
ulIdleCycleCount = 8564623
Task 2 is running
ulIdleCycleCount = 13181489
Task 1 is running
ulIdleCycleCount = 13181489
Task 2 is running
ulIdleCycleCount = 17838406
Task 1 is running
ulIdleCycleCount = 17838406
Task 2 is running

```

图 4.13 执行示例 4.7 时产生的输出

4.9 更改任务优先级

4.9.1 vTaskPrioritySet() API 函数

vTaskPrioritySet() API 函数在调度器启动后更改任务的优先级。**vTaskPrioritySet()** API 函数仅在 FreeRTOSConfig.h 中将 **INCLUDE_vTaskPrioritySet** 设置为 1 时可用。

```
void vTaskPrioritySet( TaskHandle_t xTask,  
                      UBaseType_t uxNewPriority );
```

代码清单 4.21 vTaskPrioritySet() API 函数原型

vTaskPrioritySet() 参数

- **pxTask**

正在修改优先级（目标任务）的任务句柄。有关获取任务句柄的信息，请参见 **xTaskCreate()** API 函数的 **pxCreatedTask** 参数，或 **xTaskCreateStatic()** API 函数的返回值。

任务可以通过传递 NULL 而不是有效的任务句柄来更改自身的优先级。

- **uxNewPriority**

为目标任务设置的优先级。该值会自动限制为最大可用优先级 **(configMAX_PRIORITIES - 1)**，其中 **configMAX_PRIORITIES** 是在 **FreeRTOSConfig.h** 头文件中设置的编译时常量。

4.9.2 uxTaskPriorityGet() API 函数

uxTaskPriorityGet() API 函数返回任务的优先级。**uxTaskPriorityGet()** API 函数仅在 FreeRTOSConfig.h 中将 **INCLUDE_uxTaskPriorityGet** 设置为 1 时可用。

```
UBaseType_t uxTaskPriorityGet( TaskHandle_t xTask );
```

代码清单 4.22 uxTaskPriorityGet() API 函数原型

uxTaskPriorityGet() 参数和返回值

- `pxTask`

正在查询优先级（目标任务）的任务句柄。有关如何获取任务句柄的信息，请参见

`xTaskCreate()` API 函数的 `pxCreatedTask` 参数。

任务可以通过传递 `NULL` 而不是有效的任务句柄来查询自身的优先级。

- 返回值

当前分配给被查询任务的优先级。

示例 4.8 更改任务优先级

调度器始终选择优先级最高的 *就绪* 状态任务进入 *运行中* 状态。示例 4.8 使用 `vTaskPrioritySet()` API 函数改变两个任务的相对优先级，以演示这一行为。

示例 4.8 以两个不同的优先级创建两个任务。两个任务都不调用任何可能导致它们进入阻塞状态的 API 函数，因此两者始终处于 *就绪* 状态或 *运行中* 状态。因此，具有较高相对优先级的任务将始终是被调度器选为 *运行中* 状态的任务。

示例 4.8 的行为如下：

1. 任务 1（代码清单 4.23）以最高优先级创建，因此它保证首先运行。任务 1 打印出几个字符串，然后将任务 2（代码清单 4.24）的优先级提高到高于自身。
2. 任务 2 一旦具有最高相对优先级就开始运行（进入 *运行中* 状态）。一次只能有一个任务处于 *运行中* 状态，因此当任务 2 处于 *运行中* 状态时，任务 1 处于 *就绪* 状态。
3. 任务 2 打印出一条消息，然后将自己的优先级降低到任务 1 以下。
4. 当任务 2 降低其优先级时，任务 1 再次成为最高优先级任务，因此任务 1 重新进入 *运行中* 状态，迫使任务 2 回到 *就绪* 状态。

```

void vTask1( void * pvParameters )
{
    UBaseType_t uxPriority;

    /*
     * 此任务将始终在任务 2 之前运行，因为它以更高的优先级创建。
     * 任务 1 和任务 2 都不阻塞，因此两者始终处于运行中或就绪状态。
     */

    /*
     * 查询此任务运行的优先级 - 传递 NULL 意味着"返回调用任务的优先级"。
     */
    uxPriority = uxTaskPriorityGet( NULL );

    for( ;; )
    {
        /* 打印此任务的名称。 */
        vPrintLine( "Task 1 is running" );

        /*
         * 将任务 2 的优先级设置到任务 1 的优先级以上将导致任务 2
         * 立即开始运行（因为此时任务 2 将具有两个创建任务中较高的优先级）。
         * 注意在调用 vTaskPrioritySet() 时使用了任务 2 的句柄 (xTask2Handle)。
         * 代码清单 4.25 显示了如何获取句柄。
         */
        vPrintLine( "About to raise the Task 2 priority" );
        vTaskPrioritySet( xTask2Handle, ( uxPriority + 1 ) );

        /*
         * 任务 1 仅在其优先级高于任务 2 时才会运行。
         * 因此，对于此任务到达此点，任务 2 必须已经执行并将自己的优先级
         * 降低到此任务的优先级以下。
         */
    }
}

```

代码清单 4.23 示例 4.8 中任务 1 的实现


```

void vTask2( void * pvParameters )
{
    UBaseType_t uxPriority;

    /*
     * 任务 1 将始终在此任务之前运行，因为任务 1 以更高的优先级创建。
     * 任务 1 和任务 2 都不阻塞，因此始终处于运行中或就绪状态。
     *
     * 查询此任务运行的优先级 - 传递 NULL 意味着"返回调用任务的优先级"。
     */
    uxPriority = uxTaskPriorityGet( NULL );

    for( ;; )
    {
        /*
         * 对于此任务到达此点，任务 1 必须已经运行并将此任务的优先级
         * 设置得高于其自身。
         */

        /* 打印此任务的名称。 */
        vPrintLine( "Task 2 is running" );

        /*
         * 将此任务的优先级降低回其原始值。
         * 传递 NULL 作为任务句柄意味着"更改调用任务的优先级"。
         * 将优先级设置到任务 1 以下将导致任务 1 立即重新开始运行，
         * 抢占此任务。
         */
        vPrintLine( "About to lower the Task 2 priority" );
        vTaskPrioritySet( NULL, ( uxPriority - 2 ) );
    }
}

```

代码清单 4.24 示例 4.8 中任务 2 的实现

每个任务都可以通过使用 NULL 代替有效的任务句柄来查询和设置自身的优先级。任务句柄仅在任务希望引用自身以外的任务时才需要，例如任务 1 更改任务 2 的优先级时。为了允许任务 1 这样做，任务 2 的句柄在创建任务 2 时获取并保存，如代码清单 4.25 中的注释所强调的。

```

/* 声明一个用于保存任务 2 句柄的变量。 */
TaskHandle_t xTask2Handle = NULL;

int main( void )
{
    /*
     * 以优先级 2 创建第一个任务。任务参数未使用，设置为 NULL。
     * 任务句柄也未使用，因此也设置为 NULL。
     */
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 2, NULL );
    /* 任务以优先级 2 创建 -----^。 */

    /*
     * 以优先级 1 创建第二个任务 - 低于分配给任务 1 的优先级。
     * 同样，任务参数未使用，因此设置为 NULL-
     * 但是这次需要任务句柄，因此 xTask2Handle 的地址作为最后一个参数传入。
     */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, &xTask2Handle );
    /* 任务句柄是最后一个参数 -----^^^^^^^^^^^^^^^^ */

    /* 启动调度器，以便任务开始执行。 */
    vTaskStartScheduler();

    /*
     * 如果一切正常，main() 不会到达这里，因为调度器现在将运行
     * 创建的任务。如果 main() 到达这里，则没有足够的堆内存来创建
     * 空闲任务或定时器任务（本书后面描述）。第 2 章提供了更多关于
     * 堆内存管理的信息。
     */
    for( ;; )
    {
    }
}

```

代码清单 4.25 示例 4.8 的 main() 实现

图 4.14 演示了示例 4.8 中任务的执行顺序，结果输出如图 4.15 所示。

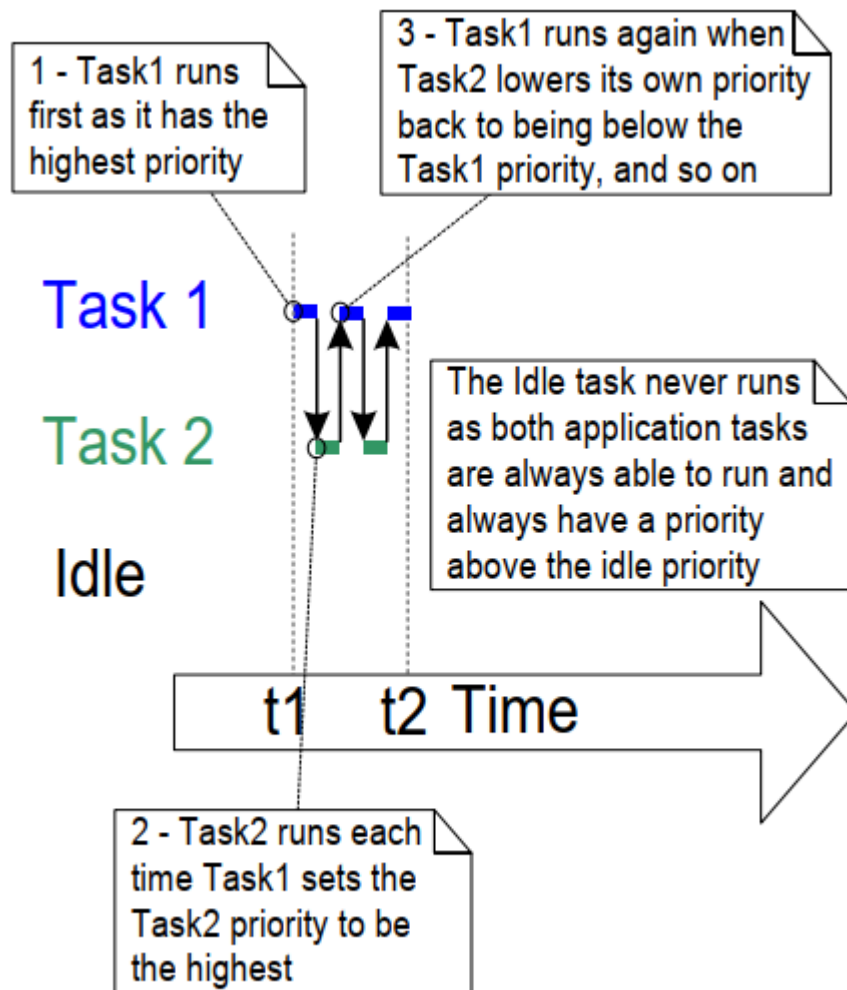


图 4.14 运行示例 4.8 时的任务执行序列

```

Task1 is running
About to raise the Task2 priority
Task2 is running
About to lower the Task2 priority
Task1 is running
About to raise the Task2 priority
Task2 is running
About to lower the Task2 priority
Task1 is running
About to raise the Task2 priority
Task2 is running
About to lower the Task2 priority
Task1 is running

```

4.10 删除任务

4.10.1 vTaskDelete() API 函数

`vTaskDelete()` API 函数删除一个任务。`vTaskDelete()` API 函数仅在 `FreeRTOSConfig.h` 中将 `INCLUDE_vTaskDelete` 设置为 1 时可用。

不建议在运行时持续创建和删除任务，因此如果您发现自己需要此函数，请考虑其他设计选项，例如重用任务。

已删除的任务不再存在，不能再进入 *运行中* 状态。

如果使用动态内存创建的任务随后删除自身，空闲任务负责释放曾为该任务分配的内存，例如任务的数据结构和栈。因此在这种情况下，应用程序必须保证空闲任务仍能获得一定的处理器时间。

注意：只有内核本身分配给任务的内存会在任务删除时自动释放。在任务实现期间分配的任何内存或其他资源，如果不再需要，必须显式释放。

```
void vTaskDelete( TaskHandle_t xTaskToDelete );
```

代码清单 4.26 vTaskDelete() API 函数原型

vTaskDelete() 参数

- `pxTaskToDelete`

要被删除（目标任务）的任务句柄。有关获取任务句柄的信息，请参见 `xTaskCreate()` API 函数的 `pxCreatedTask` 参数，以及 `xTaskCreateStatic()` API 函数的返回值。

任务可以通过传递 `NULL` 而不是有效的任务句柄来删除自身。

示例 4.9 删除任务

这是一个非常简单的示例，其行为如下。

1. 任务 1 由 `main()` 以优先级 1 创建。当它运行时，它以优先级 2 创建任务 2。任务 2 现在是最高优先级任务，因此它立即开始执行。代码清单 4.27 显示了 `main()` 的源代码。代码清单 4.28 显示了任务 1 的源代码。
2. 任务 2 除了删除自身外什么都不做。它可以通过向 `vTaskDelete()` 传递 NULL 来删除自身，但为了演示目的，它使用了自己的任务句柄。代码清单 4.29 显示了任务 2 的源代码。
3. 任务 2 被删除后，任务 1 再次成为最高优先级任务并继续执行；此时它调用 `vTaskDelay()` 阻塞一小段时间。
4. 当任务 1 处于阻塞状态时，空闲任务执行并释放分配给现已删除的任务 2 的内存。
5. 当任务 1 离开阻塞状态时，它再次成为最高优先级的就绪状态任务，因此抢占空闲任务。当它进入运行中状态时，它再次创建任务 2，如此循环。

```
int main( void )
{
    /* 以优先级 1 创建第一个任务。 */
    xTaskCreate( vTask1, "Task 1", 1000, NULL, 1, NULL );

    /* 启动调度器，以便任务开始执行。 */
    vTaskStartScheduler();

    /* main() 不应到达这里，因为调度器已启动。 */
    for( ;; )
    {
    }
}
```

代码清单 4.27 示例 4.9 的 `main()` 实现

```

TaskHandle_t xTask2Handle = NULL;

void vTask1( void * pvParameters )
{
    const TickType_t xDelay100ms = pdMS_TO_TICKS( 100UL );

    for( ;; )
    {
        /* 打印此任务的名称。 */
        vPrintLine( "Task 1 is running" );

        /*
         * 以更高优先级创建任务 2。
         * 传递 xTask2Handle 的地址作为 pxCreatedTask 参数, 以便
         * xTaskCreate 将生成的任务句柄写入该变量。
         */
        xTaskCreate( vTask2, "Task 2", 1000, NULL, 2, &xTask2Handle );

        /*
         * 任务 2 具有更高优先级。对于任务 1 到达此处, 任务 2 必须已经
         * 执行并删除了自身。
         */
        vTaskDelay( xDelay100ms );
    }
}

```

代码清单 4.28 示例 4.9 的任务 1 实现

```

void vTask2( void * pvParameters )
{
    /*
     * 任务 2 在启动后立即删除自身。
     * 为此, 它可以调用 vTaskDelete() 并使用 NULL 作为参数。
     * 为了演示目的, 它改为使用自己的任务句柄调用 vTaskDelete()。
     */
    vPrintLine( "Task 2 is running and about to delete itself" );
    vTaskDelete( xTask2Handle );
}

```

代码清单 4.29 示例 4.9 的任务 2 实现

```
C:\Temp>rtosdemo
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
Task1 is running
Task2 is running and about to delete itself
```

图 4.16 执行示例 4.9 时产生的输出

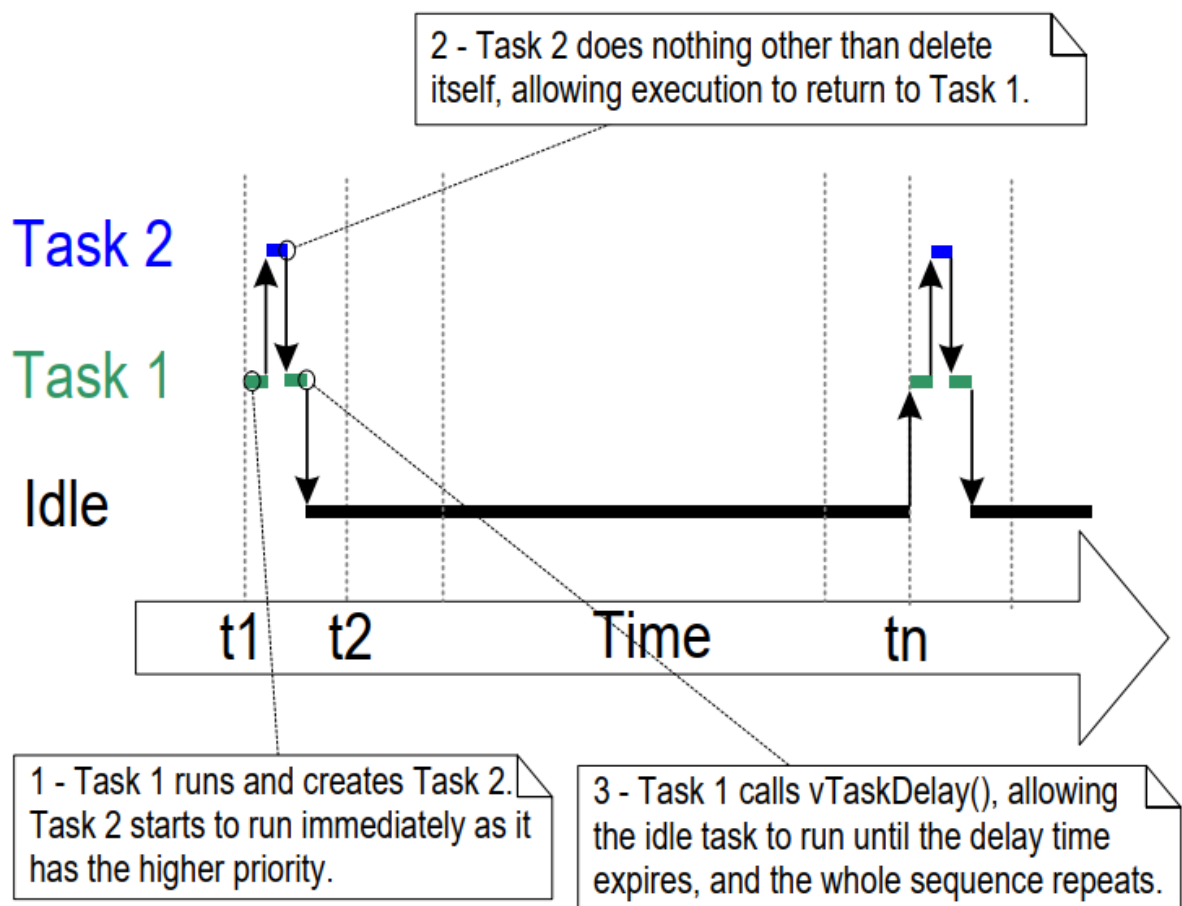


图 4.17 示例 4.9 的执行序列

4.11 线程局部存储和可重入性

线程局部存储允许应用程序开发者在每个任务的任务控制块中保存任意数据。它最常用于保存那些原本会由非可重入函数存放在全局变量中的数据。

可重入函数可以由多个线程安全调用，而不会在线程之间产生副作用。在多线程环境中使用非可重入函数而又没有线程局部存储时，必须格外小心，并在临界区内检查这些函数通过带外通道返回的结果。过度使用临界区会降低 RTOS 性能，因此通常更适合使用线程局部存储。

线程局部存储最常见的用途，是为 C 标准库和 POSIX 系统所使用的 ISO C 全局变量 `errno` 提供每线程实例。`errno` 用于保存 `strtof()`、`strtol()` 等常见标准库函数的扩展结果或错误码。

4.11.1 C 运行时线程局部存储实现

大多数嵌入式 libc 实现提供 API 以确保非可重入函数可以在多线程环境中正常工作。FreeRTOS 包含对两个常用开源库的重入 API 的支持：[newlib](#) 和 [picolibc](#)。在项目的 `FreeRTOSConfig.h` 文件中定义下列对应宏，即可启用这些预构建的 C 运行时线程局部存储实现。

- 对于 [newlib](#)，定义 `configUSE_NEWLIB_REENTRANT`
- 对于 [picolibc](#)，定义 `configUSE_PICOLIBC_TLS`

4.11.2 自定义 C 运行时线程局部存储

应用程序开发者可以在 `FreeRTOSConfig.h` 文件中定义以下宏，实现线程局部存储：

- 将 `configUSE_C_RUNTIME_TLS_SUPPORT` 定义为 1，以启用 C 运行时线程局部存储支持。
- 将 `configTLS_BLOCK_TYPE` 定义为用于保存 C 运行时线程局部存储数据的 C 类型。
- 将 `configINIT_TLS_BLOCK` 定义为初始化 C 运行时线程局部存储块时应执行的 C 代码。
- 将 `configSET_TLS_BLOCK` 定义为换入新任务时应执行的 C 代码。
- 将 `configDEINIT_TLS_BLOCK` 定义为销毁 C 运行时线程局部存储块时应执行的 C 代码。

4.11.3 应用程序线程局部存储

除 C 运行时线程局部存储外，应用程序开发者还可以定义一组随任务控制块保存的应用程序专用指针。在项目的 `FreeRTOSConfig.h` 文件中将 `configNUM_THREAD_LOCAL_STORAGE_POINTERS` 设置为非零值即可启用此功能。代码清单 4.30 中定义的 `vTaskSetThreadLocalStoragePointer()` 和 `pvTaskGetThreadLocalStoragePointer()` 函数，分别用于在运行时设置和读取各线程局部存储指针的值。

```
void * pvTaskGetThreadLocalStoragePointer( TaskHandle_t xTaskToQuery,
                                           BaseType_t xIndex )

void vTaskSetThreadLocalStoragePointer( TaskHandle_t xTaskToSet,
                                       BaseType_t xIndex,
                                       void * pvValue );
```

代码清单 4.30 线程局部存储指针 API 函数的函数原型

4.12 调度算法

4.12.1 任务状态和事件回顾

实际正在执行（占用处理器时间）的任务处于 *运行中* 状态。在单核处理器上，任何时刻都只能有一个任务处于 *运行中* 状态。也可以让多个 FreeRTOS 实例分别运行在多个核上（非对称多处理，AMP），或由 FreeRTOS 在多个核之间调度任务（对称多处理，SMP）。本书此处不讨论这些场景。

实际未运行但也不处于阻塞状态或 *挂起* 状态的任务处于 *就绪* 状态。处于就绪状态的任务可由调度器选择作为进入 *运行中* 状态的任务。调度器将始终选择优先级最高的就绪状态任务进入 *运行中* 状态。

任务可以在 *阻塞* 状态中等待事件，并在事件发生时自动返回 *就绪* 状态。定时事件在指定时刻发生，例如阻塞时间到期时，通常用于实现周期性操作或超时。同步事件则在任务或中断服务程序通过任务通知、队列、事件组、消息缓冲区、流缓冲区或某种信号量发送信息时发生，通常用于通知异步活动，例如外设收到数据。

4.12.2 选择调度算法

调度算法是决定将哪个 *就绪* 状态任务转换到 *运行中* 状态的软件例程。

到目前为止，所有示例都使用了相同的调度算法，但可以使用 `configUSE_PREEMPTION` 和 `configUSE_TIME_SLICING` 配置常量更改该算法。这两个常量都在 FreeRTOSConfig.h 中定义。

第三个配置常量 `configUSE_TICKLESS_IDLE` 也会影响调度算法，因为启用它后可能会长时间完全关闭 Tick 中断。`configUSE_TICKLESS_IDLE` 是一个高级选项，专门用于必须尽量降低功耗的应用程序。本节假设 `configUSE_TICKLESS_IDLE` 设置为 0，这也是未定义该常量时的默认值。

在所有可能的单核配置中，FreeRTOS 调度器依次选择共享同一优先级的任务。这种“轮流”策略通常称为“轮转调度”。轮转调度算法不保证处理时间在相同优先级的任务之间平均分配，只保证相同优先级的 *就绪* 状态任务轮流进入 *运行中* 状态。

调度算法	优先级调度	<code>configUSE_PREEMPTION</code>	<code>configUSE_TIME_SLICING</code>
带时间片的抢占式调度	是	1	1
不带时间片的抢占式调度	是	1	0
协作式	否	0	任意

4.12.3 带时间片的优先级抢占式调度

表 5 中显示的配置将 FreeRTOS 调度器设置为使用称为“带时间片的固定优先级抢占式调度”的调度算法，这是大多数小型 RTOS 应用程序使用的调度算法，也是本书迄今为止所有示例使用的算法。下表提供了算法名称中使用的术语说明。

用于描述调度策略的术语说明：

- 固定优先级

描述为“固定优先级”的调度算法不会更改分配给被调度任务的优先级，但也不会阻止任务自身更改自己的优先级或其他任务的优先级。

- 抢占式

抢占式调度算法会在优先级更高的任务进入 *就绪* 状态时，立即“抢占”当前处于 *运行中* 状态的任务。被抢占意味着任务在没有主动让出或阻塞的情况下，被移出 *运行中* 状态并转入 *就绪* 状态，以便另一个任务进入 *运行中* 状态。抢占可以在任何时刻发生，并不局限于 RTOS Tick 中断。

- 时间片

时间片用于在相同优先级的任务之间共享处理器时间，即使这些任务没有显式让出处理器或进入 *阻塞* 状态。采用 *时间片* 的调度算法会在每个时间片结束时检查是否还有与当前任务优先级相同的 *就绪* 状态任务；如果有，就选择另一个任务进入 *运行中* 状态。一个时间片等于两次 RTOS Tick 中断之间的时间。

图 4.18 和图 4.19 演示了使用带时间片的固定优先级抢占式调度算法时任务的调度方式。图 4.18 显示应用程序中的所有任务各自具有不同优先级时，任务进入 *运行中* 状态的顺序。图 4.19 则显示两个任务共享同一优先级时的执行顺序。

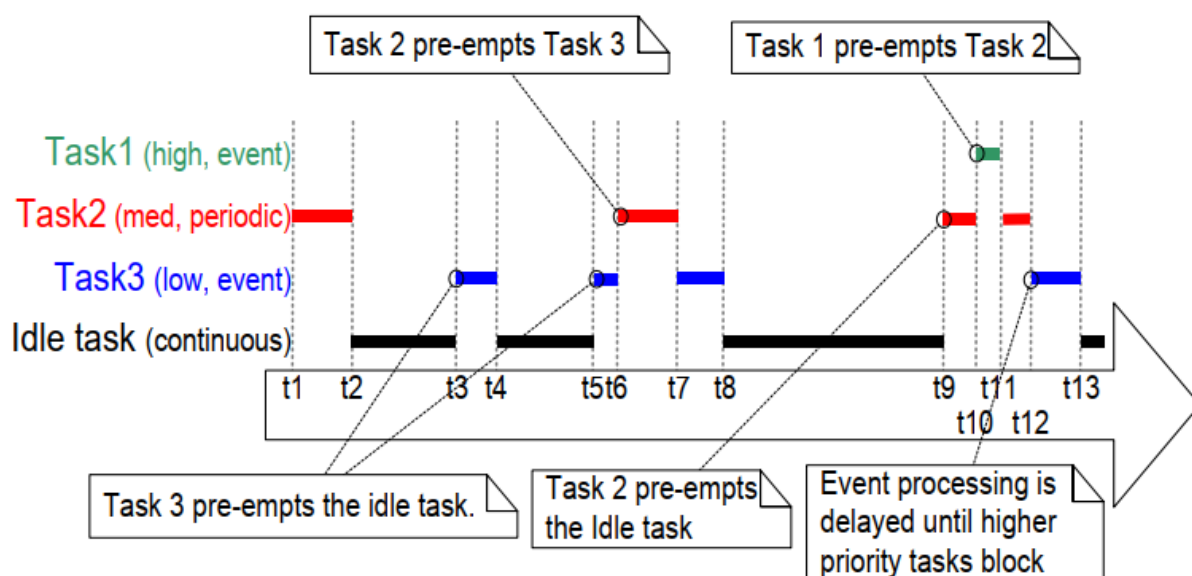


图 4.18 在一个假设应用程序中突出显示任务优先级和抢占的执行模式，其中每个任务被分配了唯一优先级

参考图 4.18:

- 空闲任务

空闲任务以最低优先级运行，因此每次更高优先级的任务进入 *就绪* 状态时都会被抢占，例如在时间 t3、t5 和 t9。

- 任务 3

任务 3 是一个事件驱动任务，以相对较低的优先级执行，但高于空闲优先级。它大部分时间处于 *阻塞* 状态等待其感兴趣的事件，每次事件发生时从 *阻塞* 状态转换到 *就绪* 状态。所有 FreeRTOS 任务间通信机制（任务通知、队列、信号量、事件组等）都可用于以这种方式发信号通知事件和解除任务阻塞。

事件在时间 t3 和 t5 发生，以及 t9 和 t12 之间的某个时间发生。在时间 t3 和 t5 发生的事件立即被处理，因为在这些时间，任务 3 是可以运行的优先级最高的任务。在 t9 和 t12 之间发生的某个事件直到 t12 才被处理，因为在此之前，较高优先级的任务 1 和任务 2 仍在执行。只有在 t12 时，任务 1 和任务 2 都处于 *阻塞* 状态，才使任务 3 成为优先级最高的 *就绪* 状态任务。

- 任务 2

任务 2 是一个周期性任务，以高于任务 3 但低于任务 1 的优先级执行。任务的周期意味着任务 2 希望在时间 t1、t6 和 t9 执行。

在时间 t6，任务 3 处于 *运行中* 状态，但任务 2 具有较高的相对优先级，因此抢占任务 3 并立即开始执行。任务 2 完成其处理并在时间 t7 重新进入 *阻塞* 状态，此时任务 3 可以重新进入 *运行中* 状态以完成其处理。任务 3 本身在时间 t8 阻塞。

- 任务 1

任务 1 也是一个事件驱动任务。它以所有任务中最高的优先级执行，因此可以抢占系统中的任何其他任务。显示的唯一任务 1 事件在时间 t10 发生，此时任务 1 抢占任务 2。只有在任务 1 在时间 t11 重新进入 *阻塞* 状态后，任务 2 才能完成其处理。

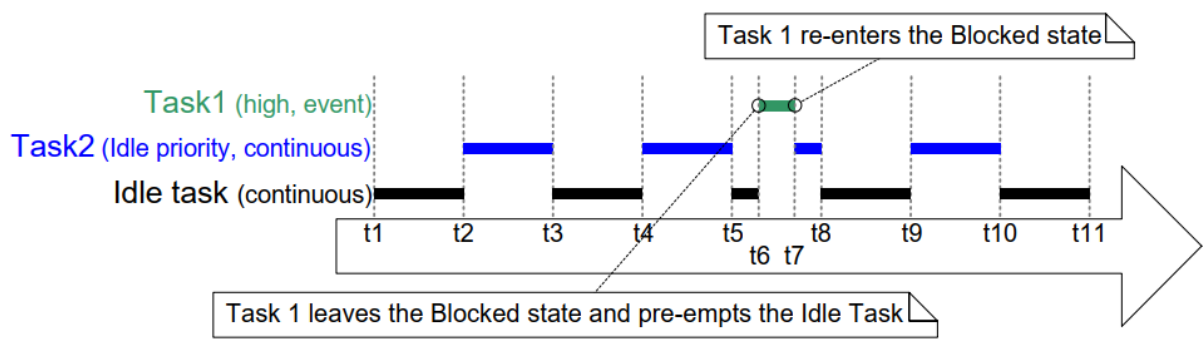


图 4.19 在一个假设应用程序中突出显示任务优先级和时间片的执行模式，其中两个任务以相同优先级运行

参考图 4.19:

- 空闲任务和任务 2

空闲任务和任务 2 都是连续处理任务，并且都具有优先级 0（最低可能优先级）。调度器仅在没有任何更高优先级的任务可以运行时才将处理时间分配给优先级 0 的任务，并通过时间片共享分配给优先级 0 任务的时间。新的时间片在每个 tick 中断时开始，在图 4.19 中发生在时间 t1、t2、t3、t4、t5、t8、t9、t10 和 t11。

空闲任务和任务 2 轮流进入 *运行中* 状态，这可能导致两个任务在同一时间片的部分时间内处于 *运行中* 状态，如 t5 和 t8 之间发生的那样。

- 任务 1

任务 1 的优先级高于空闲优先级。任务 1 是一个事件驱动任务，大部分时间处于 *阻塞* 状态等待其感兴趣的事件，每次事件发生时从 *阻塞* 状态转换到 *就绪* 状态。

感兴趣的事件在时间 t6 发生。在 t6，任务 1 成为可以运行的优先级最高的任务，因此任务 1 在一个时间片中间抢占空闲任务。事件的处理在时间 t7 完成，此时任务 1 重新进入阻塞状态。

图 4.19 显示空闲任务与应用程序创建的任务共享处理器时间。如果应用程序创建的空闲优先级任务有工作要做，而空闲任务本身没有，那么为后者分配如此多的处理器时间可能并不合适。可以使用编译时配置常量 `configIDLE_SHOULD_YIELD` 更改空闲任务的调度方式：

- 如果 `configIDLE_SHOULD_YIELD` 设置为 0，则空闲任务在其整个时间片内保持在 *运行中* 状态，除非被更高优先级的任务抢占。
- 如果 `configIDLE_SHOULD_YIELD` 设置为 1，则如果存在其他处于 *就绪* 状态的空闲优先级任务，空闲任务在其循环的每次迭代时让出（自愿放弃其分配时间片的剩余部分）。

图 4.19 所示的执行模式是当 `configIDLE_SHOULD_YIELD` 设置为 0 时观察到的。图 4.20 所示的执行模式是在相同场景下当 `configIDLE_SHOULD_YIELD` 设置为 1 时观察到的。

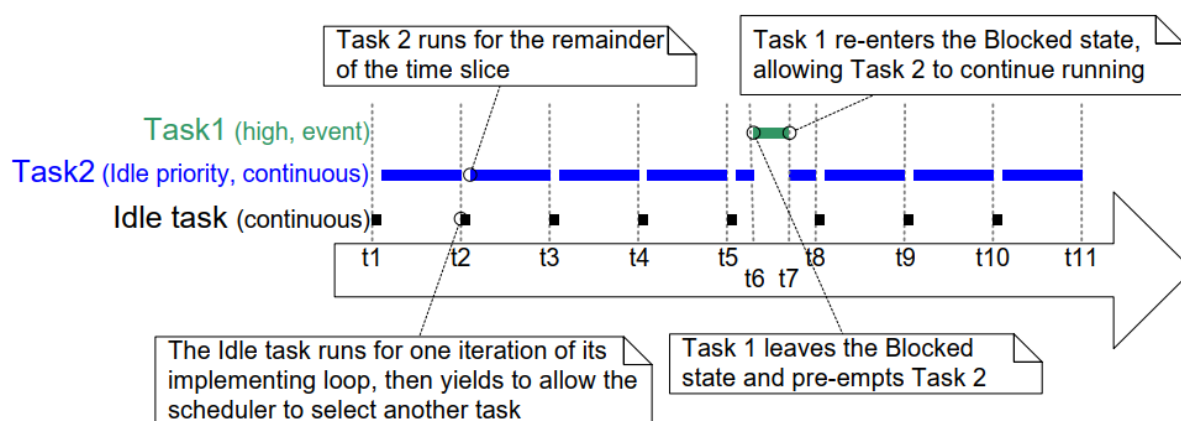


图 4.20 与图 4.19 相同场景的执行模式，但这次 `configIDLE_SHOULD_YIELD` 设置为 1

图 4.20 还表明，当 `configIDLE_SHOULD_YIELD` 设置为 1 时，在空闲任务之后进入 *运行中* 状态的任务不会获得完整的时间片，只会执行空闲任务让出后的剩余部分。

4.12.4 不带时间片的优先级抢占式调度

不带时间片的优先级抢占式调度保持与上一节描述的相同的任务选择和抢占算法，但不使用时间片在相同优先级的任务之间共享处理时间。

表 5 显示了将 FreeRTOS 调度器配置为使用不带时间片的优先级抢占式调度的 FreeRTOSConfig.h 设置。

如图 4.19 所示，如果使用时间片，并且有多个处于就绪状态的最高优先级任务可以运行，则调度器在每个 RTOS tick 中断（tick 中断标记一个时间片的结束）期间选择一个新任务进入 *运行中* 状态。如果不使用时间片，则调度器仅在以下情况下选择新任务进入 *运行中* 状态：

- 更高优先级的任务进入 *就绪* 状态。
- *运行中* 状态的任务进入 *阻塞* 或 *挂起* 状态。

不使用时间片时的任务上下文切换次数少于使用时间片时。因此，关闭时间片会导致调度器处理开销的减少。但是，关闭时间片也可能导致相同优先级的任务获得截然不同的处理时间量，图 4.21 演示了这种场景。因此，不使用时间片运行调度器被认为是一种高级技术，只应由有经验的用户使用。

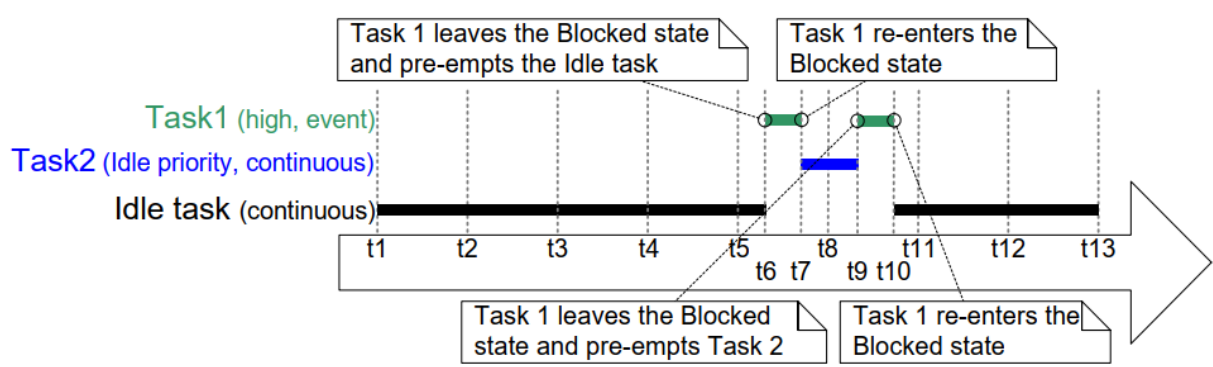


图 4.21 演示了在不使用时间片时相同优先级的任务如何获得差异巨大的处理时间量的执行模式

参考图 4.21，假设 `configIDLE_SHOULD_YIELD` 设置为 0：

- Tick 中断

Tick 中断发生在时间 t1、t2、t3、t4、t5、t8、t11、t12 和 t13。

- 任务 1

任务 1 是一个高优先级事件驱动任务，大部分时间处于 *阻塞* 状态等待其感兴趣的事件。每次事件发生时，任务 1 从 *阻塞* 状态转换到 *就绪* 状态（随后，作为优先级最高的 *就绪* 状态任务，进入 *运行中* 状态）。图 4.21 显示任务 1 在 t6 和 t7 之间处理一个事件，然后在 t9 和 t10 之间再次处理一个事件。

- 空闲任务和任务 2

空闲任务和任务 2 都是连续处理任务，并且都具有优先级 0（空闲优先级）。连续处理任务不会进入 *阻塞* 状态。

不使用时间片，因此处于 *运行中* 状态的空闲优先级任务将保持在 *运行中* 状态，直到被更高优先级的任务 1 抢占。

在图 4.21 中，空闲任务从时间 t1 开始运行，并保持在 *运行中* 状态，直到在时间 t6 被任务 1 抢占，这距离其进入 *运行中* 状态已经超过了四个完整的 tick 周期。

任务 2 从时间 t7 开始运行，即任务 1 重新进入 *阻塞* 状态等待另一个事件时。任务 2 保持在 *运行中* 状态，直到同样在时间 t9 被任务 1 抢占，这距离其进入 *运行中* 状态不到一个 tick 周期。

在时间 t10，空闲任务重新进入 *运行中* 状态，尽管它此前获得的处理器时间已经超过任务 2 的四倍。

4.12.5 协作式调度

本书侧重于抢占式调度，但 FreeRTOS 也可以使用协作式调度。表 5 显示了将 FreeRTOS 调度器配置为使用协作式调度的 FreeRTOSConfig.h 设置。

使用协作式调度器时（并假定应用程序提供的中断服务程序不会显式请求上下文切换），只有当 *运行中* 状态的任务进入 *阻塞* 状态，或调用 `taskYIELD()` 主动让出处理器（手动请求重新调度）时，才会发生上下文切换。任务不会被抢占，因此也不能使用时间片。

图 4.22 演示了协作式调度器的行为。图 4.22 中的水平虚线显示任务何时处于就绪状态。

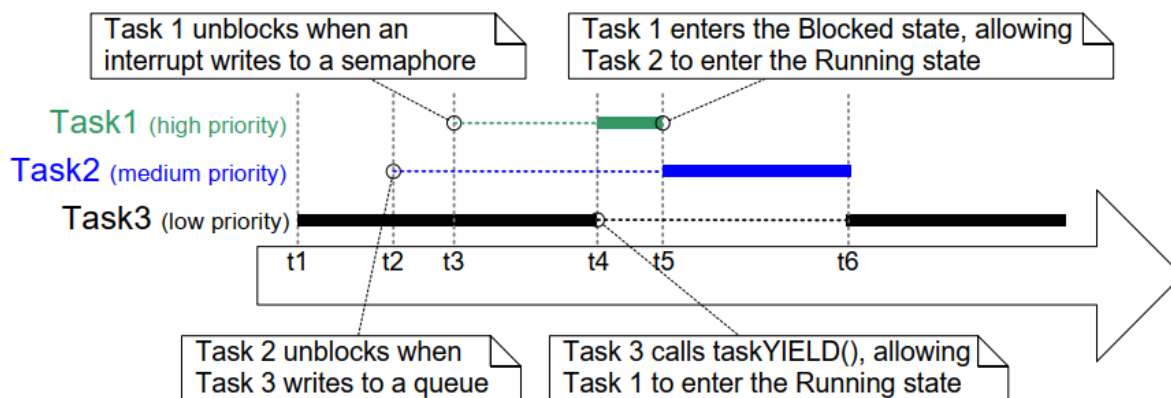


图 4.22 演示协作式调度器行为的执行模式

参考图 4.22:

- 任务 1

任务 1 具有最高优先级。它开始时处于 *阻塞* 状态，等待一个信号量。

在时间 t_3 ，一个中断释放了信号量，使任务 1 离开 *阻塞* 状态并进入 *就绪* 状态（第 6 章将介绍如何从中断中释放信号量）。

在时间 t_3 ，任务 1 是最高优先级的 *就绪* 状态任务，如果使用抢占式调度器，任务 1 将成为 *运行中* 状态任务。但是，由于使用的是协作式调度器，任务 1 保持在 *就绪* 状态，直到时间 t_4 ，即 *运行中* 状态任务调用 `taskYIELD()` 时。

- 任务 2

任务 2 的优先级介于任务 1 和任务 3 之间。它开始时处于 *阻塞* 状态，等待任务 3 在时间 t_2 发送给它的消息。

在时间 t_2 ，任务 2 是最高优先级的 *就绪* 状态任务，如果使用抢占式调度器，任务 2 将成为 *运行中* 状态任务。但是，由于使用的是协作式调度器，任务 2 保持在 *就绪* 状态，直到 *运行中* 状态的任务进入 *阻塞* 状态或调用 `taskYIELD()`。

运行中 状态的任务在时间 t_4 调用 `taskYIELD()`，但此时任务 1 是最高优先级的 *就绪* 状态任务，因此直到任务 1 在时间 t_5 重新进入 *阻塞* 状态之前，任务 2 实际上不会成为 *运行中* 状态的任务。

在时间 t6，任务 2 重新进入 *阻塞* 状态等待下一条消息，此时任务 3 再次成为最高优先级的 *就绪* 状态任务。

在多任务应用程序中，应用程序编写者必须注意一个资源不会被多个任务同时访问，因为同时访问可能破坏资源。例如，考虑以下场景，其中被访问的资源是 UART（串行端口）。两个任务向 UART 写入字符串；任务 1 写入 "abcdefghijklmnop"，任务 2 写入 "123456789"：

1. 任务 1 处于 *运行中* 状态并开始写入其字符串。它向 UART 写入 "abcdefg"，但在写入任何更多字符之前离开 *运行中* 状态。
2. 任务 2 进入 *运行中* 状态并向 UART 写入 "123456789"，然后离开 *运行中* 状态。
3. 任务 1 重新进入 *运行中* 状态并向 UART 写入其字符串的其余字符。

在该场景中，实际写入 UART 的是 "abcdefg123456789hijklmnop"。任务 1 写入的字符串没有按预期以不间断的顺序写入 UART，而是被破坏了，因为任务 2 写入 UART 的字符串出现在其中。

使用协作式调度器通常比使用抢占式调度器更容易避免由同时访问引起的问题^[4]：

- 当您使用抢占式调度器时，*运行中* 状态的任务可以在任何时候被抢占，包括当它与另一个任务共享的资源处于不一致状态时。正如 UART 示例刚刚演示的，将资源留在不一致状态可能导致数据损坏。
- 当您使用协作式调度器时，您可以控制何时切换到另一个任务。因此，您可以确保在资源处于不一致状态时不会发生切换到另一个任务。
- 在上述 UART 示例中，您可以确保任务 1 在将其整个字符串写入 UART 之前不会离开 *运行中* 状态，从而消除了字符串被另一个任务的活动破坏的可能性。

如图 4.22 所示，使用协作式调度器使系统的响应性低于使用抢占式调度器：

- 使用抢占式调度器时，一旦某个任务成为优先级最高的 *就绪* 状态任务，调度器就会立即开始运行它。实时系统通常必须在规定时间内响应高优先级事件，因此往往需要这种行为。
- 使用协作式调度器时，直到 *运行中* 状态的任务进入 *阻塞* 状态或调用 `taskYIELD()` 后，才会执行切换到已成为最高优先级 *就绪* 状态的任务。

-
1. 屏幕截图显示每个任务在下个任务执行之前恰好打印一次其消息。这是使用 FreeRTOS Windows 模拟器导致的人工场景。Windows 模拟器不是真正的实时。此外，写入 Windows 控制台需要相对较长的时间，并导致一系列 Windows 系统调用。在真正的嵌入式目标上使用快速且非阻塞的打印函数执行相同的代码，可能会导致每个任务在被切换到以允许另一个任务运行之前多次打印其字符串。↩

2. 需要注意的是，时间片的结束不是调度器可以选择新运行任务的唯一时机。正如我们将在本书中演示的，调度器还会在当前执行任务进入 *阻塞* 状态后立即选择新任务运行，或者当中断将更高优先级的任务移到 *就绪* 状态时。↩
3. 即使在使用 FreeRTOS 的特殊低功耗功能时也是如此，在这种情况下，如果应用程序创建的任何任务都无法执行，则运行 FreeRTOS 的微控制器将进入低功耗模式。↩
4. 在任务之间安全共享资源的方法将在本书后面介绍。FreeRTOS 本身提供的资源，如队列和信号量，在任务之间始终是安全的。↩

5 队列管理

5.1 引言

“队列”提供了任务到任务、任务到中断以及中断到任务的通信机制。

5.1.1 范围

本章涵盖：

- 如何创建队列。
- 队列如何管理其包含的数据。
- 如何向队列发送数据。
- 如何从队列接收数据。
- 在队列上阻塞的含义。
- 如何在多个队列上阻塞。
- 如何覆盖队列中的数据。
- 如何清空队列。
- 任务优先级在读写队列时的影响。

本章仅讨论任务到任务的通信。第 7 章将介绍任务到中断以及中断到任务的通信。

5.2 队列的特性

5.2.1 数据存储

一个队列可以容纳有限数量、大小固定的数据项^[1]。队列最多能够容纳的数据项数量称为队列的“长度”。队列长度和每个数据项的大小都在创建队列时确定。

队列通常用作先进先出（FIFO）缓冲区：数据写入队列尾部，并从队列头部移除。图 5.1 演示了向 FIFO 队列写入数据以及从中读取数据的过程。也可以把数据写入队列头部，或覆盖队列头部已有的数据。

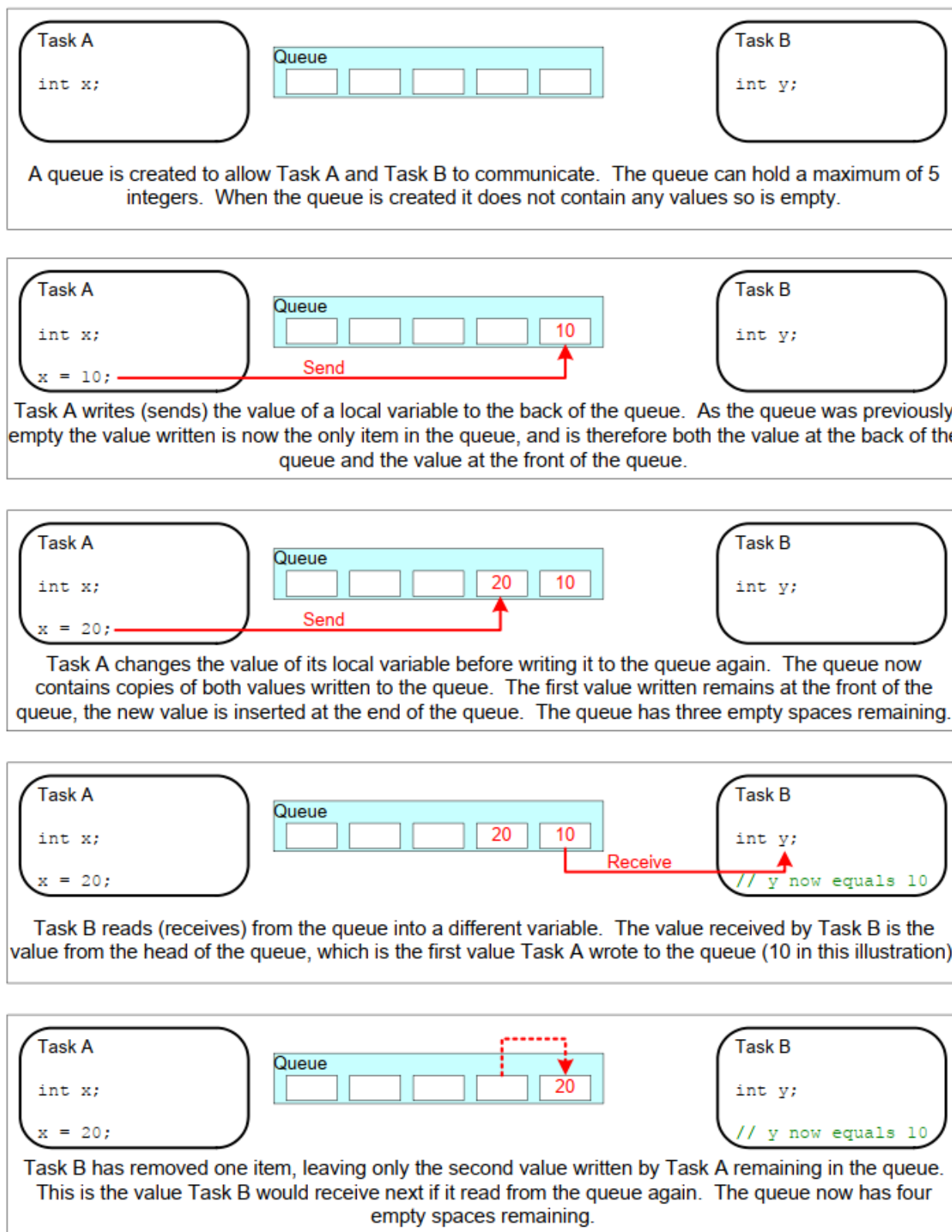


图 5.1 队列写入和读取操作的示例序列

队列行为可以通过两种方式实现：

1. 按值复制入队

按值复制入队是指将发送的数据逐字节复制到队列中。

2. 按引用入队

按引用入队是指队列只保存指向数据的指针，而不保存数据本身。

FreeRTOS 采用按值复制入队，因为它比按引用入队功能更强，也更易于使用：

- 按值复制并不妨碍队列用于按引用入队。例如，数据太大、不适合直接复制进队列时，可以改为把指向数据的指针复制进队列。
- 栈变量可以直接发送到队列，即使该变量在其声明的函数退出后将不再存在。
- 可以直接向队列发送数据，无需先分配缓冲区、把数据复制到缓冲区，再将缓冲区的引用入队。
- 发送任务可以立即重用发送给队列的变量或缓冲区。
- 发送任务和接收任务完全解耦：应用程序设计者无需考虑哪个任务“拥有”数据，也无需决定由哪个任务负责释放数据。
- RTOS 完全负责分配用于保存数据的内存。
- 在具有内存保护的系统中，对 RAM 的访问会受到限制。只有发送任务和接收任务都能访问被引用的数据时，按引用入队才可行；按值复制则允许数据跨越内存保护边界传递。

5.2.2 多个任务访问

队列本身是一种对象，任何知道该队列存在的任务或 ISR 都可以访问它。任意数量的任务可以写入同一队列，也可以从同一队列读取。实际应用中，一个队列有多个写入者很常见，但有多多个读取者则少见得多。

5.2.3 在队列读取时阻塞

任务尝试从队列读取数据时，可以选择指定“阻塞时间”。如果队列为空，任务会在这段时间内保持 *阻塞* 状态，等待队列中出现可用数据。另一个任务或中断把数据放入队列后，等待该数据的任务会自动转入 *就绪* 状态。如果数据出现前阻塞时间已经到期，任务同样会自动从 *阻塞* 状态转入 *就绪* 状态。

一个队列可以有多个读取者，因此可能有多个任务同时阻塞在同一队列上等待数据。数据到达时只会解除一个任务的阻塞，所选择的是等待任务中优先级最高的任务。如果两个或多个阻塞任务优先级相同，则解除等待时间最长的任务。

5.2.4 在队列写入时阻塞

与读取队列相同，任务写入队列时也可以指定阻塞时间。如果队列已满，该时间表示任务保持 *阻塞* 状态、等待队列出现可用空间的最长时间。

一个队列可以有多个写入者，因此队列已满时，可能有多个任务阻塞在该队列上等待完成发送操作。队列出现可用空间时只会解除一个任务的阻塞，所选择的是等待任务中优先级最高的任务。如果两个或多个阻塞任务优先级相同，则解除等待时间最长的任务。

5.2.5 在多个队列上阻塞

多个队列可以组成队列集，使任务能够进入 *阻塞* 状态，等待其中任意队列出现可用数据。第 5.6 节“从多个队列接收数据”将演示队列集的用法。

5.2.6 创建队列：静态分配与动态分配

队列通过句柄引用，句柄是 `QueueHandle_t` 类型的变量。队列必须先显式创建，才能使用。

创建队列有两个 API 函数：`xQueueCreate()` 和 `xQueueCreateStatic()`。

每个队列需要两块 RAM：第一块保存队列的数据结构，第二块保存入队的数据。`xQueueCreate()` 从堆中动态分配所需 RAM；`xQueueCreateStatic()` 则使用通过参数传入的预分配 RAM。

5.3 使用队列

5.3.1 xQueueCreate() API 函数

代码清单 5.1 给出了 `xQueueCreate()` 的函数原型。`xQueueCreateStatic()` 还有两个参数，分别指向预先分配给队列数据结构和数据存储区的内存。

```
QueueHandle_t xQueueCreate( UBaseType_t uxQueueLength, UBaseType_t uxItemSize );
```

代码清单 5.1 `xQueueCreate()` API 函数原型

`xQueueCreate()` 参数和返回值：

- `uxQueueLength`

所创建的队列在任何时候能容纳的最大项数。

- `uxItemSize`

可以存储在队列中的每个数据项的大小（以字节为单位）。

- 返回值

如果返回 `NULL`，说明没有足够的堆内存供 FreeRTOS 分配队列数据结构和存储区，队列创建失败。第 3 章提供了有关 FreeRTOS 堆的更多信息。

如果返回非 `NULL` 值，则队列创建成功，返回值是新队列的句柄。

`xQueueReset()` API 函数用于把已创建的队列恢复为空队列。

5.3.2 xQueueSendToBack() 和 xQueueSendToFront() API 函数

顾名思义，`xQueueSendToBack()` 把数据发送到队列尾部，`xQueueSendToFront()` 则把数据发送到队列头部。

`xQueueSend()` 与 `xQueueSendToBack()` 完全等价。

注意：切勿在中断服务程序中调用 `xQueueSendToFront()` 或 `xQueueSendToBack()`。应改用中断安全版本 `xQueueSendToFrontFromISR()` 和 `xQueueSendToBackFromISR()`。第 7 章将介绍这些函数。

```
BaseType_t xQueueSendToFront( QueueHandle_t xQueue,  
                             const void * pvItemToQueue,  
                             TickType_t xTicksToWait );
```

代码清单 5.2 xQueueSendToFront() API 函数原型

```
BaseType_t xQueueSendToBack( QueueHandle_t xQueue,  
                             const void * pvItemToQueue,  
                             TickType_t xTicksToWait );
```

代码清单 5.3 xQueueSendToBack() API 函数原型

`xQueueSendToFront()` 和 `xQueueSendToBack()` 函数参数和返回值

- `xQueue`

接收待发送（写入）数据的队列句柄。该句柄由创建队列时调用的 `xQueueCreate()` 或 `xQueueCreateStatic()` 返回。

- `pvItemToQueue`

指向待复制进队列的数据。

创建队列时已经设定每个数据项的大小，因此函数会从 `pvItemToQueue` 指向的位置复制相应字节数到队列存储区。

- `xTicksToWait`

队列已满时，任务保持 阻塞 状态、等待队列出现可用空间的最长时间。

如果 `xTicksToWait` 为 0 且队列已满，`xQueueSendToFront()` 和 `xQueueSendToBack()` 都会立即返回。

阻塞时间以 Tick 周期为单位，因此对应的实际时长取决于 Tick 频率。宏 `pdMS_TO_TICKS()` 可将毫秒转换为 Tick 数。

如果在 `FreeRTOSConfig.h` 中将 `INCLUDE_vTaskSuspend` 设置为 1，再把 `xTicksToWait` 设置为 `portMAX_DELAY`，任务将无限期等待而不会超时。

- 返回值

有两个可能的返回值：

- `pdPASS`

数据成功发送到队列时返回 `pdPASS`。

如果指定了非零阻塞时间，调用任务可能先进入 阻塞 状态等待可用空间，但在阻塞时间到期前成功把数据写入队列。

- `errQUEUE_FULL`（与 `pdFAIL` 的值相同）

如果队列已满、无法写入数据，则返回 `errQUEUE_FULL`。

如果指定了非零阻塞时间，调用任务会进入 阻塞 状态，等待另一个任务或中断从队列中腾出空间；但在空间出现之前，指定的阻塞时间已经到期。

5.3.3 xQueueReceive() API 函数

xQueueReceive() 从队列接收（读取）一个数据项，并将该数据项从队列中移除。

注意：切勿在中断服务程序中调用 **xQueueReceive()**。第 7 章将介绍中断安全的 **xQueueReceiveFromISR()** API 函数。

```
BaseType_t xQueueReceive( QueueHandle_t xQueue,  
                          void * const pvBuffer,  
                          TickType_t xTicksToWait );
```

代码清单 5.4 xQueueReceive() API 函数原型

xQueueReceive() 函数参数和返回值

- **xQueue**

待读取数据的队列句柄。该句柄由创建队列时调用的 **xQueueCreate()** 或 **xQueueCreateStatic()** 返回。

- **pvBuffer**

指向用于保存接收数据的内存。

创建队列时已经设定每个数据项的大小。**pvBuffer** 指向的内存必须足以容纳相应的字节数。

- **xTicksToWait**

队列为空时，任务保持 阻塞 状态、等待数据到达队列的最长时间。

如果 **xTicksToWait** 为 0 且队列为空，**xQueueReceive()** 将立即返回。

阻塞时间以 Tick 周期为单位，因此对应的实际时长取决于 Tick 频率。宏 **pdMS_TO_TICKS()** 可将毫秒转换为 Tick 数。

如果在 **FreeRTOSConfig.h** 中将 **INCLUDE_vTaskSuspend** 设置为 1，再把 **xTicksToWait** 设置为 **portMAX_DELAY**，任务将无限期等待而不会超时。

- 返回值

有两个可能的返回值：

- `pdPASS`

成功从队列读取数据时返回 `pdPASS`。

如果指定了非零阻塞时间，调用任务可能先进入 *阻塞* 状态等待数据，但在阻塞时间到期前成功从队列读到了数据。

- `errQUEUE_EMPTY`（与 `pdFAIL` 的值相同）

如果队列为空、无法读取数据，则返回 `errQUEUE_EMPTY`。

如果指定了非零阻塞时间，调用任务会进入 *阻塞* 状态，等待另一个任务或中断向队列发送数据；但在数据到达之前，阻塞时间已经到期。

5.3.4 `uxQueueMessagesWaiting()` API 函数

`uxQueueMessagesWaiting()` 查询队列中当前的数据项数量。

注意：切勿在中断服务程序中调用 `uxQueueMessagesWaiting()`。应改用中断安全的 `uxQueueMessagesWaitingFromISR()`。

```
UBaseType_t uxQueueMessagesWaiting( QueueHandle_t xQueue );
```

代码清单 5.5 `uxQueueMessagesWaiting()` API 函数原型

`uxQueueMessagesWaiting()` 函数参数和返回值

- `xQueue`

要查询的队列句柄。该句柄由创建队列时调用的 `xQueueCreate()` 或 `xQueueCreateStatic()` 返回。

- 返回值

队列中当前的数据项数量。返回 0 表示队列为空。

示例 5.1 从队列接收时阻塞

本示例演示如何创建队列、由多个任务向队列发送数据，以及从队列接收数据。队列用于保存 `int32_t` 类型的数据项。发送任务不指定阻塞时间，接收任务则指定阻塞时间。

发送任务的优先级低于接收任务。因此，队列中不会同时存在多个数据项：数据一进入队列，较高优先级的接收任务就会解除阻塞并抢占发送任务，随后取出数据，使队列再次变为空。

本示例创建代码清单 5.6 所示任务的两个实例：一个持续向队列写入值 100，另一个持续向同一队列写入值 200。这两个值通过任务参数传给相应的任务实例。

```
static void vSenderTask( void *pvParameters )
{
    int32_t lValueToSend;

    BaseType_t xStatus;

    /* 本任务会创建两个实例。待发送到队列的值通过任务参数传入，
       因而每个实例可以使用不同的值。队列用于保存 int32_t 类型的值，
       所以需要把参数转换为相应类型。 */
    lValueToSend = ( int32_t ) pvParameters;

    /* 与大多数任务一样，本任务在无限循环中实现。 */
    for( ;; )
    {
        /* 把值发送到队列。

           第一个参数是接收数据的队列。该队列在调度器启动之前创建，
           因而也早于本任务开始执行。

           第二个参数是待发送数据的地址，此处为 lValueToSend 的地址。

           第三个参数是阻塞时间，即队列已满时，任务为等待可用空间而
           保持阻塞状态的时间。本例不指定阻塞时间，因为队列中不应
           同时存在多个数据项，因此也不应变满。 */
        xStatus = xQueueSendToBack( xQueue, &lValueToSend, 0 );

        if( xStatus != pdPASS )
        {
            /* 队列已满，发送操作无法完成。队列中本不应同时存在
               多个数据项，因此这一定是错误。 */
            vPrintString( "Could not send to the queue.\r\n" );
        }
    }
}
```

代码清单 5.6 示例 5.1 使用的发送任务实现

代码清单 5.7 给出了接收任务的实现。该任务指定 100 毫秒的阻塞时间，并进入 *阻塞* 状态等待数据。当队列中出现数据，或等待 100 毫秒后仍无数据时，任务都会离开 *阻塞* 状态。本示例有两个任务持续写入队列，因此不会发生 100 毫秒超时。

```
static void vReceiverTask( void *pvParameters )
{
    /* 声明用于保存队列接收值的变量。 */
    int32_t lReceivedValue;
    BaseType_t xStatus;
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100 );

    /* 本任务同样在无限循环中实现。 */
    for( ;; )
    {
        /* 本任务会立即移除写入队列的所有数据，因此此处队列应始终为空。 */
        if( uxQueueMessagesWaiting( xQueue ) != 0 )
        {
            vPrintString( "Queue should have been empty!\r\n" );
        }

        /* 从队列接收数据。

        第一个参数是读取数据的队列。该队列在调度器启动之前创建，
        因而也早于本任务首次运行。

        第二个参数是保存接收数据的缓冲区。此处直接使用一个大小
        足以保存接收数据的变量地址。

        最后一个参数是阻塞时间，即队列为空时，任务为等待数据而
        保持阻塞状态的最长时间。 */
        xStatus = xQueueReceive( xQueue, &lReceivedValue, xTicksToWait );

        if( xStatus == pdPASS )
        {
            /* 已成功从队列接收数据，打印接收到的值。 */
            vPrintStringAndNumber( "Received = ", lReceivedValue );
        }
        else
        {
            /* 等待 100 毫秒后仍未从队列接收到数据。发送任务会持续运行
            并不断写入队列，因此这一定是错误。 */
            vPrintString( "Could not receive from the queue.\r\n" );
        }
    }
}
```

代码清单 5.7 示例 5.1 的接收任务实现

代码清单 5.8 给出了 `main()` 函数。它创建队列和三个任务，然后启动调度器。队列最多可以容纳五个 `int32_t` 值，不过由于任务的相对优先级，队列中实际不会同时存在多个数据项。

```

/* 声明 QueueHandle_t 类型的变量，用于保存三个任务共同访问的队列句柄。 */
QueueHandle_t xQueue;

int main( void )
{
    /* 创建最多可保存 5 个 int32_t 类型值的队列。 */
    xQueue = xQueueCreate( 5, sizeof( int32_t ) );

    if( xQueue != NULL )
    {
        /* 创建两个发送任务实例。通过任务参数传入写入队列的值，
           因一个任务持续写入 100，另一个任务持续写入 200。
           两个任务的优先级均为 1。 */
        xTaskCreate( vSenderTask, "Sender1", 1000, ( void * ) 100, 1, NULL );
        xTaskCreate( vSenderTask, "Sender2", 1000, ( void * ) 200, 1, NULL );

        /* 创建读取队列的任务。其优先级为 2，高于发送任务。 */
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 2, NULL );

        /* 启动调度器，使已创建的任务开始执行。 */
        vTaskStartScheduler();
    }
    else
    {
        /* 队列创建失败。 */
    }

    /* 如果一切正常，调度器会持续运行任务，main() 不会执行到这里。
       如果 main() 执行到这里，很可能是 FreeRTOS 堆内存不足，无法创建
       空闲任务。第 3 章提供了更多堆内存管理信息。 */
    for( ;; );
}

```

代码清单 5.8 示例 5.1 的 `main()` 实现

图 5.2 显示了示例 5.1 产生的输出。

```
C:\WINDOWS\system32\cmd.exe - rtosdemo
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
Received = 100
Received = 200
```

图 5.2 执行示例 5.1 时产生的输出

图 5.3 展示了相应的执行序列。

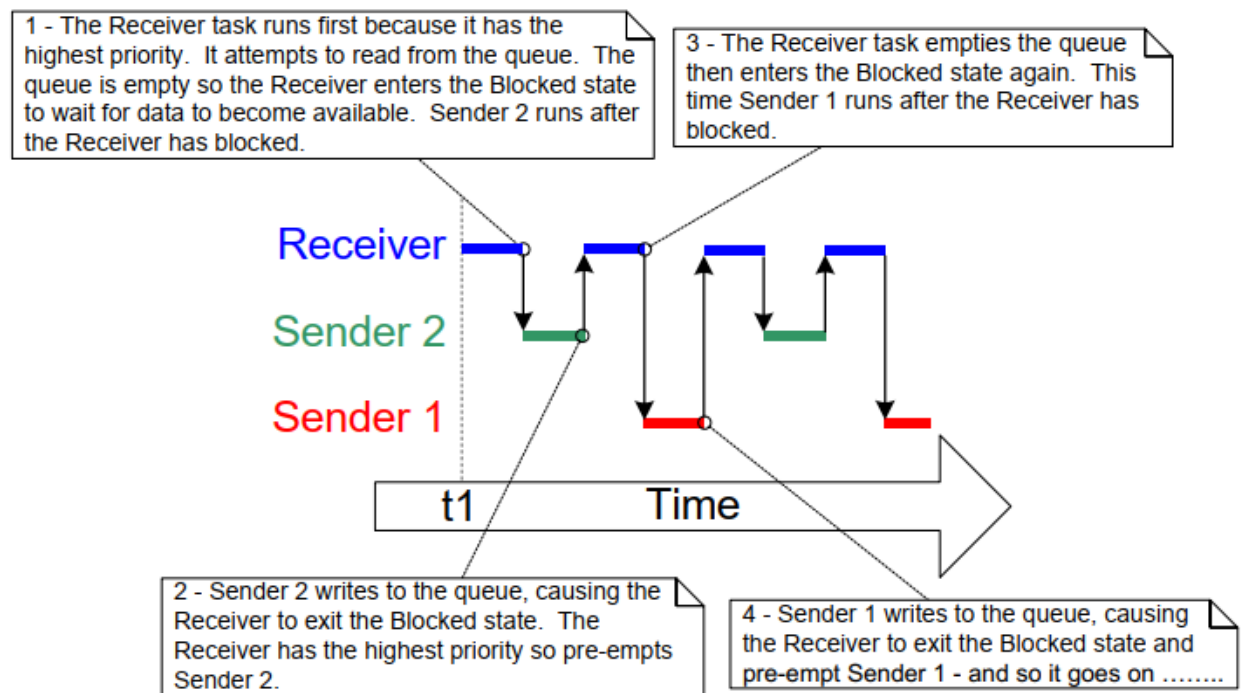


图 5.3 示例 5.1 的执行序列

5.4 从多个数据源接收数据

在 FreeRTOS 设计中，一个任务从多个数据源接收数据很常见。接收任务需要知道数据来自何处，才能决定如何处理。图 5.4 展示了一种简单的设计模式：使用单个队列传输同时包含数据值和数据来源的结构体。

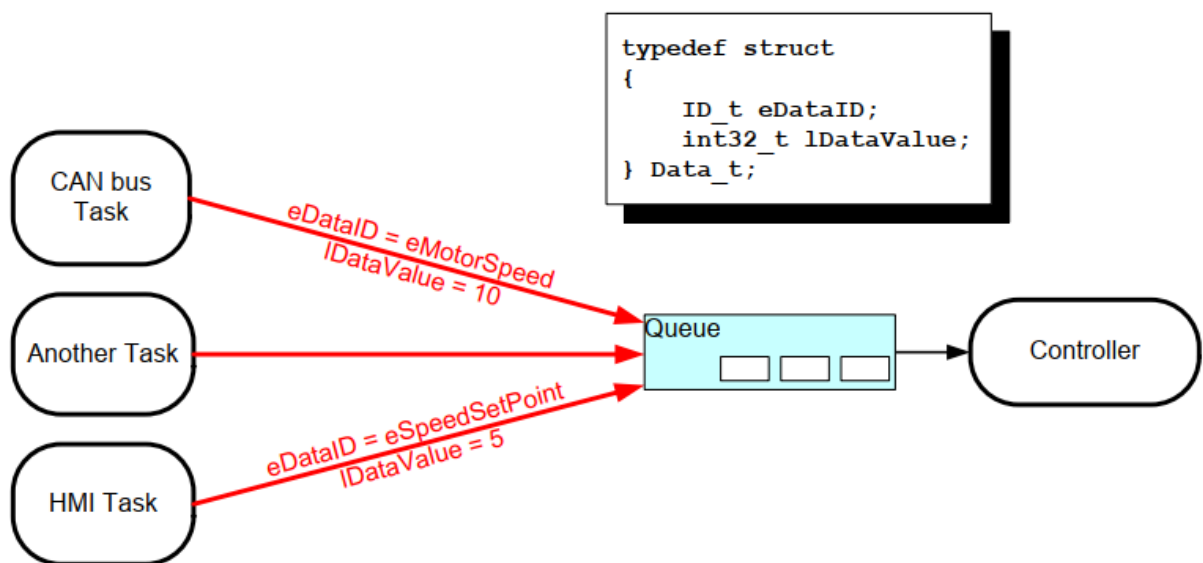


图 5.4 通过队列发送结构体的示例场景

参照图 5.4:

- 创建的队列保存 `Data_t` 类型的结构体。借助该结构体，一条队列消息可以同时携带数据值和说明数据含义的枚举值。
- 中央控制器任务执行系统的主要功能，并对通过队列传来的输入和系统状态变化作出响应。
- CAN 总线任务封装 CAN 总线接口功能。收到并解码消息后，它以 `Data_t` 结构体的形式把解码结果发送给控制器任务。结构体的 `eDataID` 成员说明数据的含义；在图示场景中，它表示电机转速。`lDataValue` 成员则携带实际的电机转速值。

- 人机界面（HMI）任务封装所有 HMI 功能。操作员可能通过多种方式输入命令和查询数值，HMI 任务需要检测并解释这些操作。输入新命令后，HMI 任务以 `Data_t` 结构体的形式把命令发送给控制器任务。结构体的 `eDataID` 成员说明数据的含义；在图示场景中，它表示新的设定值。`lDataValue` 成员则携带实际的设定值。

本书后续章节将介绍如何扩展这种设计模式，使控制器任务能够直接回复把结构体发送到队列的任务。

示例 5.2 向队列发送时阻塞，以及通过队列发送结构体

示例 5.2 与示例 5.1 类似，但任务优先级正好相反，因此接收任务的优先级低于发送任务。此外，本例创建的队列保存结构体，而不是整数。

代码清单 5.9 给出了示例 5.2 使用的结构体定义。

```
/* 定义一个枚举类型，用于标识数据来源。 */
typedef enum
{
    eSender1,
    eSender2
} DataSource_t;

/* 定义将通过队列传递的结构体类型。 */
typedef struct
{
    uint8_t ucValue;
    DataSource_t eDataSource;
} Data_t;

/* 声明两个将通过队列传递的 Data_t 类型变量。 */
static const Data_t xStructsToSend[ 2 ] =
{
    { 100, eSender1 }, /* 由发送任务 1 使用。 */
    { 200, eSender2 } /* 由发送任务 2 使用。 */
};
```

代码清单 5.9 通过队列传递的结构体定义，以及示例所用变量的声明

在示例 5.1 中，接收任务的优先级最高，因此队列中的数据项始终不会超过一个。这是因为一旦数据被放入队列，接收任务就会立即抢占发送任务。在示例 5.2 中，发送任务的优先级更高，因此队列通常处于满状态。这是因为接收任务每从队列中移除一个数据项，就会立即被某个发送任务抢占，而该发送任务随即再次填满队列。随后，发送任务重新进入阻塞状态，等待队列再次出现可用空间。

代码清单 5.10 给出了发送任务的实现。发送任务指定了 100 毫秒的阻塞时间，因此每当队列已满时，它都会进入阻塞状态等待可用空间。当队列中出现可用空间，或者等待 100 毫秒后仍无可用空间时，任务将退出阻塞状态。在本例中，接收任务会不断从队列中腾出空间，因此 100 毫秒的超时永远不会发生。

```
static void vSenderTask( void *pvParameters )
{
    BaseType_t xStatus;
    const TickType_t xTicksToWait = pdMS_TO_TICKS( 100 );

    /* 与大多数任务一样，本任务在无限循环中实现。 */
    for( ;; )
    {
        /* 向队列发送数据。

           第二个参数是待发送结构体的地址。该地址通过任务参数传入，
           因此这里直接使用 pvParameters。

           第三个参数是阻塞时间，即当队列已满时，任务为等待队列出现
           可用空间而保持阻塞状态的时间。这里必须指定阻塞时间，因为
           发送任务的优先级高于接收任务，队列预计会被填满。两个发送
           任务都处于阻塞状态时，接收任务才会从队列中移除数据项。 */
        xStatus = xQueueSendToBack( xQueue, pvParameters, xTicksToWait );

        if( xStatus != pdPASS )
        {
            /* 即使等待了 100 毫秒，发送操作仍未完成。这必定是一个错误，
               因为两个发送任务一旦都进入阻塞状态，接收任务就应当在
               队列中腾出空间。 */
            vPrintString( "Could not send to the queue.\r\n" );
        }
    }
}
```

代码清单 5.10 示例 5.2 中发送任务的实现

接收任务的优先级最低，因此只有两个发送任务都处于阻塞状态时它才会运行。发送任务仅在队列已满时进入阻塞状态，所以接收任务每次执行时，队列都应当是满的。因此，即使不指定阻塞时间，它也始终能够接收到数据。

代码清单 5.11 给出了接收任务的实现。

```

static void vReceiverTask( void *pvParameters )
{
    /* 声明用于保存从队列接收到的值的结构体。 */
    Data_t xReceivedStructure;
    BaseType_t xStatus;

    /* 本任务同样在无限循环中实现。 */
    for( ;; )
    {
        /* 由于本任务的优先级最低，它只会在发送任务处于阻塞状态时运行。
           发送任务仅在队列已满时进入阻塞状态，因此本任务始终预期队列中
           的数据项数量等于队列长度，本例中该长度为 3。 */
        if( uxQueueMessagesWaiting( xQueue ) != 3 )
        {
            vPrintString( "Queue should have been full!\r\n" );
        }

        /* 从队列接收数据。

           第二个参数是用于存放接收数据的缓冲区。本例中的缓冲区就是一个
           变量的地址，该变量的大小足以容纳接收到的结构体。

           最后一个参数是阻塞时间，即队列为空时，任务为等待数据可用而
           保持阻塞状态的最长时间。本例无须指定阻塞时间，因为本任务只会
           在队列已满时运行。 */
        xStatus = xQueueReceive( xQueue, &xReceivedStructure, 0 );

        if( xStatus == pdPASS )
        {
            /* 已成功从队列接收数据，打印接收到的值及其来源。 */
            if( xReceivedStructure.eDataSource == eSender1 )
            {
                vPrintStringAndNumber( "From Sender 1 = ",
                                       xReceivedStructure.ucValue );
            }
            else
            {
                vPrintStringAndNumber( "From Sender 2 = ",
                                       xReceivedStructure.ucValue );
            }
        }
        else
        {
            /* 未从队列接收到任何数据。这必定是一个错误，因为本任务只应
               在队列已满时运行。 */
            vPrintString( "Could not receive from the queue.\r\n" );
        }
    }
}

```

代码清单 5.11 示例 5.2 中接收任务的定义

与前一个示例相比，`main()` 只做了少量修改。队列被创建为最多容纳三个 `Data_t` 结构体，并且发送任务与接收任务的优先级关系发生了反转。代码清单 5.12 给出了 `main()` 的实现。

```

int main( void )
{
    /* 创建一个最多可容纳 3 个 Data_t 类型结构体的队列。 */
    xQueue = xQueueCreate( 3, sizeof( Data_t ) );

    if( xQueue != NULL )
    {
        /* 创建两个向队列写入数据的任务实例。任务参数用于传递该任务将
           写入队列的结构体，因此一个任务会不断向队列发送
           xStructsToSend[ 0 ]，另一个任务则不断发送 xStructsToSend[ 1 ]。
           两个任务的优先级均为 2，高于接收任务的优先级。 */
        xTaskCreate( vSenderTask, "Sender1", 1000, &(amp; xStructsToSend[ 0 ] ),
                    2, NULL );
        xTaskCreate( vSenderTask, "Sender2", 1000, &(amp; xStructsToSend[ 1 ] ),
                    2, NULL );

        /* 创建从队列读取数据的任务。该任务的优先级为 1，低于发送任务。 */
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 1, NULL );

        /* 启动调度器，使已创建的任务开始执行。 */
        vTaskStartScheduler();
    }
    else
    {
        /* 队列创建失败。 */
    }

    /* 如果一切正常，调度器此时已经开始运行任务，main() 永远不会执行到
       这里。如果 main() 确实执行到这里，很可能是因为没有足够的堆内存
       来创建空闲任务。第 3 章提供了有关堆内存管理的更多信息。 */
    for( ;; );
}

```

代码清单 5.12 示例 5.2 中 `main()` 的实现

图 5.5 展示了示例 5.2 产生的输出。

图 5.6 图例

- t1

发送任务 1 执行，并向队列发送三个数据项。

- t2

队列已满，因此发送任务 1 进入阻塞状态，等待下一次发送操作完成。此时，发送任务 2 是能够运行的最高优先级任务，因此进入运行状态。

- t3

发送任务 2 发现队列已满，因此进入阻塞状态，等待第一次发送操作完成。此时，接收任务是能够运行的最高优先级任务，因此进入运行状态。

- t4

两个优先级高于接收任务的发送任务都在等待队列出现可用空间，因此接收任务从队列中移除一个数据项后会立即被抢占。两个发送任务的优先级相同，所以调度器选择等待时间更长的任务进入运行状态，本例中即发送任务 1。

- t5

发送任务 1 向队列发送另一个数据项。队列中原本只有一个空位，因此发送任务 1 随即再次进入阻塞状态，等待下一次发送操作完成。接收任务再次成为能够运行的最高优先级任务，因此进入运行状态。

此时，发送任务 1 已向队列发送四个数据项，而发送任务 2 仍在等待发送它的第一个数据项。

- t6

两个优先级高于接收任务的发送任务都在等待队列出现可用空间，因此接收任务从队列中移除一个数据项后会立即被抢占。这一次，发送任务 2 的等待时间比发送任务 1 更长，所以发送任务 2 进入运行状态。

- t7

发送任务 2 向队列发送一个数据项。队列中原本只有一个空位，因此发送任务 2 随即进入阻塞状态，等待下一次发送操作完成。此时两个发送任务都在等待队列出现可用空间，所以接收任务成为唯一能够进入运行状态的任务。

5.5 处理大型或长度可变的数据

5.5.1 将指针入队

如果要存入队列的数据量很大，最好通过队列传递指向数据的指针，而不是逐字节地将数据本身复制进队列和复制出队列。无论从处理时间还是创建队列所需的 RAM 容量来看，传递指针都更加高效。不过，将指针入队时必须格外谨慎，确保满足以下条件：

- 明确定义所指向 RAM 的所有权。

通过指针在任务之间共享内存时，必须确保两个任务不会同时修改内存内容，也不会执行任何其他可能使内存内容失效或不一致的操作。理想情况下，指针发送到队列之前，只有发送任务能够访问该内存；从队列接收到指针之后，则只有接收任务能够访问该内存。

- 确保所指向的 RAM 始终有效。

如果所指向的内存是动态分配的，或取自预先分配的缓冲区池，则应当明确由且仅由一个任务负责释放该内存。内存释放后，任何任务都不得再尝试访问它。

绝不能通过队列传递指向任务栈上数据的指针，因为栈帧发生变化后，这些数据将不再有效。

代码清单 5.13、5.14 和 5.15 演示了如何使用队列将指向缓冲区的指针从一个任务发送到另一个任务：

- 代码清单 5.13 创建一个最多可容纳五个指针的队列。
- 代码清单 5.14 获取一个缓冲区，向其中写入字符串，然后将指向该缓冲区的指针发送到队列。
- 代码清单 5.15 从队列接收一个指向缓冲区的指针，然后打印缓冲区中的字符串。

```
/* 声明一个 QueueHandle_t 类型的变量，用于保存所创建队列的句柄。 */  
QueueHandle_t xPointerQueue;  
  
/* 创建一个最多可保存 5 个指针的队列，本例中保存字符指针。 */  
xPointerQueue = xQueueCreate( 5, sizeof( char * ) );
```

代码清单 5.13 创建保存指针的队列

```

/* 此任务获取一个缓冲区，向其中写入字符串，然后将缓冲区地址发送到
   代码清单 5.13 创建的队列。 */
void vStringSendingTask( void *pvParameters )
{
    char *pcStringToSend;
    const size_t xMaxStringLength = 50;
    BaseType_t xStringNumber = 0;

    for( ;; )
    {
        /* 获取一个至少可容纳 xMaxStringLength 个字符的缓冲区。这里未给出
           prvGetBuffer() 的实现；它可能从预先分配的缓冲区池中获取缓冲区，
           也可能直接动态分配缓冲区。 */
        pcStringToSend = ( char * ) prvGetBuffer( xMaxStringLength );

        /* 向缓冲区写入字符串。 */
        snprintf( pcStringToSend, xMaxStringLength, "String number %d\r\n",
                  xStringNumber );

        /* 递增计数器，使任务每轮迭代写入的字符串都不同。 */
        xStringNumber++;

        /* 将缓冲区地址发送到代码清单 5.13 创建的队列。缓冲区地址保存在
           pcStringToSend 变量中。 */
        xQueueSend( xPointerQueue, /* 队列句柄。 */
                   &pcStringToSend, /* 指向缓冲区的指针本身的地址。 */
                   portMAX_DELAY );
    }
}

```

代码清单 5.14 使用队列发送指向缓冲区的指针


```

/* 此任务从代码清单 5.13 创建、代码清单 5.14 写入的队列中接收缓冲区
地址。缓冲区中保存着一个字符串，任务会将其打印出来。 */

void vStringReceivingTask( void *pvParameters )
{
    char *pcReceivedString;

    for( ;; )
    {
        /* 接收缓冲区地址。 */
        xQueueReceive( xPointerQueue, /* 队列句柄。 */
                      &pcReceivedString, /* 将缓冲区地址保存到
                                             pcReceivedString 中。 */
                      portMAX_DELAY );

        /* 缓冲区中保存着字符串，将其打印出来。 */
        vPrintString( pcReceivedString );

        /* 不再需要该缓冲区，将其释放，以便回收或再次使用。 */
        prvReleaseBuffer( pcReceivedString );
    }
}

```

代码清单 5.15 使用队列接收指向缓冲区的指针

5.5.2 使用队列发送不同类型和长度的数据^[2]

本书前面演示了两种功能强大的设计模式：向队列发送结构体，以及向队列发送指针。将这两种技术结合起来，任务便可通过单个队列接收来自任意数据源、任意类型的数据。FreeRTOS+TCP 的 TCP/IP 协议栈实现提供了一个实际示例。

TCP/IP 协议栈在独立任务中运行，必须处理来自许多不同数据源的事件。不同事件类型关联的数据类型和长度也各不相同。`IPStackEvent_t` 结构体用于描述发生在 TCP/IP 任务之外的所有事件，并通过队列发送到 TCP/IP 任务。代码清单 5.16 给出了 `IPStackEvent_t` 结构体。它的 `pvData` 成员是一个指针，既可用于直接保存一个值，也可指向一个缓冲区。

```

/* TCP/IP 协议栈用于标识事件的部分枚举值。 */
typedef enum
{
    eNetworkDownEvent = 0, /* 网络接口已断开, 或需要 (重新) 连接。 */
    eNetworkRxEvent,      /* 已从网络接收到数据包。 */
    eTCPAcceptEvent,      /* 调用 FreeRTOS_accept() 以接受或等待新客户端。 */

    /* 此处还有其他事件类型, 但未在本代码清单中列出。 */

} eIPEvent_t;

/* 描述事件并通过队列发送到 TCP/IP 任务的结构体。 */
typedef struct IP_TASK_COMMANDS
{
    /* 标识事件的枚举类型, 参见上面的 eIPEvent_t 定义。 */
    eIPEvent_t eEventType;

    /* 可直接保存一个值或指向缓冲区的通用指针。 */
    void *pvData;

} IPStackEvent_t;

```

代码清单 5.16 FreeRTOS+TCP 中用于向 TCP/IP 协议栈任务发送事件的结构体

TCP/IP 事件及其关联数据的示例包括:

- **eNetworkRxEvent**: 从网络接收到一个数据包。

网络接口使用 **IPStackEvent_t** 类型的结构体向 TCP/IP 任务发送数据接收事件。结构体的 **eEventType** 成员设为 **eNetworkRxEvent**, **pvData** 成员则指向包含接收数据的缓冲区。代码清单 5.17 给出了伪代码示例。

```

void vSendRxDataToTheTCPTask( NetworkBufferDescriptor_t *pRxedData )
{
    IPStackEvent_t xEventStruct;

    /* 填充 IPStackEvent_t 结构体。接收到的数据保存在 pRxedData 中。 */
    xEventStruct.eEventType = eNetworkRxEvent;
    xEventStruct.pvData = ( void * ) pRxedData;

    /* 将 IPStackEvent_t 结构体发送到 TCP/IP 任务。 */
    xSendEventStructToIPTask( &xEventStruct );
}

```

代码清单 5.17 使用 **IPStackEvent_t** 结构体把网络接收数据发送到 TCP/IP 任务的伪代码

- **eTCPAcceptEvent**: 套接字准备接受客户端连接, 或等待客户端发起连接。

调用 `FreeRTOS_accept()` 的任务使用 `IPStackEvent_t` 类型的结构体向 TCP/IP 任务发送连接接受事件。结构体的 `eEventType` 成员设为 `eTCPAcceptEvent`，`pvData` 成员则设为正在接受连接的套接字句柄。代码清单 5.18 给出了伪代码示例。

```
void vSendAcceptRequestToTheTCPTask( Socket_t xSocket )
{
    IPStackEvent_t xEventStruct;

    /* 填充 IPStackEvent_t 结构体。 */
    xEventStruct.eEventType = eTCPAcceptEvent;
    xEventStruct.pvData = ( void * ) xSocket;

    /* 将 IPStackEvent_t 结构体发送到 TCP/IP 任务。 */
    xSendEventStructToIPTask( &xEventStruct );
}
```

代码清单 5.18 使用 `IPStackEvent_t` 结构体把正在接受连接的套接字句柄发送到 TCP/IP 任务的伪代码

- `eNetworkDownEvent`：网络需要连接或重新连接。

网络接口使用 `IPStackEvent_t` 类型的结构体向 TCP/IP 任务发送网络断开事件。结构体的 `eEventType` 成员设为 `eNetworkDownEvent`。网络断开事件不关联任何数据，因此不使用结构体的 `pvData` 成员。代码清单 5.19 给出了伪代码示例。

```
void vSendNetworkDownEventToTheTCPTask( Socket_t xSocket )
{
    IPStackEvent_t xEventStruct;

    /* 填充 IPStackEvent_t 结构体。 */
    xEventStruct.eEventType = eNetworkDownEvent;

    xEventStruct.pvData = NULL; /* 此处不使用，但为完整起见仍设为 NULL。 */

    /* 将 IPStackEvent_t 结构体发送到 TCP/IP 任务。 */
    xSendEventStructToIPTask( &xEventStruct );
}
```

代码清单 5.19 使用 `IPStackEvent_t` 结构体向 TCP/IP 任务发送网络断开事件的伪代码

代码清单 5.20 展示了 TCP/IP 任务内部接收和处理这些事件的代码。可以看出，从队列接收到 `IPStackEvent_t` 结构体后，程序根据其 `eEventType` 成员确定应当如何解释 `pvData` 成员。

```

IPStackEvent_t xReceivedEvent;

/* 在网络事件队列上阻塞，直到收到事件，或等待 xNextIPSleep 个 Tick 后
   仍未收到事件。先将 eEventType 设为 eNoEvent，以便处理 xQueueReceive()
   因超时而返回、而不是因为收到事件而返回的情况。 */
xReceivedEvent.eEventType = eNoEvent;
xQueueReceive( xNetworkEventQueue, &xReceivedEvent, xNextIPSleep );

/* 收到了哪一种事件（如果有）？ */
switch( xReceivedEvent.eEventType )
{
    case eNetworkDownEvent :
        /* 尝试（重新）建立连接。此事件不关联任何数据。 */
        prvProcessNetworkDownEvent();
        break;

    case eNetworkRxEvent:
        /* 网络接口收到了新数据包。接收到的 IPStackEvent_t 结构体的
           pvData 成员中保存着指向接收数据的指针。处理接收数据。 */
        prvHandleEthernetPacket( ( NetworkBufferDescriptor_t * )
                                ( xReceivedEvent.pvData ) );

        break;

    case eTCPAcceptEvent:
        /* FreeRTOS_accept() API 函数已被调用。接收到的 IPStackEvent_t
           结构体的 pvData 成员中保存着正在接受连接的套接字句柄。 */
        xSocket = ( FreeRTOS_Socket_t * ) ( xReceivedEvent.pvData );
        xTCPCheckNewClient( xSocket );
        break;

    /* 其他事件类型以相同方式处理，此处不再列出。 */
}

```

代码清单 5.20 接收和处理 `IPStackEvent_t` 结构体的伪代码

5.6 从多个队列接收数据

5.6.1 队列集

应用程序设计经常要求单个任务接收长度不同、含义不同以及来源不同的数据。上一节演示了如何使用一个接收结构体的队列，以简洁高效的方式满足这一要求。然而，应用程序设计者有时会受到某些约束，无法自由选择设计方案，因而必须为特定数据源使用独立队列。例如，要集成到系统中的第三方代码可能假定存在专用队列。在这种情况下，可以使用“队列集”。

队列集允许任务从多个队列接收数据，而无须逐个轮询队列来判断其中哪个队列包含数据。

与使用单个队列接收结构体的方案相比，使用队列集从多个数据源接收数据不够简洁，效率也更低。因此，建议仅在设计约束确实要求使用队列集时采用这种方案。

以下各节将介绍如何使用队列集：

- 创建队列集。
- 向队列集中添加队列。

信号量也可以添加到队列集中，本书后续章节将介绍信号量。

- 从队列集读取，以确定其中哪个队列包含数据。

当队列集中的某个成员队列接收到数据时，该队列的句柄会被发送到队列集，并在任务调用队列集读取函数时返回。因此，只要队列集返回了某个队列句柄，就可以确定该句柄所引用的队列中包含数据，任务随后便可直接读取该队列。

注意：如果一个队列是队列集的成员，那么每次从队列集接收到该队列的句柄后，都必须从该队列读取一个数据项；在从队列集接收到其句柄之前，则不得读取该队列。

要启用队列集功能，需在 `FreeRTOSConfig.h` 中将编译时配置常量 `configUSE_QUEUE_SETS` 设为 1。

5.6.2 xQueueCreateSet() API 函数

队列集必须在使用前显式创建。在本书编写时，FreeRTOS 尚未实现 `xQueueCreateSetStatic()`。不过，队列集本身也是队列，因此可以通过谨慎构造对 `xQueueCreateStatic()` 的调用，使用预先分配的内存创建队列集。

队列集由句柄引用，该句柄保存在 `QueueSetHandle_t` 类型的变量中。`xQueueCreateSet()` API 函数创建一个队列集，并返回引用所创建队列集的 `QueueSetHandle_t`。

```
QueueSetHandle_t xQueueCreateSet( const UBaseType_t uxEventQueueLength);
```

代码清单 5.21 `xQueueCreateSet()` API 函数原型

`xQueueCreateSet()` 参数和返回值

- `uxEventQueueLength`

当队列集中的某个成员队列接收到数据时，该队列的句柄会被发送到队列集。

`uxEventQueueLength` 指定所创建的队列集在任意时刻最多能够容纳多少个队列句柄。

只有当队列集中的队列接收到数据时，该队列的句柄才会被发送到队列集。已满的队列无法再接收数据，因此当队列集中的所有队列都已满时，也不会有新的队列句柄发送到队列集。由此可知，队列集在任意时刻可能需要容纳的最大数据项数，等于其中所有成员队列的长度之和。

例如，某个队列集中有三个空队列，每个队列的长度均为 5。这些成员队列总共可以接收十五个数据项（三个队列乘以每个队列五个数据项），之后才会全部填满。在这个例子中，必须将

`uxEventQueueLength` 设为 15，才能保证队列集能够容纳发送给它的每一个句柄。

信号量也可以添加到队列集中，本书后续章节将介绍信号量。计算所需的 `uxEventQueueLength` 时，二值信号量和互斥量的长度均视为 1，计数信号量的长度则等于其最大计数值。

再例如，如果一个队列集包含一个长度为 3 的队列和一个二值信号量（长度视为 1），则必须将 `uxEventQueueLength` 设为 4（3 加 1）。

- 返回值

如果返回 `NULL`，则表示队列集创建失败，因为没有足够的堆内存供 FreeRTOS 分配队列集数据结构和存储区。第 3 章提供了有关 FreeRTOS 堆的更多信息。

如果返回值不是 `NULL`，则表示队列集创建成功，返回值就是所创建队列集的句柄。

5.6.3 xQueueAddToSet() API 函数

`xQueueAddToSet()` 将队列或信号量添加到队列集。本书后续章节将介绍信号量。

```
BaseType_t xQueueAddToSet( QueueSetMemberHandle_t xQueueOrSemaphore,  
                           QueueSetHandle_t xQueueSet );
```

代码清单 5.22 `xQueueAddToSet()` API 函数原型

`xQueueAddToSet()` 参数和返回值

- `xQueueOrSemaphore`

要添加到队列集的队列或信号量句柄。

队列句柄和信号量句柄都可以强制转换为 `QueueSetMemberHandle_t` 类型。

- `xQueueSet`

目标队列集的句柄。

- 返回值

有两个可能的返回值：

1. `pdPASS`

表示队列或信号量已成功添加到队列集。

2. `pdFAIL`

表示无法将队列或信号量添加到队列集。

队列和二值信号量只能在为空时添加到队列集；计数信号量只能在计数值为零时添加。一个队列或信号量同一时刻只能属于一个队列集。

5.6.4 xQueueSelectFromSet() API 函数

`xQueueSelectFromSet()` 从队列集中读取队列或信号量句柄。

当队列集中的某个成员队列或信号量接收到数据时，该成员的句柄会被发送到队列集，并在任务调用 `xQueueSelectFromSet()` 时返回。只要 `xQueueSelectFromSet()` 返回了一个句柄，就可以确定该句柄引用的队列或信号量中有数据可用，调用任务必须随即直接读取该队列或信号量。

注意：除非 `xQueueSelectFromSet()` 已经返回了某个成员队列或信号量的句柄，否则不得读取该成员。每当 `xQueueSelectFromSet()` 返回一个成员句柄时，只能从对应的队列或信号量读取一个数据项。

```
QueueSetMemberHandle_t xQueueSelectFromSet( QueueSetHandle_t xQueueSet,  
                                             const TickType_t xTicksToWait );
```

代码清单 5.23 `xQueueSelectFromSet()` API 函数原型

`xQueueSelectFromSet()` 参数和返回值

- `xQueueSet`

要从中接收（读取）队列或信号量句柄的队列集句柄。该句柄由创建队列集时调用的 `xQueueCreateSet()` 返回。

- `xTicksToWait`

如果队列集中的所有队列和信号量都为空，该参数指定调用任务为等待队列集返回成员句柄而保持阻塞状态的最长时间。

如果 `xTicksToWait` 为零，那么当队列集中的所有队列和信号量都为空时，

`xQueueSelectFromSet()` 将立即返回。

阻塞时间以 Tick 周期为单位，因此其对应的绝对时间取决于 Tick 频率。宏 `pdMS_TO_TICKS()` 可将以毫秒为单位的时间转换为 Tick 数。

如果在 `FreeRTOSConfig.h` 中将 `INCLUDE_vTaskSuspend` 设为 1，那么把 `xTicksToWait` 设为 `portMAX_DELAY` 可使任务无限期等待（永不超时）。

- 返回值

非 `NULL` 返回值是某个已知有数据可用的队列或信号量的句柄。如果指定了阻塞时间（`xTicksToWait` 不为零），调用任务可能先进入阻塞状态，等待队列集中的队列或信号量有数据可用，并在阻塞时间到期前成功从队列集读取到句柄。返回的句柄类型为

`QueueSetMemberHandle_t`，可以强制转换为 `QueueHandle_t` 或 `SemaphoreHandle_t`。

如果返回值为 `NULL`，则表示未能从队列集读取到句柄。如果指定了阻塞时间（`xTicksToWait` 不为零），调用任务会进入阻塞状态，等待另一个任务或中断向队列集中的队列或信号量发送数据，但在数据到来之前阻塞时间已经到期。

示例 5.3 使用队列集

本例创建两个发送任务和一个接收任务。两个发送任务分别通过各自的队列向接收任务发送数据。这两个队列都被加入同一个队列集，接收任务读取队列集，以确定哪一个队列中包含数据。

任务、队列和队列集都在 `main()` 中创建，具体实现见代码清单 5.24。


```

/* 声明两个 QueueHandle_t 类型的变量。两个队列将加入同一个队列集。 */
static QueueHandle_t xQueue1 = NULL, xQueue2 = NULL;

/* 声明一个 QueueSetHandle_t 类型的变量，用于保存两个队列所加入的
   队列集。 */
static QueueSetHandle_t xQueueSet = NULL;

int main( void )
{
    /* 创建两个队列，它们都用于发送字符指针。接收任务的优先级高于发送
       任务，因此任一时刻每个队列中的数据项都不会超过一个。 */
    xQueue1 = xQueueCreate( 1, sizeof( char * ) );
    xQueue2 = xQueueCreate( 1, sizeof( char * ) );

    /* 创建队列集。队列集中将加入两个队列，每个队列可容纳一个数据项，
       因此队列集在任一时刻最多需要保存两个队列句柄（两个队列乘以
       每个队列一个数据项）。 */
    xQueueSet = xQueueCreateSet( 1 * 2 );

    /* 将两个队列加入队列集。 */
    xQueueAddToSet( xQueue1, xQueueSet );
    xQueueAddToSet( xQueue2, xQueueSet );

    /* 创建向队列发送数据的任务。 */
    xTaskCreate( vSenderTask1, "Sender1", 1000, NULL, 1, NULL );
    xTaskCreate( vSenderTask2, "Sender2", 1000, NULL, 1, NULL );

    /* 创建读取队列集的任务，用于确定两个队列中的哪一个包含数据。 */
    xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 2, NULL );

    /* 启动调度器，使已创建的任务开始执行。 */
    vTaskStartScheduler();

    /* 正常情况下 vTaskStartScheduler() 不会返回，因此下面的代码永远
       不会执行。 */
    for( ;; );
    return 0;
}

```

代码清单 5.24 示例 5.3 中 `main()` 的实现

第一个发送任务每 100 毫秒通过 `xQueue1` 向接收任务发送一个字符指针；第二个发送任务每 200 毫秒通过 `xQueue2` 发送一个字符指针。这些字符指针指向用于标识发送任务的字符串。代码清单 5.25 给出了两个发送任务的实现。

```

void vSenderTask1( void *pvParameters )
{
    const TickType_t xBlockTime = pdMS_TO_TICKS( 100 );
    const char * const pcMessage = "Message from vSenderTask1\r\n";

    /* 与大多数任务一样，本任务在无限循环中实现。 */

    for( ;; )
    {

        /* 阻塞 100 毫秒。 */
        vTaskDelay( xBlockTime );

        /* 将本任务的字符串发送到 xQueue1。尽管队列只能容纳一个数据项，
           这里也无须使用阻塞时间。这是因为读取队列的任务优先级高于
           本任务；本任务一向队列写入数据，就会被读取队列的任务抢占，
           因此等 xQueueSend() 返回时，队列已经再次变为空。阻塞时间设为 0。 */
        xQueueSend( xQueue1, &pcMessage, 0 );
    }
}

/*-----*/

void vSenderTask2( void *pvParameters )
{
    const TickType_t xBlockTime = pdMS_TO_TICKS( 200 );
    const char * const pcMessage = "Message from vSenderTask2\r\n";

    /* 与大多数任务一样，本任务在无限循环中实现。 */
    for( ;; )
    {
        /* 阻塞 200 毫秒。 */
        vTaskDelay( xBlockTime );

        /* 将本任务的字符串发送到 xQueue2。尽管队列只能容纳一个数据项，
           这里也无须使用阻塞时间。这是因为读取队列的任务优先级高于
           本任务；本任务一向队列写入数据，就会被读取队列的任务抢占，
           因此等 xQueueSend() 返回时，队列已经再次变为空。阻塞时间设为 0。 */
        xQueueSend( xQueue2, &pcMessage, 0 );
    }
}

```

代码清单 5.25 示例 5.3 使用的发送任务

两个发送任务写入的队列属于同一个队列集。每当任务向其中一个队列发送数据时，该队列的句柄都会被发送到队列集。接收任务调用 `xQueueSelectFromSet()` 从队列集读取队列句柄。收到句柄后，接收任务便知道该句柄所引用的队列中包含数据，因此直接读取对应队列。读取到的数据是指向字符串的指针，接收任务会将该字符串打印出来。

如果 `xQueueSelectFromSet()` 调用超时，它将返回 `NULL`。示例 5.3 使用无限阻塞时间调用 `xQueueSelectFromSet()`，因此该调用永远不会超时，只会返回有效的队列句柄。接收任务在使用返回值之前无须检查它是否为 `NULL`。

`xQueueSelectFromSet()` 只会在句柄所引用的队列中包含数据时返回该句柄，因此随后读取队列时无须使用阻塞时间。

代码清单 5.26 给出了接收任务的实现。

```
void vReceiverTask( void *pvParameters )
{
    QueueHandle_t xQueueThatContainsData;
    char *pcReceivedString;

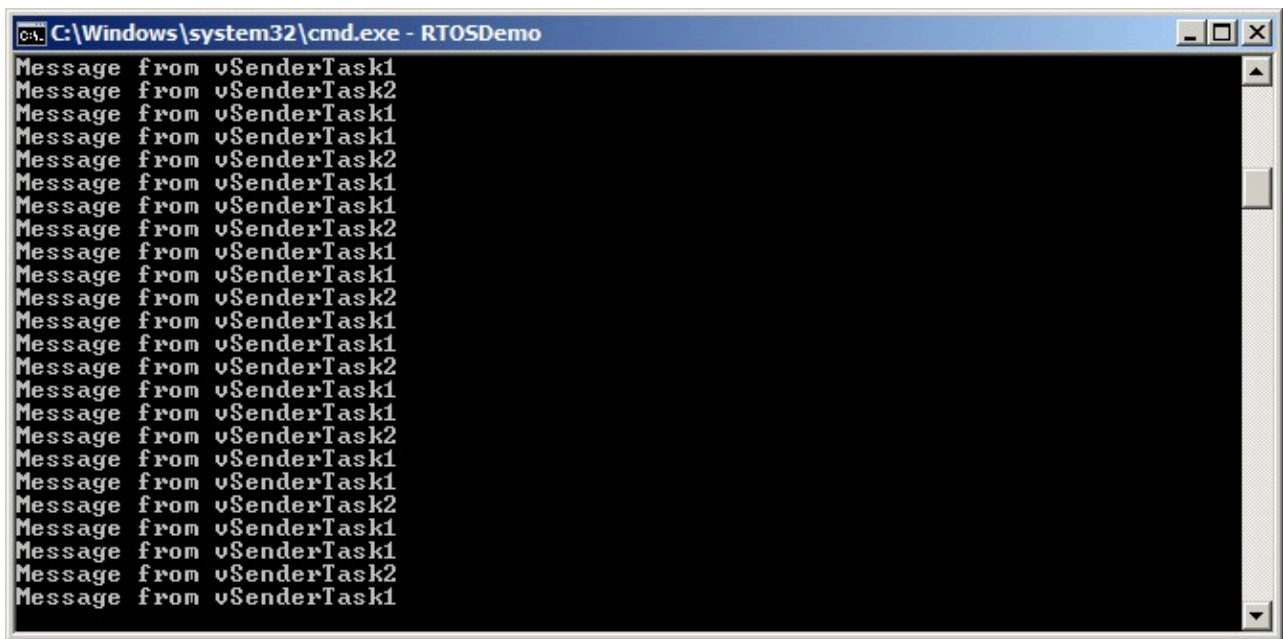
    /* 与大多数任务一样，本任务在无限循环中实现。 */
    for( ;; )
    {
        /* 在队列集上阻塞，等待其中某个队列包含数据。已知队列集的所有
           成员都是队列（其中不含信号量），因此将 xQueueSelectFromSet()
           返回的 QueueSetMemberHandle_t 值强制转换为 QueueHandle_t。 */
        xQueueThatContainsData = ( QueueHandle_t ) xQueueSelectFromSet(
                                                    xQueueSet, portMAX_DELAY );

        /* 读取队列集时使用了无限阻塞时间，因此只有当队列集中的某个队列
           包含数据时 xQueueSelectFromSet() 才会返回，xQueueThatContainsData
           不可能为 NULL。读取该队列。由于已知队列中包含数据，无须指定
           阻塞时间，因此将其设为 0。 */
        xQueueReceive( xQueueThatContainsData, &pcReceivedString, 0 );

        /* 打印从队列接收到的字符串。 */
        vPrintString( pcReceivedString );
    }
}
```

代码清单 5.26 示例 5.3 使用的接收任务

图 5.7 展示了示例 5.3 产生的输出。可以看出，接收任务从两个发送任务接收字符串。`vSenderTask1()` 使用的阻塞时间是 `vSenderTask2()` 的一半，因此前者发送的字符串打印频率是后者的两倍。



```
C:\Windows\system32\cmd.exe - RTOSDemo
Message from vSenderTask1
Message from vSenderTask2
Message from vSenderTask1
Message from vSenderTask1
Message from vSenderTask2
Message from vSenderTask1
Message from vSenderTask1
Message from vSenderTask2
Message from vSenderTask1
Message from vSenderTask1
Message from vSenderTask2
Message from vSenderTask1
Message from vSenderTask1
Message from vSenderTask2
Message from vSenderTask1
Message from vSenderTask1
Message from vSenderTask2
Message from vSenderTask1
Message from vSenderTask1
Message from vSenderTask2
Message from vSenderTask1
```

图 5.7 执行示例 5.3 时产生的输出

5.6.5 更贴近实际的队列集用例

示例 5.3 展示的是一个非常简单的用例：队列集中只包含队列，而且两个队列都用于发送字符指针。在实际应用中，队列集可能同时包含队列和信号量，各个队列保存的数据类型也可能不同。此时，必须先检查 `xQueueSelectFromSet()` 的返回值，再决定如何使用它。代码清单 5.27 演示了如何处理包含以下成员的队列集：

- 一个二值信号量。
- 一个从中读取字符指针的队列。
- 一个从中读取 `uint32_t` 值的队列。

代码清单 5.27 假定这些队列和信号量均已创建并添加到队列集中。

```

/* 用于接收字符指针的队列句柄。 */
QueueHandle_t xCharPointerQueue;

/* 用于接收 uint32_t 值的队列句柄。 */
QueueHandle_t xUint32tQueue;

/* 二值信号量的句柄。 */
SemaphoreHandle_t xBinarySemaphore;

/* 上述两个队列和二值信号量所属的队列集。 */
QueueSetHandle_t xQueueSet;

void vAMoreRealisticReceiverTask( void *pvParameters )
{
    QueueSetMemberHandle_t xHandle;
    char *pcReceivedString;
    uint32_t ulRecievedValue;
    const TickType_t xDelay100ms = pdMS_TO_TICKS( 100 );

    for( ;; )
    {
        /* 在队列集上最多阻塞 100 毫秒，等待其中某个成员包含数据。 */
        xHandle = xQueueSelectFromSet( xQueueSet, xDelay100ms );

        /* 检查 xQueueSelectFromSet() 的返回值。如果返回 NULL，说明调用已
           超时；否则返回值就是队列集中某个成员的句柄。
           QueueSetMemberHandle_t 值可以强制转换为 QueueHandle_t 或
           SemaphoreHandle_t。是否需要显式强制转换取决于编译器。 */

        if( xHandle == NULL )
        {
            /* xQueueSelectFromSet() 调用超时。 */
        }
        else if( xHandle == ( QueueSetMemberHandle_t ) xCharPointerQueue )
        {
            /* xQueueSelectFromSet() 返回了接收字符指针的队列句柄。读取该
               队列。已知队列中包含数据，因此阻塞时间设为 0。 */
            xQueueReceive( xCharPointerQueue, &pcReceivedString, 0 );

            /* 可在此处处理接收到的字符指针..... */
        }
        else if( xHandle == ( QueueSetMemberHandle_t ) xUint32tQueue )
        {
            /* xQueueSelectFromSet() 返回了接收 uint32_t 值的队列句柄。
               读取该队列。已知队列中包含数据，因此阻塞时间设为 0。 */
            xQueueReceive(xUint32tQueue, &ulRecievedValue, 0 );

            /* 可在此处处理接收到的值..... */
        }
        else if( xHandle == ( QueueSetMemberHandle_t ) xBinarySemaphore )
        {
            /* xQueueSelectFromSet() 返回了二值信号量的句柄。获取该信号量。
               已知信号量可用，因此阻塞时间设为 0。 */
            xSemaphoreTake( xBinarySemaphore, 0 );

            /* 可在此处执行获取信号量后所需的处理..... */
        }
    }
}

```

```

    }
}
}

```

代码清单 5.27 使用同时包含队列和信号量的队列集

5.7 使用队列创建邮箱

嵌入式系统领域尚未就相关术语达成共识，“邮箱”在不同 RTOS 中的含义也不尽相同。本书使用“邮箱”一词指代长度为 1 的队列。之所以把某个队列称为邮箱，是因为应用程序使用它的方式，而不是因为它在功能上与队列有所不同：

- 队列用于把数据从一个任务发送到另一个任务，或者从中断服务程序发送到任务。发送方把数据项放入队列，接收方再将该数据项从队列中移除。数据通过队列从发送方传递到接收方。
- 邮箱用于保存任何任务或中断服务程序都可以读取的数据。数据不会经由邮箱传递，而是一直保留在邮箱中，直到被覆盖。发送方覆盖邮箱中的值；接收方读取该值，但不会将其从邮箱中移除。

本章介绍两个可使队列充当邮箱的队列 API 函数。

代码清单 5.28 展示了如何创建一个用作邮箱的队列。

```

/* 邮箱可保存一个固定大小的数据项。创建邮箱（队列）时会设定该数据项的
   大小。本例创建的邮箱用于保存 Example_t 结构体。Example_t 中包含一个
   时间戳，使邮箱中的数据能够记录邮箱上次更新的时间。本例使用时间戳
   只是为了演示；邮箱可以保存应用程序开发者需要的任意数据，其中无须
   包含时间戳。 */
typedef struct xExampleStructure
{
    TickType_t xTimeStamp;
    uint32_t ulValue;
} Example_t;

/* 邮箱就是队列，因此其句柄保存在 QueueHandle_t 类型的变量中。 */
QueueHandle_t xMailbox;

void vAFunction( void )
{
    /* 创建将用作邮箱的队列。队列长度为 1，以便配合下面介绍的
       xQueueOverwrite() API 函数使用。 */
    xMailbox = xQueueCreate( 1, sizeof( Example_t ) );
}

```

代码清单 5.28 创建用作邮箱的队列

5.7.1 xQueueOverwrite() API 函数

与 `xQueueSendToBack()` 类似，`xQueueOverwrite()` API 函数也把数据发送到队列。不同之处在于，如果队列已满，`xQueueOverwrite()` 会覆盖队列中已有的数据。

`xQueueOverwrite()` 只能用于长度为 1 的队列。在覆盖模式下，函数始终写入队列头部并更新队列头指针，但不会增加待接收消息数。如果定义了 `configASSERT`，对长度大于 1 的队列调用该函数将触发断言。

注意：切勿从中断服务程序中调用 `xQueueOverwrite()`，应改用其中断安全版本 `xQueueOverwriteFromISR()`。

```
 BaseType_t xQueueOverwrite( QueueHandle_t xQueue, const void * pvItemToQueue );
```

代码清单 5.29 `xQueueOverwrite()` API 函数原型

`xQueueOverwrite()` 参数和返回值

- `xQueue`

数据将被发送（写入）到的队列句柄。该句柄由创建队列时调用的 `xQueueCreate()` 或 `xQueueCreateStatic()` 返回。

- `pvItemToQueue`

指向要复制到队列中的数据指针。

创建队列时已经设定每个数据项的大小，因此函数会从 `pvItemToQueue` 指向的位置向队列存储区复制相应数量的字节。

- 返回值

即使队列已满，`xQueueOverwrite()` 也会成功写入，因此唯一可能的返回值是 `pdPASS`。

代码清单 5.30 展示了如何使用 `xQueueOverwrite()` 写入代码清单 5.28 创建的邮箱（队列）。

```

void vUpdateMailbox( uint32_t ulNewValue )
{
    /* Example_t 已在代码清单 5.28 中定义。 */
    Example_t xData;

    /* 将新数据写入 Example_t 结构体。 */
    xData.ulValue = ulNewValue;

    /* 使用 RTOS Tick 计数作为 Example_t 结构体中保存的时间戳。 */
    xData.xTimeStamp = xTaskGetTickCount();

    /* 将结构体发送到邮箱，覆盖其中已有的数据。 */
    xQueueOverwrite( xMailbox, &xData );
}

```

代码清单 5.30 使用 `xQueueOverwrite()` API 函数

5.7.2 xQueuePeek() API 函数

`xQueuePeek()` 从队列接收（读取）一个数据项，但不会将该数据项从队列中移除。它读取队列头部的数据，却不会修改队列中保存的数据，也不会改变数据在队列中的存储顺序。

注意：切勿从中断服务程序中调用 `xQueuePeek()`，应改用其中断安全版本 `xQueuePeekFromISR()`。

`xQueuePeek()` 的函数参数和返回值与 `xQueueReceive()` 相同。

```

BaseType_t xQueuePeek( QueueHandle_t xQueue,
                      void * const pvBuffer,
                      TickType_t xTicksToWait );

```

代码清单 5.31 `xQueuePeek()` API 函数原型

代码清单 5.32 展示了如何使用 `xQueuePeek()` 接收代码清单 5.30 发布到邮箱（队列）的数据项。


```

 BaseType_t vReadMailbox( Example_t *pxData )
{
    TickType_t xPreviousTimeStamp;
    BaseType_t xDataUpdated;

    /* 本函数使用从邮箱读取到的最新值更新 Example_t 结构体。在 *pxData
       被新数据覆盖之前，先记录其中原有的时间戳。 */
    xPreviousTimeStamp = pxData->xTimeStamp;

    /* 使用邮箱中的数据更新 pxData 所指向的 Example_t 结构体。如果这里
       使用 xQueueReceive(), 邮箱会被读空，其他任务便无法再读取其中的
       数据。使用 xQueuePeek() 代替 xQueueReceive() 可确保数据仍留在邮箱中。

       这里指定了阻塞时间；如果邮箱为空，调用任务将进入阻塞状态等待
       邮箱中出现数据。由于使用无限阻塞时间，无须检查 xQueuePeek() 的
       返回值，因为只有在数据可用时 xQueuePeek() 才会返回。 */
    xQueuePeek( xMailbox, pxData, portMAX_DELAY );

    /* 如果邮箱中的值自上次调用本函数后已更新，则返回 pdTRUE；否则返回
       pdFALSE。 */
    if( pxData->xTimeStamp > xPreviousTimeStamp )
    {
        xDataUpdated = pdTRUE;
    }
    else
    {
        xDataUpdated = pdFALSE;
    }

    return xDataUpdated;
}

```

代码清单 5.32 使用 `xQueuePeek()` API 函数

1. FreeRTOS 消息缓冲区将在后续章节介绍。对于长度可变的消息，它是比队列更轻量的替代方案。↩
2. 与用队列传递长度可变的数据相比，FreeRTOS 消息缓冲区是一种更轻量的替代方案。↩

6 软件定时器管理

6.1 本章简介与范围

软件定时器用于安排函数在未来某个设定时刻执行，或以固定频率周期性执行。由软件定时器执行的函数称为软件定时器回调函数。

软件定时器由 FreeRTOS 内核实现和控制，不需要硬件支持，也与硬件定时器或硬件计数器无关。

FreeRTOS 采用经过精心设计的实现来确保高效率：除执行软件定时器回调函数之外，软件定时器不会占用任何处理时间。

软件定时器功能是可选的。要启用该功能，需要：

1. 将 FreeRTOS 源文件 `FreeRTOS/Source/timers.c` 加入项目并参与编译。
2. 在应用程序的 `FreeRTOSConfig.h` 头文件中定义以下配置常量：

- `configUSE_TIMERS`

在 `FreeRTOSConfig.h` 中将 `configUSE_TIMERS` 设为 1。

- `configTIMER_TASK_PRIORITY`

设置定时器服务任务的优先级，取值范围为 0 到 `configMAX_PRIORITIES - 1`。

- `configTIMER_QUEUE_LENGTH`

设置定时器命令队列在任意时刻最多能够容纳的待处理命令数。

- `configTIMER_TASK_STACK_DEPTH`

设置分配给定时器服务任务的栈大小，单位为字而不是字节。

6.1.1 范围

本章涵盖：

- 软件定时器特性与任务特性的比较。

- RTOS 守护任务。
- 定时器命令队列。
- 单次软件定时器与自动重载软件定时器的区别。
- 如何创建、启动、复位和更改软件定时器的周期。

6.2 软件定时器回调函数

软件定时器回调函数以 C 函数实现。它唯一的特殊之处在于函数原型：返回类型必须是 `void`，并且唯一参数必须是软件定时器句柄。代码清单 6.1 给出了回调函数原型。

```
void ATimerCallback( TimerHandle_t xTimer );
```

代码清单 6.1 软件定时器回调函数原型

软件定时器回调函数会从头到尾执行并正常返回。回调函数应当尽量简短，而且绝不能进入阻塞状态。

注意：后文将说明，软件定时器回调函数在 FreeRTOS 调度器启动时自动创建的任务上下文中执行。因此，回调函数绝不能调用会使调用任务进入阻塞状态的 FreeRTOS API 函数。可以调用 `xQueueReceive()` 之类的函数，但必须把指定阻塞时间的 `xTicksToWait` 参数设为 0。不能调用 `vTaskDelay()` 之类的函数，因为 `vTaskDelay()` 总会使调用任务进入阻塞状态。

6.3 软件定时器的属性和状态

6.3.1 软件定时器的周期

软件定时器的“周期”是从软件定时器启动到其回调函数执行之间的时间间隔。

6.3.2 单次定时器与自动重载定时器

有两种类型的软件定时器：

1. 单次定时器

启动后，单次定时器只执行一次回调函数。它可以手动重新启动，但不会自行重新启动。

2. 自动重载定时器

启动后，自动重载定时器每次到期都会自行重新启动，因此其回调函数会周期性执行。

图 6.1 展示了单次定时器与自动重载定时器的行为差异。竖直虚线表示 Tick 中断发生的时刻。

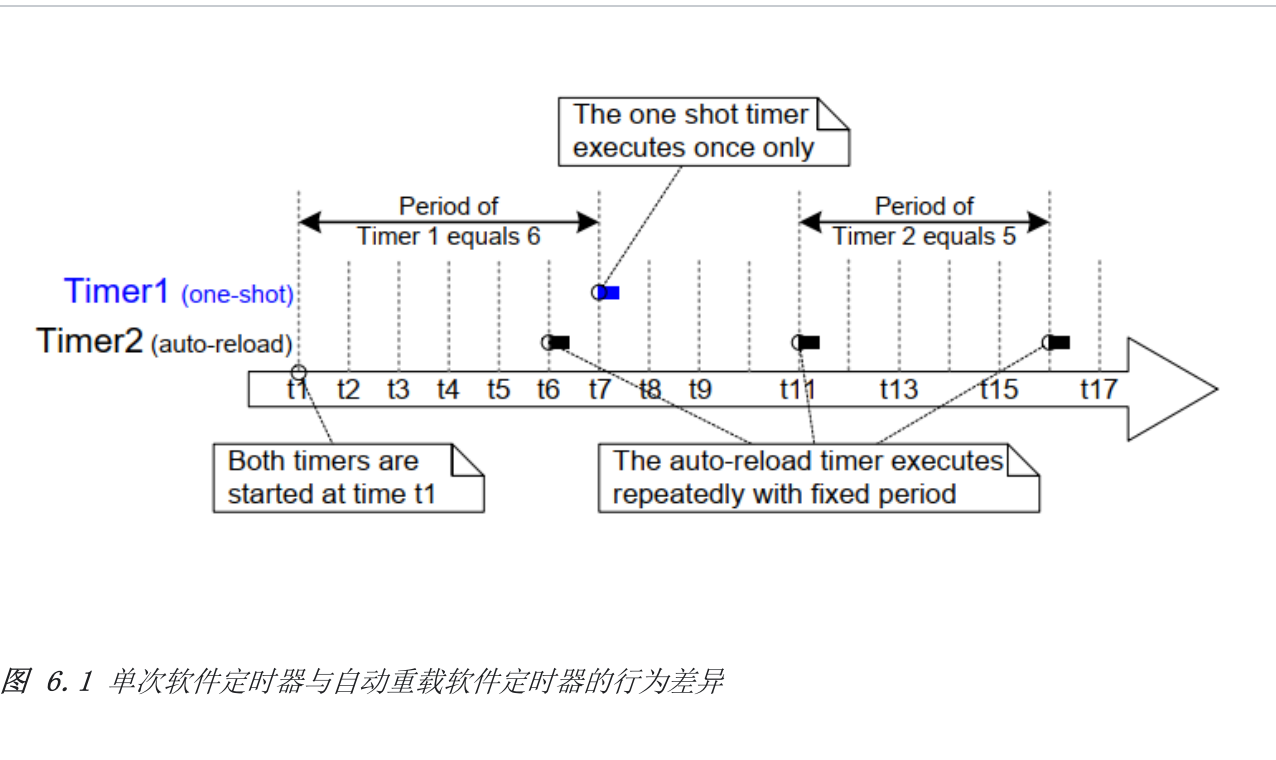


图 6.1 单次软件定时器与自动重载软件定时器的行为差异

参照图 6.1：

- 定时器 1

定时器 1 是周期为 6 个 Tick 的单次定时器。它在时刻 t1 启动，因此其回调函数会在 6 个 Tick 后，也就是时刻 t7 执行。由于它是单次定时器，回调函数不会再次执行。

- 定时器 2

定时器 2 是周期为 5 个 Tick 的自动重载定时器。它在时刻 t1 启动，因此回调函数此后每 5 个 Tick 执行一次。在图 6.1 中，对应的执行时刻为 t6、t11 和 t16。

6.3.3 软件定时器状态

软件定时器可以处于以下两种状态之一：

- 休眠状态

处于休眠状态的软件定时器仍然存在，也可以通过句柄引用，但它并未运行，因此不会执行回调函数。

- 运行状态

软件定时器进入运行状态或上次复位后，经过一个完整周期便会执行回调函数。

图 6.2 和图 6.3 分别展示了自动重载定时器和单次定时器在休眠状态与运行状态之间可能发生的转换。两图的关键区别在于定时器到期后进入的状态：自动重载定时器执行回调函数后会重新进入运行状态，而单次定时器执行回调函数后则进入休眠状态。

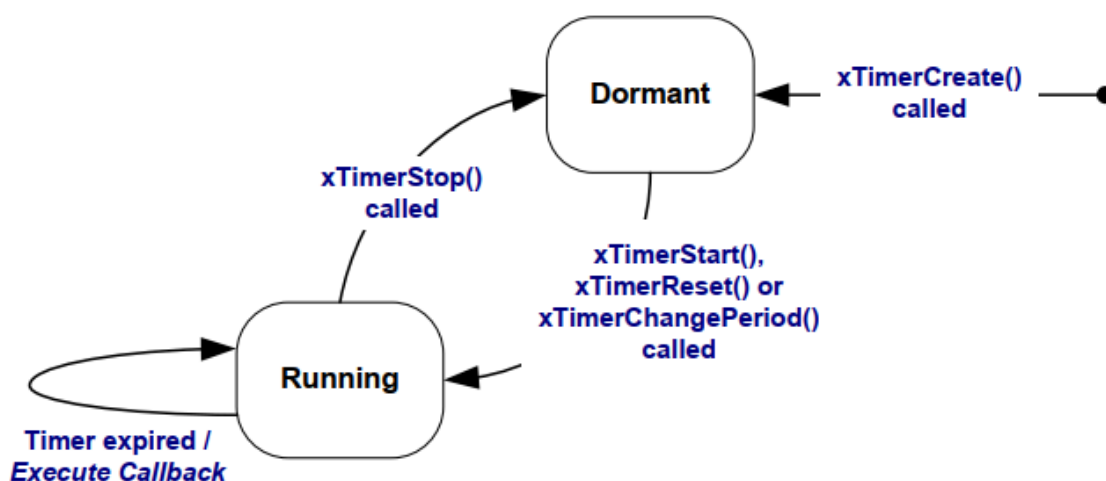


图 6.2 自动重载软件定时器的状态与转换

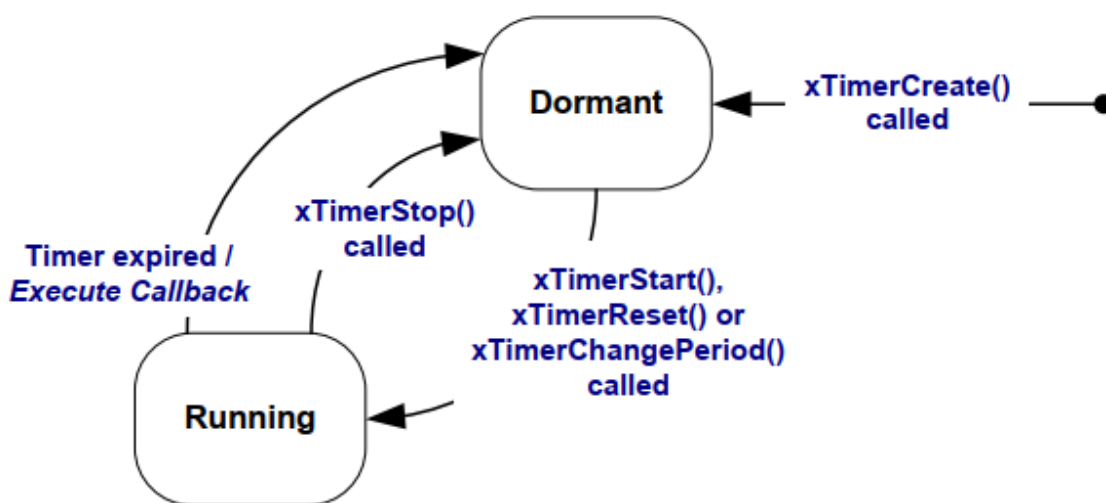


图 6.3 单次软件定时器的状态与转换

`xTimerDelete()` API 函数用于删除定时器。定时器可以在任何时刻被删除。代码清单 6.2 给出了该函数原型。

```
BaseType_t xTimerDelete( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

代码清单 6.2 `xTimerDelete()` API 函数原型

`xTimerDelete()` 参数和返回值

- `xTimer`

要删除的定时器句柄。

- `xTicksToWait`

调用 `xTimerDelete()` 时，如果定时器命令队列已满，该参数指定调用任务为等待删除命令成功发送到队列而保持阻塞状态的最长时间，单位为 Tick。如果在调度器启动前调用

`xTimerDelete()`，则忽略 `xTicksToWait`。

- 返回值

有两个可能的返回值：

- `pdPASS`

如果删除命令成功发送到定时器命令队列，则返回 `pdPASS`。

- `pdFAIL`

如果等待 `xTicksToWait` 个 Tick 后，删除命令仍无法发送到定时器命令队列，则返回 `pdFAIL`。

6.4 软件定时器的上下文

6.4.1 RTOS 守护任务（定时器服务任务）

所有软件定时器回调函数都在同一个 RTOS 守护任务（也称“定时器服务任务”）的上下文中执行^[1]。

守护任务是一个标准 FreeRTOS 任务，在调度器启动时自动创建。它的优先级和栈大小分别由编译时配置常量 `configTIMER_TASK_PRIORITY` 和 `configTIMER_TASK_STACK_DEPTH` 设置，这两个常量都在 `FreeRTOSConfig.h` 中定义。

软件定时器回调函数不得调用会使调用任务进入阻塞状态的 FreeRTOS API 函数，否则守护任务也会随之进入阻塞状态。

6.4.2 定时器命令队列

软件定时器 API 函数通过一个称为“定时器命令队列”的队列，把命令从调用任务发送到守护任务，如图 6.4 所示。命令示例包括“启动定时器”“停止定时器”和“复位定时器”。

定时器命令队列是一个标准 FreeRTOS 队列，在调度器启动时自动创建。队列长度由 `FreeRTOSConfig.h` 中的编译时配置常量 `configTIMER_QUEUE_LENGTH` 设置。

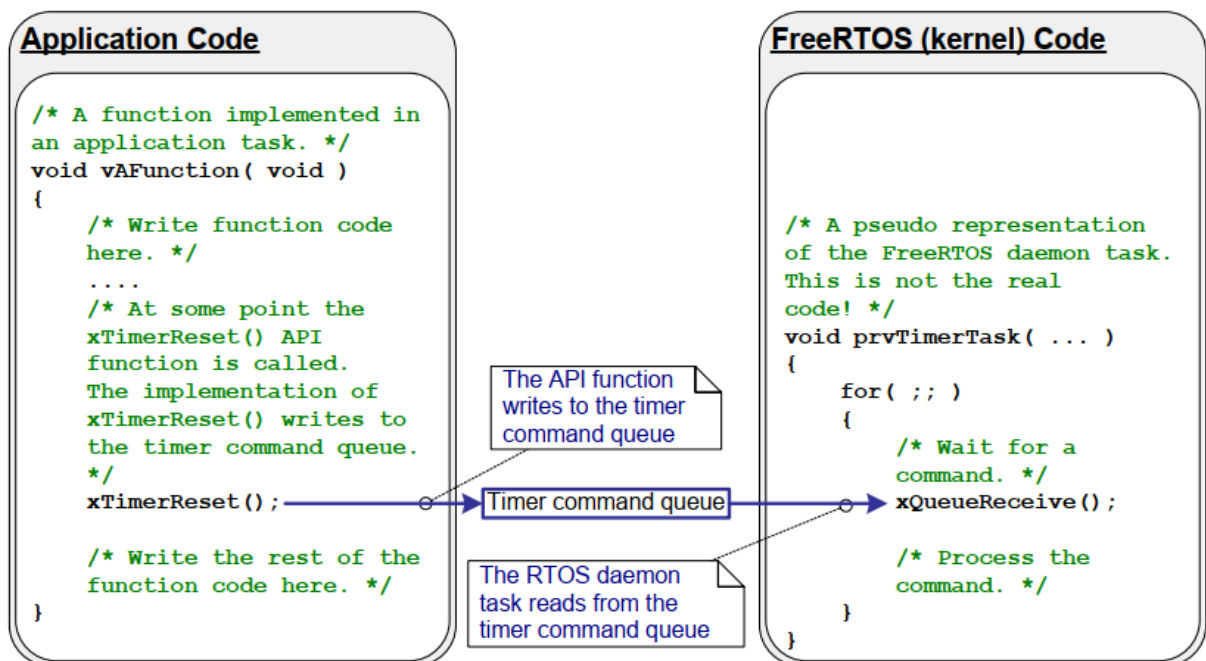


图 6.4 软件定时器 API 函数通过定时器命令队列与 RTOS 守护任务通信

6.4.3 守护任务调度

守护任务与其他 FreeRTOS 任务一样由调度器调度。只有当它成为能够运行的最高优先级任务时，才会处理命令或执行定时器回调函数。图 6.5 和图 6.6 展示了 `configTIMER_TASK_PRIORITY` 的设置如何影响执行模式。

图 6.5 展示了守护任务的优先级低于调用 `xTimerStart()` API 函数的任务时所产生的执行模式。

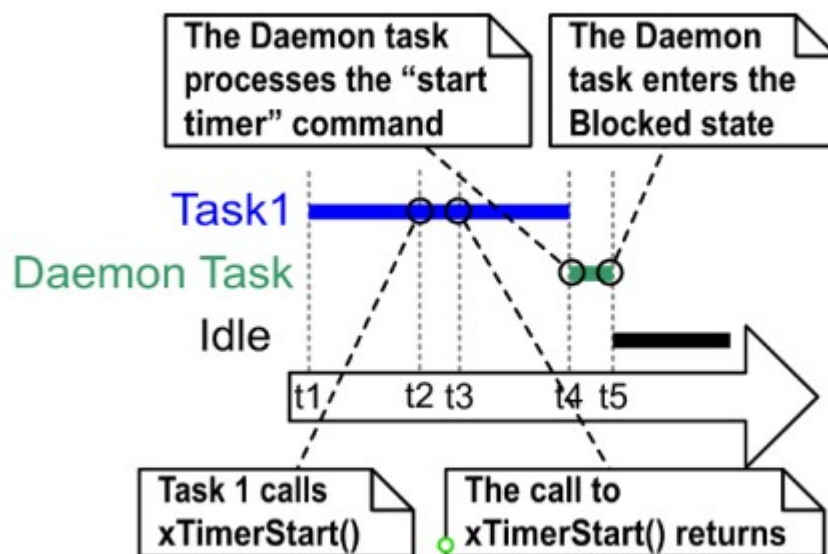


图 6.5 调用 `xTimerStart()` 的任务优先级高于守护任务时的执行模式

参照图 6.5，其中任务 1 的优先级高于守护任务，而守护任务的优先级又高于空闲任务：

1. 在时刻 t1

任务 1 处于运行状态，守护任务处于阻塞状态。

如果有命令发送到定时器命令队列，守护任务将退出阻塞状态并处理该命令；如果软件定时器到期，守护任务也会退出阻塞状态并执行相应的回调函数。

2. 在时刻 t2

任务 1 调用 `xTimerStart()`。

`xTimerStart()` 向定时器命令队列发送一条命令，使守护任务退出阻塞状态。由于任务 1 的优先级高于守护任务，守护任务不会抢占任务 1。

任务 1 仍处于运行状态，守护任务则从阻塞状态转入就绪状态。

3. 在时刻 t3

任务 1 完成 `xTimerStart()` API 函数的执行。从进入函数到函数返回，任务 1 始终处于运行状态。

4. 在时刻 t4

任务 1 调用某个使其进入阻塞状态的 API 函数。此时守护任务是就绪任务中优先级最高的任务，因此调度器选择守护任务进入运行状态。随后，守护任务开始处理任务 1 发送到定时器命令队列的命令。

注意：软件定时器的到期时间从“启动定时器”命令发送到定时器命令队列的时刻开始计算，而不是从守护任务读取并处理该命令的时刻开始计算。

5. 在时刻 t5

守护任务已处理完任务 1 发送的命令，并尝试从定时器命令队列接收更多数据。由于队列为空，守护任务重新进入阻塞状态。当有新命令发送到定时器命令队列，或某个软件定时器到期时，它会再次退出阻塞状态。

此时空闲任务是就绪任务中优先级最高的任务，因此调度器选择空闲任务进入运行状态。

图 6.6 展示了与图 6.5 类似的场景，但这一次守护任务的优先级高于调用 `xTimerStart()` 的任务。

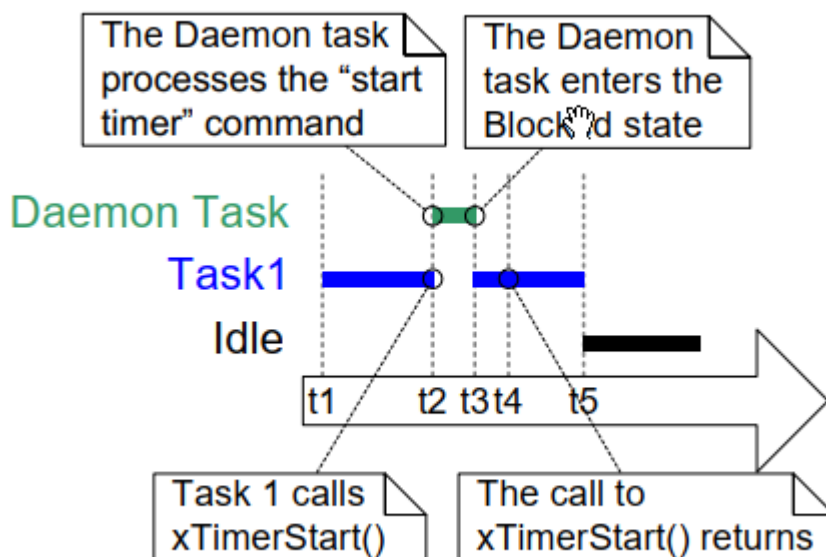


图 6.6 调用 `xTimerStart()` 的任务优先级低于守护任务时的执行模式

参照图 6.6，其中守护任务的优先级高于任务 1，而任务 1 的优先级又高于空闲任务：

1. 在时刻 `t1`

与前一种情况相同，任务 1 处于运行状态，守护任务处于阻塞状态。

2. 在时刻 `t2`

任务 1 调用 `xTimerStart()`。

`xTimerStart()` 向定时器命令队列发送一条命令，使守护任务退出阻塞状态。由于守护任务的优先级高于任务 1，调度器选择守护任务进入运行状态。

任务 1 尚未执行完 `xTimerStart()` 就被守护任务抢占，因而转入就绪状态。

守护任务开始处理任务 1 发送到定时器命令队列的命令。

3. 在时刻 `t3`

守护任务处理完任务 1 发送的命令，并尝试从定时器命令队列接收更多数据。由于队列为空，守护任务重新进入阻塞状态。

此时任务 1 是就绪任务中优先级最高的任务，因此调度器选择任务 1 再次进入运行状态。

4. 在时刻 t4

任务 1 先前在 `xTimerStart()` 返回前被守护任务抢占，现在重新进入运行状态，并从 `xTimerStart()` 返回。

5. 在时刻 t5

任务 1 调用某个使其进入阻塞状态的 API 函数。此时空闲任务成为就绪任务中优先级最高的任务，因此调度器选择空闲任务进入运行状态。

在图 6.5 所示的场景中，任务 1 把命令发送到定时器命令队列后，经过一段时间守护任务才读取并处理该命令。在图 6.6 所示的场景中，任务 1 甚至还没有从发送命令的函数返回，守护任务就已经读取并处理了命令。

发送到定时器命令队列的命令包含时间戳，用于计入应用程序任务发送命令到守护任务处理命令之间经过的时间。例如，若发送“启动定时器”命令来启动一个周期为 10 个 Tick 的定时器，时间戳可确保定时器在命令发出 10 个 Tick 后到期，而不是在守护任务处理命令 10 个 Tick 后才到期。

6.5 创建和启动软件定时器

6.5.1 xTimerCreate() API 函数

软件定时器必须先显式创建，才能使用。除了 `xTimerCreate()`，FreeRTOS 还提供 `xTimerCreateStatic()`，后者使用由应用程序在编译期静态分配的内存创建定时器。

软件定时器通过 `TimerHandle_t` 类型的变量引用。`xTimerCreate()` 用于创建软件定时器，并返回引用新定时器的 `TimerHandle_t`。软件定时器创建后处于休眠状态。

软件定时器既可以在调度器启动前创建，也可以在调度器启动后由任务创建。

[第 2.5 节“数据类型和编码风格指南”](#)介绍了这里使用的数据类型和命名约定。

```
TimerHandle_t xTimerCreate( const char * const pcTimerName,
                           const TickType_t xTimerPeriodInTicks,
                           const BaseType_t xAutoReload,
                           void * const pvTimerID,
                           TimerCallbackFunction_t pxCallbackFunction );
```

代码清单 6.3 `xTimerCreate()` API 函数原型

`xTimerCreate()` 参数和返回值

- `pcTimerName`

定时器的描述性名称。FreeRTOS 本身不会使用该名称，它仅用于辅助调试。与句柄相比，便于阅读的名称更容易帮助开发者识别定时器。

- `xTimerPeriodInTicks`

以 Tick 为单位的定时器周期。宏 `pdMS_TO_TICKS()` 可将毫秒转换为 Tick 数。该参数不能为 0。

- `xAutoReload`

将 `xAutoReload` 设为 `pdTRUE` 可创建自动重载定时器，设为 `pdFALSE` 则创建单次定时器。

- `pvTimerID`

每个软件定时器都有一个 ID 值。ID 的类型是 `void` 指针，应用程序开发者可以按需要使用它。当多个软件定时器共用同一个回调函数时，ID 尤其有用，因为它可以为每个定时器提供专属存储。本章后续示例会演示定时器 ID 的用法。

`pvTimerID` 用于设置所创建定时器的初始 ID 值。

- `pxCallbackFunction`

软件定时器回调函数是符合代码清单 6.1 所示原型的 C 函数。`pxCallbackFunction` 是指向回调函数的指针，实际使用时就是函数名，该函数将作为新定时器的回调函数。

- 返回值

如果返回 `NULL`，则表示软件定时器创建失败，因为没有足够的堆内存供 FreeRTOS 分配所需的数据结构。

如果返回值不是 `NULL`，则表示软件定时器创建成功，返回值就是新定时器的句柄。

第 3 章提供了有关堆内存管理的更多信息。

6.5.2 xTimerStart() API 函数

`xTimerStart()` 用于启动处于休眠状态的软件定时器，也可以复位（重新启动）处于运行状态的软件定时器。`xTimerStop()` 用于停止正在运行的软件定时器；停止定时器等同于使其转入休眠状态。

可以在调度器启动前调用 `xTimerStart()`，但此时软件定时器要等到调度器真正启动后才开始计时。

注意：切勿从中断服务程序中调用 `xTimerStart()`，应改用其中断安全版本 `xTimerStartFromISR()`。

```
BaseType_t xTimerStart( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

代码清单 6.4 `xTimerStart()` API 函数原型

`xTimerStart()` 参数和返回值

- `xTimer`

要启动或复位的软件定时器句柄。该句柄由创建软件定时器时调用的 `xTimerCreate()` 返回。

- `xTicksToWait`

`xTimerStart()` 通过定时器命令队列把“启动定时器”命令发送到守护任务。队列已满时，`xTicksToWait` 指定调用任务为等待队列出现可用空间而保持阻塞状态的最长时间。

如果 `xTicksToWait` 为 0 且定时器命令队列已满，`xTimerStart()` 将立即返回。

阻塞时间以 Tick 周期为单位，因此对应的绝对时间取决于 Tick 频率。宏 `pdMS_TO_TICKS()` 可将毫秒转换为 Tick 数。

如果在 `FreeRTOSConfig.h` 中将 `INCLUDE_vTaskSuspend` 设为 1，那么把 `xTicksToWait` 设为 `portMAX_DELAY` 可使调用任务无限期地保持阻塞状态，等待定时器命令队列出现可用空间。

如果在调度器启动前调用 `xTimerStart()`，则忽略 `xTicksToWait`，函数行为与该参数设为 0 时相同。

- 返回值

有两个可能的返回值：

- `pdPASS`

仅当“启动定时器”命令成功发送到定时器命令队列时，才返回 `pdPASS`。

如果守护任务的优先级高于调用 `xTimerStart()` 的任务，那么启动命令会在 `xTimerStart()` 返回前得到处理。这是因为定时器命令队列一有数据，守护任务就会抢占调用任务。

如果指定了非零阻塞时间，调用任务可能先进入阻塞状态等待队列出现可用空间，但命令在阻塞时间到期前已成功写入定时器命令队列。

- `pdFAIL`

如果队列已满，无法写入“启动定时器”命令，则返回 `pdFAIL`。

如果指定了非零阻塞时间，调用任务会进入阻塞状态，等待守护任务在定时器命令队列中腾出空间，但在空间出现之前阻塞时间已经到期。

示例 6.1 创建单次定时器和自动重载定时器

本例创建并启动一个单次定时器和一个自动重载定时器，如代码清单 6.5 所示。

```

/* 单次定时器和自动重载定时器的周期分别为 3.333 秒和 0.5 秒。 */
#define mainONE_SHOT_TIMER_PERIOD pdMS_TO_TICKS( 3333 )
#define mainAUTO_RELOAD_TIMER_PERIOD pdMS_TO_TICKS( 500 )

int main( void )
{
    TimerHandle_t xAutoReloadTimer, xOneShotTimer;
    BaseType_t xTimer1Started, xTimer2Started;

    /* 创建单次定时器, 并将新定时器的句柄保存到 xOneShotTimer。 */
    xOneShotTimer = xTimerCreate(
        /* 软件定时器的文本名称, FreeRTOS 不会使用该名称。 */
        "OneShot",
        /* 软件定时器的周期, 单位为 Tick。 */
        mainONE_SHOT_TIMER_PERIOD,
        /* 将 xAutoReload 设为 pdFALSE 可创建单次软件定时器。 */
        pdFALSE,
        /* 本例不使用定时器 ID。 */
        0,
        /* 新软件定时器使用的回调函数。 */
        prvOneShotTimerCallback );

    /* 创建自动重载定时器, 并将新定时器的句柄保存到 xAutoReloadTimer。 */
    xAutoReloadTimer = xTimerCreate(
        /* 软件定时器的文本名称, FreeRTOS 不会使用该名称。 */
        "AutoReload",
        /* 软件定时器的周期, 单位为 Tick。 */
        mainAUTO_RELOAD_TIMER_PERIOD,
        /* 将 xAutoReload 设为 pdTRUE 可创建自动重载定时器。 */
        pdTRUE,
        /* 本例不使用定时器 ID。 */
        0,
        /* 新软件定时器使用的回调函数。 */
        prvAutoReloadTimerCallback );

    /* 检查两个软件定时器是否创建成功。 */
    if( ( xOneShotTimer != NULL ) && ( xAutoReloadTimer != NULL ) )
    {
        /* 启动软件定时器, 并将阻塞时间设为 0 (不阻塞)。调度器尚未启动,
           因此这里指定的任何阻塞时间原本也会被忽略。 */
        xTimer1Started = xTimerStart( xOneShotTimer, 0 );
        xTimer2Started = xTimerStart( xAutoReloadTimer, 0 );

        /* xTimerStart() 通过定时器命令队列实现, 因此当队列已满时调用会
           失败。调度器启动后才会创建定时器服务任务, 所以发送到命令
           队列的所有命令都要等调度器启动后才能得到处理。检查两次
           xTimerStart() 调用是否都成功。 */
        if( ( xTimer1Started == pdPASS ) && ( xTimer2Started == pdPASS ) )
        {
            /* 启动调度器。 */
            vTaskStartScheduler();
        }
    }

    /* 与以往示例一样, 程序不应执行到这里。 */
}

```

```
    for( ;; );  
}
```

代码清单 6.5 创建并启动示例 6.1 使用的定时器

两个定时器的回调函数每次被调用时都只打印一条消息。单次定时器的回调函数见代码清单 6.6，自动重载定时器的回调函数见代码清单 6.7。

```
static void prvOneShotTimerCallback( TimerHandle_t xTimer )  
{  
    TickType_t xTimeNow;  
  
    /* 获取当前 Tick 计数。 */  
    xTimeNow = xTaskGetTickCount();  
  
    /* 输出字符串，显示回调函数的执行时刻。 */  
    vPrintStringAndNumber( "One-shot timer callback executing", xTimeNow );  
  
    /* 文件作用域变量。 */  
    ulCallCount++;  
}
```

代码清单 6.6 示例 6.1 中单次定时器使用的回调函数

```
static void prvAutoReloadTimerCallback( TimerHandle_t xTimer )  
{  
    TickType_t xTimeNow;  
  
    /* 获取当前 Tick 计数。 */  
    xTimeNow = xTaskGetTickCount();  
  
    /* 输出字符串，显示回调函数的执行时刻。 */  
    vPrintStringAndNumber( "Auto-reload timer callback executing", xTimeNow );  
  
    ulCallCount++;  
}
```

代码清单 6.7 示例 6.1 中自动重载定时器使用的回调函数

本例产生的输出如图 6.7 所示。自动重载定时器的回调函数每隔 500 个 Tick 执行一次，因为代码清单 6.5 将 `mainAUTO_RELOAD_TIMER_PERIOD` 设为 500。单次定时器的回调函数只在 Tick 计数达到 3333 时执行一次，因为 `mainONE_SHOT_TIMER_PERIOD` 被设为 3333。

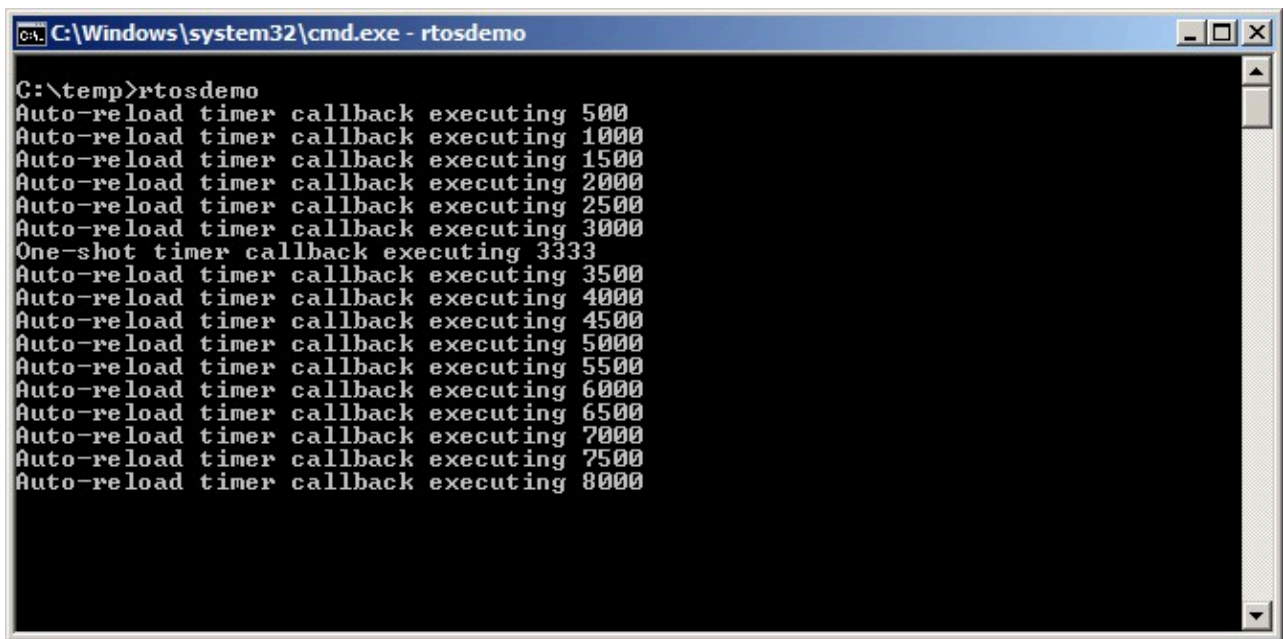


图 6.7 执行示例 6.1 时产生的输出

6.6 定时器 ID

每个软件定时器都有一个 ID。它是一个标签值，应用程序开发者可以按任意用途使用。ID 保存在 `void` 指针（`void *`）中，因此既可以直接保存整数值，也可以指向其他对象，或作为函数指针使用。

创建软件定时器时会为其设置初始 ID 值。之后可以使用 `vTimerSetTimerID()` API 函数更新 ID，并使用 `pvTimerGetTimerID()` API 函数查询 ID。

与其他软件定时器 API 函数不同，`vTimerSetTimerID()` 和 `pvTimerGetTimerID()` 会直接访问软件定时器，不会向定时器命令队列发送命令。

6.6.1 vTimerSetTimerID() API 函数

```
void vTimerSetTimerID( const TimerHandle_t xTimer, void *pvNewID );
```

代码清单 6.8 `vTimerSetTimerID()` API 函数原型

`vTimerSetTimerID()` 参数

- `xTimer`

要更新 ID 值的软件定时器句柄。该句柄由创建软件定时器时调用的 `xTimerCreate()` 返回。

- `pvNewID`

软件定时器的 ID 将被设为该值。

6.6.2 pvTimerGetTimerID() API 函数

```
void *pvTimerGetTimerID( const TimerHandle_t xTimer );
```

代码清单 6.9 `pvTimerGetTimerID()` API 函数原型

`pvTimerGetTimerID()` 参数和返回值

- `xTimer`

要查询的软件定时器句柄。该句柄由创建软件定时器时调用的 `xTimerCreate()` 返回。

- 返回值

被查询软件定时器的 ID。

示例 6.2 使用回调函数参数和软件定时器 ID

多个软件定时器可以共用同一个回调函数。此时，可以通过回调函数的参数判断究竟是哪一个软件定时器到期。

示例 6.1 使用两个独立的回调函数，分别服务于单次定时器和自动重载定时器。示例 6.2 实现了类似功能，但两个软件定时器共用同一个回调函数。

示例 6.2 的 `main()` 与示例 6.1 几乎相同，唯一区别在于创建软件定时器的代码。代码清单 6.10 展示了这部分差异，其中两个定时器都使用 `prvTimerCallback()` 作为回调函数。

```

/* 创建单次软件定时器，并将句柄保存到 xOneShotTimer。 */
xOneShotTimer = xTimerCreate( "OneShot",
                               mainONE_SHOT_TIMER_PERIOD,
                               pdFALSE,
                               /* 将定时器 ID 初始化为 NULL。 */
                               NULL,
                               /* 两个定时器都使用 prvTimerCallback()。 */
                               prvTimerCallback );

/* 创建自动重载软件定时器，并将句柄保存到 xAutoReloadTimer。 */
xAutoReloadTimer = xTimerCreate( "AutoReload",
                                  mainAUTO_RELOAD_TIMER_PERIOD,
                                  pdTRUE,
                                  /* 将定时器 ID 初始化为 NULL。 */
                                  NULL,
                                  /* 两个定时器都使用 prvTimerCallback()。 */
                                  prvTimerCallback );

```

代码清单 6.10 创建示例 6.2 使用的定时器

任一定时器到期时都会执行 `prvTimerCallback()`。该函数通过传入的参数判断本次调用是由单次定时器到期，还是由自动重载定时器到期所触发。

`prvTimerCallback()` 还演示了如何使用软件定时器 ID 保存定时器专属数据。每个软件定时器都在自己的 ID 中记录到期次数，自动重载定时器则利用该计数在第五次执行后停止自身。

`prvTimerCallback()` 的实现见代码清单 6.11。

```

static void prvTimerCallback( TimerHandle_t xTimer )
{
    TickType_t xTimeNow;
    uint32_t ulExecutionCount;

    /* 软件定时器的到期次数保存在定时器 ID 中。读取 ID，将其递增，再把
       结果保存为新的 ID 值。ID 是 void 指针，因此要强制转换为 uint32_t。 */
    ulExecutionCount = ( uint32_t ) pvTimerGetTimerID( xTimer );
    ulExecutionCount++;
    vTimerSetTimerID( xTimer, ( void * ) ulExecutionCount );

    /* 获取当前 Tick 计数。 */
    xTimeNow = xTaskGetTickCount();

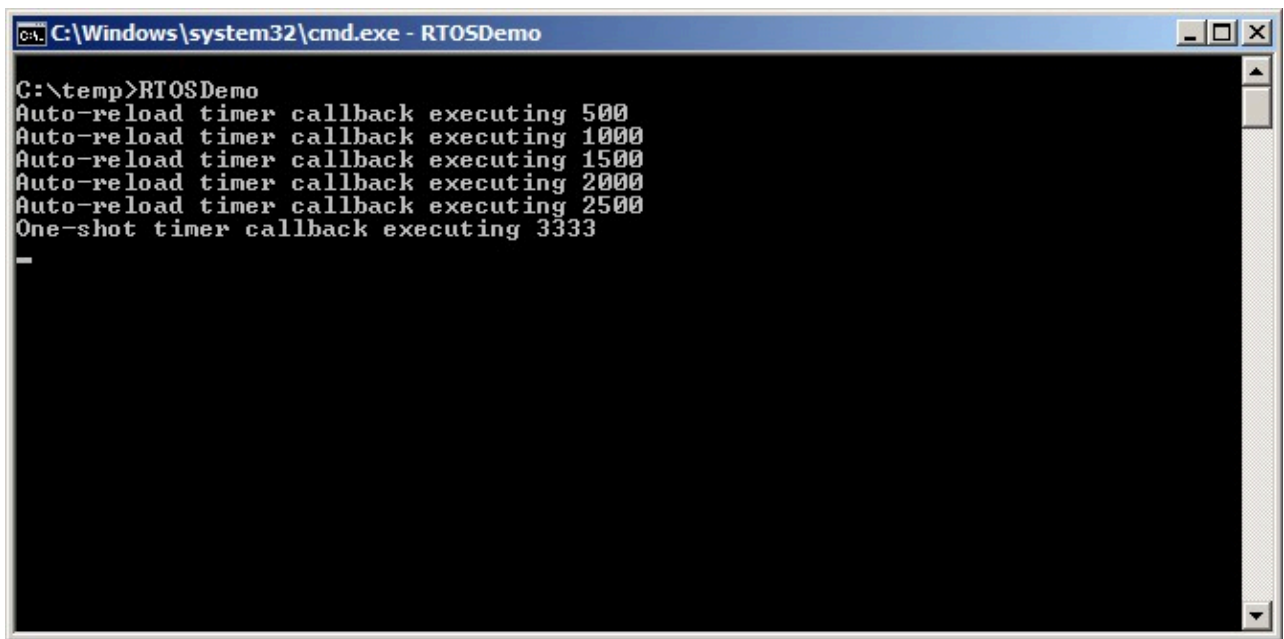
    /* 创建单次定时器时，其句柄已保存到 xOneShotTimer。将传入本函数的
       句柄与 xOneShotTimer 比较，以判断到期的是单次定时器还是自动重载
       定时器，然后输出字符串显示回调函数的执行时刻。 */
    if( xTimer == xOneShotTimer )
    {
        vPrintStringAndNumber( "One-shot timer callback executing", xTimeNow );
    }
    else
    {
        /* xTimer 不等于 xOneShotTimer，因此到期的必定是自动重载定时器。 */
        vPrintStringAndNumber( "Auto-reload timer callback executing", xTimeNow);

        if( ulExecutionCount == 5 )
        {
            /* 自动重载定时器执行 5 次后将其停止。本回调函数在 RTOS 守护
               任务的上下文中执行，因此不得调用可能使守护任务进入阻塞
               状态的函数。为此将阻塞时间设为 0。 */
            xTimerStop( xTimer, 0 );
        }
    }
}

```

代码清单 6.11 示例 6.2 使用的定时器回调函数

示例 6.2 产生的输出如图 6.8 所示。可以看出，自动重载定时器只执行了五次。



```
C:\Windows\system32\cmd.exe - RTOSDemo
C:\temp>RTOSDemo
Auto-reload timer callback executing 500
Auto-reload timer callback executing 1000
Auto-reload timer callback executing 1500
Auto-reload timer callback executing 2000
Auto-reload timer callback executing 2500
One-shot timer callback executing 3333
-
```

图 6.8 执行示例 6.2 时产生的输出

6.7 更改定时器的周期

每个官方 FreeRTOS 移植版本都附带一个或多个示例项目。大多数示例项目都具有自检功能，并通过 LED 直观显示项目状态：如果自检始终通过，LED 会缓慢翻转；一旦自检失败，LED 就会快速翻转。

一些示例项目在任务中执行自检，并使用 `vTaskDelay()` 控制 LED 翻转速率；另一些示例项目在软件定时器回调函数中执行自检，并通过定时器周期控制 LED 翻转速率。

6.7.1 xTimerChangePeriod() API 函数

`xTimerChangePeriod()` 函数用于更改软件定时器的周期。

如果使用 `xTimerChangePeriod()` 更改正在运行的定时器周期，定时器会使用新周期重新计算到期时间。新的到期时间以调用 `xTimerChangePeriod()` 的时刻为基准，而不是以定时器最初启动的时刻为基准。

如果使用 `xTimerChangePeriod()` 更改处于休眠状态（未运行）的定时器周期，定时器会计算新的到期时间并转入运行状态，也就是开始运行。

注意：切勿从中断服务程序中调用 `xTimerChangePeriod()`，应改用其中断安全版本

`xTimerChangePeriodFromISR()`。

```
BaseType_t xTimerChangePeriod( TimerHandle_t xTimer,  
                               TickType_t xNewPeriod,  
                               TickType_t xTicksToWait );
```

代码清单 6.12 `xTimerChangePeriod()` API 函数原型

`xTimerChangePeriod()` 参数和返回值

- `xTimer`

要更新周期的软件定时器句柄。该句柄由创建软件定时器时调用的 `xTimerCreate()` 返回。

- `xNewPeriod`

软件定时器的新周期，单位为 Tick。宏 `pdMS_TO_TICKS()` 可将毫秒转换为 Tick 数。

- `xTicksToWait`

`xTimerChangePeriod()` 通过定时器命令队列把“更改周期”命令发送到守护任务。队列已满时，`xTicksToWait` 指定调用任务为等待队列出现可用空间而保持阻塞状态的最长时间。

如果 `xTicksToWait` 为 0 且定时器命令队列已满，`xTimerChangePeriod()` 将立即返回。

阻塞时间以 Tick 周期为单位，宏 `pdMS_TO_TICKS()` 可将毫秒转换为 Tick 数。

如果在 `FreeRTOSConfig.h` 中将 `INCLUDE_vTaskSuspend` 设为 1，那么把 `xTicksToWait` 设为 `portMAX_DELAY` 可使调用任务无限期地保持阻塞状态，等待定时器命令队列出现可用空间。

如果在调度器启动前调用 `xTimerChangePeriod()`，则忽略 `xTicksToWait`，函数行为与该参数设为 0 时相同。

- 返回值

有两个可能的返回值：

- `pdPASS`

仅当“更改周期”命令成功发送到定时器命令队列时，才返回 `pdPASS`。

如果指定了非零阻塞时间，调用任务可能先进入阻塞状态等待队列出现可用空间，但命令在阻塞时间到期前已成功写入定时器命令队列。

- **pdFAIL**

如果队列已满，无法写入“更改周期”命令，则返回 **pdFAIL**。

如果指定了非零阻塞时间，调用任务会进入阻塞状态，等待守护任务在队列中腾出空间，但在空间出现之前阻塞时间已经到期。

代码清单 6.13 展示了一个在软件定时器回调函数中执行自检的 FreeRTOS 示例。自检失败后，示例使用 **xTimerChangePeriod()** 提高 LED 的翻转速率。执行自检的软件定时器称为“检查定时器”。

```
/* 检查定时器的初始周期为 3000 毫秒，因此 LED 每 3 秒翻转一次。如果
   自检功能发现异常状态，检查定时器的周期将缩短为 200 毫秒，使 LED
   以快得多的速率翻转。 */
const TickType_t xHealthyTimerPeriod = pdMS_TO_TICKS( 3000 );
const TickType_t xErrorTimerPeriod = pdMS_TO_TICKS( 200 );

/* 检查定时器使用的回调函数。 */
static void prvCheckTimerCallbackFunction( TimerHandle_t xTimer )
{
    static BaseType_t xErrorDetected = pdFALSE;

    if( xErrorDetected == pdFALSE )
    {
        /* 尚未检测到错误，再次运行自检函数。该函数要求示例创建的每个
           任务报告自身状态，同时检查所有任务是否仍在运行（从而能够
           正确报告状态）。 */
        if( CheckTasksAreRunningWithoutError() == pdFAIL )
        {
            /* 一个或多个任务报告了异常状态，可能已经发生错误。缩短检查
               定时器的周期，提高本回调函数的执行频率，从而提高 LED 的
               翻转速率。本回调函数在 RTOS 守护任务的上下文中执行，因此
               阻塞时间设为 0，以确保守护任务绝不会进入阻塞状态。 */
            xTimerChangePeriod(
                xTimer,          /* 要更新的定时器。 */
                xErrorTimerPeriod, /* 定时器的新周期。 */
                0 );             /* 发送该命令时不阻塞。 */
        }

        /* 记录已经检测到错误。 */
        xErrorDetected = pdTRUE;
    }

    /* 翻转 LED。LED 的翻转速率取决于本函数的调用频率，而调用频率由检查
       定时器的周期决定。如果 CheckTasksAreRunningWithoutError() 曾返回
       pdFAIL，定时器周期就会从 3000 毫秒缩短为 200 毫秒。 */
    ToggleLED();
}
```

6.8 复位软件定时器

复位软件定时器就是重新启动定时器。复位后，到期时间将以定时器被复位时刻为基准重新计算，而不再以最初启动的时刻为基准。图 6.9 展示了一个周期为 6 个 Tick 的定时器先被启动、随后复位两次，最终到期并执行回调函数的过程。

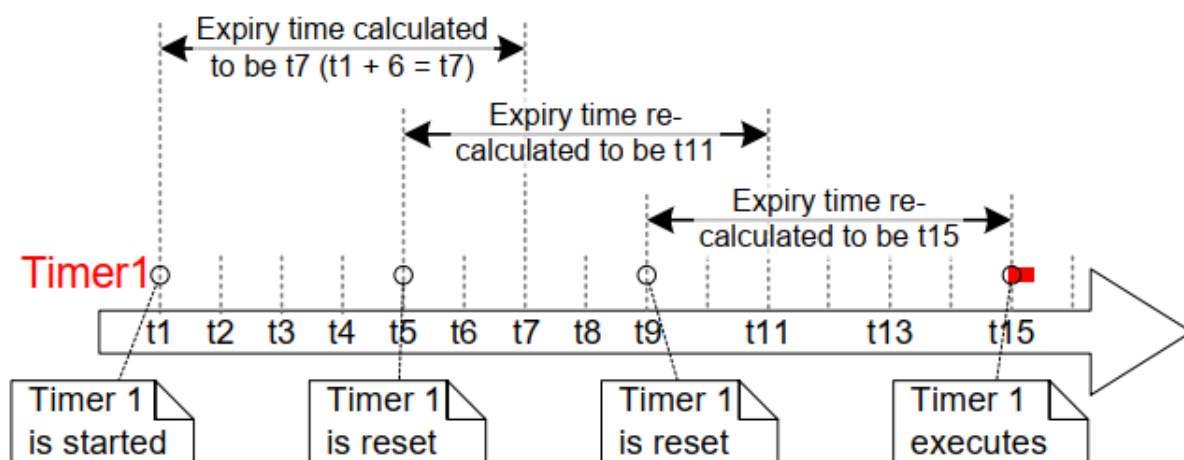


图 6.9 启动并复位周期为 6 个 Tick 的软件定时器

参照图 6.9:

- 定时器 1 在时刻 t1 启动。其周期为 6 个 Tick，因此回调函数最初计划在时刻 t7，也就是启动 6 个 Tick 后执行。
- 定时器 1 在时刻 t7 之前，也就是到期并执行回调函数之前被复位。它在时刻 t5 复位，因此回调函数的执行时刻被重新计算为 t11，即复位 6 个 Tick 后。
- 定时器 1 在时刻 t11 之前再次复位，同样发生在定时器到期并执行回调函数之前。它在时刻 t9 复位，因此回调函数的执行时刻再次被计算为 t15，即上次复位 6 个 Tick 后。
- 定时器 1 此后没有再次复位，因此会在时刻 t15 到期并执行回调函数。

6.8.1 xTimerReset() API 函数

`xTimerReset()` API 函数用于复位定时器。

`xTimerReset()` 也可用于启动处于休眠状态的定时器。

注意：切勿从中断服务程序中调用 `xTimerReset()`，应改用其中断安全版本 `xTimerResetFromISR()`。

```
BaseType_t xTimerReset( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

代码清单 6.14 `xTimerReset()` API 函数原型

`xTimerReset()` 参数和返回值

- `xTimer`

要复位或启动的软件定时器句柄。该句柄由创建软件定时器时调用的 `xTimerCreate()` 返回。

- `xTicksToWait`

`xTimerReset()` 通过定时器命令队列把“复位”命令发送到守护任务。队列已满时，`xTicksToWait` 指定调用任务为等待队列出现可用空间而保持阻塞状态的最长时间。

如果 `xTicksToWait` 为 0 且定时器命令队列已满，`xTimerReset()` 将立即返回。

如果在 `FreeRTOSConfig.h` 中将 `INCLUDE_vTaskSuspend` 设为 1，那么把 `xTicksToWait` 设为 `portMAX_DELAY` 可使调用任务无限期地保持阻塞状态，等待定时器命令队列出现可用空间。

- 返回值

有两个可能的返回值：

- `pdPASS`

仅当“复位”命令成功发送到定时器命令队列时，才返回 `pdPASS`。

如果指定了非零阻塞时间，调用任务可能先进入阻塞状态等待队列出现可用空间，但命令在阻塞时间到期前已成功写入定时器命令队列。

pdFAIL

如果队列已满，无法写入“复位”命令，则返回 pdFAIL。

如果指定了非零阻塞时间，调用任务会进入阻塞状态，等待守护任务在队列中腾出空间，但在空间出现之前阻塞时间已经到期。

示例 6.3 复位软件定时器

本例模拟手机背光的行为。背光应当：

- 在用户按键时亮起。
- 如果用户在指定时间内继续按键，则保持亮起。
- 如果指定时间内没有按键操作，则自动关闭。

使用单次软件定时器来实现此行为：

- 用户按键时打开模拟背光，在软件定时器的回调函数中关闭背光。
- 每次按键时都复位软件定时器。
- 因此，要让背光保持开启，两次按键之间的时间不得超过软件定时器周期。如果定时器到期前没有新的按键操作将其复位，回调函数就会执行并关闭背光。

变量 `xSimulatedBacklightOn` 保存背光状态：设为 `pdTRUE` 表示背光开启，设为 `pdFALSE` 表示背光关闭。

软件定时器回调函数见代码清单 6.15。

```
static void prvBacklightTimerCallback( TimerHandle_t xTimer )
{
    TickType_t xTimeNow = xTaskGetTickCount();

    /* 背光定时器已到期，关闭背光。 */
    xSimulatedBacklightOn = pdFALSE;

    /* 打印关闭背光的时刻。 */
    vPrintStringAndNumber(
        "Timer expired, turning backlight OFF at time\t\t", xTimeNow );
}
```

代码清单 6.15 示例 6.3 中单次定时器的回调函数

示例 6.3 创建一个任务来轮询键盘^[2]。该任务见代码清单 6.16。不过，由于下一段说明的原因，这段代码并不是推荐的最佳设计。

FreeRTOS 支持采用事件驱动方式设计应用程序。事件驱动设计只在事件发生时占用处理时间，不会把时间浪费在轮询尚未发生的事件上，因此效率很高。示例 6.3 无法采用事件驱动方式，因为在 FreeRTOS Windows 移植版本中处理键盘中断并不现实，所以只能使用效率较低的轮询方案。如果代码清单 6.16 展示的是中断服务程序，就应使用 `xTimerResetFromISR()` 代替 `xTimerReset()`。

```

static void vKeyHitTask( void *pvParameters )
{
    const TickType_t xShortDelay = pdMS_TO_TICKS( 50 );
    TickType_t xTimeNow;

    vPrintString( "Press a key to turn the backlight on.\r\n" );

    /* 理想情况下，应用程序应采用事件驱动设计，并通过中断处理按键操作。
    但 FreeRTOS Windows 移植版本不适合使用键盘中断，因此本任务通过
    轮询检测按键。 */
    for( ;; )
    {
        /* 是否有按键被按下？ */
        if( _kbhit() != 0 )
        {
            /* 检测到按键，记录当前时刻。 */
            xTimeNow = xTaskGetTickCount();

            if( xSimulatedBacklightOn == pdFALSE )
            {
                /* 背光原本处于关闭状态，将其打开并打印开启时刻。 */
                xSimulatedBacklightOn = pdTRUE;
                vPrintStringAndNumber(
                    "Key pressed, turning backlight ON at time\t\t",
                    xTimeNow );
            }
            else
            {
                /* 背光已经开启，打印消息说明即将复位定时器以及复位时刻。 */
                vPrintStringAndNumber(
                    "Key pressed, resetting software timer at time\t\t",
                    xTimeNow );
            }

            /* 复位软件定时器。如果背光原本关闭，此调用会启动定时器；如果
            背光原本开启，此调用会重新启动定时器。实际应用可能在中断
            中读取按键；如果本函数是中断服务程序，就必须使用
            xTimerResetFromISR() 代替 xTimerReset()。 */
            xTimerReset( xBacklightTimer, xShortDelay );

            /* 读取并丢弃按下的键，本简单示例不需要该按键值。 */
            ( void ) _getch();
        }
    }
}

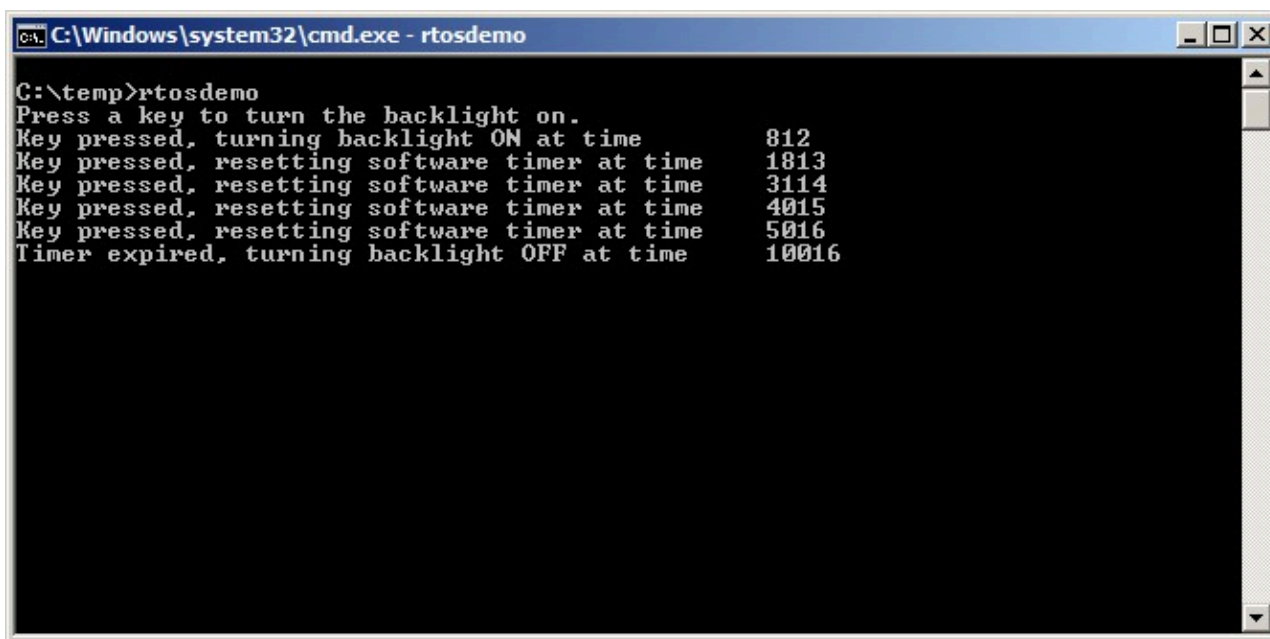
```

代码清单 6.16 示例 6.3 中用于复位软件定时器的任务

执行示例 6.3 时产生的输出如图 6.10 所示。参照图 6.10:

- 第一次按键发生在 Tick 计数为 812 时。此时背光开启，单次定时器启动。

- 随后的按键分别发生在 Tick 计数为 1813、3114、4015 和 5016 时。每次按键都会在定时器到期前将其复位。
- 定时器在 Tick 计数为 10016 时到期，此时背光关闭。



```
C:\temp>rtosdemo
Press a key to turn the backlight on.
Key pressed, turning backlight ON at time      812
Key pressed, resetting software timer at time 1813
Key pressed, resetting software timer at time 3114
Key pressed, resetting software timer at time 4015
Key pressed, resetting software timer at time 5016
Timer expired, turning backlight OFF at time 10016
```

图 6.10 执行示例 6.3 时产生的输出

由图 6.10 可见，定时器周期为 5000 个 Tick；背光恰好在最后一次按键之后 5000 个 Tick 关闭，也就是在定时器最后一次复位后 5000 个 Tick 关闭。

1. 该任务以前称为“定时器服务任务”，因为最初只用于执行软件定时器回调函数。现在同一任务还承担其他用途，因此改用更通用的名称“RTOS 守护任务”。↩
2. 向 Windows 控制台输出内容以及从控制台读取按键都会触发 Windows 系统调用。Windows 系统调用，包括访问控制台、磁盘或 TCP/IP 协议栈，可能对 FreeRTOS Windows 移植版本的行为产生不利影响，通常应当避免。↩

7 中断管理

7.1 引言

7.1.1 事件

嵌入式实时系统必须响应来自外部环境的事件。例如，以太网外设收到数据包（事件）后，可能需要将其传递给 TCP/IP 协议栈进行处理（操作）。稍具复杂度的系统都必须处理来自多个数据源的事件，而每类事件的处理开销和响应时间要求各不相同。因此，需要针对每种情况确定最佳的事件处理策略：

- 如何检测事件？通常使用中断，但输入也可以轮询。
- 使用中断时，应当在中断服务程序（ISR）内部完成多少处理，又应当把多少处理放到 ISR 外部？通常应使每个 ISR 尽可能短。
- 如何把事件传递给主体（非 ISR）代码，以及如何组织这部分代码，才能妥善处理可能异步发生的事件？

FreeRTOS 不会强制应用程序设计者采用某一种事件处理策略，但它提供了相应功能，使选定的策略能够以简洁且易于维护的方式实现。

区分任务优先级和中断优先级非常重要：

- 任务是一种软件特性，与运行 FreeRTOS 的硬件无关。任务优先级由应用程序开发者在软件中指定，软件算法（调度器）决定哪个任务进入运行状态。
- 中断服务程序虽然由软件编写，但中断本身是硬件特性，因为硬件决定哪个 ISR 在何时运行。任务只能在没有 ISR 运行时执行，因此即使是最低优先级的中断也能中断最高优先级的任务，而任务无法抢占 ISR。

所有能够运行 FreeRTOS 的架构都支持中断，但中断入口和中断优先级分配的具体方式因架构而异。

7.1.2 范围

本章涵盖：

- 哪些 FreeRTOS API 函数可以在中断服务程序中使用。

- 如何把中断处理延迟到任务中执行。
- 如何创建和使用二值信号量与计数信号量。
- 二值信号量与计数信号量之间的区别。
- 如何使用队列向中断服务程序传入数据以及从中传出数据。
- 某些 FreeRTOS 移植版本提供的中断嵌套模型。

7.2 从 ISR 中使用 FreeRTOS API

7.2.1 中断安全 API

中断服务程序（ISR）经常需要使用 FreeRTOS API 提供的功能，但许多 FreeRTOS API 函数执行的操作在 ISR 中没有意义。最明显的例子是把调用 API 的任务置于阻塞状态：从 ISR 调用 API 时，并不存在可以被阻塞的调用任务。FreeRTOS 为部分 API 函数分别提供任务版本和 ISR 版本，以解决这一问题。供 ISR 使用的版本在函数名末尾带有 `FromISR`。

注意：切勿从 ISR 调用函数名中不含 `FromISR` 的 FreeRTOS API 函数。

7.2.2 使用单独的中断安全 API 的好处

独立的中断安全 API 能提高任务代码和 ISR 代码的效率，同时简化中断入口。要理解其中原因，可以设想另一种方案：每个 API 函数只提供一个版本，并允许从任务和 ISR 调用。如果采用这种方案：

- API 函数需要额外逻辑判断调用来自任务还是 ISR。这会引入新的执行路径，使函数更长、更复杂，也更难测试。
- 某些参数在从任务调用时会显得多余，另一些参数则在从 ISR 调用时没有意义。
- 每个 FreeRTOS 移植版本都需要提供一种机制来确定执行上下文（任务或 ISR）。
- 对于难以判断执行上下文的架构，还需要额外、低效、复杂且非标准的中断入口代码，才能让软件获知当前处于任务还是 ISR 上下文。

7.2.3 使用单独的中断安全 API 的缺点

为部分 API 函数提供两个版本提高了任务和 ISR 的效率，但也带来一个问题：有时某个并非 FreeRTOS API、但内部调用了 FreeRTOS API 的函数，需要同时从任务和 ISR 调用。

这通常只在集成第三方代码时才会成为问题，因为此时应用程序开发者无法控制原有软件设计。可以采用以下方法之一解决：

- 将中断处理延迟到任务^[1]，使 API 函数只在任务上下文中调用。
- 如果使用支持中断嵌套的 FreeRTOS 移植版本，可使用函数名以 `FromISR` 结尾的 API 版本，因为该版本既能从任务调用，也能从 ISR 调用。反过来则不成立：函数名不以 `FromISR` 结尾的 API 不得从 ISR 调用。
- 第三方代码通常包含一个 RTOS 抽象层，可以实现该抽象层来测试调用函数的上下文（任务或中断），然后调用适合该上下文的 API 函数。

7.2.4 xHigherPriorityTaskWoken 参数

本节介绍 `xHigherPriorityTaskWoken` 参数。如果暂时没有完全理解也不必担心，后续各节会给出实际示例。

如果中断引发了上下文切换，那么中断退出后运行的任务可能与进入中断前运行的任务不同：中断打断了一个任务，却返回到另一个任务。

某些 FreeRTOS API 函数能够把任务从阻塞状态转入就绪状态。例如，如果有任务阻塞在某个队列上等待数据，`xQueueSendToBack()` 向该队列写入数据后就会解除该任务的阻塞。

如果被 FreeRTOS API 函数解除阻塞的任务优先级高于当前运行任务，那么根据 FreeRTOS 调度策略，应当切换到这个更高优先级的任务。切换实际发生的时机取决于调用 API 的上下文：

- 如果从任务调用 API 函数：

如果在 `FreeRTOSConfig.h` 中将 `configUSE_PREEMPTION` 设为 1，则 API 函数会在返回前自动切换到更高优先级的任务。图 6.6 已展示这种情况：向定时器命令队列写入数据后，写队列函数还没有返回，系统就切换到了 RTOS 守护任务。

- 如果从中断调用 API 函数：

系统不会在中断安全 API 内部自动切换到更高优先级的任务，而是通过一个变量通知应用程序应当执行上下文切换。函数名以 `FromISR` 结尾的中断安全 API 都为此提供一个名为 `pxHigherPriorityTaskWoken` 的指针参数。

如果需要执行上下文切换，中断安全 API 会把 `*pxHigherPriorityTaskWoken` 设为 `pdTRUE`。为了可靠检测这一结果，必须在首次使用前把 `pxHigherPriorityTaskWoken` 所指向的变量初始化为 `pdFALSE`。

如果应用程序没有从 ISR 请求上下文切换，则更高优先级的任务会保持就绪状态，直到调度器下一次运行，最坏情况下要等到下一次 Tick 中断。

FreeRTOS API 只会把 `*pxHigherPriorityTaskWoken` 设为 `pdTRUE`，绝不会将其重置为 `pdFALSE`。因此，如果一个 ISR 调用多个 FreeRTOS API，可以把同一个变量传给每次调用的 `pxHigherPriorityTaskWoken` 参数，而且只需在首次使用前把该变量初始化为 `pdFALSE`。

中断安全 API 不在函数内部自动执行上下文切换，原因包括：

- 避免不必要的上下文切换

在任务真正需要处理数据之前，中断可能已经发生多次。例如，一个任务负责处理由中断驱动的 UART 接收到的字符串，只有收到完整字符串后才需要运行。如果每收到一个字符就切换到该任务，会造成浪费。

- 控制执行顺序

中断可能零星发生，而且时机不可预测。经验丰富的 FreeRTOS 用户有时希望在应用程序的特定位置暂时避免这种不可预测的任务切换，尽管也可以通过锁定 FreeRTOS 调度器实现。

- 可移植性

这是可以在所有 FreeRTOS 移植版本上使用的最简单机制。

- 效率

面向小型处理器架构的移植版本可能只允许在 ISR 末尾请求上下文切换。取消这一限制需要额外而复杂的代码。当前机制还允许在同一个 ISR 中多次调用 FreeRTOS API，而不会产生多个上下文切换请求。

- 在 RTOS Tick 中断中执行

后文将介绍如何把应用程序代码加入 RTOS Tick 中断。在 Tick 中断内部尝试上下文切换的结果取决于所用 FreeRTOS 移植版本，即使在最好的情况下，也会造成一次不必要的调度器调用。

`pxHigherPriorityTaskWoken` 参数是可选的。如果不需要，可将其设为 `NULL`。

7.2.5 portYIELD_FROM_ISR() 和 portEND_SWITCHING_ISR() 宏

本节介绍用于从 ISR 请求上下文切换的宏。后续各节会给出实际示例。

`taskYIELD()` 是任务用于请求上下文切换的宏。`portYIELD_FROM_ISR()` 和 `portEND_SWITCHING_ISR()` 则是它的中断安全版本，两者用法相同，执行的操作也相同^[2]。某些 FreeRTOS 移植版本只提供其中一个宏，较新的移植版本通常两者都提供。本书示例使用 `portYIELD_FROM_ISR()`。

```
portEND_SWITCHING_ISR( xHigherPriorityTaskWoken );
```

代码清单 7.1 `portEND_SWITCHING_ISR()` 宏

```
portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
```

代码清单 7.2 `portYIELD_FROM_ISR()` 宏

中断安全 API 输出的 `xHigherPriorityTaskWoken` 变量可以直接传给 `portYIELD_FROM_ISR()`。

如果传给 `portYIELD_FROM_ISR()` 的 `xHigherPriorityTaskWoken` 为 `pdFALSE`（零），宏不会请求上下文切换，也不会产生任何效果。如果该参数不为 `pdFALSE`，宏将请求上下文切换，当前运行任务可能随之改变。中断始终返回到切换后的运行任务，即使它与进入中断前的运行任务不同。

大多数 FreeRTOS 移植版本允许在 ISR 内的任意位置调用 `portYIELD_FROM_ISR()`。少数移植版本（主要面向小型架构）只允许在 ISR 的最末尾调用 `portYIELD_FROM_ISR()`。

7.3 延迟中断处理

通常认为最佳实践是保持 ISR 尽可能短。原因包括：

- 即使任务已被分配非常高的优先级，它们也只有在硬件没有服务任何中断时才会运行。
- ISR 可能会干扰任务的启动时间和执行时间，增加时间“抖动”。
- 根据运行 FreeRTOS 的架构，在执行 ISR 时可能无法接受任何新中断，或至少是部分新中断。
- 应用程序编写者需要考虑并防范任务和 ISR 同时访问变量、外设和内存缓冲区等资源带来的后果。
- 某些 FreeRTOS 移植版本允许中断嵌套，但嵌套会增加复杂性并降低可预测性。中断越短，发生嵌套的可能性就越小。

中断服务程序必须记录中断原因并清除中断。中断所需的其他处理通常可以在任务中执行，从而让中断服务程序尽快退出。这称为“延迟中断处理”，因为中断所需的处理从 ISR 延迟到了任务中。

将中断处理延迟到任务中，还允许应用程序开发者相对于其他任务安排处理优先级，并使用全部 FreeRTOS API 函数。

如果延迟处理中断的任务优先级高于其他所有任务，那么它会立即执行，就像处理工作仍在 ISR 中完成一样。图 7.1 展示了这种情形，其中任务 1 是普通应用程序任务，任务 2 是延迟处理中断的任务。

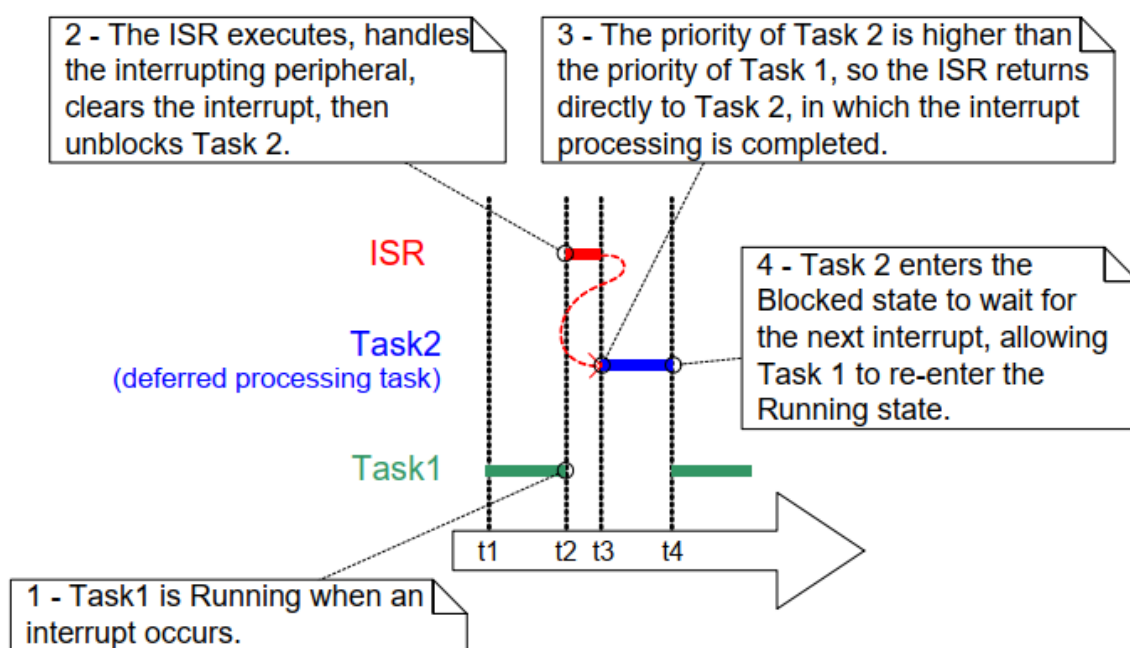


图 7.1 在高优先级任务中完成中断处理

在图 7.1 中，中断处理从时刻 t2 开始，到时刻 t4 才真正结束，但只有 t2 到 t3 之间的时间在 ISR 中度过。如果不采用延迟中断处理，t2 到 t4 之间的全部时间都将在 ISR 中度过。

对于何时应在 ISR 中完成中断所需的全部处理、何时应把部分处理延迟到任务中，并没有绝对规则。在以下情况下，延迟到任务中处理尤其有用：

- 中断所需的处理并非微不足道。例如，如果只需保存模数转换结果，那么几乎肯定应在 ISR 中完成；但如果还要用软件滤波器处理转换结果，则最好在任务中执行滤波。
- 中断处理需要执行无法在 ISR 中完成的操作，例如向控制台输出或分配内存。

- 中断处理不具备确定性，也就是无法预先知道处理需要多长时间。

以下各节将说明并演示本章至此介绍的概念，包括可用于实现延迟中断处理的 FreeRTOS 功能。

7.4 用于同步的二值信号量

中断每次发生时，都可以通过中断安全 API 给出二值信号量并解除一个任务的阻塞，从而有效地使该任务与中断同步。这样，大部分中断事件处理都能放在同步任务中完成，ISR 只保留极快且极短的部分。

如前一节所述，二值信号量可用于把中断处理延迟到任务中^[3]。

如图 7.1 所示，如果中断处理对时序要求极高，可以把延迟处理任务的优先级设得足够高，使它始终抢占系统中的其他任务。随后在 ISR 中调用 `portYIELD_FROM_ISR()`，即可确保 ISR 直接返回到延迟处理中断的任务。这样，整个事件处理过程在时间上连续、不被打断，效果就像全部工作都在 ISR 中完成一样。图 7.2 仍采用图 7.1 的场景，但更新了图中文字，以说明如何用信号量控制延迟处理任务的执行。

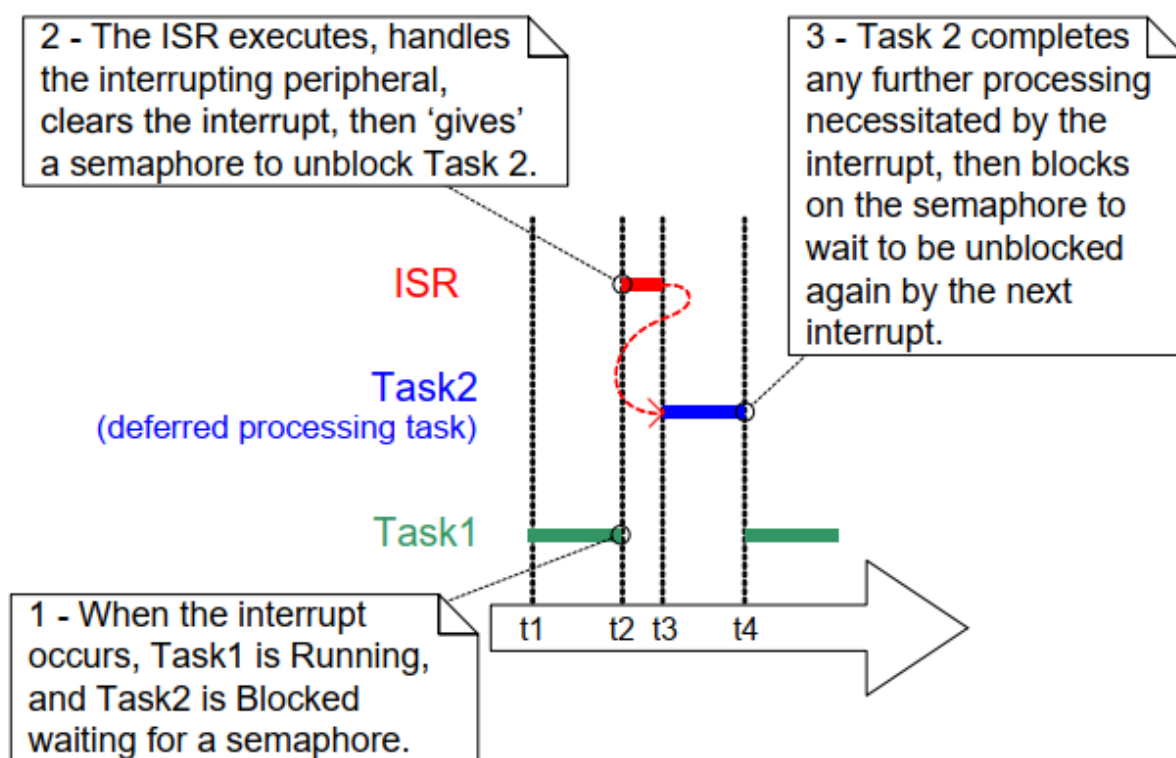


图 7.2 使用二值信号量实现延迟中断处理

延迟处理任务通过阻塞式“获取”信号量进入阻塞状态，等待事件发生。事件发生时，ISR“给出”同一个信号量以解除任务的阻塞，使所需的事件处理得以继续。

“获取”和“给出”信号量是两个概念，其具体含义取决于使用场景。在这里的中断同步场景中，可以把二值信号量看作长度为 1 的队列。队列最多容纳一个项目，所以不是空就是满，这也是“二值”一词的由来。延迟处理中断的任务调用 `xSemaphoreTake()`，相当于尝试从队列读取数据，并指定一段阻塞时间；如果队列为空，任务便进入阻塞状态。事件发生时，ISR 调用 `xSemaphoreGiveFromISR()`，把一个令牌（信号量）放入队列，使队列变满。任务因而离开阻塞状态并取走令牌，队列再次变空。任务处理完事件后再次尝试读取队列；由于队列为空，任务重新进入阻塞状态，等待下一个事件。图 7.3 展示了这一过程。

图 7.3 中，中断即使从未“获取”过信号量，也会“给出”信号量；任务则“获取”信号量，却从不将其归还。因此，这种场景在概念上更像队列的写入和读取。它经常令人困惑，因为它不遵循其他信号量使用场景的规则。在那些场景中，获取信号量的任务必须始终归还信号量，例如第 8 章“资源管理”介绍的用法。

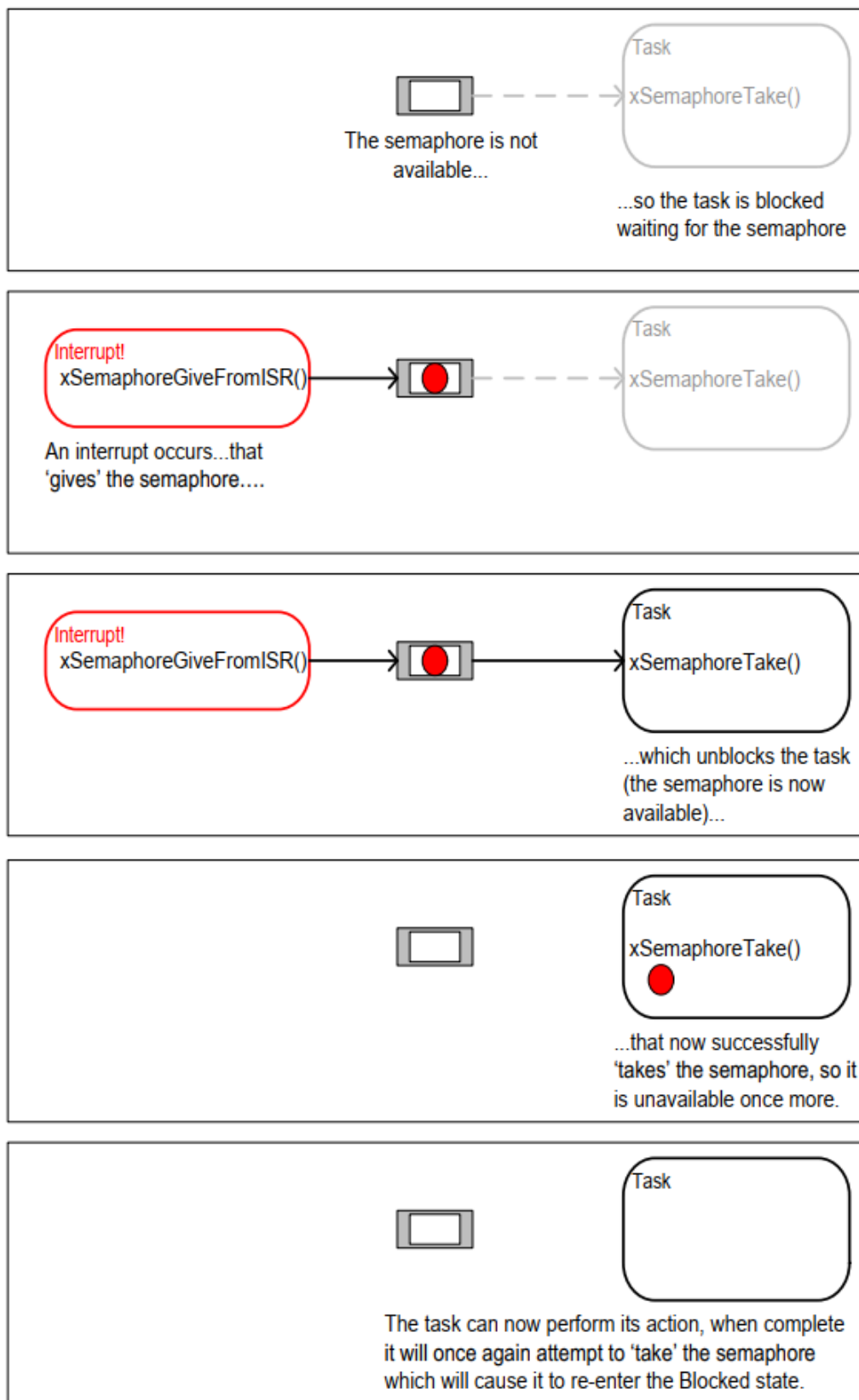


图 7.3 使用二值信号量同步任务与中断

7.4.1 xSemaphoreCreateBinary() API 函数

所有类型的 FreeRTOS 信号量句柄都保存在 `SemaphoreHandle_t` 类型的变量中。使用信号量之前必须先创建它。要创建二值信号量，可以使用 `xSemaphoreCreateBinary()` API 函数^[4]。该函数从 FreeRTOS 堆中动态分配信号量所需的内存；FreeRTOS 还提供 `xSemaphoreCreateBinaryStatic()`，使用调用者预先提供的内存创建二值信号量。

```
SemaphoreHandle_t xSemaphoreCreateBinary( void );
```

代码清单 7.3 `xSemaphoreCreateBinary()` API 函数原型

`xSemaphoreCreateBinary()` 返回值

- 返回值

如果返回 `NULL`，则信号量创建失败，因为 FreeRTOS 没有足够的堆内存用于分配信号量数据结构。

如果返回非 `NULL` 值，则表示信号量创建成功。该返回值是所创建信号量的句柄。

7.4.2 xSemaphoreTake() API 函数

“获取”信号量意味着“取得”或“接收”信号量。只有信号量可用时才能将其获取。

除递归互斥量外，所有类型的 FreeRTOS 信号量都可以用 `xSemaphoreTake()` 函数“获取”。

不得从中断服务程序中调用 `xSemaphoreTake()`。

```
BaseType_t xSemaphoreTake( SemaphoreHandle_t xSemaphore, TickType_t xTicksToWait );
```

代码清单 7.4 `xSemaphoreTake()` API 函数原型

`xSemaphoreTake()` 参数和返回值

- `xSemaphore`

要“获取”的信号量。

信号量通过 `SemaphoreHandle_t` 类型的变量引用。它必须在使用前显式创建。

- `xTicksToWait`

如果信号量尚不可用，任务为等待信号量而保持阻塞状态的最长时间。

如果 `xTicksToWait` 为零，则当信号量不可用时，`xSemaphoreTake()` 将立即返回。

阻塞时间以 Tick 周期为单位，因此它表示的绝对时间取决于 Tick 频率。宏 `pdMS_TO_TICKS()` 可将毫秒转换为 Tick 数。

如果将 `xTicksToWait` 设为 `portMAX_DELAY`，并在 `FreeRTOSConfig.h` 中将 `INCLUDE_vTaskSuspend` 设为 1，任务将无限期等待，不会超时。

- 返回值

有两种可能的返回值：

- `pdPASS`

仅当 `xSemaphoreTake()` 调用成功获取信号量时才返回 `pdPASS`。

如果指定了阻塞时间（`xTicksToWait` 不为零），而信号量又没有立即可用，调用任务可能进入阻塞状态等待；只要信号量在阻塞时间到期前变为可用，调用便会成功。

- `pdFALSE`

信号量不可用，获取失败。

如果指定了阻塞时间（`xTicksToWait` 不为零），调用任务会进入阻塞状态等待信号量，但阻塞时间在信号量变为可用之前已经到期。

7.4.3 xSemaphoreGiveFromISR() API 函数

可以使用 `xSemaphoreGiveFromISR()` 函数“给出”二值信号量和计数信号量^[5]。

`xSemaphoreGiveFromISR()` 是 `xSemaphoreGive()` 的中断安全版本，因此具有本章开头描述的 `pxHigherPriorityTaskWoken` 参数。

```
BaseType_t xSemaphoreGiveFromISR( SemaphoreHandle_t xSemaphore,  
                                   BaseType_t *pxHigherPriorityTaskWoken );
```

代码清单 7.5 `xSemaphoreGiveFromISR()` API 函数原型

`xSemaphoreGiveFromISR()` 参数和返回值

- `xSemaphore`

要“给出”的信号量。

信号量通过 `SemaphoreHandle_t` 类型的变量引用，并且必须在使用前显式创建。

- `pxHigherPriorityTaskWoken`

一个信号量上可能阻塞着一个或多个等待它变为可用的任务。调用 `xSemaphoreGiveFromISR()` 可以使信号量变为可用，从而让等待它的任务离开阻塞状态。如果被解除阻塞的任务优先级高于当前执行的任务（即被中断的任务），`xSemaphoreGiveFromISR()` 会在内部把 `*pxHigherPriorityTaskWoken` 设为 `pdTRUE`。

如果 `xSemaphoreGiveFromISR()` 把该值设为 `pdTRUE`，通常应在退出中断前执行上下文切换，确保中断直接返回到优先级最高的就绪状态任务。

- 返回值

有两种可能的返回值：

- `pdPASS`

仅当 `xSemaphoreGiveFromISR()` 调用成功时才返回 `pdPASS`。

- `pdFAIL`

如果信号量已经可用，就无法再次给出，此时 `xSemaphoreGiveFromISR()` 返回 `pdFAIL`。

示例 7.1 使用二值信号量同步任务与中断

本示例使用二值信号量从中断服务程序中解除任务的阻塞，从而有效地使任务与中断同步。

一个简单的周期性任务每 500 毫秒生成一次软件中断。这里为方便起见使用软件中断，因为在某些目标环境中挂接真实中断较为复杂。代码清单 7.6 给出了周期性任务的实现。请注意，该任务在生成中断前后各输出一条消息，以便从示例输出中观察执行顺序。

```

/* 本示例使用的软件中断编号。所示代码来自 Windows 项目，其中编号 0 到 2
   由 FreeRTOS Windows 移植版本自身使用，因此 3 是应用程序可用的第一个
   编号。 */
#define mainINTERRUPT_NUMBER 3

static void vPeriodicTask( void *pvParameters )
{
    const TickType_t xDelay500ms = pdMS_TO_TICKS( 500UL );

    /* 与大多数任务一样，本任务在无限循环中实现。 */
    for( ;; )
    {
        /* 阻塞到应当再次生成软件中断时。 */
        vTaskDelay( xDelay500ms );

        /* 生成中断，并在生成前后分别输出一条消息，以便从输出中看出执行顺序。

           生成软件中断所用的语法取决于当前 FreeRTOS 移植版本。以下语法仅适用于
           FreeRTOS Windows 移植版本，其中此类中断只是模拟出来的。 */
        vPrintString( "Periodic task - About to generate an interrupt.\r\n" );
        vPortGenerateSimulatedInterrupt( mainINTERRUPT_NUMBER );
        vPrintString( "Periodic task - Interrupt generated.\r\n\r\n\r\n" );
    }
}

```

代码清单 7.6 示例 7.1 中周期性生成软件中断的任务实现

代码清单 7.7 给出了延迟处理中断的任务实现，也就是使用二值信号量与软件中断同步的任务。任务每次循环时同样会输出一条消息，以便运行示例时从输出中清楚看出任务与中断的执行顺序。

需要注意，代码清单 7.7 的代码足以满足示例 7.1，因为其中断由软件生成；但对于硬件外设生成中断的场景，这种实现还不够完善。下一小节将说明如何调整代码结构，使其适用于硬件中断。

```

static void vHandlerTask( void *pvParameters )
{
    /* 与大多数任务一样，本任务在无限循环中实现。 */
    for( ;; )
    {
        /* 使用信号量等待事件。信号量在调度器启动之前创建，因此早于本任务首次
           运行。任务会无限期阻塞，这意味着只有成功获取信号量后此函数调用才会
           返回，所以无需检查 xSemaphoreTake() 的返回值。 */
        xSemaphoreTake( xBinarySemaphore, portMAX_DELAY );

        /* 执行到这里说明事件必定已经发生。处理事件（本例只输出一条消息）。 */
        vPrintString( "Handler task - Processing event.\r\n" );
    }
}

```

代码清单 7.7 示例 7.1 中延迟处理中断的任务（与中断同步的任务）实现

代码清单 7.8 给出了 ISR。它几乎不做其他工作，只负责“给出”信号量，解除延迟处理任务的阻塞。

请注意如何使用 `xHigherPriorityTaskWoken` 变量。它在调用 `xSemaphoreGiveFromISR()` 之前设置为 `pdFALSE`，然后在调用 `portYIELD_FROM_ISR()` 时用作参数。如果 `xHigherPriorityTaskWoken` 等于 `pdTRUE`，则将在 `portYIELD_FROM_ISR()` 宏内部请求上下文切换。

这里的 ISR 原型和请求上下文切换的宏适用于 FreeRTOS Windows 移植版本，其他移植版本可能有所不同。请参阅 FreeRTOS.org 网站上针对相应移植版本的文档页面，以及 FreeRTOS 下载包中的示例，确认所用移植版本要求的语法。

与运行 FreeRTOS 的大多数架构不同，FreeRTOS Windows 移植版本要求 ISR 返回一个值。该移植版本的 `portYIELD_FROM_ISR()` 宏内部包含返回语句，因此代码清单 7.8 没有显式写出返回值。

```
static uint32_t ulExampleInterruptHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken;

    /* 必须把 xHigherPriorityTaskWoken 参数初始化为 pdFALSE。如果需要进行
       上下文切换，中断安全 API 函数会在内部将其设为 pdTRUE。 */
    xHigherPriorityTaskWoken = pdFALSE;

    /* “给出”信号量以解除任务的阻塞，并将 xHigherPriorityTaskWoken 的地址
       作为中断安全 API 函数的 pxHigherPriorityTaskWoken 参数传入。 */
    xSemaphoreGiveFromISR( xBinarySemaphore, &xHigherPriorityTaskWoken );

    /* 将 xHigherPriorityTaskWoken 的值传给 portYIELD_FROM_ISR()。如果
       xSemaphoreGiveFromISR() 在内部把 xHigherPriorityTaskWoken 设为
       pdTRUE，调用 portYIELD_FROM_ISR() 将请求上下文切换。如果该值仍为
       pdFALSE，调用 portYIELD_FROM_ISR() 不会产生任何效果。与大多数
       FreeRTOS 移植版本不同，Windows 移植版本要求 ISR 返回一个值；返回
       语句位于 Windows 版本的 portYIELD_FROM_ISR() 内部。 */
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

代码清单 7.8 示例 7.1 所用软件中断的 ISR

`main()` 函数负责创建二值信号量和任务、安装中断处理程序并启动调度器。代码清单 7.9 给出了它的实现。

安装中断处理程序的函数调用语法特定于 FreeRTOS Windows 移植版本，其他移植版本可能有所不同。请参阅 FreeRTOS.org 网站上针对相应移植版本的文档页面，以及 FreeRTOS 下载包中的示例，确认所用移植版本要求的语法。

```

int main( void )
{
    /* 使用信号量之前必须显式创建它。本例创建一个二值信号量。 */
    xBinarySemaphore = xSemaphoreCreateBinary();

    /* 检查信号量是否创建成功。 */
    if( xBinarySemaphore != NULL )
    {
        /* 创建“处理程序”任务，也就是延迟处理中断并与中断同步的任务。该任务以
           较高优先级创建，以确保退出中断后立即运行。本例选择优先级 3。 */
        xTaskCreate( vHandlerTask, "Handler", 1000, NULL, 3, NULL );

        /* 创建周期性生成软件中断的任务。它的优先级低于处理程序任务，以确保
           处理程序任务每次离开阻塞状态时都能抢占它。 */
        xTaskCreate( vPeriodicTask, "Periodic", 1000, NULL, 1, NULL );

        /* 安装软件中断处理程序。所需语法取决于当前 FreeRTOS 移植版本。这里
           所示的语法仅适用于 FreeRTOS Windows 移植版本，其中此类中断只是
           模拟出来的。 */
        vPortSetInterruptHandler( mainINTERRUPT_NUMBER,
                                   ulExampleInterruptHandler );

        /* 启动调度器，使已创建的任务开始执行。 */
        vTaskStartScheduler();
    }

    /* 正常情况下不应执行到下面这一行。 */
    for( ;; );
}

```

代码清单 7.9 示例 7.1 的 `main()` 实现

图 7.4 是示例 7.1 产生的输出。与预期一致，`vHandlerTask()` 在中断生成后立即进入运行状态，因此它的输出把周期性任务的两条输出分隔开来。图 7.5 给出了进一步说明。

```
C:\WINDOWS\system32\cmd.exe - rtosdemo
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.
```

图 7.4 执行示例 7.1 时产生的输出

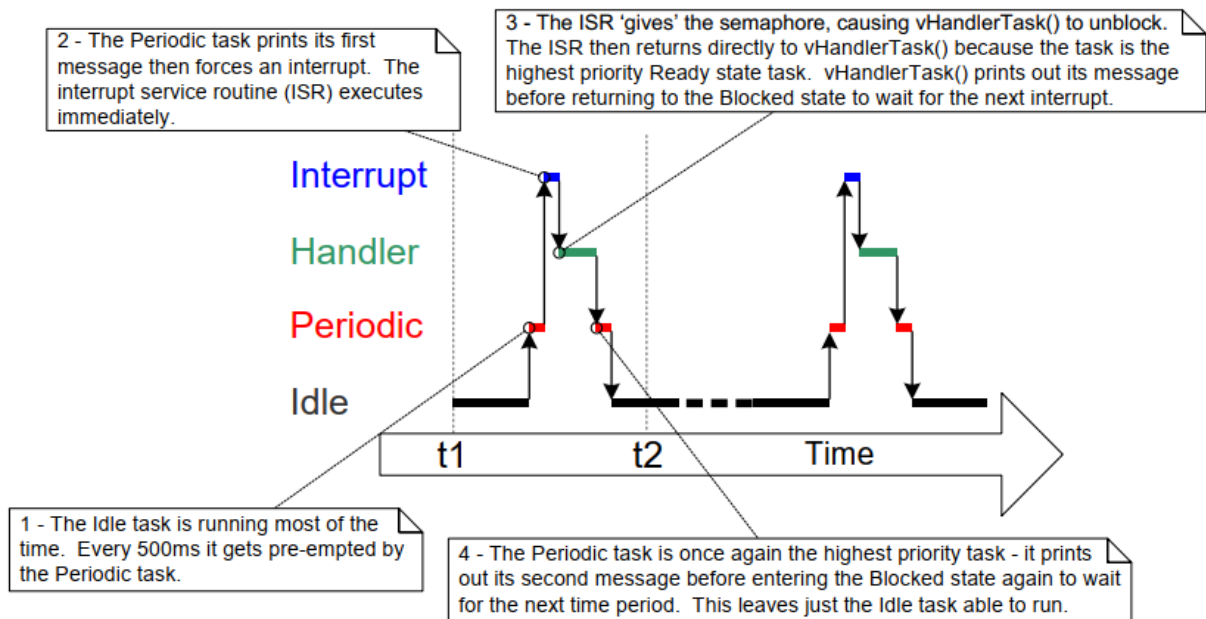


图 7.5 执行示例 7.1 时的执行顺序

7.4.4 改进示例 7.1 中使用的任务的实现

示例 7.1 使用二值信号量来同步任务与中断。执行顺序如下：

1. 中断发生。
2. ISR 执行并“给出”信号量，以解除任务的阻塞。
3. 任务在 ISR 之后立即执行，并“获取”信号量。
4. 任务处理事件，然后再次尝试“获取”信号量。由于另一个中断尚未发生，信号量不可用，任务进入阻塞状态。

示例 7.1 的任务结构只适用于中断频率相对较低的情况。要理解原因，可以设想任务尚未处理完第一个中断时，第二个和第三个中断已经发生：

- 第二个 ISR 执行时，信号量为空，因此 ISR 可以给出信号量。任务处理完第一个事件后，会立即处理第二个事件。图 7.6 展示了这种情形。
 - 第三个 ISR 执行时，信号量已经可用，ISR 无法再次给出信号量，因此任务不会知道第三个事件已经发生。图 7.7 展示了这种情形。
-

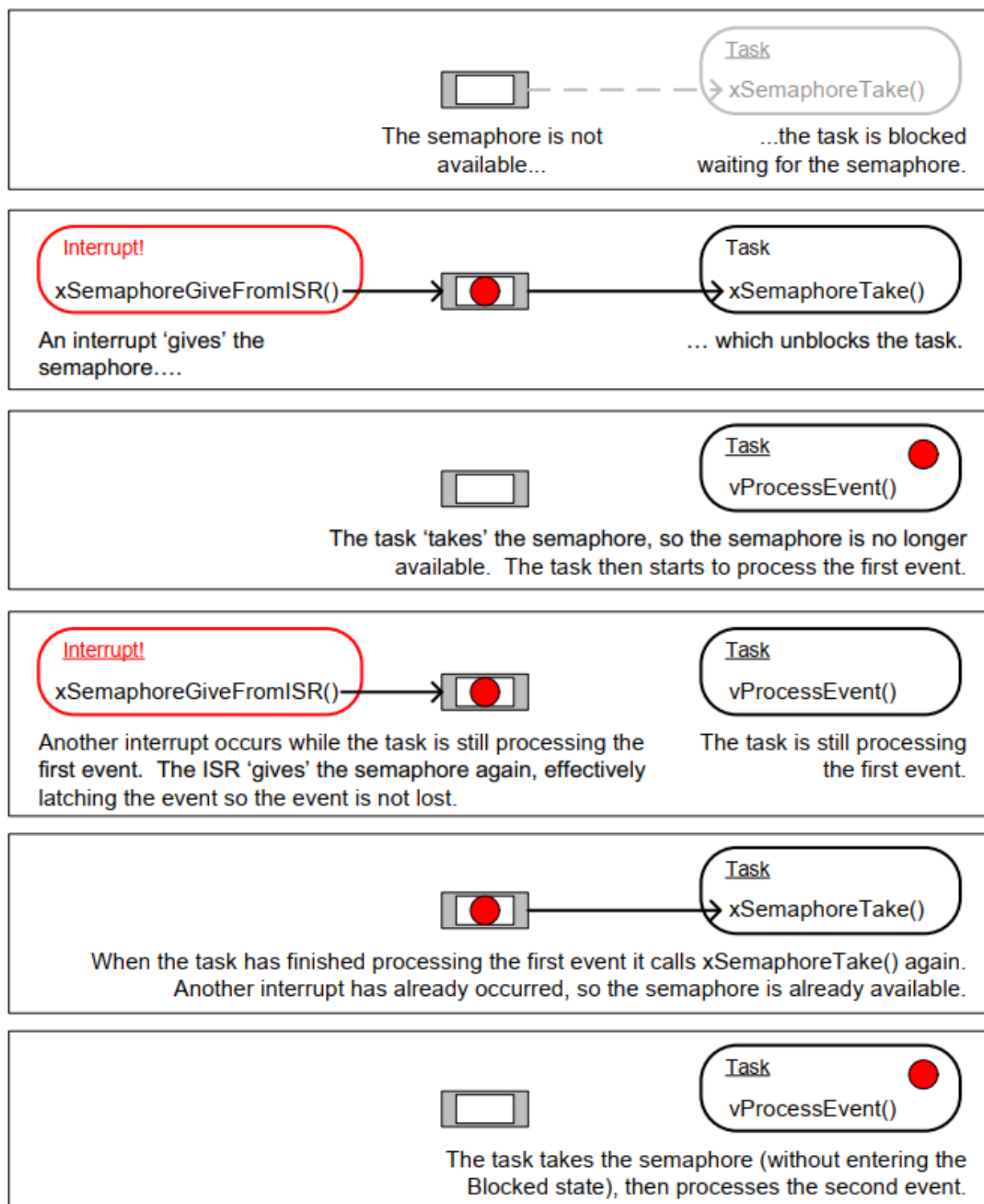


图 7.6 任务处理完第一个事件前又发生一次中断的情形

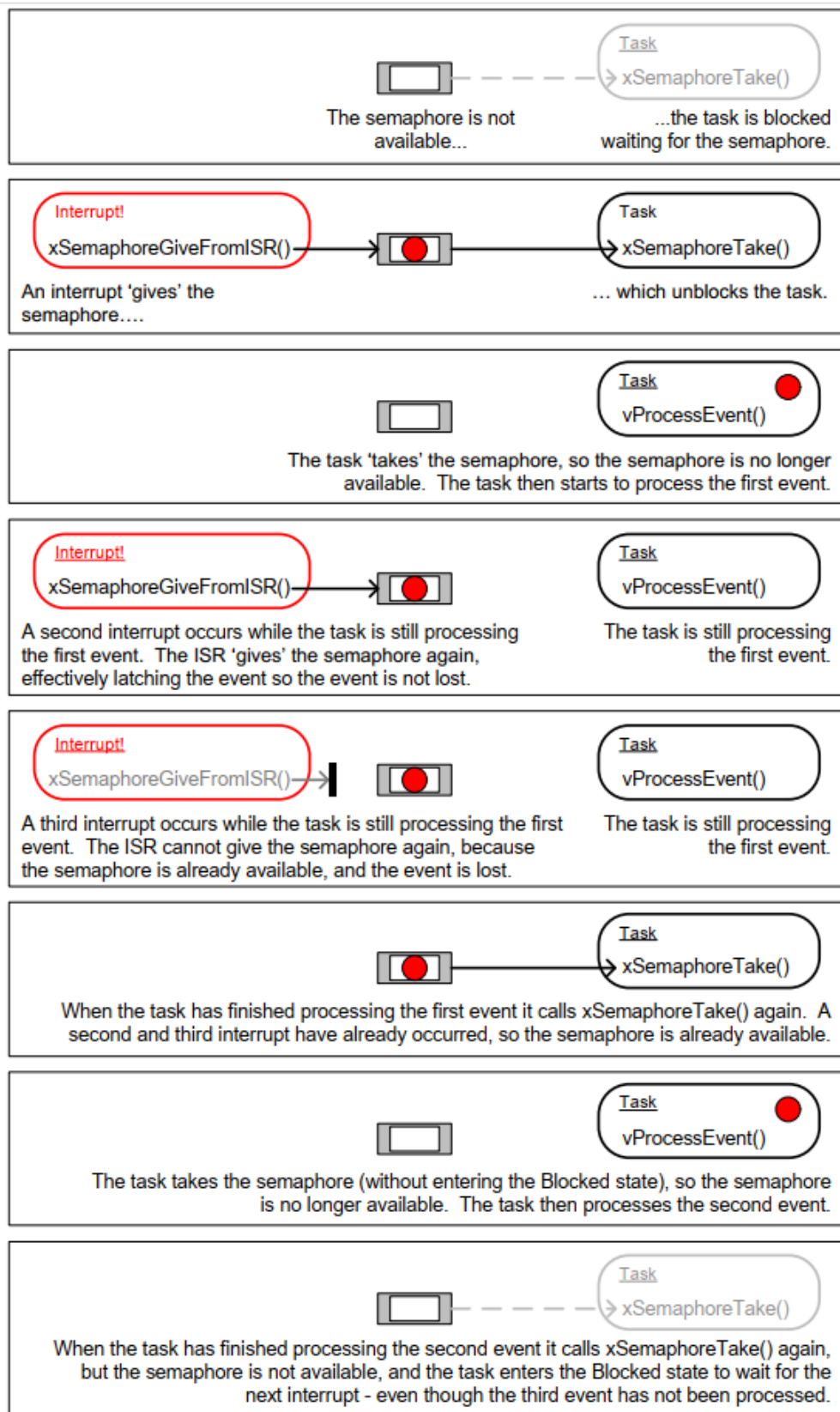


图 7.7 任务处理完第一个事件前又发生两次中断的情形

代码清单 7.7 所示的延迟中断处理任务，在相邻两次 `xSemaphoreTake()` 调用之间只处理一个事件。对于示例 7.1，这样已经足够，因为事件由软件中断产生，发生时刻可以预测。但在实际应用中，中断通常由硬件产生，时机不可预测。因此，为尽量避免遗漏中断，延迟中断处理任务应在相邻两次 `xSemaphoreTake()` 调用之间处理所有已经到达的事件^[6]。代码清单 7.10 以 UART 接收处理程序为例展示了这种结构。它假定 UART 每接收一个字符就产生一次接收中断，并把收到的字符放入硬件 FIFO（硬件缓冲区）。

示例 7.1 的延迟中断处理任务还有一个弱点：调用 `xSemaphoreTake()` 时没有设置超时。任务把 `portMAX_DELAY` 作为 `xTicksToWait` 参数传入，因此会无限期等待信号量变为可用。示例代码经常采用无限等待，因为这样结构更简单、更易理解；但在实际应用中，这通常不是良好做法，因为系统会更难从错误中恢复。例如，假设任务正在等待中断给出信号量，而硬件错误又阻止了中断产生：

- 如果任务无限期等待，它不会察觉错误状态，只会一直等待。
- 如果任务设置了超时，超时到期时 `xSemaphoreTake()` 会返回 `pdFAIL`，任务便可检测并清除错误。代码清单 7.10 也演示了这种处理方式。

```

static void vUARTReceiveHandlerTask( void *pvParameters )
{
    /* xMaxExpectedBlockTime 保存两次中断之间预期的最长时间。 */
    const TickType_t xMaxExpectedBlockTime = pdMS_TO_TICKS( 500 );

    /* 与大多数任务一样，本任务在无限循环中实现。 */
    for( ;; )
    {
        /* UART 接收 (Rx) 中断会“给出”信号量。等待下一次中断的时间最多为
           xMaxExpectedBlockTime 个 Tick。 */
        if( xSemaphoreTake( xBinarySemaphore, xMaxExpectedBlockTime ) == pdPASS )
        {
            /* 已获取信号量。在再次调用 xSemaphoreTake() 前处理所有待处理的 Rx
               事件。每个 Rx 事件都会把一个字符放入 UART 接收 FIFO; 这里假定
               UART_RxCount() 返回 FIFO 中的字符数。 */
            while( UART_RxCount() > 0 )
            {
                /* 假定 UART_ProcessNextRxEvent() 处理一个 Rx 字符，使 FIFO 中
                   的字符数减 1。 */
                UART_ProcessNextRxEvent();
            }

            /* 已无待处理的 Rx 事件 (FIFO 中没有字符)，因此返回循环并调用
               xSemaphoreTake() 等待下一次中断。从此处到调用 xSemaphoreTake()
               之间发生的任何中断都会锁存在信号量中，不会丢失。 */
        }
        else
        {
            /* 未在预期时间内收到事件。检查 UART 是否存在可能阻止其继续生成
               中断的错误状态，并在必要时清除错误。 */
            UART_ClearErrors();
        }
    }
}

```

代码清单 7.10 以 UART 接收处理程序为例的延迟中断处理任务推荐结构

7.5 计数信号量

正如二值信号量可视为长度为 1 的队列，计数信号量也可视为长度大于 1 的队列。任务并不关心队列中存储的数据，只关心其中的项目数。必须在 `FreeRTOSConfig.h` 中将 `configUSE_COUNTING_SEMAPHORES` 设为 1，才能使用计数信号量。

每“给出”一次计数信号量，它所对应的队列就会多占用一个位置。队列中的项目数就是信号量的“计数”值。

计数信号量通常用于两件事：

1. 对事件计数^[7]

在这种场景中，事件处理程序每遇到一个事件就“给出”一次信号量，使计数值加 1。任务每处理一个事件就“获取”一次信号量，使计数值减 1。因此，计数值等于已发生事件数与已处理事件数之差。图 7.8 展示了这一机制。

用于计数事件的计数信号量创建时初始计数值为零。

2. 资源管理

在这种场景中，计数值表示可用资源的数量。任务要取得资源的控制权，必须先获取信号量，使信号量计数值减 1。计数值为零时，表示没有空闲资源。任务用完资源后“给出”信号量，使计数值加 1。

用于管理资源的计数信号量，其初始计数值应等于可用资源数量。第 8 章将介绍如何使用信号量管理资源。

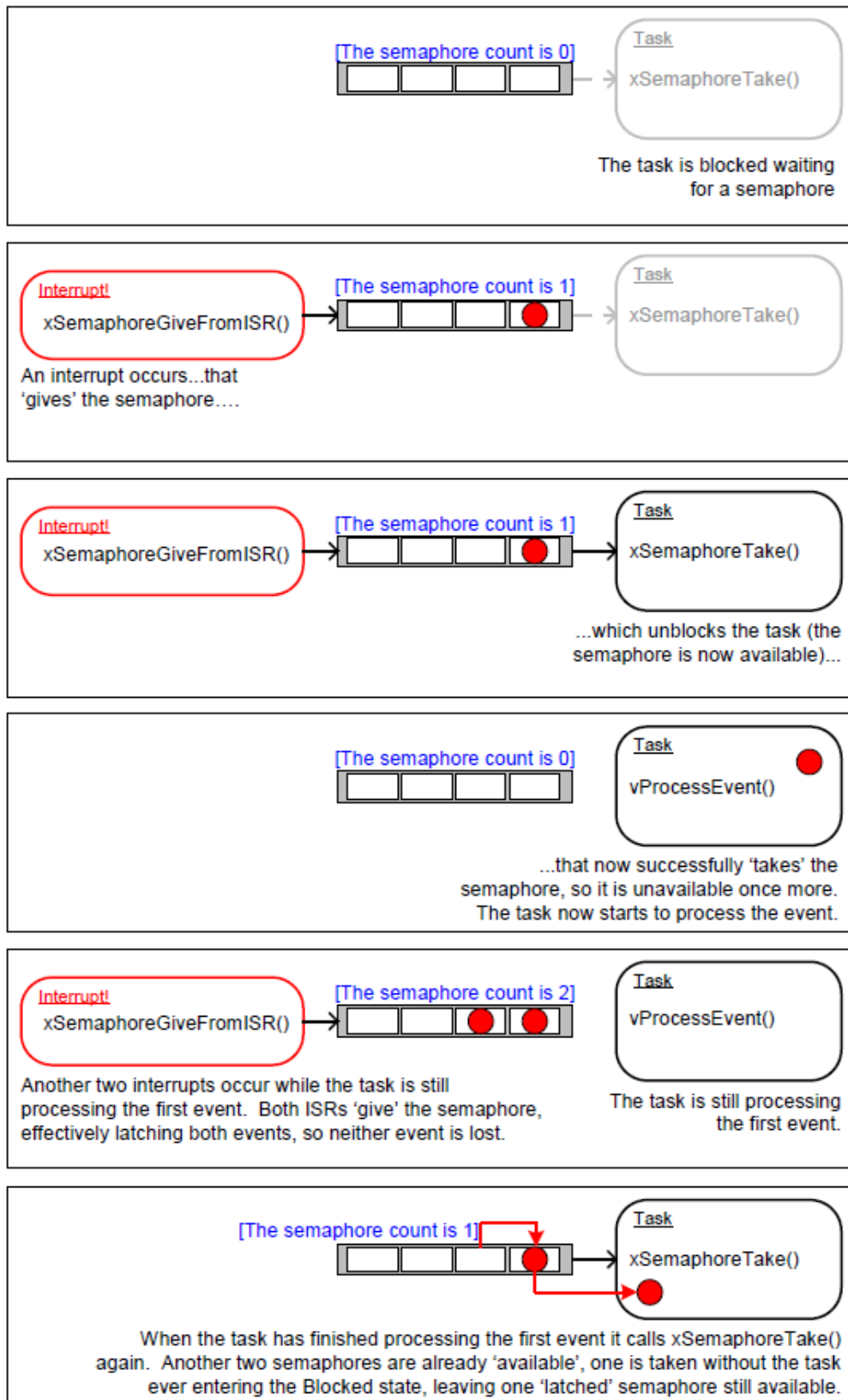


图 7.8 使用计数信号量对事件计数

7.5.1 xSemaphoreCreateCounting() API 函数

所有类型的 FreeRTOS 信号量句柄都保存在 `SemaphoreHandle_t` 类型的变量中。使用信号量之前必须先创建它。要创建计数信号量，可以使用 `xSemaphoreCreateCounting()` API 函数。该函数从 FreeRTOS 堆中动态分配信号量所需的内存；FreeRTOS 还提供 `xSemaphoreCreateCountingStatic()`，使用调用者预先提供的内存创建计数信号量。

```
SemaphoreHandle_t xSemaphoreCreateCounting( UBaseType_t uxMaxCount,
                                             UBaseType_t uxInitialCount );
```

代码清单 7.11 `xSemaphoreCreateCounting()` API 函数原型

`xSemaphoreCreateCounting()` 参数和返回值

- `uxMaxCount`

信号量可以达到的最大计数值。沿用队列类比，`uxMaxCount` 实际上就是队列长度。

信号量用于对事件计数或锁存事件时，`uxMaxCount` 是能够锁存的最大事件数。

当信号量用于管理对一组资源的访问时，`uxMaxCount` 应设置为可用的资源总数。

- `uxInitialCount`

信号量创建后的初始计数值。

信号量用于对事件计数或锁存事件时，应把 `uxInitialCount` 设为零，因为创建信号量时假定尚未发生任何事件。

信号量用于管理一组资源的访问时，应把 `uxInitialCount` 设为 `uxMaxCount`，因为创建信号量时假定所有资源均可用。

- 返回值

如果返回 `NULL`，则信号量创建失败，因为 FreeRTOS 没有足够的堆内存用于分配信号量数据结构。第 3 章提供了更多堆内存管理信息。

如果返回非 `NULL` 值，则表示信号量创建成功。该返回值是所创建信号量的句柄。

示例 7.2 使用计数信号量同步任务与中断

示例 7.2 使用计数信号量取代二值信号量，改进了示例 7.1。`main()` 不再调用 `xSemaphoreCreateBinary()`，而改为调用 `xSemaphoreCreateCounting()`。代码清单 7.12 给出了新的 API 调用。

```
/* 使用信号量之前必须显式创建它。本例创建一个计数信号量，其最大计数值为 10，
   初始计数值为 0。 */
xCountingSemaphore = xSemaphoreCreateCounting( 10, 0 );
```

代码清单 7.12 示例 7.2 中创建计数信号量的 `xSemaphoreCreateCounting()` 调用

为模拟多个事件快速连续发生，中断服务程序改为在每次中断中多次“给出”信号量。每个事件都会锁存在信号量的计数值中。代码清单 7.13 给出了修改后的中断服务程序。

```
static uint32_t ulExampleInterruptHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken;

    /* 必须把 xHigherPriorityTaskWoken 参数初始化为 pdFALSE。如果需要进行
       上下文切换，中断安全 API 函数会在内部将其设为 pdTRUE。 */
    xHigherPriorityTaskWoken = pdFALSE;

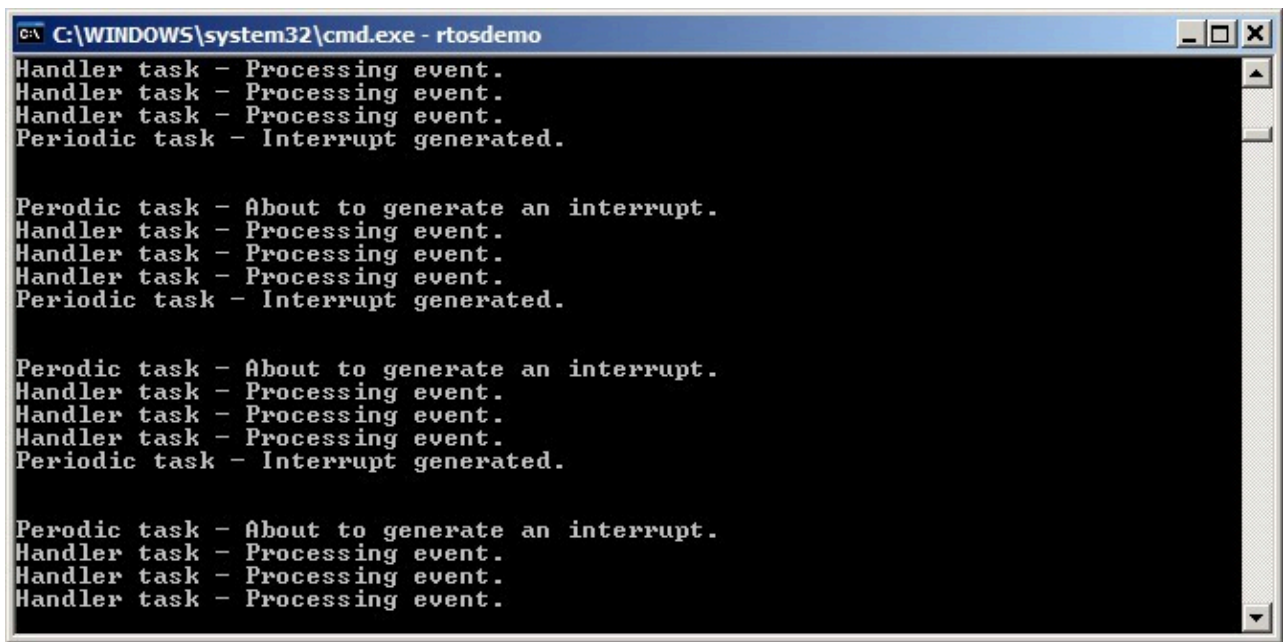
    /* 多次“给出”信号量。第一次会解除延迟中断处理任务的阻塞，后续几次用于
       演示信号量如何锁存事件，让延迟处理中断的任务依次处理各个事件而不发生
       遗漏。虽然本例在一次中断中模拟这些事件，但效果相当于处理器接收到多次
       中断。 */
    xSemaphoreGiveFromISR( xCountingSemaphore, &xHigherPriorityTaskWoken );
    xSemaphoreGiveFromISR( xCountingSemaphore, &xHigherPriorityTaskWoken );
    xSemaphoreGiveFromISR( xCountingSemaphore, &xHigherPriorityTaskWoken );

    /* 将 xHigherPriorityTaskWoken 的值传给 portYIELD_FROM_ISR()。如果
       xSemaphoreGiveFromISR() 在内部把 xHigherPriorityTaskWoken 设为
       pdTRUE，调用 portYIELD_FROM_ISR() 将请求上下文切换。如果该值仍为
       pdFALSE，调用 portYIELD_FROM_ISR() 不会产生任何效果。与大多数
       FreeRTOS 移植版本不同，Windows 移植版本要求 ISR 返回一个值；返回
       语句位于 Windows 版本的 portYIELD_FROM_ISR() 内部。 */
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

代码清单 7.13 示例 7.2 使用的中断服务程序实现

其他函数与示例 7.1 相同，无需修改。

图 7.9 是执行示例 7.2 时产生的输出。可以看到，每次产生中断后，延迟处理中断的任务都会处理全部三个模拟事件。事件锁存在信号量计数值中，因此任务能够逐个处理而不会遗漏。



```
C:\WINDOWS\system32\cmd.exe - rtosdemo
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
```

图 7.9 执行示例 7.2 时产生的输出

7.6 将工作延迟到 RTOS 守护任务

前面介绍的延迟中断处理示例要求应用程序开发者为每个采用这种技术的中断创建一个任务。也可以使用 `xTimerPendFunctionCallFromISR()` ^[8] API 函数，把中断处理延迟到 RTOS 守护任务中执行，从而不必为每个中断分别创建任务。把中断处理延迟到守护任务称为“集中式延迟中断处理”。

第 6 章介绍了软件定时器相关的 FreeRTOS API 如何通过定时器命令队列向守护任务发送命令。`xTimerPendFunctionCall()` 和 `xTimerPendFunctionCallFromISR()` 使用同一个队列向守护任务发送“执行函数”命令，指定的函数随后在守护任务上下文中执行。

集中式延迟中断处理的优点包括：

- 更低的资源使用

不必为每个需要延迟处理的中断创建单独的任务。

- 简化的用户模型

延迟中断处理函数就是普通的 C 函数。

集中式延迟中断处理的缺点包括：

- 灵活性较低

无法为各个延迟中断处理函数单独设置优先级。所有这类函数都以守护任务的优先级执行。如第 6 章所述，守护任务优先级由 `FreeRTOSConfig.h` 中的编译时配置常量

`configTIMER_TASK_PRIORITY` 设置。

- 确定性较低

`xTimerPendFunctionCallFromISR()` 把命令发送到定时器命令队列尾部。守护任务会先处理队列中已有的命令，再处理该函数发出的“执行函数”命令。

不同中断的时序约束各异，因此同一个应用程序通常会同时使用两种延迟中断处理方法。

7.6.1 xTimerPendFunctionCallFromISR() API 函数

`xTimerPendFunctionCallFromISR()` 是 `xTimerPendFunctionCall()` 的中断安全版本。这两个 API 函数都可以让应用程序提供的函数由 RTOS 守护任务执行。待执行函数及其输入参数通过定时器命令队列发送给守护任务，因此函数实际何时执行，取决于守护任务相对于其他应用程序任务的优先级。

```
BaseType_t xTimerPendFunctionCallFromISR( PendedFunction_t
                                           xFunctionToPend,
                                           void *pvParameter1,
                                           uint32_t ulParameter2,
                                           BaseType_t *pxHigherPriorityTaskWoken );
```

代码清单 7.14 `xTimerPendFunctionCallFromISR()` API 函数原型

```
void vPendableFunction( void *pvParameter1, uint32_t ulParameter2 );
```

代码清单 7.15 传给 `xTimerPendFunctionCallFromISR()` 的函数必须符合的原型

`xTimerPendFunctionCallFromISR()` 参数和返回值

- `xFunctionToPend`

指向要在守护任务中执行的函数的指针，实际使用时就是函数名。函数原型必须与代码清单 7.15 相同。

- `pvParameter1`

守护任务执行指定函数时，作为该函数 `pvParameter1` 参数传入的值。该参数类型为 `void *`，因此可用于传递任意数据类型。例如，可以把整数直接转换为 `void *`，也可以让 `void *` 指向一个结构体。

- `ulParameter2`

守护任务执行指定函数时，作为该函数 `ulParameter2` 参数传入的值。

- `pxHigherPriorityTaskWoken`

`xTimerPendFunctionCallFromISR()` 会写入定时器命令队列。如果 RTOS 守护任务正处于阻塞状态，等待队列中出现数据，这次写入会使它离开阻塞状态。如果守护任务的优先级高于当前执行的任务（即被中断的任务），`xTimerPendFunctionCallFromISR()` 会在内部把 `*pxHigherPriorityTaskWoken` 设为 `pdTRUE`。

如果 `xTimerPendFunctionCallFromISR()` 把该值设为 `pdTRUE`，必须在退出中断前执行上下文切换。这样，中断会直接返回到守护任务，因为此时它是优先级最高的就绪状态任务。

- 返回值

有两种可能的返回值：

- `pdPASS`

如果“执行函数”命令已成功写入定时器命令队列，则返回 `pdPASS`。

- `pdFAIL`

如果定时器命令队列已满，无法写入“执行函数”命令，则返回 `pdFAIL`。第 6 章介绍了如何设置定时器命令队列长度。

示例 7.3 集中式延迟中断处理

示例 7.3 实现了与示例 7.1 类似的功能，但既不使用信号量，也不专门创建任务来完成中断所需的处理，而是由 RTOS 守护任务执行这些工作。

代码清单 7.16 给出了示例 7.3 的中断服务程序。它调用 `xTimerPendFunctionCallFromISR()`，把函数 `vDeferredHandlingFunction()` 的指针传给守护任务。延迟的中断处理由 `vDeferredHandlingFunction()` 完成。

中断服务程序每执行一次，就把变量 `ulParameterValue` 加 1。调用 `xTimerPendFunctionCallFromISR()` 时，该变量作为 `ulParameter2` 传入；随后守护任务执行 `vDeferredHandlingFunction()` 时，同一个值又成为 `vDeferredHandlingFunction()` 的 `ulParameter2` 参数。本示例不使用另一个参数 `pvParameter1`。

```
static uint32_t ulExampleInterruptHandler( void )
{
    static uint32_t ulParameterValue = 0;
    BaseType_t xHigherPriorityTaskWoken;

    /* 必须把 xHigherPriorityTaskWoken 参数初始化为 pdFALSE。如果需要进行
       上下文切换，中断安全 API 函数会在内部将其设为 pdTRUE。 */
    xHigherPriorityTaskWoken = pdFALSE;

    /* 把中断延迟处理函数的指针发送给守护任务。延迟处理函数的 pvParameter1
       参数未使用，因此设为 NULL。ulParameter2 参数用于传递一个数值，该值
       在中断处理程序每次执行时加 1。 */
    xTimerPendFunctionCallFromISR( vDeferredHandlingFunction, /* 要执行的函数 */
                                   NULL, /* 未使用 */
                                   ulParameterValue, /* 递增值。 */
                                   &xHigherPriorityTaskWoken );

    ulParameterValue++;

    /* 将 xHigherPriorityTaskWoken 的值传给 portYIELD_FROM_ISR()。如果
       xTimerPendFunctionCallFromISR() 在内部把它设为 pdTRUE，调用
       portYIELD_FROM_ISR() 将请求上下文切换。如果该值仍为 pdFALSE，调用
       portYIELD_FROM_ISR() 不会产生任何效果。与大多数 FreeRTOS 移植版本
       不同，Windows 移植版本要求 ISR 返回一个值；返回语句位于 Windows
       版本的 portYIELD_FROM_ISR() 内部。 */
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

代码清单 7.16 示例 7.3 使用的软件中断处理程序

代码清单 7.17 给出了 `vDeferredHandlingFunction()` 的实现。该函数输出一段固定文本和 `ulParameter2` 参数的值。

即使本示例只实际使用一个参数，`vDeferredHandlingFunction()` 仍必须符合代码清单 7.15 所示的原型。

```
static void vDeferredHandlingFunction( void *pvParameter1, uint32_t ulParameter2 )
{
    /* 处理事件。本例只输出一条消息和 ulParameter2 的值，不使用
       pvParameter1。 */
    vPrintStringAndNumber( "Handler function - Processing event ", ulParameter2 );
}
```

代码清单 7.17 示例 7.3 中执行中断所需处理的函数

代码清单 7.18 给出了示例 7.3 的 `main()` 函数。它比示例 7.1 的 `main()` 更简单，因为无需创建信号量，也无需创建专门执行延迟中断处理的任务。

`vPeriodicTask()` 是周期性生成软件中断的任务。创建它时使用低于守护任务的优先级，以确保守护任务一离开阻塞状态就能抢占该任务。

```
int main( void )
{
    /* 生成软件中断的任务以低于守护任务的优先级创建。守护任务优先级由
       FreeRTOSConfig.h 中的编译时配置常量 configTIMER_TASK_PRIORITY
       设置。 */
    const UBaseType_t ulPeriodicTaskPriority = configTIMER_TASK_PRIORITY - 1;

    /* 创建周期性生成软件中断的任务。 */
    xTaskCreate( vPeriodicTask, "Periodic", 1000, NULL, ulPeriodicTaskPriority,
                NULL );

    /* 安装软件中断处理程序。所需语法取决于当前 FreeRTOS 移植版本。这里
       所示的语法仅适用于 FreeRTOS Windows 移植版本，其中此类中断只是
       模拟出来的。 */
    vPortSetInterruptHandler( mainINTERRUPT_NUMBER, ulExampleInterruptHandler );

    /* 启动调度器，使已创建的任务开始执行。 */
    vTaskStartScheduler();

    /* 正常情况下不应执行到下面这一行。 */
    for( ;; );
}
```

代码清单 7.18 示例 7.3 的 `main()` 实现

图 7.10 是示例 7.3 产生的输出。守护任务优先级高于生成软件中断的任务，因此中断一生成，守护任务就会立即执行 `vDeferredHandlingFunction()`。于是，`vDeferredHandlingFunction()` 的输出出现在周期性任务的两条输出之间，效果与使用信号量解除专用延迟中断处理任务的阻塞相同。图 7.11 给出了进一步说明。

```
C:\Windows\system32\cmd.exe - rtsdemo

C:\temp>rtsdemo
Periodic task - About to generate an interrupt.
Handler function - Processing event 0
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler function - Processing event 1
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler function - Processing event 2
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler function - Processing event 3
Periodic task - Interrupt generated.
```

图 7.10 执行示例 7.3 时产生的输出

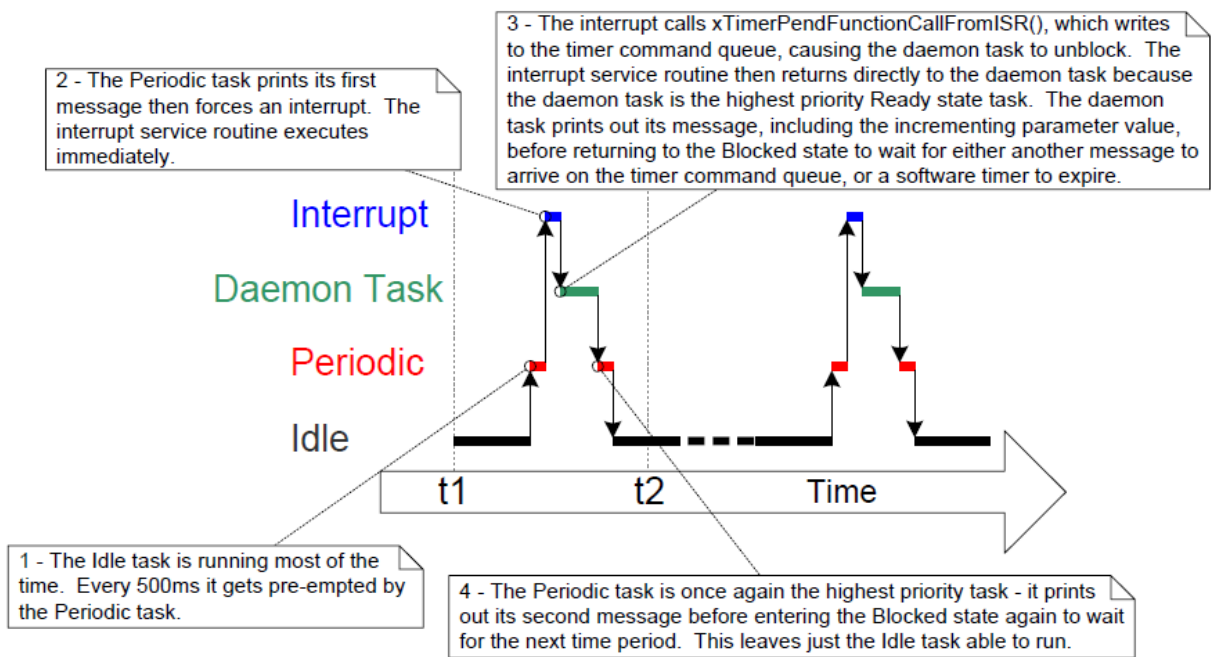


图 7.11 执行示例 7.3 时的执行顺序

7.7 在中断服务程序中使用队列

二值信号量和计数信号量用于传递事件，队列则既能传递事件，也能传输数据。

`xQueueSendToFrontFromISR()` 是 `xQueueSendToFront()` 的中断安全版本，`xQueueSendToBackFromISR()` 是 `xQueueSendToBack()` 的中断安全版本，`xQueueReceiveFromISR()` 则是 `xQueueReceive()` 的中断安全版本。

7.7.1 xQueueSendToFrontFromISR() 和 xQueueSendToBackFromISR() API 函数

```
BaseType_t xQueueSendToFrontFromISR( QueueHandle_t xQueue,
                                     const void *pvItemToQueue
                                     BaseType_t *pxHigherPriorityTaskWoken );
```

代码清单 7.19 `xQueueSendToFrontFromISR()` API 函数原型

```
BaseType_t xQueueSendToBackFromISR( QueueHandle_t xQueue,
                                     const void *pvItemToQueue
                                     BaseType_t *pxHigherPriorityTaskWoken );
```

代码清单 7.20 `xQueueSendToBackFromISR()` API 函数原型

`xQueueSendFromISR()` 和 `xQueueSendToBackFromISR()` 功能相同。

`xQueueSendToFrontFromISR()` 和 `xQueueSendToBackFromISR()` 参数和返回值

- `xQueue`

要向其中发送（写入）数据的队列句柄。该句柄由创建队列时调用的 `xQueueCreate()` 返回。

- `pvItemToQueue`

指向要放入队列的项目。

创建队列时已经定义了每个项目的大小，因此函数会从 `pvItemToQueue` 指向的位置复制相应字节数到队列存储区。

- `pxHigherPriorityTaskWoken`

一个队列上可能阻塞着一个或多个等待数据到达的任务。调用 `xQueueSendToFrontFromISR()` 或 `xQueueSendToBackFromISR()` 可以使数据变为可用，让这些任务离开阻塞状态。如果被解除阻塞的任务优先级高于当前执行的任务（即被中断的任务），API 函数会在内部把

`*pxHigherPriorityTaskWoken` 设为 `pdTRUE`。

如果 `xQueueSendToFrontFromISR()` 或 `xQueueSendToBackFromISR()` 把该值设为 `pdTRUE`，应在退出中断前执行上下文切换，确保中断直接返回到优先级最高的就绪状态任务。

- 返回值

有两种可能的返回值：

- `pdPASS`

仅当数据已成功发送到队列时才返回 `pdPASS`。

- `errQUEUE_FULL`

如果由于队列已满而无法发送数据，则返回 `errQUEUE_FULL`。

7.7.2 从 ISR 使用队列时的注意事项

队列为从中断向任务传递数据提供了简便方法，但当数据以较高频率到达时，这种方法效率不高。

FreeRTOS 下载包中的许多演示应用程序都包含一个简单的 UART 驱动程序，它通过队列逐个传出 UART 接收 ISR 收到的字符。这些演示使用队列有两个目的：展示如何从 ISR 使用队列，以及有意增加系统负载以测试 FreeRTOS 移植版本。这样的 ISR 并不代表高效设计。除非数据到达速度很慢，否则不建议在生产代码中照搬这种技术。更适合生产代码的高效方法包括：

- 使用直接内存访问（DMA）硬件接收并缓冲字符。这种方法几乎没有软件开销。检测到传输结束中断后，再用直接任务通知^[9]解除负责处理缓冲区的任务的阻塞。
- 把每个接收到的字符复制到线程安全的 RAM 缓冲区^[10]。收到完整消息或检测到传输结束中断后，同样可以用直接任务通知解除负责处理缓冲区的任务的阻塞。
- 直接在 ISR 中处理收到的字符，再通过队列把处理结果而不是原始数据发送给任务。图 5.4 已经演示了这种方法。

示例 7.4 在中断中向队列发送数据并从队列接收数据

本示例演示如何在同一个中断中使用 `xQueueSendToBackFromISR()` 和 `xQueueReceiveFromISR()`。与前面的示例一样，为方便起见，这里使用软件生成中断。

程序创建一个周期性任务，每 200 毫秒向队列发送五个数字。只有五个值全部发送完毕后，任务才生成软件中断。代码清单 7.21 给出了该任务的实现。

```
static void vIntegerGenerator( void *pvParameters )
{
    TickType_t xLastExecutionTime;
    uint32_t ulValueToSend = 0;
    int i;

    /* 初始化 vTaskDelayUntil() 调用使用的变量。 */
    xLastExecutionTime = xTaskGetTickCount();

    for( ;; )
    {
        /* 这是一个周期性任务。阻塞到下一次运行时刻，每 200 毫秒执行一次。 */
        vTaskDelayUntil( &xLastExecutionTime, pdMS_TO_TICKS( 200 ) );

        /* 向队列发送五个数字，每个值都比前一个大 1。中断服务程序会从队列中
           读取这些数字，并且每次都会清空队列，因此即使不指定阻塞时间，也能
           保证本任务成功写入全部五个值。 */
        for( i = 0; i < 5; i++ )
        {
            xQueueSendToBack( xIntegerQueue, &ulValueToSend, 0 );
            ulValueToSend++;
        }

        /* 生成中断，让中断服务程序从队列读取这些值。生成软件中断所用的语法
           取决于当前 FreeRTOS 移植版本。以下语法仅适用于 FreeRTOS Windows
           移植版本，其中此类中断只是模拟出来的。 */
        vPrintString( "Generator task - About to generate an interrupt.\r\n" );
        vPortGenerateSimulatedInterrupt( mainINTERRUPT_NUMBER );
        vPrintString( "Generator task - Interrupt generated.\r\n\r\n\r\n" );
    }
}
```

代码清单 7.21 示例 7.4 中写队列任务的实现

中断服务程序反复调用 `xQueueReceiveFromISR()`，直到读出周期性任务写入的全部值，队列变空。每个接收值的最低两位用作字符串数组的索引，然后调用 `xQueueSendFromISR()`，把对应字符串的指针发送到另一个队列。代码清单 7.22 给出了中断服务程序的实现。

```

static uint32_t ulExampleInterruptHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken;
    uint32_t ulReceivedNumber;

    /* 字符串声明为 static const, 以确保它们不在中断服务程序的栈上分配,
       因而在中断服务程序未执行时仍然存在。 */

    static const char *pcStrings[] =
    {
        "String 0\r\n",
        "String 1\r\n",
        "String 2\r\n",
        "String 3\r\n"
    };

    /* 与往常一样, 把 xHigherPriorityTaskWoken 初始化为 pdFALSE, 以便检测
       中断安全 API 函数是否在内部将其设为 pdTRUE。中断安全 API 只能把
       xHigherPriorityTaskWoken 设为 pdTRUE, 因此 xQueueReceiveFromISR()
       和 xQueueSendToBackFromISR() 调用可以安全地共用同一个变量。 */
    xHigherPriorityTaskWoken = pdFALSE;

    /* 不断读取队列, 直到队列为空。 */
    while( xQueueReceiveFromISR( xIntegerQueue,
                                &ulReceivedNumber,
                                &xHigherPriorityTaskWoken ) != errQUEUE_EMPTY )
    {
        /* 只保留接收值的最低两位 (取值为 0 到 3), 再把截断后的值作为
           pcStrings[] 数组索引, 选出要发送到另一个队列的字符串 (char *)。 */
        ulReceivedNumber &= 0x03;
        xQueueSendToBackFromISR( xStringQueue,
                                &pcStrings[ ulReceivedNumber ],
                                &xHigherPriorityTaskWoken );
    }

    /* 如果从 xIntegerQueue 接收数据使一个任务离开阻塞状态, 并且该任务的
       优先级高于运行状态任务, xQueueReceiveFromISR() 会在内部把
       xHigherPriorityTaskWoken 设为 pdTRUE。

       如果向 xStringQueue 发送数据使一个任务离开阻塞状态, 并且该任务的
       优先级高于运行状态任务, xQueueSendToBackFromISR() 会在内部把
       xHigherPriorityTaskWoken 设为 pdTRUE。

       xHigherPriorityTaskWoken 作为参数传给 portYIELD_FROM_ISR()。如果它
       等于 pdTRUE, 调用 portYIELD_FROM_ISR() 将请求上下文切换; 如果仍为
       pdFALSE, 则调用不会产生任何效果。

       Windows 移植版本使用的 portYIELD_FROM_ISR() 实现包含返回语句, 因此
       本函数没有显式返回值。 */
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}

```

代码清单 7.22 示例 7.4 使用的中断服务程序实现

负责接收中断服务程序所发字符指针的任务会阻塞在队列上，直到消息到达，并输出收到的每个字符串。代码清单 7.23 给出了该任务的实现。

```
static void vStringPrinter( void *pvParameters )
{
    char *pcString;

    for( ;; )
    {
        /* 阻塞在队列上，等待数据到达。 */
        xQueueReceive( xStringQueue, &pcString, portMAX_DELAY );

        /* 输出收到的字符串。 */
        vPrintString( pcString );
    }
}
```

代码清单 7.23 示例 7.4 中输出从中断服务程序收到的字符串的任务

与往常一样，`main()` 在启动调度器之前创建所需的队列和任务。代码清单 7.24 给出了它的实现。

```
int main( void )
{
    /* 使用队列之前必须先创建它。本例创建两个队列：一个保存 uint32_t 类型
       的变量，另一个保存 char * 类型的变量；两者最多都可容纳 10 个项目。
       实际应用应检查返回值，确保队列创建成功。 */
    xIntegerQueue = xQueueCreate( 10, sizeof( uint32_t ) );
    xStringQueue = xQueueCreate( 10, sizeof( char * ) );

    /* 创建通过队列向中断服务程序传递整数的任务，优先级为 1。 */
    xTaskCreate( vIntegerGenerator, "IntGen", 1000, NULL, 1, NULL );

    /* 创建负责输出中断服务程序所发字符串的任务，使用更高的优先级 2。 */
    xTaskCreate( vStringPrinter, "String", 1000, NULL, 2, NULL );

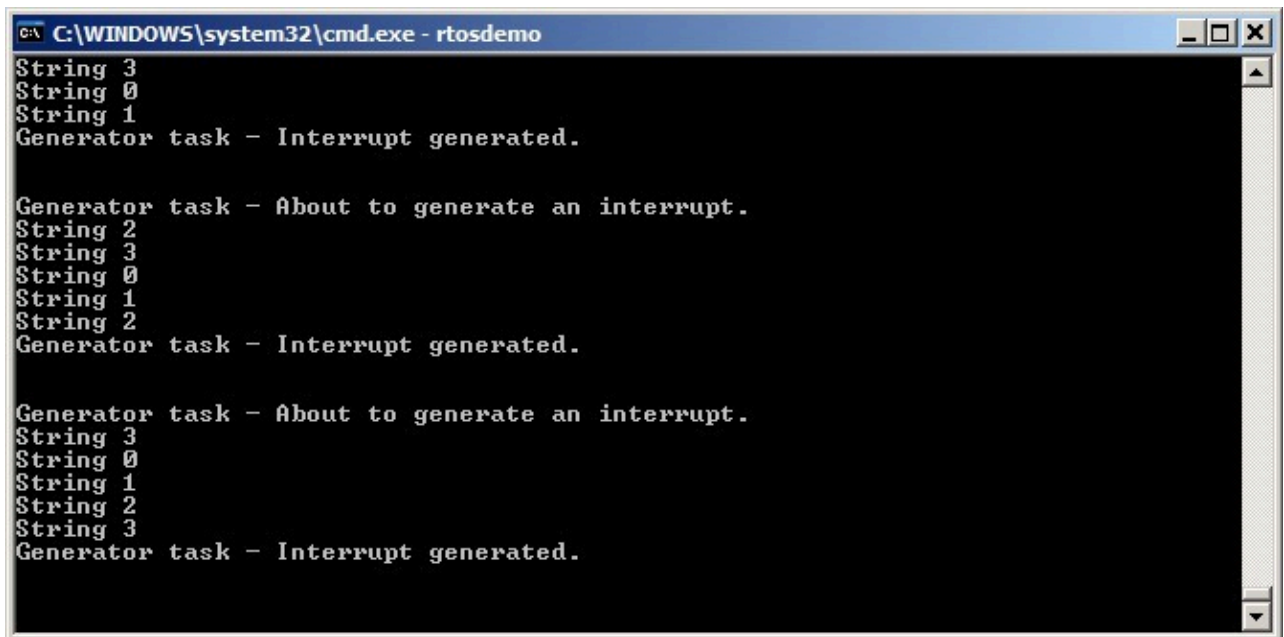
    /* 安装软件中断处理程序。所需语法取决于当前 FreeRTOS 移植版本。这里
       所示的语法仅适用于 FreeRTOS Windows 移植版本，其中此类中断只是
       模拟出来的。 */
    vPortSetInterruptHandler( mainINTERRUPT_NUMBER, ulExampleInterruptHandler );

    /* 启动调度器，使已创建的任务开始执行。 */
    vTaskStartScheduler();

    /* 如果一切正常，调度器此时正在运行任务，main() 永远不会执行到这里。
       如果确实执行到这里，很可能是因为堆内存不足，无法创建空闲任务。
       第 3 章提供了更多堆内存管理信息。 */
    for( ;; );
}
```

代码清单 7.24 示例 7.4 的 `main()` 函数

图 7.12 是执行示例 7.4 时产生的输出。可以看到，中断接收了全部五个整数，并相应产生五个字符串。图 7.13 给出了进一步说明。



```
C:\WINDOWS\system32\cmd.exe - rtosdemo
String 3
String 0
String 1
Generator task - Interrupt generated.

Generator task - About to generate an interrupt.
String 2
String 3
String 0
String 1
String 2
Generator task - Interrupt generated.

Generator task - About to generate an interrupt.
String 3
String 0
String 1
String 2
String 3
Generator task - Interrupt generated.
```

图 7.12 执行示例 7.4 时产生的输出

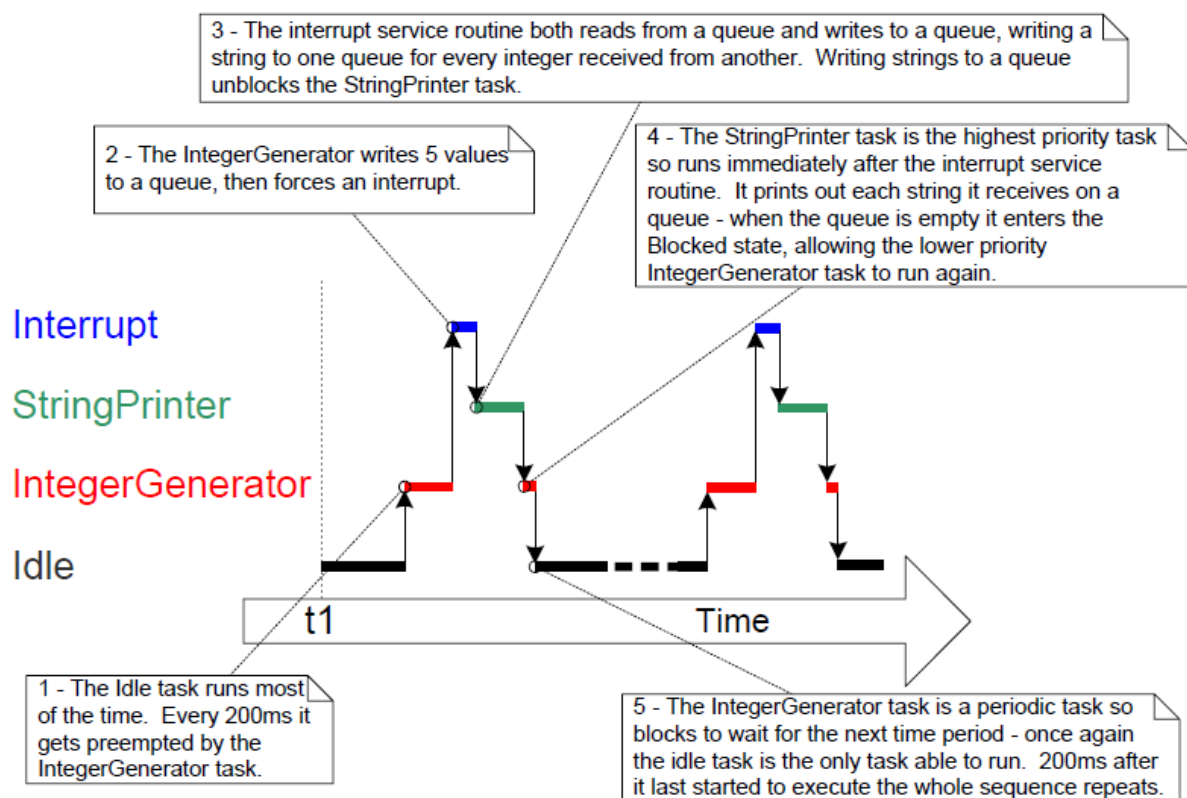


图 7.13 执行示例 7.4 时的执行顺序

7.8 中断嵌套

任务优先级和中断优先级经常被混淆。本节讨论的是中断优先级，也就是各个中断服务程序（ISR）彼此之间的执行优先级。任务优先级与中断优先级没有任何关系：硬件决定 ISR 何时执行，软件决定任务何时执行。硬件中断触发的 ISR 可以中断任务，但任务不能抢占 ISR。

支持中断嵌套的 FreeRTOS 移植版本要求在 `FreeRTOSConfig.h` 中定义下面一个或两个常量。`configMAX_SYSCALL_INTERRUPT_PRIORITY` 和 `configMAX_API_CALL_INTERRUPT_PRIORITY` 定义的是同一项属性：较旧的移植版本使用前者，较新的移植版本使用后者。

控制中断嵌套的常量

- `configMAX_SYSCALL_INTERRUPT_PRIORITY` 或 `configMAX_API_CALL_INTERRUPT_PRIORITY`

设置能够调用中断安全 FreeRTOS API 函数的最高中断优先级。

- `configKERNEL_INTERRUPT_PRIORITY`

设置 Tick 中断使用的中断优先级，并且必须始终设为可用的最低中断优先级。

如果当前 FreeRTOS 移植版本不使用 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 常量，那么所有调用中断安全 FreeRTOS API 的中断也必须以 `configKERNEL_INTERRUPT_PRIORITY` 定义的优先级执行。

每个中断源都有一个数字优先级和一个逻辑优先级：

- 数字优先级

数字优先级就是分配给中断的优先级编号。例如，中断被设为优先级 7，其数字优先级就是 7；被设为优先级 200，其数字优先级就是 200。

- 逻辑优先级

逻辑优先级表示一个中断相对于其他中断的优先顺序。

如果两个优先级不同的中断同时发生，处理器先执行逻辑优先级较高的 ISR，再执行逻辑优先级较低的 ISR。

一个中断可以抢占（嵌套于）逻辑优先级更低的中断，但不能抢占逻辑优先级相同或更高的中断。

数字优先级与逻辑优先级之间的关系取决于处理器架构。在某些处理器上，数字优先级越大，逻辑优先级越高；在另一些处理器上，数字优先级越大，逻辑优先级反而越低。

把 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 设为逻辑上高于 `configKERNEL_INTERRUPT_PRIORITY` 的中断优先级，即可建立完整的中断嵌套模型。图 7.14 展示了以下场景：

- 处理器有七个唯一的中断优先级。
 - 分配数字优先级 7 的中断具有比分配数字优先级 1 的中断更高的逻辑优先级。
 - `configKERNEL_INTERRUPT_PRIORITY` 设置为 1。
 - `configMAX_SYSCALL_INTERRUPT_PRIORITY` 设置为 3。
-

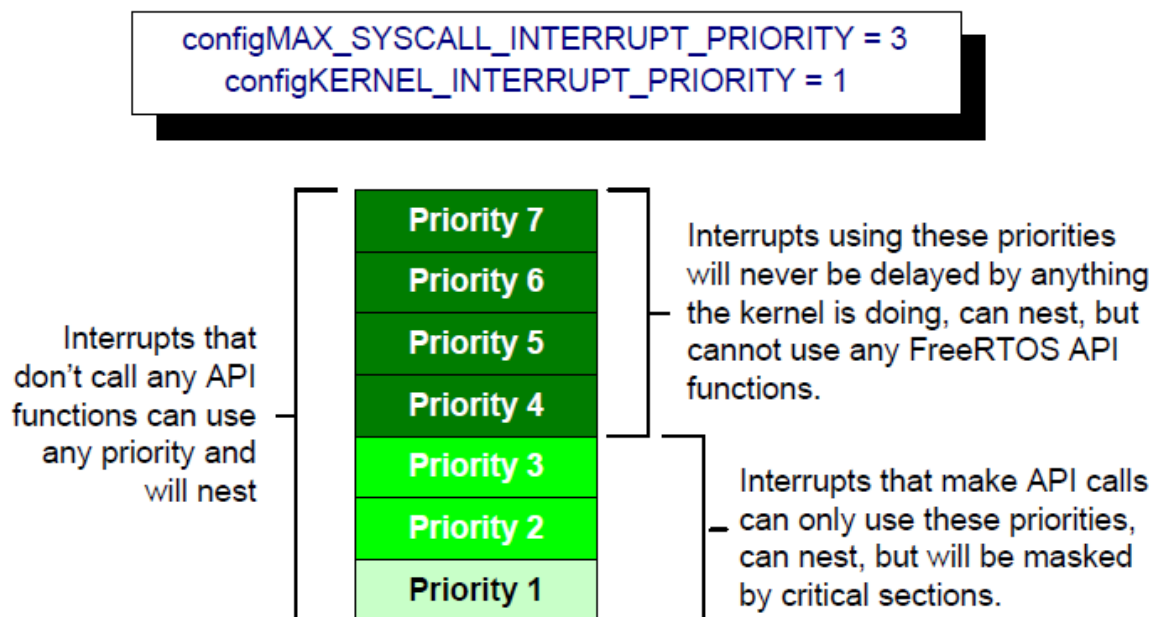


图 7.14 影响中断嵌套行为的常量

参见图 7.14:

- 当内核或应用程序处于临界区时，优先级为 1 到 3（含）的中断会被禁止执行。以这些优先级运行的 ISR 可以调用中断安全 FreeRTOS API。第 8 章将介绍临界区。
- 优先级为 4 或更高的中断不受临界区影响。因此，在硬件能力允许的范围内，无论调度器正在做什么，这些中断都能立即执行。以这些优先级运行的 ISR 不能调用任何 FreeRTOS API。
- 对时序精度要求极高的功能（例如电机控制）通常使用逻辑上高于 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 的优先级，以确保调度器不会给中断响应时间引入抖动。

7.8.1 ARM Cortex-M^[11] 和 ARM GIC 用户注意事项

Cortex-M 处理器的中断配置容易令人困惑，也很容易出错。为帮助开发者发现问题，FreeRTOS Cortex-M 移植版本会自动检查中断配置，但只有定义了 `configASSERT()` 才会执行这些检查。第 11.2 节将介绍 `configASSERT()`。

ARM Cortex 内核和 ARM 通用中断控制器（GIC）用数值较小的优先级编号表示逻辑优先级较高的中断。这看似违反直觉，也很容易忘记。要给中断分配较低的逻辑优先级，必须使用较大的数值；要分配较高的逻辑优先级，则必须使用较小的数值。

Cortex-M 中断控制器最多允许使用 8 位表示每个中断的优先级，因此 255 是可表示的最低逻辑优先级，0 是最高逻辑优先级。不过，Cortex-M 微控制器通常只实现这 8 位中的一部分，实际位数取决于微控制器系列。

如果只实现 8 个优先级位中的一部分，可用的是字节中的最高有效位，最低有效位没有实现。未实现位可以取任意值，但通常设为 1。图 7.15 展示了在只实现 4 个优先级位的 Cortex-M 微控制器中，如何存储二进制优先级 101。



图 7.15 二进制优先级 101 在实现 4 个优先级位的 Cortex-M 微控制器中的存储方式

图 7.15 中，由于最低 4 位没有实现，二进制值 101 被左移到最高有效的 4 位，未实现位则设为 1。某些库函数要求传入已经左移到已实现（最高有效）位的优先级值。使用这类函数时，图 7.15 的优先级可指定为十进制 95：把二进制 101 左移 4 位得到 101nnnn（n 表示未实现位），再把未实现位设为 1，得到二进制 1011111，也就是十进制 95。

另一些库函数要求传入左移之前的优先级值。使用这类函数时，图 7.15 的优先级必须指定为十进制 5，即无需移位的二进制 101。

`configMAX_SYSCALL_INTERRUPT_PRIORITY` 和 `configKERNEL_INTERRUPT_PRIORITY` 必须采用可直接写入 Cortex-M 寄存器的形式，也就是把优先级值左移到已实现位之后的形式。

`configKERNEL_INTERRUPT_PRIORITY` 必须始终设为可用的最低中断优先级。未实现的优先级位可以设为 1，因此无论实际实现了多少位，该常量始终可以设为 255。

Cortex-M 中断的默认优先级为 0，也就是可用的最高优先级。Cortex-M 硬件不允许把 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 设为 0，因此任何调用 FreeRTOS API 的中断都绝不能保留默认优先级。

1. 下一节将介绍延迟中断处理。↩
2. 从历史上看，要求用汇编代码封装中断处理程序的 FreeRTOS 移植版本使用 `portEND_SWITCHING_ISR()`；允许整个中断处理程序使用 C 编写的移植版本则使用 `portYIELD_FROM_ISR()`。↩
3. 使用直接任务通知从中断中解除任务的阻塞，比使用二值信号量更高效。直接任务通知将在第 10 章“任务通知”中介绍。↩
4. 某些信号量 API 函数实际上是宏，而不是函数。为简单起见，本书中统称为函数。↩
5. 下一节将介绍计数信号量。↩
6. 也可以使用计数信号量或直接任务通知来计数事件。下一节将介绍计数信号量，第 10 章“任务通知”将介绍直接任务通知。直接任务通知在运行时间和 RAM 占用方面效率最高，因此是首选方法。↩
7. 使用直接任务通知对事件计数比使用计数信号量更高效。第 10 章将介绍直接任务通知。↩
8. 守护任务最初称为定时器服务任务，因为它起初只用于执行软件定时器回调函数。因此，`xTimerPendFunctionCall()` 实现在 `timers.c` 中。按照函数名前缀采用实现文件名称的惯例，该函数以 `Timer` 为前缀。↩
9. 直接任务通知是从 ISR 中解除任务阻塞最高效的方法。第 10 章“任务通知”将介绍这一功能。↩
10. FreeRTOS+TCP 提供的“流缓冲区” (<https://www.FreeRTOS.org/tcp>) 可用于此目的。↩
11. 本节只有部分内容适用于 Cortex-M0 和 Cortex-M0+ 内核。↩

8 资源管理

8.1 本章简介与范围

在多任务系统中，如果一个任务开始访问某项资源，却在访问完成前被切换出运行状态，就可能发生错误。如果资源此时处于不一致状态，其他任务或中断再访问同一资源，便可能导致数据损坏或类似问题。

以下是一些示例：

- 访问外设

假设两个任务都要向液晶显示器（LCD）写入数据：

1. 任务 A 开始执行，并向 LCD 写入字符串 `Hello world`。
2. 任务 A 只输出了开头的 `Hello w`，随后被任务 B 抢占。
3. 任务 B 把 `Abort, Retry, Fail?` 写入 LCD，然后进入阻塞状态。
4. 任务 A 从被抢占的位置继续执行，输出剩余的 `orld`。

LCD 最终显示的是已经混杂的字符串 `Hello wAbort, Retry, Fail?orld`。

- 读-修改-写操作

代码清单 8.1 给出一行 C 代码，以及编译器通常生成的汇编代码。可以看到，程序先把 `PORTA` 的值从内存读入寄存器，在寄存器中修改，再写回内存。这就是读-修改-写操作。

```
/* 待编译的 C 代码。 */
PORTA |= 0x01;

/* 编译 C 代码后生成的汇编代码。 */
LOAD R1,[#PORTA] ; Read a value from PORTA into R1
MOVE R2,#0x01    ; Move the absolute constant 1 into R2
OR R1,R2         ; Bitwise OR R1 (PORTA) with R2 (constant 1)
STORE R1,[#PORTA] ; Store the new value back to PORTA
```

代码清单 8.1 读-修改-写操作示例

这是一个“非原子”操作，因为它需要多条指令才能完成，并且可能在中途被打断。假设两个任务都要更新名为 `PORTA` 的内存映射寄存器：

1. 任务 A 把 `PORTA` 的值载入寄存器，完成操作的“读”阶段。
2. 任务 A 尚未完成“修改”和“写”阶段，就被任务 B 抢占。
3. 任务 B 更新 `PORTA` 的值，然后进入阻塞状态。
4. 任务 A 从被抢占的位置继续执行。它修改寄存器中保存的旧 `PORTA` 副本，再把结果写回 `PORTA`。

在这种情况下，任务 A 修改并写回的是过时的 `PORTA` 值。任务 B 的修改发生在任务 A 读取 `PORTA` 之后、写回之前，因此任务 A 最后的写入覆盖了任务 B 的修改，实际破坏了 `PORTA` 寄存器值。

本例使用外设寄存器，但对普通变量执行读-修改-写操作时，原理完全相同。

- 对变量的非原子访问

更新结构体的多个成员，或者更新大于处理器自然字长的变量，例如在 16 位机器上更新 32 位变量，都是非原子访问。如果访问过程被打断，可能造成数据丢失或损坏。

- 函数可重入性

如果一个函数可以由多个任务安全调用，或者可以同时由任务和中断安全调用，就称它是“可重入的”。可重入函数也称为“线程安全”函数，因为多个执行线程调用它时，不会破坏数据或逻辑状态。

每个任务都有自己的栈和一组处理器寄存器值。如果函数只访问当前调用者栈上或寄存器中的数据，它就是可重入且线程安全的。代码清单 8.2 展示可重入函数，代码清单 8.3 展示不可重入函数。

如果应用程序使用 newlib C 库，必须在 `FreeRTOSConfig.h` 中把 `configUSE_NEWLIB_REENTRANT` 设为 1，确保正确分配 newlib 所需的线程局部存储。

如果应用程序使用 picolibc C 库，必须在 `FreeRTOSConfig.h` 中把 `configUSE_PICOLIBC_TLS` 设为 1，确保正确分配 picolibc 所需的线程局部存储。

如果应用程序使用其他 C 库，并且需要线程局部存储（TLS），则必须在 `FreeRTOSConfig.h` 中把 `configUSE_C_RUNTIME_TLS_SUPPORT` 设为 1，并实现以下宏：

- `configTLS_BLOCK_TYPE`：定义每个任务 TLS 块的类型。

- **configINIT_TLS_BLOCK**：初始化每个任务的 TLS 块。
- **configSET_TLS_BLOCK**：更新当前 TLS 块。上下文切换时会调用它，以确保使用正确的 TLS 块。
- **configDEINIT_TLS_BLOCK**：释放 TLS 块。

```
/* 参数通过栈或处理器寄存器传入函数。两种方式都是安全的，因为调用该函数的
   每个任务或中断都有自己的栈和寄存器值，所以也都有自己的 lVar1 副本。 */
long lAddOneHundred( long lVar1 )
{
    /* 根据编译器和优化级别，这个函数作用域变量同样会分配到栈或寄存器中。
       调用该函数的每个任务或中断都有自己的 lVar2 副本。 */
    long lVar2;

    lVar2 = lVar1 + 100;
    return lVar2;
}
```

代码清单 8.2 可重入函数示例

```
/* lVar1 是全局变量，因此调用 lNonsenseFunction 的所有任务都会访问它的
   同一个副本。 */
long lVar1;

long lNonsenseFunction( void )
{
    /* lState 是静态变量，不会在栈上分配。调用该函数的所有任务都会访问它的
       同一个副本。 */
    static long lState = 0;
    long lReturn;

    switch( lState )
    {
        case 0 : lReturn = lVar1 + 10;
                 lState = 1;
                 break;

        case 1 : lReturn = lVar1 + 20;
                 lState = 0;
                 break;
    }
}
```

代码清单 8.3 不可重入函数示例

8.1.1 互斥

为了始终保持数据一致性，任务之间共享的资源，以及任务与中断共享的资源，都必须采用“互斥”技术管理访问。目标是确保某个任务一旦开始访问不可重入、非线程安全的共享资源，就能独占该资源，直到资源恢复一致状态。

FreeRTOS 提供了多种实现互斥的机制。不过，只要条件允许，最好的方法仍是在设计上避免共享资源，让每项资源只由一个任务访问，尽管这在实际系统中并不总能做到。

8.1.2 范围

本章涵盖：

- 何时需要资源管理与访问控制，以及为什么需要。
- 什么是临界区。
- 互斥的含义。
- 挂起调度器的含义。
- 如何使用互斥量。
- 如何创建和使用门卫任务。
- 什么是优先级反转，以及优先级继承如何减少（但不能消除）其影响。

8.2 临界区与挂起调度器

8.2.1 基本临界区

基本临界区是由宏 `taskENTER_CRITICAL()` 和 `taskEXIT_CRITICAL()` 包围的一段代码。临界区也称为临界段。

`taskENTER_CRITICAL()` 和 `taskEXIT_CRITICAL()` 都不接收参数，也不返回值^[1]。代码清单 8.4 展示了它们的用法。

```

/* 把 PORTA 寄存器访问放在临界区内，确保访问过程不会被打断。进入临界区。 */
taskENTER_CRITICAL();

/* 从调用 taskENTER_CRITICAL() 到调用 taskEXIT_CRITICAL() 之间不会切换到
   其他任务。在允许中断嵌套的 FreeRTOS 移植版本中，中断仍可能执行，但仅限
   逻辑优先级高于 configMAX_SYSCALL_INTERRUPT_PRIORITY 所设值的中断；
   这些中断不得调用 FreeRTOS API 函数。 */
PORTA |= 0x01;

/* PORTA 访问已经完成，现在可以安全退出临界区。 */
taskEXIT_CRITICAL();

```

代码清单 8.4 使用临界区保护寄存器访问

本书配套示例使用 `vPrintString()` 函数向标准输出写入字符串；使用 FreeRTOS Windows 移植版本时，标准输出就是终端窗口。许多不同任务都会调用 `vPrintString()`，因此理论上可以用临界区保护标准输出访问，如代码清单 8.5 所示。

```

void vPrintString( const char *pcString )
{
    /* 把字符串写入 stdout，并以临界区作为一种简单粗糙的互斥方法。 */
    taskENTER_CRITICAL();
    {
        printf( "%s", pcString );
        fflush( stdout );
    }
    taskEXIT_CRITICAL();
}

```

代码清单 8.5 `vPrintString()` 的一种可能实现

以这种方式使用临界区，是一种非常粗糙的互斥方法。它会完全禁用中断；对于支持中断嵌套的 FreeRTOS 移植版本，则屏蔽逻辑优先级不高于 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 的中断。抢占式上下文切换只能由中断触发，因此只要相关中断保持禁用，调用 `taskENTER_CRITICAL()` 的任务就会一直处于运行状态，直到退出临界区。

基本临界区必须非常短，否则会损害中断响应时间。每次调用 `taskENTER_CRITICAL()` 都必须与一次 `taskEXIT_CRITICAL()` 严格配对。因此，不应像代码清单 8.5 那样用临界区保护标准输出（`stdout`，即程序写出数据的流），因为写终端可能耗时较长。本章后续示例将介绍其他方案。

临界区可以安全嵌套，因为内核会记录嵌套深度。只有嵌套深度回到零，也就是每次先前的 `taskENTER_CRITICAL()` 都有一次对应的 `taskEXIT_CRITICAL()` 时，系统才真正退出临界区。

对任务而言，调用 `taskENTER_CRITICAL()` 和 `taskEXIT_CRITICAL()` 是更改 FreeRTOS 所在处理器中断使能状态的唯一合法方式。通过其他方式更改中断使能状态，会破坏这两个宏维护的嵌套计数。

`taskENTER_CRITICAL()` 和 `taskEXIT_CRITICAL()` 的名称不以 `FromISR` 结尾，因此不得从中断服务程序调用。`taskENTER_CRITICAL_FROM_ISR()` 和 `taskEXIT_CRITICAL_FROM_ISR()` 分别是它们的中断安全版本。只有允许中断嵌套的 FreeRTOS 移植版本才提供这些宏，因为在不支持嵌套的移植版本中并不需要它们。

`taskENTER_CRITICAL_FROM_ISR()` 返回一个值，该值必须传给与之配对的 `taskEXIT_CRITICAL_FROM_ISR()` 调用。代码清单 8.6 展示了这种用法。

```
void vAnInterruptServiceRoutine( void )
{
    /* 声明变量，用于保存 taskENTER_CRITICAL_FROM_ISR() 的返回值。 */
    UBaseType_t uxSavedInterruptStatus;

    /* ISR 的这一部分可以被任何更高优先级的中断打断。 */

    /* 使用 taskENTER_CRITICAL_FROM_ISR() 保护 ISR 中的一段代码。保存其
       返回值，以便传给配对的 taskEXIT_CRITICAL_FROM_ISR() 调用。 */
    uxSavedInterruptStatus = taskENTER_CRITICAL_FROM_ISR();

    /* ISR 的这一部分位于 taskENTER_CRITICAL_FROM_ISR() 与
       taskEXIT_CRITICAL_FROM_ISR() 调用之间，因此只能被逻辑优先级高于
       configMAX_SYSCALL_INTERRUPT_PRIORITY 所设值的中断打断。 */

    /* 调用 taskEXIT_CRITICAL_FROM_ISR() 退出临界区，并传入与之配对的
       taskENTER_CRITICAL_FROM_ISR() 所返回的值。 */
    taskEXIT_CRITICAL_FROM_ISR( uxSavedInterruptStatus );

    /* ISR 的这一部分可以被任何更高优先级的中断打断。 */
}
```

代码清单 8.6 在中断服务程序中使用临界区

如果进入和退出临界区所花的时间，比真正受保护的代码还长，就得不偿失。基本临界区的进入和退出都非常快，而且耗时确定，因此特别适合保护很短的代码段。

8.2.2 挂起（或锁定）调度器

也可以通过挂起调度器来建立临界区。挂起调度器有时也称为“锁定”调度器。

基本临界区可以防止其他任务和中断访问受保护代码；通过挂起调度器建立的临界区则只防止其他任务访问，因为中断仍保持使能。

如果临界区太长，不适合通过禁用中断来保护，可以改为挂起调度器。不过，调度器挂起期间发生的中断活动，可能使恢复（或“解除挂起”）调度器成为耗时较长的操作。因此，需要根据具体情况选择更合适的方法。

8.2.3 vTaskSuspendAll() API 函数

```
void vTaskSuspendAll( void );
```

代码清单 8.7 `vTaskSuspendAll()` API 函数原型

调用 `vTaskSuspendAll()` 可以挂起调度器。调度器挂起后不会发生上下文切换，但中断仍然使能。如果中断在此期间请求上下文切换，该请求会暂存起来，直到调度器恢复时才执行。

调度器挂起期间不得调用 FreeRTOS API 函数。

8.2.4 xTaskResumeAll() API 函数

```
BaseType_t xTaskResumeAll( void );
```

代码清单 8.8 `xTaskResumeAll()` API 函数原型

调用 `xTaskResumeAll()` 可以恢复（解除挂起）调度器。

`xTaskResumeAll()` 返回值

- 返回值

调度器挂起期间请求的上下文切换会暂存起来，只在调度器恢复时执行。如果暂存的上下文切换在

`xTaskResumeAll()` 返回前已经执行，函数返回 `pdTRUE`；否则返回 `pdFALSE`。

`vTaskSuspendAll()` 和 `xTaskResumeAll()` 可以安全嵌套，因为内核会记录嵌套深度。只有嵌套深度回到零，也就是每次先前的 `vTaskSuspendAll()` 都有一次对应的 `xTaskResumeAll()` 时，调度器才会真正恢复。

代码清单 8.9 给出了 `vPrintString()` 的实际实现，它通过挂起调度器保护终端输出访问。

```
void vPrintString( const char *pcString )
{
    /* 把字符串写入 stdout, 并通过挂起调度器实现互斥。 */
    vTaskSuspendScheduler();
    {
        printf( "%s", pcString );
        fflush( stdout );
    }
    xTaskResumeScheduler();
}
```

8.3 互斥量（与二值信号量）

互斥量是一种特殊的二值信号量，用于控制两个或多个任务对共享资源的访问。MUTEX 一词源自“MUTual EXclusion”。必须在 `FreeRTOSConfig.h` 中把 `configUSE_MUTEXES` 设为 1，才能使用互斥量。

在互斥场景中，可以把互斥量看作与共享资源绑定的令牌。任务要合法访问资源，必须先成功“获取”令牌，成为令牌持有者。用完资源后，持有者必须“归还”令牌。只有令牌归还后，其他任务才能获取令牌并安全访问同一资源。没有持有令牌的任务不得访问共享资源。图 8.1 展示了这一机制。

互斥量和二值信号量虽然具有许多共同特性，但图 8.1 的互斥场景与图 7.6 的同步场景完全不同。主要区别在于获取信号量之后如何处理：

- 用于互斥的信号量必须始终归还。
 - 用于同步的信号量通常在获取后直接消耗，不再归还。
-

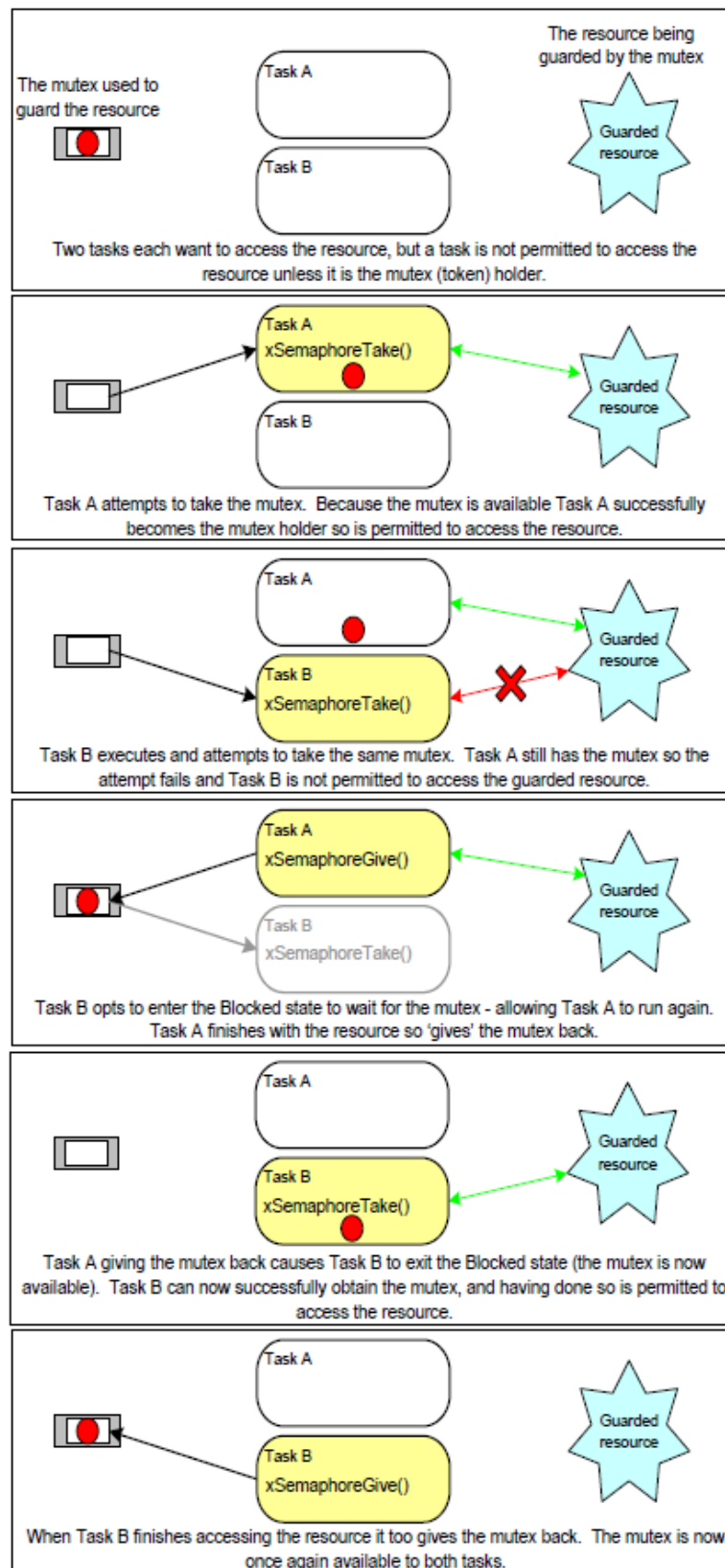


图 8.1 使用互斥量实现互斥

这种机制完全依赖应用程序开发者遵守约定。系统并不会从技术上阻止任务随时访问资源，只是所有任务都“约定”：只有成为互斥量持有者后才访问资源。

8.3.1 xSemaphoreCreateMutex() API 函数

互斥量属于信号量，因此句柄同样保存在 `SemaphoreHandle_t` 类型的变量中。使用互斥量之前必须先创建它。`xSemaphoreCreateMutex()` 从 FreeRTOS 堆中动态分配所需内存；FreeRTOS 还提供 `xSemaphoreCreateMutexStatic()`，使用调用者预先提供的内存创建互斥量。

```
SemaphoreHandle_t xSemaphoreCreateMutex( void );
```

代码清单 8.10 `xSemaphoreCreateMutex()` API 函数原型

`xSemaphoreCreateMutex()` 返回值

- 返回值

如果返回 `NULL`，则互斥量创建失败，因为 FreeRTOS 没有足够的堆内存用于分配互斥量数据结构。第 3 章提供了更多堆内存管理信息。

非 `NULL` 返回值表示互斥量创建成功。该返回值是所创建互斥量的句柄。

示例 8.1 重写 `vPrintString()` 以使用信号量

本示例创建 `vPrintString()` 的新版本 `prvNewPrintString()`，并由多个任务调用。它的功能与 `vPrintString()` 相同，但使用互斥量而不是锁定调度器来控制标准输出访问。代码清单 8.11 给出了它的实现。

```

static void prvNewPrintString( const char *pcString )
{
    /* 互斥量在调度器启动前创建，因此本任务执行时它已经存在。

    尝试获取互斥量。如果互斥量未立即可用，就无限期阻塞等待。
    xSemaphoreTake() 只有成功获取互斥量后才会返回，因此无需检查返回值。
    如果使用其他等待时间，则必须确认 xSemaphoreTake() 返回 pdTRUE 后，
    才能访问共享资源，本例中的共享资源是标准输出。如本书前文所述，生产
    代码不建议使用无限期超时。 */
    xSemaphoreTake( xMutex, portMAX_DELAY );
    {
        /* 只有成功获取互斥量后，下面的代码才会执行。任一时刻只能有一个任务
        持有互斥量，因此现在可以安全访问标准输出。 */
        printf( "%s", pcString );
        fflush( stdout );

        /* 必须归还互斥量！ */
    }
    xSemaphoreGive( xMutex );
}

```

代码清单 8.11 `prvNewPrintString()` 的实现

`prvPrintTask()` 的两个任务实例会反复调用 `prvNewPrintString()`，每次调用之间随机延迟一段时间。任务参数用于向每个实例传入不同的字符串。代码清单 8.12 给出了 `prvPrintTask()` 的实现。

```

static void prvPrintTask( void *pvParameters )
{
    char *pcStringToPrint;
    const TickType_t xMaxBlockTimeTicks = 0x20;

    /* 本任务会创建两个实例。待输出字符串通过任务参数传入，再转换为所需类型。 */
    pcStringToPrint = ( char * ) pvParameters;

    for( ;; )
    {
        /* 使用新定义的函数输出字符串。 */
        prvNewPrintString( pcStringToPrint );

        /* 等待一段伪随机时间。注意，rand() 不一定可重入，但本例并不关心它返回
        的具体值，因此影响不大。安全性要求更高的应用应使用确认可重入的
        rand() 版本，或者用临界区保护 rand() 调用。 */
        vTaskDelay( ( rand() % xMaxBlockTimeTicks ) );
    }
}

```

代码清单 8.12 示例 8.1 的 `prvPrintTask()` 实现

与往常一样，`main()` 只负责创建互斥量和任务，然后启动调度器。代码清单 8.13 给出了它的实现。

`prvPrintTask()` 的两个实例使用不同优先级，因此低优先级任务有时会被高优先级任务抢占。互斥量保证每个任务独占终端，即使发生抢占，显示的字符串仍然完整。减小 `xMaxBlockTimeTicks` 所设的最长阻塞时间，可以提高抢占发生的频率。

在 FreeRTOS Windows 移植版本上运行示例 8.1 时，需要注意：

- 调用 `printf()` 会产生 Windows 系统调用。Windows 系统调用不受 FreeRTOS 控制，可能引入不确定性。
- 由于 Windows 系统调用的执行方式，即使不使用互斥量，也很少能观察到字符串损坏。

```
int main( void )
{
    /* 使用信号量之前必须显式创建它。本例创建互斥量类型的信号量。 */
    xMutex = xSemaphoreCreateMutex();

    /* 创建任务之前，检查信号量是否创建成功。 */
    if( xMutex != NULL )
    {
        /* 创建两个向 stdout 写入数据的任务实例。待写入字符串作为任务参数传入。
           两个任务使用不同优先级，因此会发生抢占。 */
        xTaskCreate( prvPrintTask, "Print1", 1000,
                    "Task 1 *****\r\n",
                    1, NULL );

        xTaskCreate( prvPrintTask, "Print2", 1000,
                    "Task 2 ----- \r\n",
                    2, NULL );

        /* 启动调度器，使已创建的任务开始执行。 */
        vTaskStartScheduler();
    }

    /* 如果一切正常，调度器此时正在运行任务，main() 永远不会执行到这里。
       如果确实执行到这里，很可能是因为堆内存不足，无法创建空闲任务。
       第 3 章提供了更多堆内存管理信息。 */
    for( ;; );
}
```

代码清单 8.13 示例 8.1 的 `main()` 实现

图 8.2 是执行示例 8.1 时产生的输出，图 8.3 展示了一种可能的执行顺序。

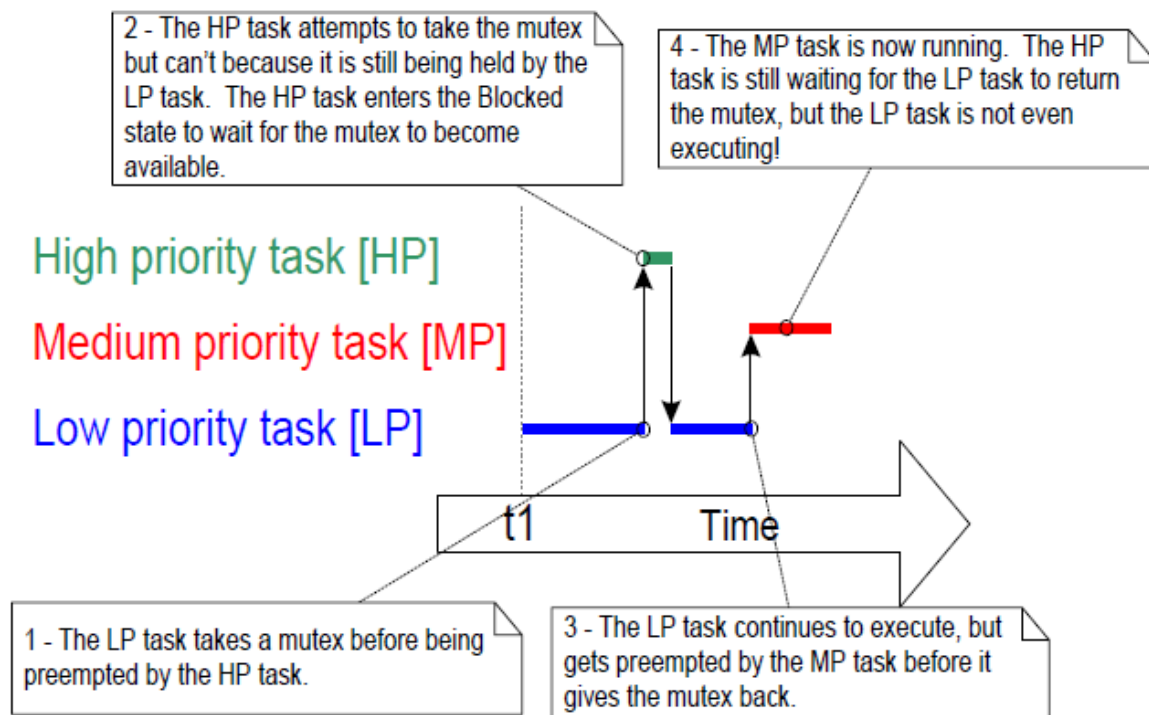


图 8.4 最坏情况下的优先级反转

优先级反转可能造成严重问题，但在小型嵌入式系统中，通常可以在设计阶段规划资源访问方式，从而避免它。

8.3.3 优先级继承

FreeRTOS 互斥量与二值信号量非常相似，区别在于互斥量带有基本的“优先级继承”机制，而二值信号量没有。优先级继承用于尽量减小优先级反转的负面影响。它不能消除优先级反转，只能确保反转持续时间有界。不过，优先级继承会增加系统时序分析的复杂度，因此不应依赖它来保证系统正确运行。

优先级继承会暂时提高互斥量持有者的优先级，使其等于所有等待该互斥量的任务中的最高优先级。换言之，持有互斥量的低优先级任务“继承”了等待者的优先级。图 8.5 展示了这一过程。互斥量归还后，持有者的优先级会自动恢复为原始值。

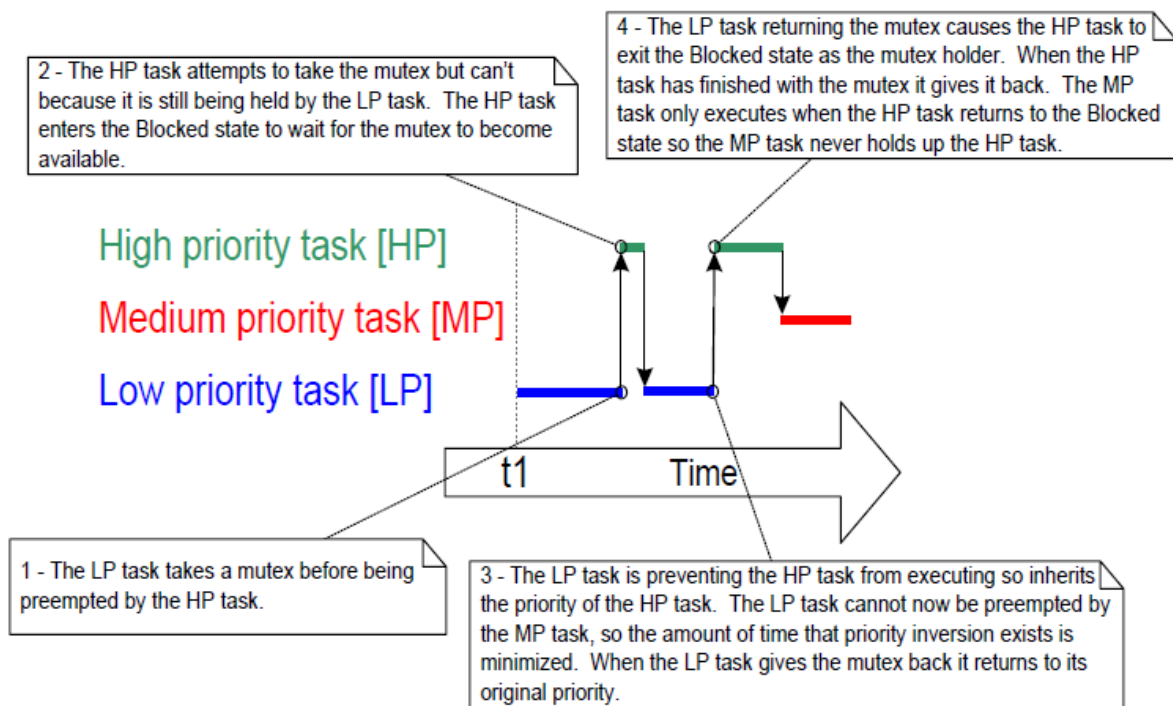


图 8.5 优先级继承减小优先级反转的影响

如上所述，优先级继承会改变使用互斥量的任务优先级，因此不得从中断服务程序中使用互斥量。

FreeRTOS 实现的是一种基础优先级继承机制，设计目标是节省空间和处理器周期。完整实现需要明显更多的数据和运算，才能随时确定继承优先级，尤其是在一个任务同时持有多个互斥量时。

需要记住这种优先级继承机制的具体行为：

- 如果任务尚未释放已持有的互斥量，又获取另一个互斥量，它继承的优先级可能继续提高。
- 任务会保持所继承的最高优先级，直到释放自己持有的全部互斥量，与互斥量的释放顺序无关。
- 如果任务持有多个互斥量，即使等待其中某个互斥量的任务停止等待（例如超时），持有者仍会保持所继承的最高优先级。

8.3.4 死锁（或称致命拥抱）

“死锁”是使用互斥量实现互斥时的另一个潜在问题，有时也被形象地称为“致命拥抱”。

当两个任务都在等待对方持有的资源，因而谁也无法继续时，就发生了死锁。假设任务 A 和任务 B 都必须同时获得互斥量 X 和互斥量 Y 才能执行某项操作：

1. 任务 A 执行并成功获取互斥量 X。

2. 任务 A 被任务 B 抢占。
3. 任务 B 成功获取互斥量 Y，随后尝试获取互斥量 X。但 X 由任务 A 持有，因此任务 B 进入阻塞状态，等待 X 被释放。
4. 任务 A 继续执行并尝试获取互斥量 Y。但 Y 由任务 B 持有，因此任务 A 也进入阻塞状态，等待 Y 被释放。

最终，任务 A 等待任务 B 持有的互斥量，任务 B 又等待任务 A 持有的互斥量。两个任务都无法继续，系统发生死锁。

与优先级反转一样，避免死锁的最好方法是在设计阶段识别其可能性，并从系统结构上保证它不会发生。尤其需要注意，本书前文已经指出，让任务无限期待互斥量通常不是良好做法。更稳妥的方法是设置一个略长于正常最大等待时间的超时；如果超时前仍未获得互斥量，就说明设计或运行状态存在错误，可能正是死锁。

在小型嵌入式系统中，死锁通常并不难处理，因为设计者往往能够掌握整个应用程序，从而识别并消除可能发生死锁的路径。

8.3.5 递归互斥量

任务也可能让自己陷入死锁。如果任务尚未归还某个互斥量，又再次尝试获取同一个互斥量，就可能发生这种情况：

1. 一个任务成功获取了互斥量。
2. 在持有互斥量的同时，该任务调用了库函数。
3. 库函数内部尝试获取同一个互斥量，于是进入阻塞状态等待互斥量变为可用。

最终，该任务阻塞等待互斥量被归还，而它自己恰恰就是互斥量持有者。任务在等待自己释放资源，于是发生死锁。

使用递归互斥量代替标准互斥量，可以避免这类自死锁。同一任务可以多次“获取”递归互斥量；只有每一次获取都有一次对应的“归还”后，互斥量才真正释放。

标准互斥量和递归互斥量的创建和使用方式类似：

- 标准互斥量用 `xSemaphoreCreateMutex()` 创建，递归互斥量用 `xSemaphoreCreateRecursiveMutex()` 创建。两个 API 函数的原型相同。

- 标准互斥量用 `xSemaphoreTake()` “获取”，递归互斥量用 `xSemaphoreTakeRecursive()` “获取”。两个 API 函数的原型相同。
- 标准互斥量用 `xSemaphoreGive()` “归还”，递归互斥量用 `xSemaphoreGiveRecursive()` “归还”。两个 API 函数的原型相同。

代码清单 8.14 展示了如何创建和使用递归互斥量。

```

/* 递归互斥量保存在 SemaphoreHandle_t 类型的变量中。 */
SemaphoreHandle_t xRecursiveMutex;

/* 创建并使用递归互斥量的任务实现。 */
void vTaskFunction( void *pvParameters )
{
    const TickType_t xMaxBlock20ms = pdMS_TO_TICKS( 20 );

    /* 使用递归互斥量之前必须显式创建它。 */
    xRecursiveMutex = xSemaphoreCreateRecursiveMutex();

    /* 检查信号量是否创建成功。第 11.2 节将介绍 configASSERT()。 */
    configASSERT( xRecursiveMutex );

    /* 与大多数任务一样，本任务在无限循环中实现。 */
    for( ;; )
    {
        /* ... */

        /* 获取递归互斥量。 */
        if( xSemaphoreTakeRecursive( xRecursiveMutex, xMaxBlock20ms ) == pdPASS )
        {
            /* 已成功获取递归互斥量，任务现在可以访问受其保护的资源。由于只获取
               过一次互斥量，此时递归调用计数，也就是嵌套调用
               xSemaphoreTakeRecursive() 的次数，为 1。 */

            /* 任务已经持有递归互斥量，又再次获取它。在实际应用中，这通常只会
               发生在任务调用的子函数中，因为没有理由有意重复获取同一互斥量。
               调用任务已经是持有者，所以第二次调用 xSemaphoreTakeRecursive()
               只会把递归调用计数增加到 2。 */
            xSemaphoreTakeRecursive( xRecursiveMutex, xMaxBlock20ms );

            /* ... */

            /* 任务访问完受保护资源后开始归还互斥量。此时递归调用计数为 2，
               所以第一次调用 xSemaphoreGiveRecursive() 不会真正释放互斥量，
               只会把计数减回 1。 */
            xSemaphoreGiveRecursive( xRecursiveMutex );

            /* 下一次调用 xSemaphoreGiveRecursive() 把递归调用计数减为 0，
               此时递归互斥量才真正归还。 */
            xSemaphoreGiveRecursive( xRecursiveMutex );

            /* 现在，每次先前的 xSemaphoreTakeRecursive() 调用都有一次对应的
               xSemaphoreGiveRecursive() 调用，任务不再是互斥量持有者。 */
        }
    }
}

```

代码清单 8.14 创建并使用递归互斥量

8.3.6 互斥量与任务调度

如果两个优先级不同的任务使用同一个互斥量，FreeRTOS 调度策略会明确决定执行顺序：能够运行的最高优先级任务进入运行状态。例如，高优先级任务因等待低优先级任务持有的互斥量而阻塞时，低优先级任务一旦归还互斥量，高优先级任务就会抢占它，随后成为新的互斥量持有者。图 8.5 已展示这种情形。

但当两个任务优先级相同时，人们经常误判执行顺序。假设任务 1 和任务 2 优先级相同，任务 1 因等待任务 2 持有的互斥量而阻塞；任务 2 “归还”互斥量时，任务 1 不会抢占任务 2。任务 2 会继续处于运行状态，任务 1 只会从阻塞状态转入就绪状态。图 8.6 展示了这种情形，竖线表示 Tick 中断发生的时刻。

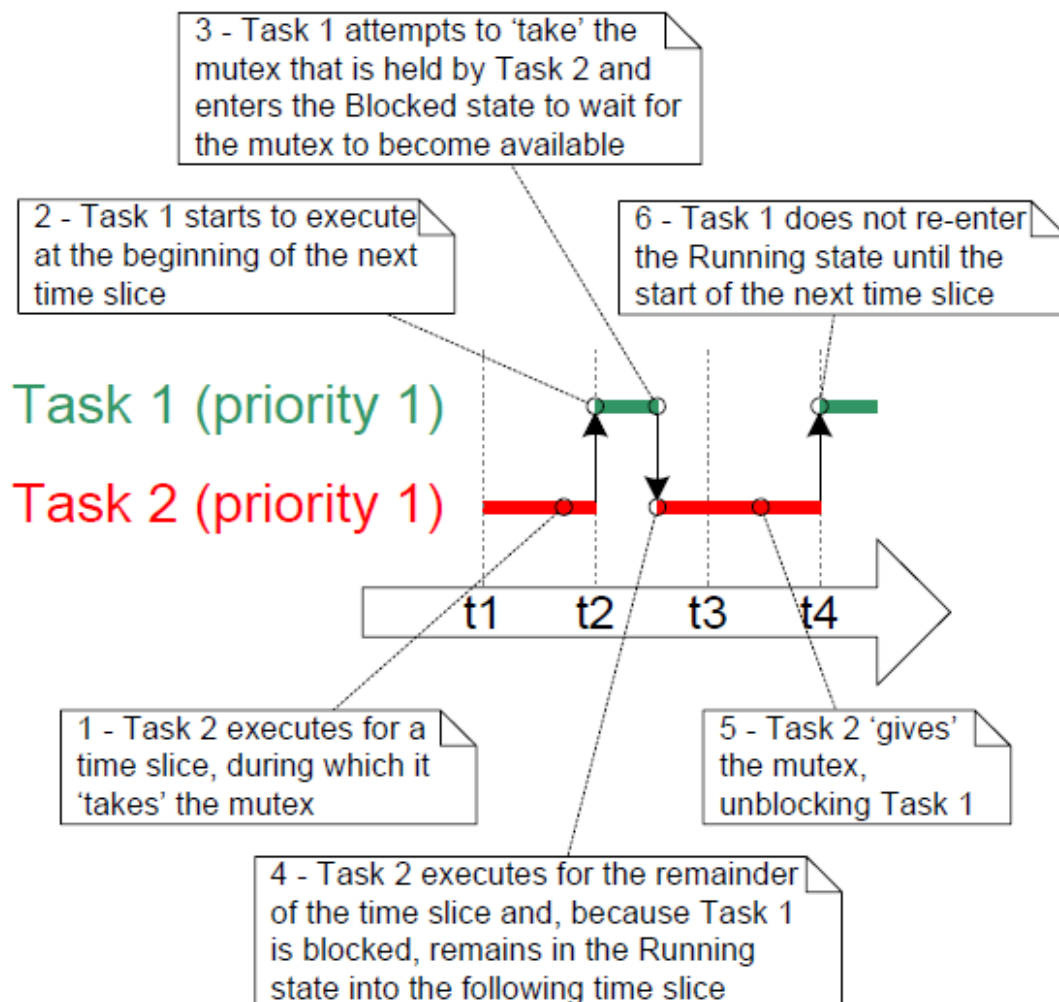


图 8.6 相同优先级任务使用同一互斥量时的一种可能执行顺序

在图 8.6 的场景中，互斥量变为可用时，FreeRTOS 调度器不会立即让任务 1 进入运行状态，原因如下：

- 任务 1 和任务 2 优先级相同。假设在 `FreeRTOSConfig.h` 中把 `configUSE_TIME_SLICING` 设为 1，那么除非任务 2 进入阻塞状态，否则下一次 Tick 中断之前不应切换到任务 1。
- 如果任务在紧密循环中使用互斥量，而且每次“归还”互斥量都会触发上下文切换，那么每个任务只能短暂处于运行状态。两个或多个任务会快速来回切换，浪费处理器时间。

如果多个相同优先级的任务在紧密循环中使用同一互斥量，需要特别保证它们获得大致相等的处理时间。图 8.7 说明了处理时间可能失衡的原因，其中展示了以相同优先级创建代码清单 8.15 所示任务的两个实例时，一种可能的执行顺序。

```
/* 在紧密循环中使用互斥量的任务实现。任务先在局部缓冲区中生成字符串，再把
   字符串写入显示器。显示器访问由互斥量保护。 */

void vATask( void *pvParameter )
{
    extern SemaphoreHandle_t xMutex;
    char cTextBuffer[ 128 ];

    for( ;; )
    {
        /* 生成字符串，这是一个快速操作。 */
        vGenerateTextInALocalBuffer( cTextBuffer );

        /* 获取用于保护显示器访问的互斥量。 */
        xSemaphoreTake( xMutex, portMAX_DELAY );

        /* 把生成的文本写入显示器，这是一个慢速操作。 */
        vCopyTextToFrameBuffer( cTextBuffer );

        /* 文本已写入显示器，归还互斥量。 */
        xSemaphoreGive( xMutex );
    }
}
```

代码清单 8.15 在紧密循环中使用互斥量的任务

代码清单 8.15 的注释指出，生成字符串很快，更新显示器却很慢。任务更新显示器时一直持有互斥量，因此在大部分运行时间内，互斥量都由该任务占用。

图 8.7 中的竖线表示 Tick 中断发生的时刻。

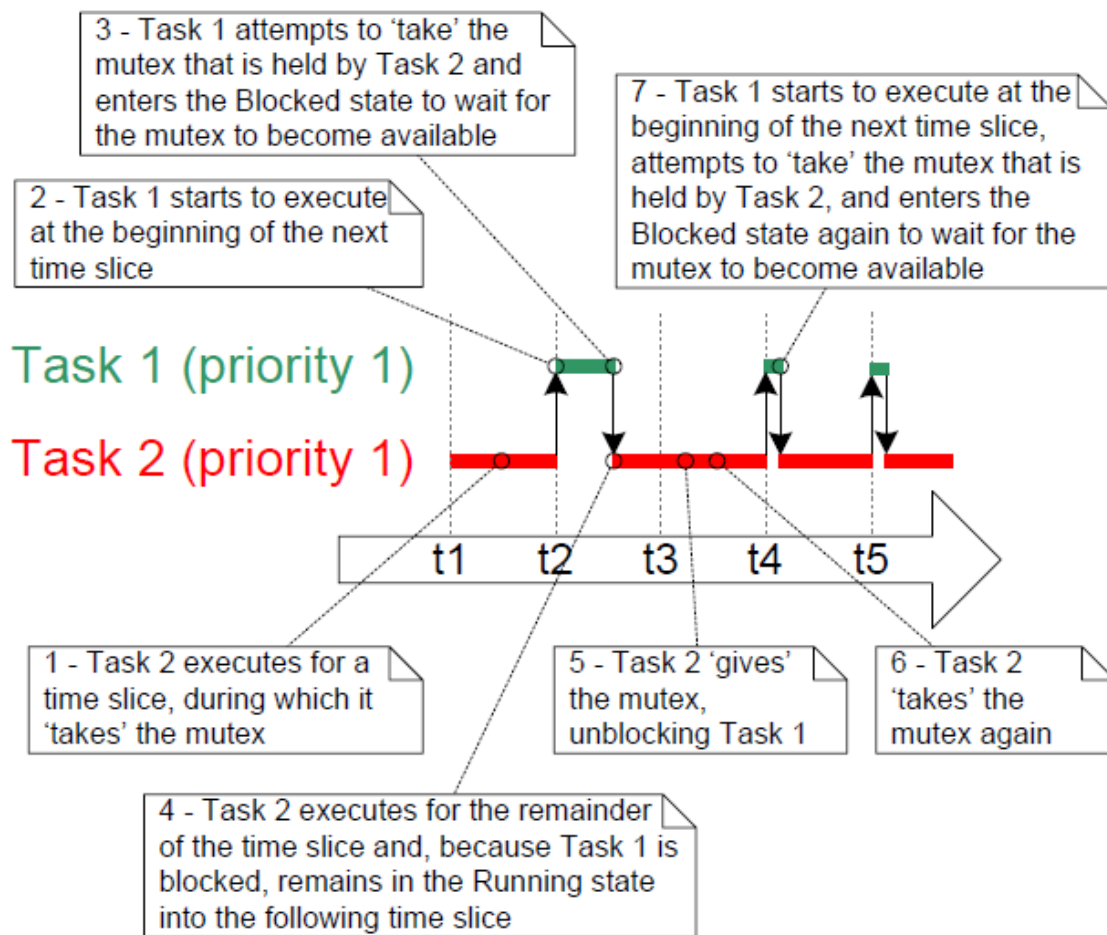


图 8.7 以相同优先级创建代码清单 8.15 中两个任务实例时可能出现的执行顺序

图 8.7 的步骤 7 显示任务 1 再次进入阻塞状态，这一过程发生在 `xSemaphoreTake()` API 函数内部。

图 8.7 表明，任务 1 会一直无法获取互斥量，直到某个时间片恰好开始于任务 2 暂时没有持有互斥量的短暂间隙。

可以在 `xSemaphoreGive()` 之后调用 `taskYIELD()`，避免图 8.7 的情况。代码清单 8.16 展示了一种优化：只有任务持有互斥量期间 Tick 计数发生变化时，才调用 `taskYIELD()`。

```

void vFunction( void *pvParameter )
{
    extern SemaphoreHandle_t xMutex;
    char cTextBuffer[ 128 ];
    TickType_t xTimeAtWhichMutexWasTaken;

    for( ;; )
    {
        /* 生成字符串，这是一个快速操作。 */
        vGenerateTextInALocalBuffer( cTextBuffer );

        /* 获取用于保护显示器访问的互斥量。 */
        xSemaphoreTake( xMutex, portMAX_DELAY );

        /* 记录获取互斥量时的时间。 */
        xTimeAtWhichMutexWasTaken = xTaskGetTickCount();

        /* 把生成的文本写入显示器，这是一个慢速操作。 */
        vCopyTextToFrameBuffer( cTextBuffer );

        /* 文本已写入显示器，归还互斥量。 */
        xSemaphoreGive( xMutex );

        /* 如果每次循环都调用 taskYIELD()，本任务每次只能短暂处于运行状态，
           处理器时间会浪费在任务间的快速切换上。因此，只有持有互斥量期间
           Tick 计数发生变化时才调用 taskYIELD()。 */
        if( xTaskGetTickCount() != xTimeAtWhichMutexWasTaken )
        {
            taskYIELD();
        }
    }
}

```

代码清单 8.16 在改善循环中互斥量使用任务的处理时间公平性时，避免因任务切换过快而浪费处理时间

8.4 门卫任务

门卫任务提供了一种简洁的互斥实现方式，不会带来优先级反转或死锁风险。

门卫任务是唯一拥有某项资源的任务。只有门卫任务可以直接访问该资源；其他任务若要使用资源，必须通过门卫任务提供的服务间接完成。

8.4.1 重写 vPrintString() 以使用门卫任务

示例 8.2 给出 `vPrintString()` 的另一种实现，这次由门卫任务管理标准输出访问。其他任务需要输出消息时，不再直接调用打印函数，而是把消息发送给门卫任务。

门卫任务使用 FreeRTOS 队列串行化标准输出访问。由于只有它能够直接访问标准输出，其内部实现不需要再考虑互斥。

门卫任务大部分时间处于阻塞状态，等待消息进入队列。消息到达后，它把消息写入标准输出，再回到阻塞状态等待下一条消息。代码清单 8.18 给出了门卫任务的实现。

中断也能向队列发送数据，因此中断服务程序可以安全地借助门卫任务向终端输出消息。本示例使用 Tick 钩子函数，每 200 个 Tick 发送一条消息。

Tick 钩子（或 Tick 回调）是内核在每次 Tick 中断期间调用的函数。要使用 Tick 钩子函数：

1. 在 `FreeRTOSConfig.h` 中把 `configUSE_TICK_HOOK` 设为 1。
2. 按代码清单 8.17 所示的准确函数名和原型实现钩子函数。

```
void vApplicationTickHook( void );
```

代码清单 8.17 Tick 钩子函数的名称和原型

Tick 钩子函数在 Tick 中断上下文中执行，因此必须非常短，只能使用少量栈空间，并且不得调用名称不以 `FromISR` 结尾的 FreeRTOS API 函数。

调度器总会在 Tick 钩子函数结束后立即运行。因此，从 Tick 钩子调用中断安全 FreeRTOS API 时，无需使用 `pxHigherPriorityTaskWoken` 参数，可以将其设为 `NULL`。

```
static void prvStdioGatekeeperTask( void *pvParameters )
{
    char *pcMessageToPrint;

    /* 只有本任务可以写标准输出。其他任务需要输出字符串时，不直接访问标准
       输出，而是把字符串发送给本任务。由于标准输出只由本任务访问，任务
       实现内部无需考虑互斥或串行化问题。 */
    for( ;; )
    {
        /* 等待消息到达。这里指定无限阻塞时间，因此无需检查返回值；只有成功
           接收到消息后，函数才会返回。 */
        xQueueReceive( xPrintQueue, &pcMessageToPrint, portMAX_DELAY );

        /* 输出收到的字符串。 */
        printf( "%s", pcMessageToPrint );
        fflush( stdout );

        /* 返回循环，等待下一条消息。 */
    }
}
```

代码清单 8.18 门卫任务

示例 8.2 打印任务的另一种实现

代码清单 8.19 给出了写队列任务。与前一个示例一样，程序创建两个任务实例，并通过任务参数指定各实例要写入队列的字符串。

```
static void prvPrintTask( void *pvParameters )
{
    int iIndexToString;
    const TickType_t xMaxBlockTimeTicks = 0x20;

    /* 本任务会创建两个实例。任务参数用于传入字符串数组的索引，再转换为所需
       类型。 */
    iIndexToString = ( int ) pvParameters;

    for( ;; )
    {
        /* 不直接输出字符串，而是通过队列把字符串指针传给门卫任务。队列在
           调度器启动前创建，因此本任务首次执行时它已经存在。队列中应当始终
           有空位，所以这里不指定阻塞时间。 */
        xQueueSendToBack( xPrintQueue, &(amp; pcStringsToPrint[ iIndexToString ]), 0 );

        /* 等待一段伪随机时间。注意，rand() 不一定可重入，但本例并不关心它返回
           的具体值，因此影响不大。安全性要求更高的应用应使用确认可重入的
           rand() 版本，或者用临界区保护 rand() 调用。 */
        vTaskDelay( ( rand() % xMaxBlockTimeTicks ) );
    }
}
```

代码清单 8.19 示例 8.2 的打印任务实现

Tick 钩子函数记录自己的调用次数，每当计数达到 200 时，就向门卫任务发送消息。仅为演示不同 API 的用法，Tick 钩子把消息写入队列前端，而任务把消息写入队列后端。代码清单 8.20 给出了 Tick 钩子的实现。


```

void vApplicationTickHook( void )
{
    static int iCount = 0;

    /* 每 200 个 Tick 输出一条消息。消息不直接写出，而是发送给门卫任务。 */
    iCount++;

    if( iCount >= 200 )
    {
        /* xQueueSendToFrontFromISR() 从 Tick 钩子中调用，所以无需使用第三个
           参数 xHigherPriorityTaskWoken，将其设为 NULL。 */
        xQueueSendToFrontFromISR( xPrintQueue,
                                   &( pcStringsToPrint[ 2 ] ),
                                   NULL );

        /* 重置计数，准备在 200 个 Tick 后再次输出字符串。 */
        iCount = 0;
    }
}

```

代码清单 8.20 Tick 钩子实现

与往常一样，`main()` 创建运行示例所需的队列和任务，然后启动调度器。代码清单 8.21 给出了它的实现。

```

/* 定义任务和中断要通过门卫任务输出的字符串。 */
static char *pcStringsToPrint[] =
{
    "Task 1 *****\r\n",
    "Task 2 ----- \r\n",
    "Message printed from the tick hook interrupt #####\r\n"
};

/*-----*/

/* 声明 QueueHandle_t 类型的变量。打印任务和 Tick 中断通过该队列向门卫任务
   发送消息。 */
QueueHandle_t xPrintQueue;

/*-----*/

int main( void )
{
    /* 使用队列之前必须显式创建它。本队列最多容纳 5 个字符指针。 */
    xPrintQueue = xQueueCreate( 5, sizeof( char * ) );

    /* 检查队列是否创建成功。 */
    if( xPrintQueue != NULL )
    {
        /* 创建两个向门卫任务发送消息的任务实例。任务使用的字符串索引通过任务
           参数，也就是 xTaskCreate() 的第 4 个参数传入。两个任务使用不同
           优先级，因此高优先级任务偶尔会抢占低优先级任务。 */
        xTaskCreate( prvPrintTask, "Print1", 1000, ( void * ) 0, 1, NULL );
        xTaskCreate( prvPrintTask, "Print2", 1000, ( void * ) 1, 2, NULL );

        /* 创建门卫任务。只有该任务可以直接访问标准输出。 */
        xTaskCreate( prvStdioGatekeeperTask, "Gatekeeper", 1000, NULL, 0, NULL );

        /* 启动调度器，使已创建的任务开始执行。 */
        vTaskStartScheduler();
    }

    /* 如果一切正常，调度器此时正在运行任务，main() 永远不会执行到这里。
       如果确实执行到这里，很可能是因为堆内存不足，无法创建空闲任务。
       第 3 章提供了更多堆内存管理信息。 */
    for( ;; );
}

```

代码清单 8.21 示例 8.2 的 `main()` 实现

图 8.8 是执行示例 8.2 时产生的输出。可以看到，来自任务和中断的字符串都被完整输出，没有损坏。

```
C:\WINDOWS\system32\cmd.exe - rtosdemo
Message printed from the tick hook interrupt #####
Message printed from the tick hook interrupt #####
Task 2 -----
Task 1 *****
Message printed from the tick hook interrupt #####
Task 1 *****
Message printed from the tick hook interrupt #####
Task 2 -----
Message printed from the tick hook interrupt #####
Task 2 -----
Task 1 *****
Task 1 *****
Message printed from the tick hook interrupt #####
Message printed from the tick hook interrupt #####
Task 2 -----
Task 1 *****
Message printed from the tick hook interrupt #####
Task 2 -----
Task 2 -----
Task 2 -----
Task 1 *****
Task 2 -----
Message printed from the tick hook interrupt #####
Task 1 *****
```

图 8.8 执行示例 8.2 时产生的输出

门卫任务的优先级低于打印任务，因此发给门卫的消息会留在队列中，直到两个打印任务都进入阻塞状态。有些场景更适合给门卫任务较高优先级，使消息立即得到处理；代价是门卫任务会延迟较低优先级任务，直到自己完成对受保护资源的访问。

1. 函数式宏并不会像真正的函数那样“返回值”。本书对宏使用“返回值”一词，只是为了便于把宏按函数来理解。↩

9 事件组

9.1 本章简介与范围

前文已经指出，实时嵌入式系统必须响应事件并采取相应动作。前几章介绍了 FreeRTOS 中用于把事件传递给任务的功能，例如信号量和队列。它们都具有以下特性：

- 允许任务进入阻塞状态，等待单个事件发生。
- 事件发生时，只解除一个任务的阻塞。被解除阻塞的是等待该事件的任务中优先级最高的任务。

事件组是 FreeRTOS 提供的另一种任务事件通知机制。与队列和信号量不同：

- 事件组允许任务进入阻塞状态，等待一个或多个事件的组合发生。
- 当某个事件或事件组合发生时，事件组会解除所有等待同一事件或事件组合的任务的阻塞。

事件组的这些独特属性使其适合用于同步多个任务、向多个任务广播事件、让任务在阻塞状态下等待一组事件中的任意一个发生，以及等待多个操作全部完成。

事件组还有助于减少应用程序的 RAM 占用，因为很多情况下可以用一个事件组替代多个二值信号量。

事件组功能是可选的。要在项目中使用事件组，请把 FreeRTOS 源文件 `event_groups.c` 加入构建。

9.1.1 范围

本章旨在帮助读者充分理解：

- 事件组的实际用途。
- 与 FreeRTOS 其他功能相比，事件组的优点和缺点。
- 如何设置事件组中的位。
- 如何进入阻塞状态，等待事件组中的位被置位。
- 如何使用事件组同步一组任务。

9.2 事件组的特性

9.2.1 事件组、事件标志和事件位

事件“标志”是一个布尔值（1 或 0），用于表示某个事件是否已经发生。事件“组”则是一组事件标志。

事件标志只能是 1 或 0，因此单个事件标志的状态可以存储在一个位中，而事件组中全部事件标志的状态可以存储在一个变量中。事件组内每个事件标志的状态，由 `EventBits_t` 类型变量中的一个位表示。因此，事件标志也称为事件“位”。如果 `EventBits_t` 变量中的某一位被置为 1，则该位代表的事件已经发生；如果某一位为 0，则该位代表的事件尚未发生。

图 9.1 展示了各个事件标志如何映射到 `EventBits_t` 类型变量中的各个位。

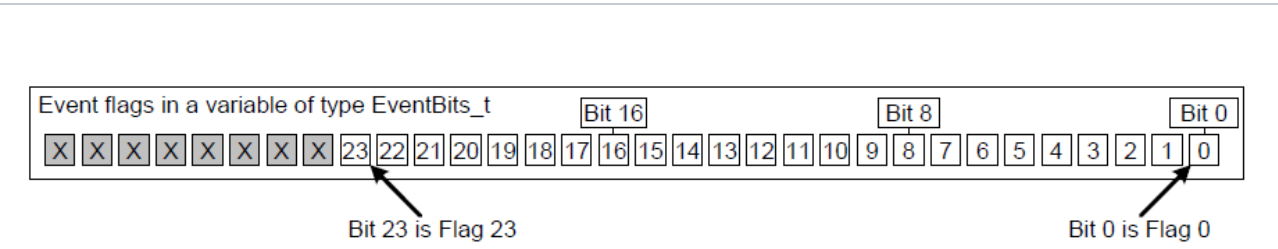


图 9.1 事件标志到 `EventBits_t` 类型变量中位编号的映射

例如，如果事件组的值为 0x92（二进制 1001 0010），那么只有事件位 1、4 和 7 被置位，也就是说，只有这些位所代表的事件已经发生。图 9.2 展示了一个 `EventBits_t` 类型变量：事件位 1、4 和 7 被置位，其他事件位均已清零，因此事件组的值为 0x92。

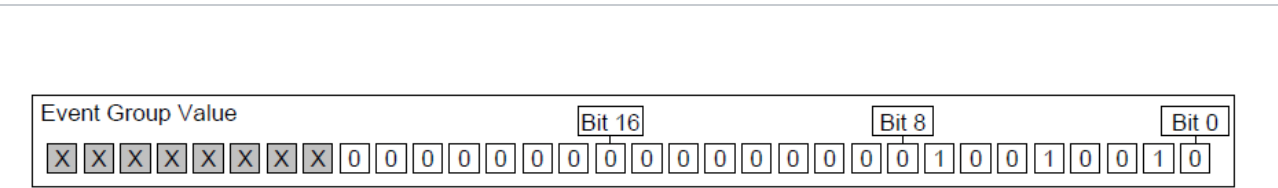


图 9.2 仅事件位 1、4 和 7 被置位、其他事件标志均已清零的事件组，其值为 0x92

事件组中各个位的含义由应用程序编写者自行定义。例如，应用程序编写者可以创建一个事件组，然后：

- 定义事件组中的位 0 表示“已经从网络接收到消息”。

- 定义事件组中的位 1 表示“已有消息准备发送到网络”。
- 定义事件组中的位 2 表示“中止当前网络连接”。

9.2.2 深入了解 EventBits_t 数据类型

事件组中可用事件位的数量取决于 FreeRTOSConfig.h 中的编译期配置常量 `configTICK_TYPE_WIDTH_IN_BITS`^[1]：

- 如果 `configTICK_TYPE_WIDTH_IN_BITS` 为 `TICK_TYPE_WIDTH_16_BITS`，则每个事件组包含 8 个可用事件位。
- 如果 `configTICK_TYPE_WIDTH_IN_BITS` 为 `TICK_TYPE_WIDTH_32_BITS`，则每个事件组包含 24 个可用事件位。
- 如果 `configTICK_TYPE_WIDTH_IN_BITS` 为 `TICK_TYPE_WIDTH_64_BITS`，则每个事件组包含 56 个可用事件位。

9.2.3 多任务访问

事件组本身就是一种对象，任何知道其存在的任务或 ISR 都可以访问它。任意数量的任务都可以在同一个事件组中置位，也可以读取同一个事件组中的位。

9.2.4 使用事件组的实际示例

FreeRTOS+TCP 协议栈的实现提供了一个实际示例，说明事件组如何同时简化设计并尽量减少资源占用。

TCP 套接字必须响应许多不同事件，例如接受事件、绑定事件、读取事件和关闭事件。套接字在某一时刻可能遇到哪些事件，取决于套接字当前的状态。例如，如果套接字已经创建但尚未绑定地址，那么它可能接收到绑定事件，却不会接收到读取事件，因为没有地址的套接字无法读取数据。

FreeRTOS+TCP 套接字的状态保存在名为 `FreeRTOS_Socket_t` 的结构体中。该结构体包含一个事件组，其中为套接字必须处理的每种事件分别定义了一个事件位。需要阻塞等待某个事件或一组事件的 FreeRTOS+TCP API 调用，只需阻塞在这个事件组上。

该事件组还包含一个“中止”位，因此无论套接字当时正在等待哪种事件，都可以中止 TCP 连接。

9.3 使用事件组管理事件

9.3.1 xEventGroupCreate() API 函数

事件组必须先显式创建才能使用。FreeRTOS 还提供 `xEventGroupCreateStatic()` 函数，它使用调用者提供的 `StaticEventGroup_t` 缓冲区创建事件组，而不从 FreeRTOS 堆中动态分配内存。

事件组通过 `EventGroupHandle_t` 类型的变量引用。`xEventGroupCreate()` API 函数用于创建事件组，并返回一个用于引用所创建事件组的 `EventGroupHandle_t`。

```
EventGroupHandle_t xEventGroupCreate( void );
```

代码清单 9.1 `xEventGroupCreate()` API 函数原型

`xEventGroupCreate()` 返回值

- 返回值

如果返回 `NULL`，则事件组创建失败，因为 FreeRTOS 没有足够的堆内存用于分配事件组数据结构。第 3 章提供了更多堆内存管理信息。

非 `NULL` 返回值表示事件组创建成功。该返回值应保存为所创建事件组的句柄。

9.3.2 xEventGroupSetBits() API 函数

`xEventGroupSetBits()` API 函数用于设置事件组中的一个或多个位，通常用来通知任务：被置位的一个或多个位所代表的事件已经发生。

注意：切勿从中断服务程序调用 `xEventGroupSetBits()`，应改用其中断安全版本 `xEventGroupSetBitsFromISR()`。

```
EventBits_t xEventGroupSetBits( EventGroupHandle_t xEventGroup,  
const EventBits_t uxBitsToSet );
```

代码清单 9.2 `xEventGroupSetBits()` API 函数原型

`xEventGroupSetBits()` 参数和返回值

- `xEventGroup`

要在其中设置位的事件组句柄。该句柄由创建事件组时调用的 `xEventGroupCreate()` 返回。

- `uxBitsToSet`

一个位掩码，用于指定要在事件组中置为 1 的一个或多个事件位。事件组的原值与 `uxBitsToSet` 按位或后，得到更新后的事件组值。

例如，把 `uxBitsToSet` 设为 0x04（二进制 0100），会使事件组中的事件位 2 被置位（如果尚未置位），而其他事件位保持不变。

- 返回值

`xEventGroupSetBits()` 返回时事件组的值。请注意，返回值中不一定仍然包含 `uxBitsToSet` 指定的置位状态，因为这些位可能已经被另一个任务再次清零。

9.3.3 xEventGroupSetBitsFromISR() API 函数

`xEventGroupSetBitsFromISR()` 是 `xEventGroupSetBits()` 的中断安全版本。

“给出”信号量是一项确定性操作，因为事先可以确定：给出一个信号量最多只会使一个任务离开阻塞状态。设置事件组中的位时，事先无法确定会有多少任务离开阻塞状态，因此在事件组中置位不是确定性操作。

FreeRTOS 的设计与实现规范不允许在中断服务程序内或中断被禁用时执行非确定性操作。因此，`xEventGroupSetBitsFromISR()` 不会直接在中断服务程序中设置事件位，而是把这项操作延迟到 RTOS 守护任务中执行。

```
BaseType_t xEventGroupSetBitsFromISR( EventGroupHandle_t xEventGroup,
                                       const EventBits_t uxBitsToSet,
                                       BaseType_t *pxHigherPriorityTaskWoken );
```

代码清单 9.3 `xEventGroupSetBitsFromISR()` API 函数原型

`xEventGroupSetBitsFromISR()` 参数和返回值

- `xEventGroup`

要在其中设置位的事件组句柄。该句柄由创建事件组时调用的 `xEventGroupCreate()` 返回。

- `uxBitsToSet`

一个位掩码，用于指定要在事件组中置为 1 的一个或多个事件位。事件组的原值与 `uxBitsToSet` 按位或后，得到更新后的事件组值。

例如，把 `uxBitsToSet` 设为 0x05（二进制 0101），会使事件组中的事件位 2 和事件位 0 被置位（如果尚未置位），而其他事件位保持不变。

- `pxHigherPriorityTaskWoken`

`xEventGroupSetBitsFromISR()` 不会直接在中断服务程序中设置事件位，而是通过向定时器命令队列发送命令，把这项操作延迟到 RTOS 守护任务中执行。如果守护任务原本处于阻塞状态，等待定时器命令队列中出现数据，那么写入定时器命令队列会使守护任务离开阻塞状态。如果守护任务的优先级高于当前正在执行的任务（即被中断的任务），`xEventGroupSetBitsFromISR()` 会在内部把 `*pxHigherPriorityTaskWoken` 设为 `pdTRUE`。

如果 `xEventGroupSetBitsFromISR()` 把该值设为 `pdTRUE`，则应在退出中断前执行上下文切换。这样可确保中断直接返回到守护任务，因为守护任务将是处于就绪状态的最高优先级任务。

- 返回值

可能有两种返回值：

- 只有在数据成功发送到定时器命令队列时才返回 `pdPASS`。
- 如果定时器命令队列已满，导致“设置位”命令无法写入队列，则返回 `pdFALSE`。

9.3.4 xEventGroupWaitBits() API 函数

`xEventGroupWaitBits()` API 函数允许任务读取事件组的值；如果指定的事件位尚未置位，还可以选择让任务进入阻塞状态，等待事件组中的一个或多个事件位被置位。

```
EventBits_t xEventGroupWaitBits( EventGroupHandle_t xEventGroup,  
                                const EventBits_t uxBitsToWaitFor,  
                                const BaseType_t xClearOnExit,  
                                const BaseType_t xWaitForAllBits,  
                                TickType_t xTicksToWait );
```

代码清单 9.4 `xEventGroupWaitBits()` API 函数原型

调度器用来判断任务是否进入阻塞状态以及何时离开阻塞状态的条件，称为“解除阻塞条件”。该条件由 `uxBitsToWaitFor` 和 `xWaitForAllBits` 两个参数的取值共同指定：

- `uxBitsToWaitFor` 指定要检查事件组中的哪些事件位。
- `xWaitForAllBits` 指定使用按位或测试，还是按位与测试。

如果调用 `xEventGroupWaitBits()` 时任务的解除阻塞条件已经满足，任务不会进入阻塞状态。

表 6 给出了会使任务进入或离开阻塞状态的条件示例。表中只显示事件组值和 `uxBitsToWaitFor` 值最低的 4 个二进制位，假定这两个值的其他位均为 0。

事件组 现有值	<code>uxBitsToWaitFor</code> 值	<code>xWaitForAllBits</code> 值	结果行为
0000	0101	<code>pdFALSE</code>	事件组中的位 0 和位 2 均未置位，因此调用任务进入阻塞状态；位 0 或位 2 中任意一个被置位时，任务离开阻塞状态。
0100	0101	<code>pdTRUE</code>	事件组中的位 0 和位 2 尚未同时置位，因此调用任务进入阻塞状态；位 0 和位 2 都被置位时，任务离开阻塞状态。
0100	0110	<code>pdFALSE</code>	<code>xWaitForAllBits</code> 为 <code>pdFALSE</code> ，而 <code>uxBitsToWaitFor</code> 指定的两个位中已有一个置位，因此调用任务不会进入阻塞状态。
0100	0110	<code>pdTRUE</code>	<code>xWaitForAllBits</code> 为 <code>pdTRUE</code> ，而 <code>uxBitsToWaitFor</code> 指定的两个位中只有一个置位，因此调用任务进入阻塞状态；位 1 和位 2 都被置位时，任务离开阻塞状态。

表 6 `uxBitsToWaitFor` 和 `xWaitForAllBits` 参数的作用

调用任务通过 `uxBitsToWaitFor` 参数指定要检查的位，而且通常需要在解除阻塞条件满足后把这些位清零。可以使用 `xEventGroupClearBits()` API 函数清除事件位，但在以下情况下，手动调用该函数清除事件位会在应用程序代码中造成竞态条件：

- 多个任务使用同一个事件组。
- 事件组中的位由另一个任务或中断服务程序设置。

`xClearOnExit` 参数用于避免这些潜在的竞态条件。如果把 `xClearOnExit` 设为 `pdTRUE`，那么从调用任务的角度看，检查并清除事件位是一项原子操作，不会被其他任务或中断打断。

`xEventGroupWaitBits()` 参数和返回值

- `xEventGroup`

包含待读取事件位的事件组句柄。该句柄由创建事件组时调用的 `xEventGroupCreate()` 返回。

- `uxBitsToWaitFor`

一个位掩码，用于指定要在事件组中检查的一个或多个事件位。

例如，如果调用任务希望等待事件组中的事件位 0 和/或事件位 2 被置位，则把

`uxBitsToWaitFor` 设为 0x05（二进制 0101）。更多示例请参见表 6。

- `xClearOnExit`

如果调用任务的解除阻塞条件已经满足，并且 `xClearOnExit` 为 `pdTRUE`，那么调用任务退出 `xEventGroupWaitBits()` API 函数之前，事件组中由 `uxBitsToWaitFor` 指定的事件位会被清零。

如果 `xClearOnExit` 为 `pdFALSE`，则 `xEventGroupWaitBits()` API 函数不会修改事件组中各事件位的状态。

- `xWaitForAllBits`

`uxBitsToWaitFor` 参数指定要在事件组中检查的事件位。`xWaitForAllBits` 则指定：当 `uxBitsToWaitFor` 所指定的一个或多个事件位被置位时，就把调用任务移出阻塞状态；还是只有在指定的所有事件位均被置位时，才把任务移出阻塞状态。

如果 `xWaitForAllBits` 为 `pdFALSE`，那么因等待解除阻塞条件满足而进入阻塞状态的任务，会在 `uxBitsToWaitFor` 指定的任意事件位被置位时离开阻塞状态，或者在 `xTicksToWait` 指定的超时时间到期时离开阻塞状态。

如果 `xWaitForAllBits` 为 `pdTRUE`，那么因等待解除阻塞条件满足而进入阻塞状态的任务，只有在 `uxBitsToWaitFor` 指定的全部事件位都被置位时才会离开阻塞状态，或者在 `xTicksToWait` 指定的超时时间到期时离开阻塞状态。

相关示例请参见表 6。

- `xTicksToWait`

任务为了等待解除阻塞条件满足而保持阻塞状态的最长时间。

如果 `xTicksToWait` 为 0，或者调用 `xEventGroupWaitBits()` 时解除阻塞条件已经满足，`xEventGroupWaitBits()` 会立即返回。

阻塞时间以 Tick 周期为单位，因此它所代表的绝对时间取决于 Tick 频率。宏

`pdMS_TO_TICKS()` 可把以毫秒表示的时间转换为以 Tick 表示的时间。

如果 FreeRTOSConfig.h 中的 `INCLUDE_vTaskSuspend` 设为 1，把 `xTicksToWait` 设为 `portMAX_DELAY` 会使任务无限期等待，不会超时。

- 返回值

如果 `xEventGroupWaitBits()` 因调用任务的解除阻塞条件得到满足而返回，则返回值是条件满足时的事件组值，也就是在 `xClearOnExit` 为 `pdTRUE` 时自动清除任何位之前的值。此时，返回值也满足解除阻塞条件。

如果 `xEventGroupWaitBits()` 因 `xTicksToWait` 参数指定的阻塞时间到期而返回，则返回值是阻塞时间到期时的事件组值。此时，返回值不满足解除阻塞条件。

9.3.5 xEventGroupGetStaticBuffer() API 函数

`xEventGroupGetStaticBuffer()` API 函数用于获取指向静态创建事件组缓冲区的指针。该缓冲区就是创建事件组时传入的缓冲区。

注意：切勿从中断服务程序调用 `xEventGroupGetStaticBuffer()`。

```
BaseType_t xEventGroupGetStaticBuffer( EventGroupHandle_t xEventGroup,  
StaticEventGroup_t ** ppxEventGroupBuffer );
```

代码清单 9.5 `xEventGroupGetStaticBuffer()` API 函数原型

`xEventGroupGetStaticBuffer()` 参数和返回值

- `xEventGroup`

要获取其缓冲区的事件组。该事件组必须由 `xEventGroupCreateStatic()` 创建。

- `ppxEventGroupBuffer`

用于返回指向事件组数据结构缓冲区的指针。该缓冲区就是创建事件组时传入的缓冲区。

- 返回值

可能有两种返回值：

- 如果成功获取缓冲区，则返回 `pdTRUE`。
- 如果未能获取缓冲区，则返回 `pdFALSE`。

示例 9.1 试验事件组

本示例演示如何：

- 创建事件组。
- 从中断服务程序设置事件组中的位。
- 从任务设置事件组中的位。
- 阻塞在事件组上。

本示例先把 `xEventGroupWaitBits()` 的 `xWaitForAllBits` 参数设为 `pdFALSE` 执行一次，再把它设为 `pdTRUE` 执行一次，以展示该参数的作用。

事件位 0 和事件位 1 由任务设置，事件位 2 由中断服务程序设置。代码清单 9.6 使用 `#define` 语句为这三个位定义了便于理解的名称。

```
/* 事件组中各事件位的定义。 */  
#define mainFIRST_TASK_BIT ( 1UL << 0UL ) /* 事件位 0, 由任务设置 */  
#define mainSECOND_TASK_BIT ( 1UL << 1UL ) /* 事件位 1, 由任务设置 */  
#define mainISR_BIT ( 1UL << 2UL ) /* 事件位 2, 由 ISR 设置 */
```

代码清单 9.6 示例 9.1 使用的事件位定义

代码清单 9.7 展示了设置事件位 0 和事件位 1 的任务实现。任务在循环中反复设置一个位、再设置另一个位，每次调用 `xEventGroupSetBits()` 之间延迟 200 毫秒。每次设置位之前都会输出一条字符串，以便从控制台观察执行顺序。

```

static void vEventBitSettingTask( void *pvParameters )
{
    const TickType_t xDelay200ms = pdMS_TO_TICKS( 200UL ), xDontBlock = 0;

    for( ;; )
    {
        /* 开始下一轮循环前短暂延迟。 */
        vTaskDelay( xDelay200ms );

        /* 输出消息，说明任务即将设置事件位 0，然后设置事件位 0。 */
        vPrintString( "Bit setting task -\t about to set bit 0.\r\n" );
        xEventGroupSetBits( xEventGroup, mainFIRST_TASK_BIT );

        /* 设置另一个位之前短暂延迟。 */
        vTaskDelay( xDelay200ms );

        /* 输出消息，说明任务即将设置事件位 1，然后设置事件位 1。 */
        vPrintString( "Bit setting task -\t about to set bit 1.\r\n" );
        xEventGroupSetBits( xEventGroup, mainSECOND_TASK_BIT );
    }
}

```

代码清单 9.7 示例 9.1 中设置事件组两个位的任务

代码清单 9.8 展示了在事件组中设置位 2 的中断服务程序实现。设置该位之前同样会输出一条字符串，以便从控制台观察执行顺序。不过，不能直接在中断服务程序中执行控制台输出，因此这里使用 `xTimerPendFunctionCallFromISR()`，在 RTOS 守护任务的上下文中完成输出。

与前面的示例一样，中断服务程序由一个简单的周期任务触发，该任务强制产生软件中断。本示例每 500 毫秒产生一次中断。

```

static uint32_t ulEventBitSettingISR( void )
{
    /* 字符串不在中断服务程序中直接输出，而是发送给 RTOS 守护任务输出。
       因此将其声明为 static，确保编译器不会把字符串分配在 ISR 栈上，
       因为守护任务输出字符串时，ISR 的栈帧已经不存在。 */
    static const char *pcString = "Bit setting ISR -\t about to set bit 2.\r\n";
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    /* 输出一条消息，说明即将设置位 2。不能从 ISR 直接输出消息，因此挂起
       一个函数调用，让它在 RTOS 守护任务的上下文中执行，从而延迟实际输出。 */
    xTimerPendFunctionCallFromISR( vPrintStringFromDaemonTask,
                                    ( void * ) pcString,
                                    0,
                                    &xHigherPriorityTaskWoken );

    /* 设置事件组中的位 2。 */
    xEventGroupSetBitsFromISR( xEventGroup,
                               mainISR_BIT,
                               &xHigherPriorityTaskWoken );

    /* xTimerPendFunctionCallFromISR() 和 xEventGroupSetBitsFromISR() 都会
       写入定时器命令队列，并且都使用同一个 xHigherPriorityTaskWoken 变量。
       如果写入定时器命令队列使 RTOS 守护任务离开阻塞状态，而且 RTOS
       守护任务的优先级高于当前正在执行的任务（即被本中断打断的任务），
       则 xHigherPriorityTaskWoken 会被设为 pdTRUE。

       xHigherPriorityTaskWoken 作为参数传给 portYIELD_FROM_ISR()。如果
       xHigherPriorityTaskWoken 等于 pdTRUE，调用 portYIELD_FROM_ISR()
       会请求上下文切换；如果它仍为 pdFALSE，则调用不会产生任何效果。

       Windows 移植版本使用的 portYIELD_FROM_ISR() 实现包含 return 语句，
       因此本函数没有显式返回值。 */

    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}

```

代码清单 9.8 示例 9.1 中设置事件组位 2 的 ISR

代码清单 9.9 展示了调用 `xEventGroupWaitBits()` 并阻塞在事件组上的任务实现。对于事件组中每一个已经置位的位，任务都会输出一条字符串。

`xEventGroupWaitBits()` 的 `xClearOnExit` 参数设为 `pdTRUE`，因此，使 `xEventGroupWaitBits()` 返回的一个或多个事件位会在函数返回前自动清零。

```

static void vEventBitReadingTask( void *pvParameters )
{
    EventBits_t xEventGroupValue;
    const EventBits_t xBitsToWaitFor = ( mainFIRST_TASK_BIT |
                                         mainSECOND_TASK_BIT |
                                         mainISR_BIT );

    for( ;; )
    {
        /* 阻塞等待事件组中的事件位被置位。 */
        xEventGroupValue = xEventGroupWaitBits( /* 要读取的事件组 */
                                                xEventGroup,

                                                /* 要检查的位 */
                                                xBitsToWaitFor,

                                                /* 如果解除阻塞条件满足,
                                                则在退出时清除这些位 */
                                                pdTRUE,

                                                /* 不等待所有位。本示例第二次
                                                执行时将该参数设为 pdTRUE。 */
                                                pdFALSE,

                                                /* 永不超时。 */
                                                portMAX_DELAY );

        /* 对每一个已经置位的位输出一条消息。 */
        if( ( xEventGroupValue & mainFIRST_TASK_BIT ) != 0 )
        {
            vPrintString( "Bit reading task -\t Event bit 0 was set\r\n" );
        }

        if( ( xEventGroupValue & mainSECOND_TASK_BIT ) != 0 )
        {
            vPrintString( "Bit reading task -\t Event bit 1 was set\r\n" );
        }

        if( ( xEventGroupValue & mainISR_BIT ) != 0 )
        {
            vPrintString( "Bit reading task -\t Event bit 2 was set\r\n" );
        }
    }
}

```

代码清单 9.9 示例 9.1 中阻塞等待事件位被置位的任务

main() 函数先创建事件组和任务，然后启动调度器，其实现见代码清单 9.10。读取事件组的任务优先级高于写入事件组的任务，因此每当读取任务的解除阻塞条件满足时，它都会抢占写入任务。


```

int main( void )
{
    /* 事件组必须先创建才能使用。 */
    xEventGroup = xEventGroupCreate();

    /* 创建在事件组中设置事件位的任务。 */
    xTaskCreate( vEventBitSettingTask, "Bit Setter", 1000, NULL, 1, NULL );

    /* 创建等待事件位被置位的任务。 */
    xTaskCreate( vEventBitReadingTask, "Bit Reader", 1000, NULL, 2, NULL );

    /* 创建用于周期性产生软件中断的任务。 */
    xTaskCreate( vInterruptGenerator, "Int Gen", 1000, NULL, 3, NULL );

    /* 安装软件中断处理程序。所需语法取决于使用的 FreeRTOS 移植版本。
       此处所示语法只能用于 FreeRTOS Windows 移植版本，其中的中断
       仅为模拟中断。 */
    vPortSetInterruptHandler( mainINTERRUPT_NUMBER, ulEventBitSettingISR );

    /* 启动调度器，使创建的任务开始执行。 */
    vTaskStartScheduler();

    /* 程序永远不应执行到下一行。 */
    for( ;; );
    return 0;
}

```

代码清单 9.10 在示例 9.1 中创建事件组和任务

图 9.3 展示了把 `xEventGroupWaitBits()` 的 `xWaitForAllBits` 参数设为 `pdFALSE` 时，执行示例 9.1 产生的输出。由于 `xWaitForAllBits` 为 `pdFALSE`，每当任意事件位被置位时，读取事件组的任务都会立即离开阻塞状态并执行。

```
C:\Windows\system32\cmd.exe - rtsdemo
Bit setting task -      about to set bit 1.
Bit reading task -      event bit 1 was set

Bit setting task -      about to set bit 0.
Bit reading task -      event bit 0 was set

Bit setting task -      about to set bit 1.
Bit reading task -      event bit 1 was set

Bit setting ISR -       about to set bit 2.
Bit reading task -      event bit 2 was set

Bit setting task -      about to set bit 0.
Bit reading task -      event bit 0 was set

Bit setting task -      about to set bit 1.
Bit reading task -      event bit 1 was set

Bit setting ISR -       about to set bit 2.
Bit reading task -      event bit 2 was set

Bit setting task -      about to set bit 0.
Bit reading task -      event bit 0 was set
```

图 9.3 把 `xWaitForAllBits` 设为 `pdFALSE` 时执行示例 9.1 产生的输出

图 9.4 展示了把 `xEventGroupWaitBits()` 的 `xWaitForAllBits` 参数设为 `pdTRUE` 时，执行示例 9.1 产生的输出。由于 `xWaitForAllBits` 为 `pdTRUE`，只有在三个事件位全部被置位后，读取事件组的任务才会离开阻塞状态。

```
C:\Windows\system32\cmd.exe - rtsdemo
Bit setting task -      about to set bit 1.
Bit setting task -      about to set bit 0.
Bit setting ISR -       about to set bit 2.
Bit reading task -      event bit 0 was set
Bit reading task -      event bit 1 was set
Bit reading task -      event bit 2 was set

Bit setting task -      about to set bit 1.
Bit setting task -      about to set bit 0.
Bit setting ISR -       about to set bit 2.
Bit reading task -      event bit 0 was set
Bit reading task -      event bit 1 was set
Bit reading task -      event bit 2 was set

Bit setting task -      about to set bit 1.
Bit setting task -      about to set bit 0.
Bit setting task -      about to set bit 1.
Bit setting ISR -       about to set bit 2.
Bit reading task -      event bit 0 was set
Bit reading task -      event bit 1 was set
Bit reading task -      event bit 2 was set

Bit setting task -      about to set bit 0.
Bit setting task -      about to set bit 1.
```

图 9.4 把 `xWaitForAllBits` 设为 `pdTRUE` 时执行示例 9.1 产生的输出

9.4 使用事件组同步任务

应用程序的设计有时要求两个或多个任务彼此同步。例如，设想任务 A 接收到一个事件后，把该事件所需的部分处理工作交给任务 B、任务 C 和任务 D。如果任务 B、C、D 全部完成对上一个事件的处理之前，任务 A 不能接收下一个事件，那么这四个任务就必须彼此同步。每个任务完成自己的处理工作后到达同步点，在其他任务也完成处理之前不能继续执行。只有当四个任务全部到达各自的同步点后，任务 A 才能接收下一个事件。

FreeRTOS+TCP 的一个演示项目给出了更具体的任务同步示例。该演示让两个任务共享一个 TCP 套接字：一个任务向套接字发送数据，另一个任务从同一套接字接收数据^[2]。在确认另一个任务不会再次访问套接字之前，任何一个任务关闭套接字都不安全。如果其中一个任务希望关闭套接字，它必须先把这一意图通知另一个任务，然后等待另一个任务停止使用套接字。代码清单 9.11 的伪代码展示了发送任务希望关闭套接字时的情形。

代码清单 9.11 所示情形很简单，因为只有两个任务需要彼此同步。但可以看出，如果还有其他任务执行依赖套接字保持打开的处理工作，这种情形会迅速变得复杂，并要求更多任务参与同步。

```

void SocketTxTask( void *pvParameters )
{
    xSocket_t xSocket;
    uint32_t ulTxCount = 0UL;

    for( ;; )
    {
        /* 创建新套接字。本任务向该套接字发送数据，另一个任务从中接收数据。 */
        xSocket = FreeRTOS_socket( ... );

        /* 连接套接字。 */
        FreeRTOS_connect( xSocket, ... );

        /* 使用队列把套接字发送给接收数据的任务。 */
        xQueueSend( xSocketPassingQueue, &xSocket, portMAX_DELAY );

        /* 向套接字发送 1000 条消息，然后关闭套接字。 */
        for( ulTxCount = 0; ulTxCount < 1000; ulTxCount++ )
        {
            if( FreeRTOS_send( xSocket, ... ) < 0 )
            {
                /* 发生意外错误，退出循环，随后关闭套接字。 */
                break;
            }
        }

        /* 通知 Rx 任务：Tx 任务希望关闭套接字。 */
        TxTaskWantsToCloseSocket();

        /* 这是 Tx 任务的同步点。Tx 任务在此等待 Rx 任务到达它的同步点。
           只有当 Rx 任务不再使用套接字时，它才会到达同步点，此时可以
           安全关闭套接字。 */
        xEventGroupSync( ... );

        /* 两个任务都不再使用套接字。关闭连接，然后关闭套接字。 */
        FreeRTOS_shutdown( xSocket, ... );
        WaitForSocketToDisconnect();
        FreeRTOS_closesocket( xSocket );
    }
}
/*-----*/

void SocketRxTask( void *pvParameters )
{
    xSocket_t xSocket;

    for( ;; )
    {
        /* 等待接收由 Tx 任务创建并连接的套接字。 */
        xQueueReceive( xSocketPassingQueue, &xSocket, portMAX_DELAY );

        /* 持续从套接字接收数据，直到 Tx 任务希望关闭套接字。 */
        while( TxTaskWantsToCloseSocket() == pdFALSE )
        {
            /* 接收并处理数据。 */
            FreeRTOS_recv( xSocket, ... );
        }
    }
}

```

```

    ProcessReceivedData();
}

/* 这是 Rx 任务的同步点。只有不再使用套接字时, Rx 任务才会到达
   这里, 因此此时 Tx 任务可以安全关闭套接字。 */
xEventGroupSync( ... );
}
}

```

代码清单 9.11 两个任务相互同步的伪代码, 用于确保关闭共享 TCP 套接字之前两个任务都不再使用它

可以使用事件组创建同步点:

- 为必须参与同步的每个任务, 在事件组中分别分配一个唯一的事件位。
- 每个任务到达同步点时, 设置代表自己的事件位。
- 设置自己的事件位后, 每个任务都阻塞在事件组上, 等待代表其他所有同步任务的事件位也被置位。

但是, 不能在这种场景中组合使用 `xEventGroupSetBits()` 和 `xEventGroupWaitBits()` API 函数。否则, 设置事件位 (表示任务已到达同步点) 与检查事件位 (判断其他同步任务是否已到达同步点) 会成为两个独立操作。要理解这为何会导致问题, 可以考虑任务 A、任务 B 和任务 C 尝试使用一个事件组进行同步的场景:

1. 任务 A 和任务 B 已经到达同步点, 因此它们的事件位已在事件组中置位。两个任务都处于阻塞状态, 等待任务 C 的事件位也被置位。
2. 任务 C 到达同步点, 并调用 `xEventGroupSetBits()` 在事件组中设置自己的位。任务 C 的位一经设置, 任务 A 和任务 B 就会离开阻塞状态, 并清除全部三个事件位。
3. 随后任务 C 调用 `xEventGroupWaitBits()`, 等待全部三个事件位被置位。但此时三个事件位都已经被清除, 任务 A 和任务 B 也已经离开各自的同步点, 因此同步失败。

要成功使用事件组创建同步点, 设置事件位和随后检查事件位必须作为一项不可中断的操作执行。FreeRTOS 为此提供了 `xEventGroupSync()` API 函数。

9.4.1 xEventGroupSync() API 函数

`xEventGroupSync()` 使两个或多个任务能够借助事件组彼此同步。该函数让任务在事件组中设置一个或多个事件位, 然后等待同一事件组中的一组事件位被置位, 并把这两个步骤作为一项不可中断的操作执行。

`xEventGroupSync()` 的 `uxBitsToWaitFor` 参数指定调用任务的解除阻塞条件。如果函数因该条件得到满足而返回，那么在 `xEventGroupSync()` 返回前，`uxBitsToWaitFor` 指定的事件位会被清零。

```
EventBits_t xEventGroupSync( EventGroupHandle_t xEventGroup,  
                             const EventBits_t uxBitsToSet,  
                             const EventBits_t uxBitsToWaitFor,  
                             TickType_t xTicksToWait );
```

代码清单 9.12 `xEventGroupSync()` API 函数原型

`xEventGroupSync()` 参数和返回值

- `xEventGroup`

要在其中设置并随后检查事件位的事件组句柄。该句柄由创建事件组时调用的 `xEventGroupCreate()` 返回。

- `uxBitsToSet`

一个位掩码，用于指定要在事件组中置为 1 的一个或多个事件位。事件组的原值与 `uxBitsToSet` 按位或后，得到更新后的事件组值。

例如，把 `uxBitsToSet` 设为 0x04（二进制 0100），会使事件位 2 被置位（如果尚未置位），而其他事件位保持不变。

- `uxBitsToWaitFor`

一个位掩码，用于指定要在事件组中检查的一个或多个事件位。

例如，如果调用任务希望等待事件组中的事件位 0、1 和 2 都被置位，则把 `uxBitsToWaitFor` 设为 0x07（二进制 111）。

- `xTicksToWait`

任务为了等待解除阻塞条件满足而保持阻塞状态的最长时间。

如果 `xTicksToWait` 为 0，或者调用 `xEventGroupSync()` 时解除阻塞条件已经满足，`xEventGroupSync()` 会立即返回。

阻塞时间以 Tick 周期为单位，因此它所代表的绝对时间取决于 Tick 频率。宏

`pdMS_TO_TICKS()` 可把以毫秒表示的时间转换为以 Tick 表示的时间。

如果 FreeRTOSConfig.h 中的 `INCLUDE_vTaskSuspend` 设为 1，把 `xTicksToWait` 设为 `portMAX_DELAY` 会使任务无限期等待，不会超时。

- 返回值

如果 `xEventGroupSync()` 因调用任务的解除阻塞条件得到满足而返回，则返回值是条件满足时的事件组值，也就是所有事件位自动清零之前的值。此时，返回值也满足调用任务的解除阻塞条件。

如果 `xEventGroupSync()` 因 `xTicksToWait` 参数指定的阻塞时间到期而返回，则返回值是阻塞时间到期时的事件组值。此时，返回值不满足调用任务的解除阻塞条件。

示例 9.2 同步任务

示例 9.2 使用 `xEventGroupSync()` 同步同一个任务实现的三个实例。通过任务参数向每个实例传入一个事件位，任务调用 `xEventGroupSync()` 时将设置该事件位。

任务在调用 `xEventGroupSync()` 前输出一条消息，并在 `xEventGroupSync()` 返回后再次输出消息。每条消息都包含时间戳，因此可以从输出中观察执行顺序。示例使用伪随机延迟，避免所有任务同时到达同步点。

任务实现见代码清单 9.13。

```

static void vSyncingTask( void *pvParameters )
{
    const TickType_t xMaxDelay = pdMS_TO_TICKS( 4000UL );
    const TickType_t xMinDelay = pdMS_TO_TICKS( 200UL );
    TickType_t xDelayTime;
    EventBits_t uxThisTasksSyncBit;
    const EventBits_t uxAllSyncBits = ( mainFIRST_TASK_BIT |
                                         mainSECOND_TASK_BIT |
                                         mainTHIRD_TASK_BIT );

    /* 创建本任务的三个实例，每个任务在同步时使用不同的事件位。通过任务
       参数向每个任务实例传入要使用的事件位，并把它保存到
       uxThisTasksSyncBit 变量中。 */
    uxThisTasksSyncBit = ( EventBits_t ) pvParameters;

    for( ;; )
    {
        /* 延迟一段伪随机时间，以模拟本任务执行某项操作所花的时间。这样可
           防止三个任务实例同时到达同步点，从而更容易观察示例的行为。 */
        xDelayTime = ( rand() % xMaxDelay ) + xMinDelay;
        vTaskDelay( xDelayTime );

        /* 输出消息，说明本任务已经到达同步点。pcTaskGetTaskName() 是一个
           API 函数，返回创建任务时为其指定的名称。 */
        vPrintTwoStrings( pcTaskGetTaskName( NULL ), "reached sync point" );

        /* 等待所有任务到达各自的同步点。 */
        xEventGroupSync( /* 用于同步的事件组。 */
                        xEventGroup,

                        /* 本任务设置该位，表示自己已经到达同步点。 */
                        uxThisTasksSyncBit,

                        /* 要等待的位，每个参与同步的任务对应一个位。 */
                        uxAllSyncBits,

                        /* 无限期等待三个任务全部到达同步点。 */
                        portMAX_DELAY );

        /* 输出消息，说明本任务已经越过同步点。这里使用无限期延迟，因此只有
           在所有任务都到达各自的同步点后，才会执行下一行。 */
        vPrintTwoStrings( pcTaskGetTaskName( NULL ), "exited sync point" );
    }
}

```

代码清单 9.13 示例 9.2 使用的任务实现

`main()` 函数创建事件组和全部三个任务，然后启动调度器，其实现见代码清单 9.14。


```

/* 事件组中各事件位的定义。 */

#define mainFIRST_TASK_BIT ( 1UL << 0UL ) /* 事件位 0, 由第 1 个任务设置 */
#define mainSECOND_TASK_BIT( 1UL << 1UL ) /* 事件位 1, 由第 2 个任务设置 */
#define mainTHIRD_TASK_BIT ( 1UL << 2UL ) /* 事件位 2, 由第 3 个任务设置 */

/* 声明用于同步三个任务的事件组。 */
EventGroupHandle_t xEventGroup;

int main( void )
{
    /* 事件组必须先创建才能使用。 */
    xEventGroup = xEventGroupCreate();

    /* 创建任务的三个实例。每个任务都有不同的名称，随后会输出这些名称，以便
       直观看出正在执行的是哪个任务。通过任务参数传入任务到达同步点时要使用
       的事件位。 */
    xTaskCreate( vSyncingTask, "Task 1", 1000, mainFIRST_TASK_BIT, 1, NULL );
    xTaskCreate( vSyncingTask, "Task 2", 1000, mainSECOND_TASK_BIT, 1, NULL );
    xTaskCreate( vSyncingTask, "Task 3", 1000, mainTHIRD_TASK_BIT, 1, NULL );

    /* 启动调度器，使创建的任务开始执行。 */
    vTaskStartScheduler();

    /* 与以往一样，程序永远不应执行到下一行。 */
    for( ;; );
    return 0;
}

```

代码清单 9.14 示例 9.2 使用的 `main()` 函数

图 9.5 展示了执行示例 9.2 时产生的输出。可以看到，虽然各任务在不同的伪随机时刻到达同步点，但它们会在同一时刻^[3]离开同步点，也就是最后一个任务到达同步点的时刻。

```
C:\Windows\system32\cmd.exe - rtosdemo
At time 211664: Task 1 reached sync point
At time 211664: Task 1 exited sync point
At time 211664: Task 2 exited sync point
At time 211664: Task 3 exited sync point
At time 212702: Task 2 reached sync point
At time 214400: Task 1 reached sync point
At time 215439: Task 3 reached sync point
At time 215439: Task 3 exited sync point
At time 215439: Task 2 exited sync point
At time 215440: Task 1 exited sync point
At time 217671: Task 2 reached sync point
At time 218622: Task 1 reached sync point
At time 219402: Task 3 reached sync point
At time 219402: Task 3 exited sync point
At time 219402: Task 2 exited sync point
At time 219402: Task 1 exited sync point
At time 220189: Task 2 reached sync point
At time 222656: Task 3 reached sync point
At time 222673: Task 1 reached sync point
At time 222673: Task 1 exited sync point
At time 222673: Task 2 exited sync point
At time 222673: Task 3 exited sync point
At time 223252: Task 1 reached sync point
At time 223682: Task 3 reached sync point
```

图 9.5 执行示例 9.2 时产生的输出

1. `configTICK_TYPE_WIDTH_IN_BITS` 用于配置保存 RTOS Tick 计数的数据类型，看起来似乎与事件组功能无关。它之所以会影响 `EventBits_t` 类型，是 FreeRTOS 内部实现方式造成的。尽管把 `configTICK_TYPE_WIDTH_IN_BITS` 设为 `TICK_TYPE_WIDTH_16_BITS` 可能有其优势，但只有当 FreeRTOS 运行所在的体系结构处理 16 位类型比处理 32 位类型更高效时，才应这样配置。↩
2. 在编写本书时，这是让多个任务共享同一个 FreeRTOS+TCP 套接字的唯一方式。↩
3. 图 9.5 展示的是示例在 FreeRTOS Windows 移植版本中的运行情况。该移植版本不具备真正的实时行为，尤其是在调用 Windows 系统函数向控制台输出时，因此图中会出现一些时序偏差。↩

10 任务通知

10.1 简介

FreeRTOS 应用程序通常由一系列相互独立的任务组成，这些任务彼此通信，共同实现系统功能。任务通知是一种高效机制，允许一个任务直接通知另一个任务。

10.1.1 通过中介对象通信

本书已经介绍了多种任务间通信方式。此前介绍的方法都要求先创建通信对象，例如队列、事件组和各种类型的信号量。

使用通信对象时，事件和数据不是直接发送给接收任务或接收 ISR，而是发送到通信对象。同样，任务和 ISR 从通信对象接收事件和数据，而不是直接从发送事件或数据的任务或 ISR 接收。图 10.1 展示了这一过程。

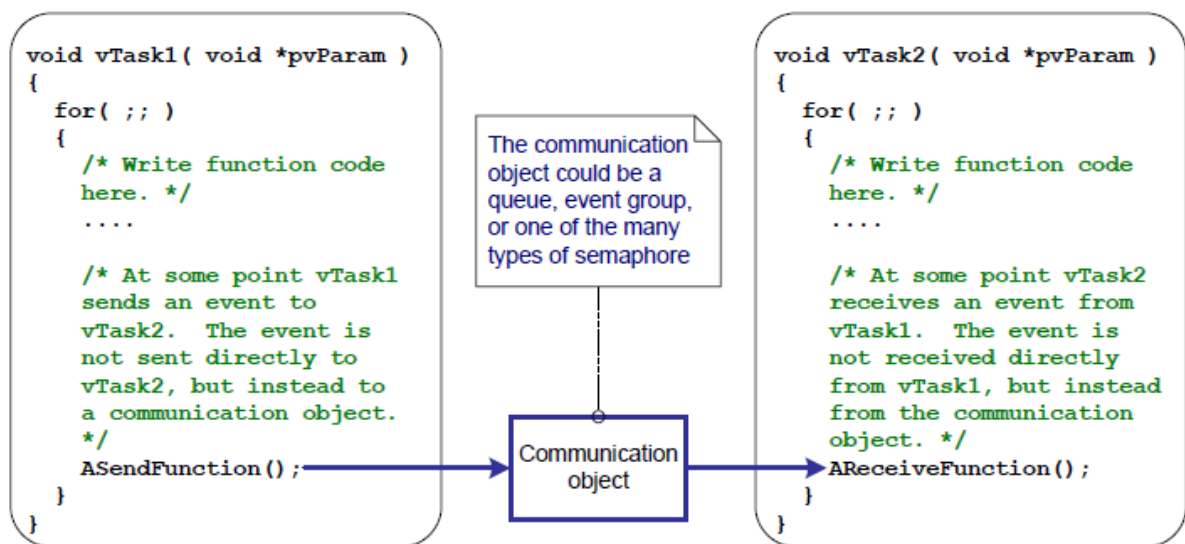


图 10.1 使用通信对象把事件从一个任务发送给另一个任务

10.1.2 任务通知：直接面向任务的通信

“任务通知”让任务无需单独的通信对象即可与其他任务交互，并与 ISR 同步。借助任务通知，任务或 ISR 可以把事件直接发送给接收任务。图 10.2 展示了这一过程。

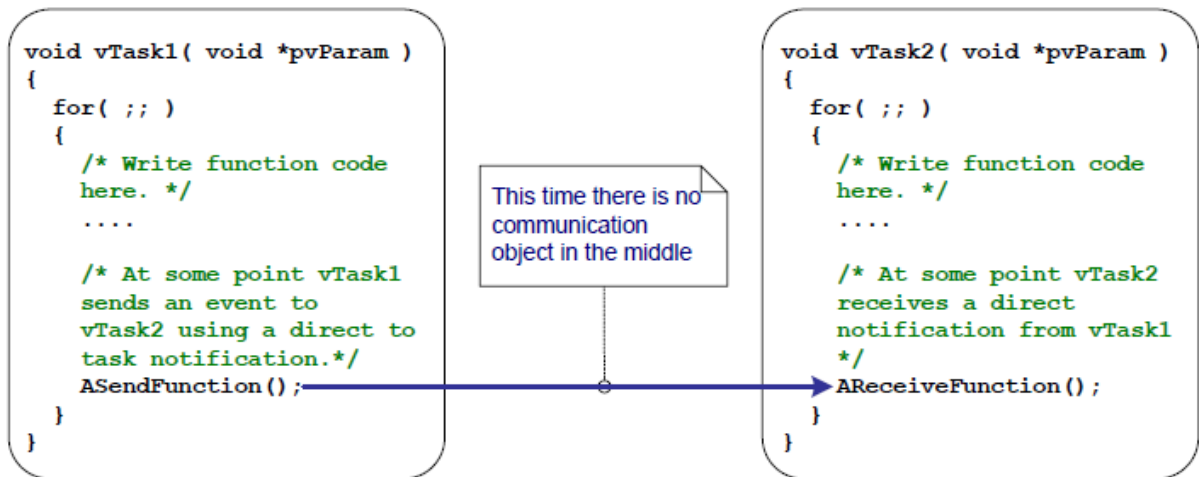


图 10.2 使用任务通知把事件直接从一个任务发送给另一个任务

任务通知功能是可选的。要启用任务通知，请在 FreeRTOSConfig.h 中把 `configUSE_TASK_NOTIFICATIONS` 设为 1。

当 `configUSE_TASK_NOTIFICATIONS` 为 1 时，每个任务至少有一个“通知状态”和一个“通知值”。通知状态可以是“待处理”或“非待处理”，通知值是一个 32 位无符号整数。任务收到通知时，其通知状态变为待处理；任务读取通知值时，其通知状态变为非待处理。如果把 `configTASK_NOTIFICATION_ARRAY_ENTRIES` 设为大于 1 的值，每个任务就拥有多个按索引标识的通知状态和通知值。

任务可以进入阻塞状态，等待其通知状态变为待处理，并可选择设置超时时间。

10.1.3 范围

本章讨论：

- 任务的通知状态和通知值。
- 如何以及何时用任务通知替代信号量等通信对象。

- 使用任务通知替代通信对象的优点。

10.2 任务通知的优点与局限

10.2.1 任务通知的性能优势

使用任务通知向任务发送事件或数据，明显快于使用队列、信号量或事件组执行等效操作。

10.2.2 任务通知的 RAM 占用优势

同样，使用任务通知向任务发送事件或数据，所需 RAM 明显少于使用队列、信号量或事件组执行等效操作。这是因为每个通信对象都必须先创建才能使用，而启用任务通知只会产生固定开销。每个任务的任务通知占用 `configTASK_NOTIFICATION_ARRAY_ENTRIES` * 5 字节 RAM。

`configTASK_NOTIFICATION_ARRAY_ENTRIES` 的默认值为 1，因此默认情况下每个任务的任务通知占用 5 字节。

10.2.3 任务通知的局限

任务通知比通信对象更快、占用的 RAM 更少，但并不适用于所有场景。以下场景不能使用任务通知：

- 向 ISR 发送事件或数据

通信对象既可将事件和数据从 ISR 发送给任务，也可从任务发送给 ISR。

任务通知可将事件和数据从 ISR 发送给任务，但不能从任务发送给 ISR。

- 允许多个任务接收

任何知道通信对象句柄的任务或 ISR 都可以访问该对象。任意数量的任务和 ISR 都能处理发送到某个通信对象的事件或数据。

任务通知直接发送给接收任务，因此只有通知的目标任务能够处理它。不过，这在实际应用中很少构成限制，因为多个任务和 ISR 向同一通信对象发送数据很常见，而多个任务和 ISR 从同一通信对象接收数据并不常见。

- 缓冲多个数据项

队列是一种可同时保存多个数据项的通信对象。已经发送到队列但尚未被接收的数据，会缓存在队列对象中。

任务通知通过更新接收任务的通知值来发送数据，而任务的通知值一次只能保存一个值。

- 向多个任务广播

事件组是一种可同时向多个任务发送事件的通信对象。

任务通知直接发送给接收任务，因此只有接收任务能够处理它。

- 在阻塞状态下等待发送完成

如果通信对象暂时无法再写入数据或事件，例如队列已满，那么尝试写入该对象的任务可以选择进入阻塞状态，等待写操作完成。

如果任务尝试向一个已经有待处理通知的任务发送通知，那么发送任务无法进入阻塞状态，等待接收任务复位其通知状态。后文会看到，在实际使用任务通知的场景中，这很少构成限制。

10.3 使用任务通知

10.3.1 任务通知 API 选项

任务通知功能非常强大，通常可以替代二值信号量、计数信号量、事件组，有时甚至可以替代队列。使用 `xTaskNotify()` API 函数发送任务通知，并使用 `xTaskNotifyWait()` API 函数接收任务通知，即可覆盖这些使用场景。

不过，大多数情况下并不需要 `xTaskNotify()` 和 `xTaskNotifyWait()` 提供的全部灵活性，使用更简单的函数已经足够。因此，FreeRTOS 提供 `xTaskNotifyGive()` 作为 `xTaskNotify()` 更简单但灵活性较低的替代函数，并提供 `ulTaskNotifyTake()` 作为 `xTaskNotifyWait()` 更简单但灵活性较低的替代函数。

任务通知系统不限于单个通知。配置参数 `configTASK_NOTIFICATION_ARRAY_ENTRIES` 默认值为 1；如果把它设为大于 1 的值，每个任务内部都会创建一个通知数组，从而可以按索引管理通知。每个任务通知 API 都有索引版本。调用非索引版本时访问的是 `notification[0]`，也就是数组中的第一个通知。索引版本的函数名带有 `Indexed` 后缀，例如 `xTaskNotify` 的索引版本是 `xTaskNotifyIndexed`。为简化说明，本书只使用各函数的非索引版本。

任务通知 API 实际上由宏实现，这些宏调用对应 API 类型底层的 `Generic` 版本。为简化表述，本书仍把这些 API 宏称为函数。

10.3.1.1 API 函数完整列表 ²⁷

- `xTaskNotifyGive`
- `xTaskNotifyGiveIndexed`

- `vTaskNotifyGiveFromISR`
- `vTaskNotifyGiveIndexedFromISR`
- `ulTaskNotifyTake`
- `ulTaskNotifyTakeIndexed`
- `xTaskNotify`
- `xTaskNotifyIndexed`
- `xTaskNotifyWait`
- `xTaskNotifyWaitIndexed`
- `xTaskNotifyStateClear`
- `xTaskNotifyStateClearIndexed`
- `ulTaskNotifyValueClear`
- `ulTaskNotifyValueClearIndexed`
- `xTaskNotifyAndQueryIndexedFromISR`
- `xTaskNotifyAndQueryFromISR`
- `xTaskNotifyFromISR`
- `xTaskNotifyIndexedFromISR`
- `xTaskNotifyAndQuery`
- `xTaskNotifyAndQueryIndexed`

(27): 这些函数实际上由宏实现。

注意：没有用于接收通知的 `FromISR` 函数，因为通知总是发送给任务，而中断并不隶属于任何任务。

10.3.2 xTaskNotifyGive() API 函数

`xTaskNotifyGive()` 直接向任务发送通知，并递增接收任务的通知值。如果接收任务的通知状态尚非待处理，调用 `xTaskNotifyGive()` 会把它设为待处理。

`xTaskNotifyGive()` 使任务通知能够作为二值信号量或计数信号量更轻量、更快速的替代方案。

```
BaseType_t xTaskNotifyGive( TaskHandle_t xTaskToNotify );
BaseType_t xTaskNotifyGiveIndexed( TaskHandle_t xTaskToNotify, UBaseType_t uxIndexToNotify );
```

代码清单 10.1 `xTaskNotifyGive()` API 函数原型

`xTaskNotifyGive()` / `xTaskNotifyGiveIndexed()` 参数和返回值

- `xTaskToNotify`

接收通知的任务句柄。有关如何获取任务句柄的信息，请参见 `xTaskCreate()` API 函数的 `pxCreatedTask` 参数。

- `uxIndexToNotify`

通知数组中的索引。

- 返回值

`xTaskNotifyGive()` 是一个调用 `xTaskNotify()` 的宏。该宏传给 `xTaskNotify()` 的参数决定了唯一可能的返回值是 `pdPASS`。本章稍后介绍 `xTaskNotify()`。

10.3.3 `vTaskNotifyGiveFromISR()` API 函数

`vTaskNotifyGiveFromISR()` 是可在中断服务程序中使用的 `xTaskNotifyGive()` 版本。

```
void vTaskNotifyGiveFromISR( TaskHandle_t xTaskToNotify,  
                             BaseType_t *pxHigherPriorityTaskWoken );
```

代码清单 10.2 `vTaskNotifyGiveFromISR()` API 函数原型

`vTaskNotifyGiveFromISR()` 参数和返回值

- `xTaskToNotify`

接收通知的任务句柄。有关如何获取任务句柄的信息，请参见 `xTaskCreate()` API 函数的 `pxCreatedTask` 参数。

- `pxHigherPriorityTaskWoken`

如果接收通知的任务正处于阻塞状态，等待接收通知，那么发送通知会使该任务离开阻塞状态。

如果调用 `vTaskNotifyGiveFromISR()` 使任务离开阻塞状态，而且该任务的优先级高于当前正在执行的任务（即被中断的任务），`vTaskNotifyGiveFromISR()` 会在内部把 `*pxHigherPriorityTaskWoken` 设为 `pdTRUE`。

如果 `vTaskNotifyGiveFromISR()` 把该值设为 `pdTRUE`，则应在退出中断前执行上下文切换，以确保中断直接返回到处于就绪状态的最高优先级任务。

与所有中断安全 API 函数一样，使用 `pxHigherPriorityTaskWoken` 参数之前必须先把它设为 `pdFALSE`。

10.3.4 ulTaskNotifyTake() API 函数

`ulTaskNotifyTake()` 允许任务进入阻塞状态，等待其通知值大于 0，并在返回前递减通知值或将其清零。

`ulTaskNotifyTake()` 使任务通知能够作为二值信号量或计数信号量更轻量、更快速的替代方案。

```
uint32_t ulTaskNotifyTake( BaseType_t xClearCountOnExit, TickType_t
xTicksToWait );
```

代码清单 10.3 `ulTaskNotifyTake()` API 函数原型

`ulTaskNotifyTake()` 参数和返回值

- `xClearCountOnExit`

如果 `xClearCountOnExit` 为 `pdTRUE`，调用任务的通知值会在 `ulTaskNotifyTake()` 返回前清零。

如果 `xClearCountOnExit` 为 `pdFALSE`，而调用任务的通知值大于 0，那么通知值会在 `ulTaskNotifyTake()` 返回前递减。

- `xTicksToWait`

调用任务为了等待通知值大于 0 而保持阻塞状态的最长时间。

阻塞时间以 Tick 周期为单位，因此它所代表的绝对时间取决于 Tick 频率。宏

`pdMS_TO_TICKS()` 可把以毫秒表示的时间转换为以 Tick 表示的时间。

如果 `FreeRTOSConfig.h` 中的 `INCLUDE_vTaskSuspend` 设为 1，把 `xTicksToWait` 设为 `portMAX_DELAY` 会使任务无限期等待，不会超时。

- 返回值

返回值是按照 `xClearCountOnExit` 参数的设置，将调用任务的通知值清零或递减之前的值。

如果指定了阻塞时间，而且返回值非 0，那么调用任务可能曾进入阻塞状态等待通知值大于 0，随后通知值在阻塞时间到期前得到更新。

如果指定了阻塞时间，而且返回值为 0，那么调用任务曾进入阻塞状态等待通知值大于 0，但在此之前指定的阻塞时间已经到期。

示例 10.1 使用任务通知替代信号量：方法 1

示例 7.1 使用二值信号量从中断服务程序解除任务的阻塞，相当于让任务与中断同步。本示例复现示例 7.1 的功能，但使用直接任务通知替代二值信号量。

代码清单 10.4 展示了与中断同步的任务实现。示例 7.1 中对 `xSemaphoreTake()` 的调用被替换为对 `ulTaskNotifyTake()` 的调用。

`ulTaskNotifyTake()` 的 `xClearCountOnExit` 参数设为 `pdTRUE`，因此接收任务的通知值会在函数返回前清零。所以，每次调用 `ulTaskNotifyTake()` 之间都必须处理所有已经出现的事件。在示例 7.1 中，由于使用二值信号量，必须从硬件确定待处理事件的数量，这并非总是可行。示例 10.1 则由 `ulTaskNotifyTake()` 返回待处理事件的数量。

两次调用 `ulTaskNotifyTake()` 之间发生的中断事件会锁存在任务的通知值中；如果调用任务已经有待处理通知，`ulTaskNotifyTake()` 会立即返回。

```

/* 周期任务产生软件中断的频率。 */
const TickType_t xInterruptFrequency = pdMS_TO_TICKS( 500UL );

static void vHandlerTask( void *pvParameters )
{
    /* xMaxExpectedBlockTime 略长于两次事件之间预计的最长时间。 */
    const TickType_t xMaxExpectedBlockTime = xInterruptFrequency +
                                              pdMS_TO_TICKS( 10 );

    uint32_t ulEventsToProcess;

    /* 与大多数任务一样，本任务在无限循环中实现。 */
    for( ;; )
    {
        /* 等待接收中断服务程序直接发给本任务的通知。 */
        ulEventsToProcess = ulTaskNotifyTake( pdTRUE, xMaxExpectedBlockTime );
        if( ulEventsToProcess != 0 )
        {
            /* 执行到这里说明至少发生了一个事件。循环处理所有待处理事件，
               本例只为每个事件输出一条消息。 */
            while( ulEventsToProcess > 0 )
            {
                vPrintString( "Handler task - Processing event.\r\n" );
                ulEventsToProcess--;
            }
        }
        else
        {
            /* 执行到这里说明中断没有在预期时间内到达。在实际应用中，
               可能需要执行某些错误恢复操作。 */
        }
    }
}

```

代码清单 10.4 示例 10.1 中延迟执行中断处理的任务实现（与中断同步的任务）

用于产生软件中断的周期任务会在产生中断前后分别输出一条消息，以便从输出中观察执行顺序。

代码清单 10.5 展示了中断处理程序。除了直接向延迟执行中断处理的任务发送通知外，它几乎不做其他工作。

```

static uint32_t ulExampleInterruptHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken;

    /* 必须把 xHigherPriorityTaskWoken 参数初始化为 pdFALSE。如果需要上下文
       切换，中断安全 API 函数会在内部把它设为 pdTRUE。 */
    xHigherPriorityTaskWoken = pdFALSE;

    /* 直接向延迟执行中断处理的任务发送通知。 */
    vTaskNotifyGiveFromISR( /* 接收通知的任务句柄。创建任务时已经保存该句柄。 */
                           xHandlerTask,

                           /* 按通常方式使用 xHigherPriorityTaskWoken。 */
                           &xHigherPriorityTaskWoken );

    /* 把 xHigherPriorityTaskWoken 传给 portYIELD_FROM_ISR()。如果
       vTaskNotifyGiveFromISR() 在内部把它设为 pdTRUE，调用
       portYIELD_FROM_ISR() 会请求上下文切换；如果它仍为 pdFALSE，则调用
       不会产生任何效果。Windows 移植版本的 portYIELD_FROM_ISR() 实现
       包含 return 语句，因此本函数没有显式返回值。 */
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}

```

代码清单 10.5 示例 10.1 使用的中断服务程序实现

图 10.3 展示了执行示例 10.1 时产生的输出，它与执行示例 7.1 时的输出完全相同。中断一产生，**vHandlerTask()** 就进入运行状态，因此该任务的输出会插入周期任务的输出之间。图 10.4 给出了进一步说明。

```

C:\WINDOWS\system32\cmd.exe - rtosdemo
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

```

图 10.3 执行示例 10.1 时产生的输出（与示例 7.1 相同）

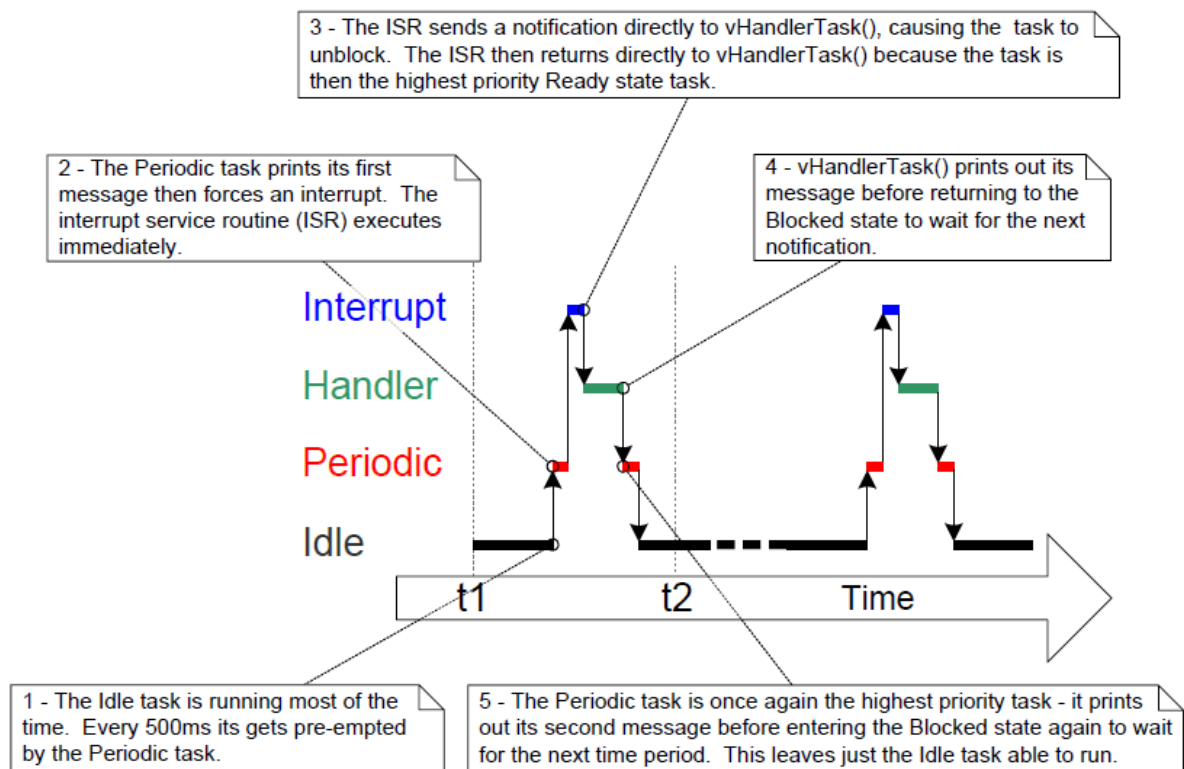


图 10.4 执行示例 10.1 时的执行顺序

示例 10.2 使用任务通知替代信号量：方法 2

示例 10.1 把 `ulTaskNotifyTake()` 的 `xClearOnExit` 参数设为 `pdTRUE`。示例 10.2 对示例 10.1 稍作修改，展示把该参数改为 `pdFALSE` 后的行为。

当 `xClearOnExit` 为 `pdFALSE` 时，调用 `ulTaskNotifyTake()` 只会把调用任务的通知值减 1，而不会将其清零。因此，通知计数等于已经发生的事件数减去已经处理的事件数。这样可以从两个方面简化 `vHandlerTask()` 的结构：

1. 等待处理的事件数量保存在通知值中，无需再保存到局部变量。
2. 每次调用 `ulTaskNotifyTake()` 之间只需处理一个事件。

示例 10.2 使用的 `vHandlerTask()` 实现见代码清单 10.6。

```
static void vHandlerTask( void *pvParameters )
{
    /* xMaxExpectedBlockTime 略长于两次事件之间预计的最长时间。 */
    const TickType_t xMaxExpectedBlockTime = xInterruptFrequency +
                                              pdMS_TO_TICKS( 10 );

    /* 与大多数任务一样，本任务在无限循环中实现。 */
    for( ;; )
    {
        /* 等待接收中断服务程序直接发给本任务的通知。xClearCountOnExit 参数
           现在为 pdFALSE，因此 ulTaskNotifyTake() 会递减任务的通知值，
           而不会将其清零。 */
        if( ulTaskNotifyTake( pdFALSE, xMaxExpectedBlockTime ) != 0 )
        {
            /* 执行到这里说明发生了一个事件。处理该事件，本例只输出一条消息。 */
            vPrintString( "Handler task - Processing event.\r\n" );
        }
        else
        {
            /* 执行到这里说明中断没有在预期时间内到达。在实际应用中，
               可能需要执行某些错误恢复操作。 */
        }
    }
}
```

代码清单 10.6 示例 10.2 中延迟执行中断处理的任务实现（与中断同步的任务）

为便于演示，中断服务程序也经过修改，每次中断会发送多个任务通知，从而模拟多个中断高频发生。示例 10.2 使用的中断服务程序实现见代码清单 10.7。

```

static uint32_t ulExampleInterruptHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken;

    xHigherPriorityTaskWoken = pdFALSE;

    /* 多次向处理任务发送通知。第一次“给出”通知会解除任务的阻塞，后续“给出”
       用于展示接收任务如何使用通知值对事件进行计数（锁存），从而逐个处理。 */
    vTaskNotifyGiveFromISR( xHandlerTask, &xHigherPriorityTaskWoken );
    vTaskNotifyGiveFromISR( xHandlerTask, &xHigherPriorityTaskWoken );
    vTaskNotifyGiveFromISR( xHandlerTask, &xHigherPriorityTaskWoken );

    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}

```

代码清单 10.7 示例 10.2 使用的中断服务程序实现

图 10.5 展示了执行示例 10.2 时产生的输出。可以看到，每次产生中断时，`vHandlerTask()` 都会处理全部三个事件。

```

C:\WINDOWS\system32\cmd.exe - rtosdemo
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.

```

图 10.5 执行示例 10.2 时产生的输出

10.3.5 xTaskNotify() 和 xTaskNotifyFromISR() API 函数

`xTaskNotify()` 是功能更强的 `xTaskNotifyGive()` 版本，可以通过以下任一种方式更新接收任务的通知值：

- 递增接收任务的通知值，此时 `xTaskNotify()` 等效于 `xTaskNotifyGive()`。
- 设置接收任务通知值中的一个或多个位，使任务通知值能够作为事件组更轻量、更快速的替代方案。
- 仅当接收任务自上次更新后已经读取通知值时，才向通知值写入一个全新的数值。这使任务通知值能够提供类似长度为 1 的队列的功能。
- 即使接收任务自上次更新后尚未读取通知值，也向通知值写入一个全新的数值。这使任务通知值能够提供类似 `xQueueOverwrite()` API 函数的功能，产生的行为有时称为“邮箱”。

`xTaskNotify()` 比 `xTaskNotifyGive()` 更灵活、功能更强，因此使用起来也稍微复杂一些。

`xTaskNotifyFromISR()` 是可在中断服务程序中使用的 `xTaskNotify()` 版本，所以它多了一个 `pxHigherPriorityTaskWoken` 参数。

如果接收任务的通知状态尚非待处理，调用 `xTaskNotify()` 总会把它设为待处理。

```
BaseType_t xTaskNotify( TaskHandle_t xTaskToNotify,
                        uint32_t ulValue,
                        eNotifyAction eAction );

BaseType_t xTaskNotifyFromISR( TaskHandle_t xTaskToNotify,
                               uint32_t ulValue,
                               eNotifyAction eAction,
                               BaseType_t *pxHigherPriorityTaskWoken );
```

代码清单 10.8 `xTaskNotify()` 和 `xTaskNotifyFromISR()` API 函数原型

`xTaskNotify()` 参数和返回值

- `xTaskToNotify`

接收通知的任务句柄。有关如何获取任务句柄的信息，请参见 `xTaskCreate()` API 函数的 `pxCreatedTask` 参数。

- `ulValue`

`ulValue` 的用途取决于 `eNotifyAction` 的取值，详见下文。

- `eNotifyAction`

一个枚举类型，指定如何更新接收任务的通知值，详见下文。

- 返回值

除下文指出的一种情况外，`xTaskNotify()` 都返回 `pdPASS`。

`xTaskNotify()` 的有效 `eNotifyAction` 参数值及其对接收任务通知值的影响

- `eNoAction`

把接收任务的通知状态设为待处理，但不更新通知值。此时不使用 `xTaskNotify()` 的 `ulValue` 参数。

`eNoAction` 使任务通知能够作为二值信号量更快、更轻量的替代方案。

- `eSetBits`

接收任务的通知值与 `xTaskNotify()` 的 `ulValue` 参数按位或。例如，如果 `ulValue` 为 `0x01`，则接收任务通知值中的位 0 被置位；如果 `ulValue` 为 `0x06`（二进制 `0110`），则位 1 和位 2 被置位。

`eSetBits` 使任务通知能够作为事件组更快、更轻量的替代方案。

- `eIncrement`

递增接收任务的通知值。此时不使用 `xTaskNotify()` 的 `ulValue` 参数。

`eIncrement` 使任务通知能够作为二值信号量或计数信号量更快、更轻量的替代方案，并且等效于更简单的 `xTaskNotifyGive()` API 函数。

- `eSetValueWithoutOverwrite`

如果调用 `xTaskNotify()` 之前接收任务已有待处理通知，则不执行任何操作，并返回 `pdFAIL`。

如果调用前接收任务没有待处理通知，则把接收任务的通知值设为 `xTaskNotify()` 的 `ulValue` 参数值。

- `eSetValueWithOverwrite`

无论调用 `xTaskNotify()` 前接收任务是否已有待处理通知，都把接收任务的通知值设为 `xTaskNotify()` 的 `ulValue` 参数值。

10.3.6 xTaskNotifyWait() API 函数

`xTaskNotifyWait()` 是功能更强的 `ulTaskNotifyTake()` 版本。如果调用任务的通知状态尚非待处理，它允许任务等待该状态变为待处理，并可选择设置超时时间。`xTaskNotifyWait()` 还可以选择在进入函数和退出函数时，分别清除调用任务通知值中的指定位。

```
BaseType_t xTaskNotifyWait( uint32_t    ulBitsToClearOnEntry,
                           uint32_t    ulBitsToClearOnExit,
                           uint32_t    *pulNotificationValue,
                           TickType_t  xTicksToWait );
```

代码清单 10.9 `xTaskNotifyWait()` API 函数原型

`xTaskNotifyWait()` 参数和返回值

- `ulBitsToClearOnEntry`

如果调用 `xTaskNotifyWait()` 前调用任务没有待处理通知，那么进入函数时，会清除任务通知值中由 `ulBitsToClearOnEntry` 置位的各个位。

例如，如果 `ulBitsToClearOnEntry` 为 0x01，则清除任务通知值中的位 0。把

`ulBitsToClearOnEntry` 设为 0xffffffff (`ULONG_MAX`) 会清除通知值中的所有位，相当于把通知值清零。

- `ulBitsToClearOnExit`

如果调用任务因为收到通知而退出 `xTaskNotifyWait()`，或者调用函数时已经有待处理通知，那么退出函数前，会清除任务通知值中由 `ulBitsToClearOnExit` 置位的各个位。

这些位只会在任务通知值保存到 `*pulNotificationValue` 之后才被清除。

例如，如果 `ulBitsToClearOnExit` 为 0x03，则函数退出前会清除任务通知值中的位 0 和位 1。

把 `ulBitsToClearOnExit` 设为 0xffffffff (`ULONG_MAX`) 会清除通知值中的所有位，相当于把通知值清零。

- `pulNotificationValue`

用于传出任务的通知值。复制到 `*pulNotificationValue` 的值，是按照

`ulBitsToClearOnExit` 清除任何位之前的任务通知值。

`pulNotificationValue` 是可选参数，不需要通知值时可设为 `NULL`。

- `xTicksToWait`

调用任务为了等待通知状态变为待处理而保持阻塞状态的最长时间。

阻塞时间以 Tick 周期为单位，因此它所代表的绝对时间取决于 Tick 频率。宏

`pdMS_TO_TICKS()` 可把以毫秒表示的时间转换为以 Tick 表示的时间。

如果 `FreeRTOSConfig.h` 中的 `INCLUDE_vTaskSuspend` 设为 1，把 `xTicksToWait` 设为 `portMAX_DELAY` 会使任务无限期等待，不会超时。

- 返回值

可能有两种返回值：

- `pdTRUE`

表示 `xTaskNotifyWait()` 因收到通知而返回，或者调用该函数时调用任务已经有待处理通知。

如果指定了非 0 阻塞时间，调用任务可能曾进入阻塞状态等待通知状态变为待处理，随后通知状态在阻塞时间到期前变为待处理。

- `pdFALSE`

表示 `xTaskNotifyWait()` 返回时，调用任务没有收到任务通知。

如果 `xTicksToWait` 非 0，调用任务曾保持在阻塞状态等待通知状态变为待处理，但在此之前指定的阻塞时间已经到期。

10.3.7 在外设驱动程序中使用任务通知：UART 示例

外设驱动库为硬件接口提供执行常见操作的函数。经常配有驱动库的外设包括通用异步收发器（UART）、串行外设接口（SPI）、模数转换器（ADC）和以太网端口。这类库通常提供初始化外设、向外设发送数据以及从外设接收数据等函数。

某些外设操作需要较长时间才能完成，例如高精度 ADC 转换，或者通过 UART 发送大型数据包。驱动库函数可以通过轮询，也就是反复读取外设状态寄存器，来判断操作何时完成。但这种轮询几乎总是在浪费资源：没有执行有效处理时却占用了 100% 的处理器时间。在多任务系统中，这种浪费尤其严重，因为轮询外设的任务可能阻止真正有工作要做的低优先级任务执行。

为了避免浪费处理时间，高效且支持 RTOS 的设备驱动程序应由中断驱动，并允许发起长时间操作的任务进入阻塞状态等待操作完成。这样，在该任务处于阻塞状态时，较低优先级任务可以执行，没有任务会在无法有效利用处理器时占用处理时间。

支持 RTOS 的驱动库通常使用二值信号量使任务进入阻塞状态。代码清单 10.10 的伪代码展示了这种技术，其中给出了一个支持 RTOS、通过 UART 端口发送数据的库函数框架。在代码清单 10.10 中：

- `xUART` 是描述 UART 外设并保存状态信息的结构体，其 `xTxSemaphore` 成员是 `SemaphoreHandle_t` 类型变量。这里假定信号量已经创建。
- `xUART_Send()` 函数不包含互斥逻辑。如果多个任务要使用该函数，应用程序编写者必须在应用层管理互斥。例如，可以要求任务先获取互斥量，再调用 `xUART_Send()`。
- 启动 UART 传输后，使用 `xSemaphoreTake()` API 函数使调用任务进入阻塞状态。
- 传输完成时会执行 UART 外设的发送结束中断服务程序，该程序调用 `xSemaphoreGiveFromISR()` API 函数，使任务离开阻塞状态。

```

/* 向 UART 发送数据的驱动库函数。 */

BaseType_t xUART_Send( xUART *pxUARTInstance,
                        uint8_t *pucDataSource,
                        size_t uxLength )
{
    BaseType_t xReturn;

    /* 尝试以 0 超时获取信号量，确保 UART 发送信号量当前不可用。 */
    xSemaphoreTake( pxUARTInstance->TxSemaphore, 0 );

    /* 启动传输。 */
    UART_low_level_send( pxUARTInstance, pucDataSource, uxLength );

    /* 阻塞在信号量上，等待传输完成。如果获得信号量，xReturn 会被设为
       pdPASS；如果获取信号量超时，xReturn 会被设为 pdFAIL。请注意，如果
       中断发生在调用 UART_low_level_send() 和调用 xSemaphoreTake() 之间，
       事件会锁存在二值信号量中，xSemaphoreTake() 随后会立即返回。 */
    xReturn = xSemaphoreTake( pxUARTInstance->TxSemaphore,
                             pxUARTInstance->TxTimeout );

    return xReturn;
}

/*-----*/

/* UART 发送结束中断的服务程序，在最后一个字节发送给 UART 后执行。 */
void xUART_TransmitEndISR( xUART *pxUARTInstance )
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    /* 清除中断。 */
    UART_low_level_interrupt_clear( pxUARTInstance );

    /* 给出 Tx 信号量，表示传输结束。如果有任务阻塞等待该信号量，则使该任务
       离开阻塞状态。 */
    xSemaphoreGiveFromISR( pxUARTInstance->TxSemaphore,
                           &xHigherPriorityTaskWoken );
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}

```

代码清单 10.10 在驱动库发送函数中使用二值信号量的伪代码

代码清单 10.10 所示技术完全可行，而且十分常见，但也有一些缺点：

- 驱动库使用多个信号量，会增加 RAM 占用。
- 信号量必须先创建才能使用，因此使用信号量的驱动库必须经过显式初始化才能使用。
- 信号量是适用于多种场景的通用对象，包含允许任意数量任务进入阻塞状态等待信号量，以及在信号量可用时以确定性方式选择解除哪个任务阻塞的逻辑。执行这些逻辑需要时间，而代码清单 10.10 的场景在任意时刻最多只有一个任务等待信号量，因此这些处理开销并无必要。

代码清单 10.11 展示了如何使用任务通知替代二值信号量，从而避免这些缺点。

注意：如果驱动库使用任务通知，其文档必须明确说明，调用库函数可能改变调用任务的通知状态和通知值。

在代码清单 10.11 中：

- `xUART` 结构体的 `xTxSemaphore` 成员替换为 `xTaskToNotify` 成员。`xTaskToNotify` 是 `TaskHandle_t` 类型变量，用于保存等待 UART 操作完成的任务句柄。
- 使用 FreeRTOS API 函数 `xTaskGetCurrentTaskHandle()` 获取处于运行状态的任务句柄。
- 驱动库不创建任何 FreeRTOS 对象，因此不会产生额外 RAM 开销，也不需要显式初始化。
- 任务通知直接发送给等待 UART 操作完成的任务，因此不会执行不必要的逻辑。

任务和中断服务程序都会访问 `xUART` 结构体的 `xTaskToNotify` 成员，因此必须考虑处理器如何更新该值：

- 如果一次内存写操作即可更新 `xTaskToNotify`，就可以像代码清单 10.11 那样在临界区外更新。例如，当 `xTaskToNotify` 是 32 位变量且 FreeRTOS 运行在 32 位处理器上时，就是这种情况。
- 如果更新 `xTaskToNotify` 需要多次内存写操作，就只能在临界区内更新，否则中断服务程序可能在它处于不一致状态时访问它。例如，当 `xTaskToNotify` 是 32 位变量，而 FreeRTOS 运行在 16 位处理器上时，需要两次 16 位内存写操作才能更新全部 32 位。

在 FreeRTOS 内部实现中，`TaskHandle_t` 是指针，因此 `sizeof( TaskHandle_t )` 始终等于 `sizeof( void * )`。

```

/* 向 UART 发送数据的驱动库函数。 */
BaseType_t xUART_Send( xUART *pxUARTInstance,
                        uint8_t *pucDataSource,
                        size_t uxLength )
{
    BaseType_t xReturn;

    /* 保存调用本函数的任务句柄。正文说明了下一行是否需要临界区保护。 */
    pxUARTInstance->xTaskToNotify = xTaskGetCurrentTaskHandle();

    /* 调用 ulTaskNotifyTake(), 把 xClearCountOnExit 设为 pdTRUE, 并把阻塞时间
       设为 0 (不阻塞), 确保调用任务当前没有待处理通知。 */
    ulTaskNotifyTake( pdTRUE, 0 );

    /* 启动传输。 */
    UART_low_level_send( pxUARTInstance, pucDataSource, uxLength );

    /* 阻塞等待传输完成通知。如果收到通知, xReturn 为 1, 因为 ISR 已把本任务
       的通知值递增到 1 (pdTRUE)。如果操作超时, xReturn 为 0 (pdFALSE),
       因为本任务的通知值自上面清零后没有改变。请注意, 如果 ISR 在调用
       UART_low_level_send() 与 ulTaskNotifyTake() 之间执行, 事件会锁存在任务
       的通知值中, ulTaskNotifyTake() 随后会立即返回。 */
    xReturn = ( BaseType_t ) ulTaskNotifyTake( pdTRUE,
                                                pxUARTInstance->xTxTimeout );

    return xReturn;
}

/*-----*/

/* UART 发送完最后一个字节后执行的 ISR。 */
void xUART_TransmitEndISR( xUART *pxUARTInstance )
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    /* 除非有任务正在等待通知, 否则不应执行本函数。使用断言检查该条件。
       这一步并非绝对必要, 但有助于调试。第 12.2 节介绍 configASSERT()。 */
    configASSERT( pxUARTInstance->xTaskToNotify != NULL );

    /* 清除中断。 */
    UART_low_level_interrupt_clear( pxUARTInstance );

    /* 直接向调用 xUART_Send() 的任务发送通知。如果该任务正阻塞等待通知,
       则使其离开阻塞状态。 */
    vTaskNotifyGiveFromISR( pxUARTInstance->xTaskToNotify,
                            &xHigherPriorityTaskWoken );

    /* 现在没有任务等待通知。把 xUART 结构体的 xTaskToNotify 成员恢复为 NULL。
       这一步并非绝对必要, 但有助于调试。 */
    pxUARTInstance->xTaskToNotify = NULL;
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}

```

代码清单 10.11 在驱动库发送函数中使用任务通知的伪代码

任务通知也可以在接收函数中替代信号量。代码清单 10.12 的伪代码给出了一个支持 RTOS、从 UART 端口接收数据的库函数框架。参见代码清单 10.12:

- `xUART_Receive()` 函数不包含互斥逻辑。如果多个任务要使用该函数,应用程序编写者必须在应用层管理互斥。例如,可以要求任务先获取互斥量,再调用 `xUART_Receive()`。
- UART 接收中断服务程序把 UART 接收到的字符放入 RAM 缓冲区, `xUART_Receive()` 函数则从该缓冲区返回字符。
- `xUART_Receive()` 的 `uxWantedBytes` 参数指定要接收的字符数。如果 RAM 缓冲区中尚无足够字符,调用任务就进入阻塞状态,等待通知缓冲区中的字符数已经增加。`while()` 循环反复执行这一过程,直到接收缓冲区包含所需数量的字符,或者发生超时。
- 调用任务可能多次进入阻塞状态,因此必须根据调用 `xUART_Receive()` 后已经过去的时间调整阻塞时间,确保函数内部花费的总时间不超过 `xUART` 结构体的 `xRxTimeout` 成员所指定的时间。阻塞时间通过 FreeRTOS 辅助函数 `vTaskSetTimeOutState()` 和 `xTaskCheckForTimeOut()` 调整。


```

/* 从 UART 接收数据的驱动库函数。 */

size_t xUART_Receive( xUART *pxUARTInstance,
                      uint8_t *pucBuffer,
                      size_t uxWantedBytes )
{
    size_t uxReceived = 0;
    TickType_t xTicksToWait;
    TimeOut_t xTimeOut;

    /* 记录进入本函数的时间。 */
    vTaskSetTimeOutState( &xTimeOut );

    /* xTicksToWait 是超时值, 初始值为该 UART 实例的最长接收超时时间。 */
    xTicksToWait = pxUARTInstance->RxTimeout;

    /* 保存调用本函数的任务句柄。正文说明了下一行是否需要临界区保护。 */
    pxUARTInstance->xTaskToNotify = xTaskGetCurrentTaskHandle();

    /* 循环, 直到缓冲区包含所需数量的字节, 或者发生超时。 */
    while( UART_bytes_in_rx_buffer( pxUARTInstance ) < uxWantedBytes )
    {
        /* 检查是否超时, 并根据本函数目前已经花费的时间调整 xTicksToWait。 */
        if( xTaskCheckForTimeOut( &xTimeOut, &xTicksToWait ) != pdFALSE )
        {
            /* 在获得所需数量的字节前已经超时, 退出循环。 */
            break;
        }

        /* 接收缓冲区中的字节数仍然不足。等待最多 xTicksToWait 个 Tick, 直到
           收到通知, 说明接收中断服务程序已把更多数据放入缓冲区。调用本函数
           时任务是否已有待处理通知并不重要; 如果有, 只会让 while 循环额外
           迭代一次。 */
        ulTaskNotifyTake( pdTRUE, xTicksToWait );
    }

    /* 没有任务在等待接收通知, 因此把 xTaskToNotify 恢复为 NULL。正文说明了
       下一行是否需要临界区保护。 */
    pxUARTInstance->xTaskToNotify = NULL;

    /* 尝试从接收缓冲区读取 uxWantedBytes 个字节到 pucBuffer, 并返回实际读取
       的字节数, 该值可能小于 uxWantedBytes。 */
    uxReceived = UART_read_from_receive_buffer( pxUARTInstance,
                                                pucBuffer,
                                                uxWantedBytes );

    return uxReceived;
}

/*-----*/

/* UART 接收中断的中断服务程序。 */
void xUART_ReceiveISR( xUART *pxUARTInstance )
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    /* 把接收到的数据复制到该 UART 的接收缓冲区, 并清除中断。 */

```

```

UART_low_level_receive( pxUARTInstance );

/* 如果有任务正在等待新数据通知，则立即通知它。 */
if( pxUARTInstance->xTaskToNotify != NULL )
{
    vTaskNotifyGiveFromISR( pxUARTInstance->xTaskToNotify,
                           &xHigherPriorityTaskWoken );
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
}

```

代码清单 10.12 在驱动库接收函数中使用任务通知的伪代码

10.3.8 在外设驱动程序中使用任务通知：ADC 示例

上一节演示了如何使用 `vTaskNotifyGiveFromISR()` 从中断向任务发送通知。`vTaskNotifyGiveFromISR()` 使用简单，但功能有限：它只能发送没有附带数值的任务通知事件，不能发送数据。本节演示如何使用 `xTaskNotifyFromISR()` 随任务通知事件发送数据。代码清单 10.13 的伪代码给出了一个支持 RTOS 的模数转换器（ADC）中断服务程序框架。其中：

- 假定至少每 50 毫秒启动一次 ADC 转换。
- `ADC_ConversionEndISR()` 是 ADC 转换结束中断的服务程序，每当新的 ADC 值可用时都会执行。
- `vADCTask()` 实现的任务负责处理 ADC 生成的每个值。假定创建该任务时已经把任务句柄保存到 `xADCTaskToNotify`。
- `ADC_ConversionEndISR()` 调用 `xTaskNotifyFromISR()`，把 `eAction` 参数设为 `eSetValueWithoutOverwrite`，向 `vADCTask()` 发送任务通知，并把 ADC 转换结果写入任务的通知值。
- `vADCTask()` 使用 `xTaskNotifyWait()` 等待新 ADC 值可用的通知，并从任务通知值中取得 ADC 转换结果。

```

/* 使用 ADC 的任务。 */
void vADCTask( void *pvParameters )
{
    uint32_t ulADCValue;
    BaseType_t xResult;

    /* 触发 ADC 转换的频率。 */
    const TickType_t xADCConversionFrequency = pdMS_TO_TICKS( 50 );

    for( ;; )
    {
        /* 等待下一个 ADC 转换结果。 */
        xResult = xTaskNotifyWait(
            /* 新 ADC 值会覆盖旧值, 因此等待新通知值前无需清除任何位。 */
            0,
            /* 后续 ADC 值会覆盖现有值, 因此退出 xTaskNotifyWait() 前
               无需清除任何位。 */
            0,
            /* 用于接收任务通知值 (即最新 ADC 转换结果) 的变量地址。 */
            &ulADCValue,
            /* 每隔 xADCConversionFrequency 个 Tick 应收到一个新 ADC 值。 */
            xADCConversionFrequency * 2 );

        if( xResult == pdPASS )
        {
            /* 收到新的 ADC 值, 立即处理。 */
            ProcessADCResult( ulADCValue );
        }
        else
        {
            /* xTaskNotifyWait() 没有在预期时间内返回, 可能是触发 ADC 转换的
               输入或 ADC 本身出现问题。在此处理错误。 */
        }
    }
}

/*-----*/

/* 每次 ADC 转换完成时执行的中断服务程序。 */
void ADC_ConversionEndISR( xADC *pxADCInstance )
{
    uint32_t ulConversionResult;
    BaseType_t xHigherPriorityTaskWoken = pdFALSE, xResult;

    /* 读取新的 ADC 值并清除中断。 */
    ulConversionResult = ADC_low_level_read( pxADCInstance );

    /* 把通知和 ADC 转换结果直接发送给 vADCTask()。 */
    xResult = xTaskNotifyFromISR( xADCTaskToNotify, /* xTaskToNotify 参数 */
                                  ulConversionResult, /* ulValue 参数 */
                                  eSetValueWithoutOverwrite, /* eAction 参数。 */
                                  &xHigherPriorityTaskWoken );

    /* 如果 xTaskNotifyFromISR() 返回 pdFAIL, 说明任务无法跟上 ADC 值的产生
       速度。第 12.2 节介绍 configASSERT()。 */
    configASSERT( xResult == pdPASS );
}

```

```
portYIELD_FROM_ISR( xHigherPriorityTaskWoken );  
}
```

代码清单 10.13 使用任务通知向任务传递数值的伪代码

10.3.9 在应用程序中直接使用任务通知

本节通过一个假想应用程序进一步展示任务通知的能力。该应用包含以下功能：

- 应用程序通过速度较慢的互联网连接向远程数据服务器发送数据，并从中请求数据。下文把远程数据服务器称为云服务器。
- 向云服务器请求数据后，请求任务必须进入阻塞状态，等待收到请求的数据。
- 向云服务器发送数据后，发送任务必须进入阻塞状态，等待云服务器确认已经正确接收数据。

图 10.6 展示了软件设计示意图。其中：

- 与云服务器建立并处理多个互联网连接的复杂逻辑被封装在单个 FreeRTOS 任务中。该任务在 FreeRTOS 应用程序内充当代理服务器，称为服务器任务。
- 应用任务调用 `CloudRead()` 从云服务器读取数据。`CloudRead()` 不直接与云服务器通信，而是通过队列把读取请求发送给服务器任务，再通过任务通知从服务器任务接收请求的数据。
- 应用任务调用 `CloudWrite()` 向云服务器写入数据。`CloudWrite()` 不直接与云服务器通信，而是通过队列把写入请求发送给服务器任务，再通过任务通知从服务器任务接收写操作结果。

`CloudRead()` 和 `CloudWrite()` 函数发送给服务器任务的结构体见代码清单 10.14。

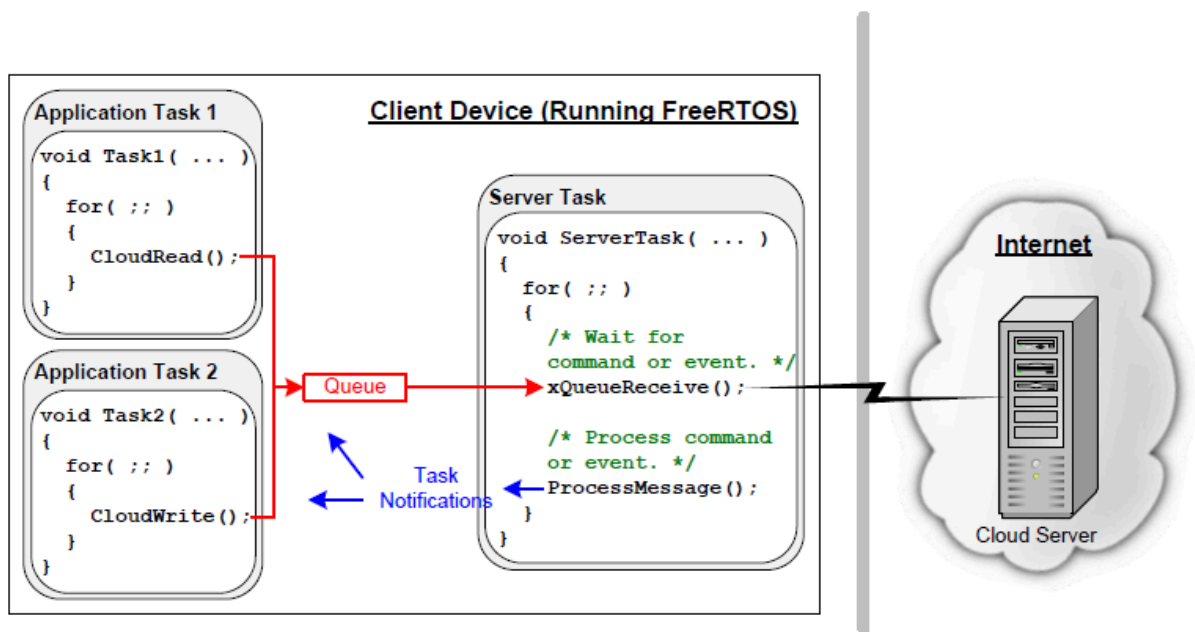


图 10.6 应用任务与云服务器之间的往返通信路径

```
typedef enum CloudOperations
{
    eRead, /* 从云服务器接收数据。 */
    eWrite /* 向云服务器发送数据。 */
} Operation_t;

typedef struct CloudCommand
{
    Operation_t eOperation; /* 要执行的操作（读取或写入）。 */
    uint32_t ulDataID; /* 标识要读取或写入的数据。 */
    uint32_t ulDataValue; /* 仅在向云服务器写入数据时使用。 */
    TaskHandle_t xTaskToNotify; /* 执行该操作的任务句柄。 */
} CloudCommand_t;
```

代码清单 10.14 通过队列发送给服务器任务的结构体和数据类型

代码清单 10.15 给出了 `CloudRead()` 的伪代码。该函数把请求发送给服务器任务，然后调用 `xTaskNotifyWait()` 进入阻塞状态，直到收到请求数据已经可用的通知。

代码清单 10.16 展示了服务器任务如何处理读取请求。服务器任务从云服务器取得数据后，调用 `xTaskNotify()`，把 `eAction` 参数设为 `eSetValueWithOverwrite`，从而解除应用任务的阻塞，并把接收到的数据发送给应用任务。

代码清单 10.16 展示的是简化场景，假定 `GetCloudData()` 无需等待即可从云服务器取得数值。

```

/* ulDataID 标识要读取的数据。pulValue 保存一个变量的地址，从云服务器接收
到的数据将写入该变量。 */
BaseType_t CloudRead( uint32_t ulDataID, uint32_t *pulValue )
{
    CloudCommand_t xRequest;
    BaseType_t xReturn;

    /* 为本次读取请求正确设置 CloudCommand_t 结构体成员。 */
    xRequest.eOperation = eRead; /* 这是读取数据的请求。 */
    xRequest.ulDataID = ulDataID; /* 标识要读取数据的代码。 */
    xRequest.xTaskToNotify = xTaskGetCurrentTaskHandle(); /* 调用任务的句柄。 */

    /* 以 0 阻塞时间读取通知值，确保当前没有待处理通知，然后把结构体发送给
    服务器任务。 */
    xTaskNotifyWait( 0, 0, NULL, 0 );
    xQueueSend( xServerTaskQueue, &xRequest, portMAX_DELAY );

    /* 等待服务器任务的通知。服务器任务把从云服务器取得的值直接写入本任务
    的通知值，因此进入和退出 xTaskNotifyWait() 时都无需清除通知值中的位。
    接收值写入 *pulValue，所以把 pulValue 作为通知值的写入地址传入。 */
    xReturn = xTaskNotifyWait( 0, /* 进入时不清除任何位 */
                              0, /* 退出时不清除任何位 */
                              pulValue, /* 把通知值写入 *pulValue */
                              pdMS_TO_TICKS( 250 ) ); /* 最多等待 250ms */

    /* xReturn 为 pdPASS 表示已取得数值；为 pdFAIL 表示请求超时。 */
    return xReturn;
}

```

代码清单 10.15 **CloudRead** API 函数的实现

```

void ServerTask( void *pvParameters )
{
    CloudCommand_t xCommand;
    uint32_t ulReceivedValue;

    for( ;; )
    {
        /* 等待接收任务发来的下一个 CloudCommand_t 结构体。 */
        xQueueReceive( xServerTaskQueue, &xCommand, portMAX_DELAY );

        switch( xCommand.eOperation ) /* 这是读取请求还是写入请求? */
        {
            case eRead:

                /* 从远程云服务器取得请求的数据项。 */
                ulReceivedValue = GetCloudData( xCommand.ulDataID );

                /* 调用 xTaskNotify(), 向发出请求的任务同时发送通知和从云服务器
                   取得的值。任务句柄从 CloudCommand_t 结构体中取得。 */
                xTaskNotify( xCommand.xTaskToNotify, /* 任务句柄位于结构体中 */
                            ulReceivedValue, /* 把云端数据作为通知值发送 */
                            eSetValueWithOverwrite );

                break;

            /* 其他 switch 分支写在这里。 */

        }
    }
}

```

代码清单 10.16 服务器任务处理读取请求

代码清单 10.17 给出了 `CloudWrite()` 的伪代码。为便于演示，`CloudWrite()` 返回一个按位状态码，其中每个位都有独立含义。代码清单开头的 `#define` 语句定义了四个示例状态位。

任务先清除四个状态位，再把请求发送给服务器任务，然后调用 `xTaskNotifyWait()` 进入阻塞状态等待状态通知。

```

/* 云端写操作使用的状态位。 */
#define SEND_SUCCESSFUL_BIT ( 0x01 << 0 )
#define OPERATION_TIMED_OUT_BIT ( 0x01 << 1 )
#define NO_INTERNET_CONNECTION_BIT ( 0x01 << 2 )
#define CANNOT_LOCATE_CLOUD_SERVER_BIT ( 0x01 << 3 )

/* 四个状态位均置位的掩码。 */
#define CLOUD_WRITE_STATUS_BIT_MASK ( SEND_SUCCESSFUL_BIT |
                                       OPERATION_TIMED_OUT_BIT |
                                       NO_INTERNET_CONNECTION_BIT |
                                       CANNOT_LOCATE_CLOUD_SERVER_BIT )

uint32_t CloudWrite( uint32_t ulDataID, uint32_t ulDataValue )
{
    CloudCommand_t xRequest;
    uint32_t ulNotificationValue;

    /* 为本次写入请求正确设置 CloudCommand_t 结构体成员。 */
    xRequest.eOperation = eWrite; /* 这是写入数据的请求 */
    xRequest.ulDataID = ulDataID; /* 标识要写入数据的代码 */
    xRequest.ulDataValue = ulDataValue; /* 写入云服务器的数据值。 */
    xRequest.xTaskToNotify = xTaskGetCurrentTaskHandle(); /* 调用任务的句柄。 */

    /* 调用 xTaskNotifyWait(), 把 ulBitsToClearOnExit 设为
       CLOUD_WRITE_STATUS_BIT_MASK, 阻塞时间设为 0, 清除与写操作有关的四个
       状态位。不需要当前通知值, 所以 pulNotificationValue 设为 NULL。 */
    xTaskNotifyWait( 0, CLOUD_WRITE_STATUS_BIT_MASK, NULL, 0 );

    /* 把请求发送给服务器任务。 */
    xQueueSend( xServerTaskQueue, &xRequest, portMAX_DELAY );

    /* 等待服务器任务的通知。服务器任务把按位状态码写入本任务的通知值,
       再写入 ulNotificationValue。 */
    xTaskNotifyWait( 0, /* 进入时不清除任何位。 */
                   CLOUD_WRITE_STATUS_BIT_MASK, /* 退出时把相关位清零。 */
                   &ulNotificationValue, /* 通知值。 */
                   pdMS_TO_TICKS( 250 ) ); /* 最多等待 250ms。 */

    /* 把状态码返回给调用任务。 */
    return ( ulNotificationValue & CLOUD_WRITE_STATUS_BIT_MASK );
}

```

代码清单 10.17 `CloudWrite` API 函数的实现

代码清单 10.18 展示了服务器任务如何处理写入请求。服务器任务把数据发送给云服务器后，调用 `xTaskNotify()`，把 `eAction` 参数设为 `eSetBits`，从而解除应用任务的阻塞，并把按位状态码发送给应用任务。接收任务的通知值中，只有 `CLOUD_WRITE_STATUS_BIT_MASK` 常量定义的位会改变，因此其他位仍可用于其他目的。

代码清单 10.18 展示的是简化场景，假定 `SetCloudData()` 无需等待即可获得远程云服务器的确认。


```

void ServerTask( void *pvParameters )
{
    CloudCommand_t xCommand;
    uint32_t ulBitwiseStatusCode;

    for( ;; )
    {
        /* 等待下一条消息。 */
        xQueueReceive( xServerTaskQueue, &xCommand, portMAX_DELAY );

        /* 这是读取请求还是写入请求? */
        switch( xCommand.eOperation )
        {
            case eWrite:

                /* 把数据发送给远程云服务器。SetCloudData() 返回按位状态码, 该状态码
                   只使用 CLOUD_WRITE_STATUS_BIT_MASK 定义的位 (见代码清单 10.17)。 */
                ulBitwiseStatusCode = SetCloudData( xCommand.ulDataID,
                                                    xCommand.ulDataValue );

                /* 向发出写入请求的任务发送通知。使用 eSetBits 操作, 把
                   ulBitwiseStatusCode 中置位的状态位设置到接收任务的通知值中,
                   其他位保持不变。任务句柄从 CloudCommand_t 结构体中取得。 */
                xTaskNotify( xCommand.xTaskToNotify, /* 任务句柄位于结构体中。 */
                            ulBitwiseStatusCode, /* 把云端数据作为通知值发送。 */
                            eSetBits );

                break;

            /* 其他 switch 分支写在这里。 */

        }
    }
}

```

代码清单 10.18 服务器任务处理发送请求

11 低功耗支持

11.1 节能简介

FreeRTOS 提供了两种简单的低功耗机制：空闲任务钩子和无 Tick 空闲模式。

一种常见的降低功耗方法，是在空闲任务钩子中让运行 FreeRTOS 的微控制器进入低功耗状态。不过，这种方法的节能效果受到限制，因为系统必须周期性退出低功耗状态以处理 Tick 中断，然后再次进入低功耗状态。此外，如果 Tick 中断频率过高，也就是系统过于频繁地从空闲状态唤醒，那么除非使用最浅的节能模式，否则每个 Tick 进入和退出低功耗状态所消耗的能量与时间，可能超过节省的能量。

FreeRTOS 支持让微控制器周期性进入和退出低功耗状态。FreeRTOS 的无 Tick 空闲模式会在空闲期间停止周期性 Tick 中断；所谓空闲期间，是指没有任何应用任务能够执行的时段。这样，MCU 可以一直保持深度节能状态，直到发生中断，或者 RTOS 内核需要把某个任务转入就绪状态。重新启动 Tick 中断时，内核会相应校正 RTOS Tick 计数值。FreeRTOS 无 Tick 模式的基本原理是：当 MCU 正在执行空闲任务时，让它进入低功耗模式，从而降低系统功耗。

11.2 FreeRTOS 睡眠模式

FreeRTOS 支持三种睡眠模式状态：

1. **eAbortSleep**：表示某个任务已进入就绪状态、上下文切换正等待执行，或者调度器挂起期间已经发生了 Tick 中断且该中断正等待处理。该状态通知 RTOS 放弃进入睡眠模式。
2. **eStandardSleep**：允许系统进入睡眠模式，但睡眠时间不会超过预计的空闲时间。
3. **eNoTasksWaitingTimeout**：没有任何任务正在等待超时，因此可以安全进入只能由外部中断或复位退出的睡眠模式。

11.3 启用内置无 Tick 空闲功能及相关函数

对于支持该功能的移植版本，在 FreeRTOSConfig.h 中把 **configUSE_TICKLESS_IDLE** 定义为 1，即可启用内置无 Tick 空闲功能。对于任何 FreeRTOS 移植版本，包括已经内置相关实现的版本，都可以在 FreeRTOSConfig.h 中把 **configUSE_TICKLESS_IDLE** 定义为 2，以提供用户自定义的无 Tick 空闲功能。

启用无 Tick 空闲功能后，内核会在同时满足以下两个条件时调用 `portSUPPRESS_TICKS_AND_SLEEP()` 宏：

1. 空闲任务是唯一能够运行的任务，因为所有应用任务都处于阻塞状态或挂起状态。
2. 在内核需要把某个应用任务移出阻塞状态之前，至少还会经过 n 个完整的 Tick 周期。 n 由 `FreeRTOSConfig.h` 中的 `configEXPECTED_IDLE_TIME_BEFORE_SLEEP` 定义设置。

11.3.1 portSUPPRESS_TICKS_AND_SLEEP() 宏

```
portSUPPRESS_TICKS_AND_SLEEP( xExpectedIdleTime )
```

代码清单 11.1 `portSUPPRESS_TICKS_AND_SLEEP` 宏的原型

`portSUPPRESS_TICKS_AND_SLEEP()` 的 `xExpectedIdleTime` 参数值，等于下一个任务应被转入就绪状态之前的 Tick 周期总数。因此，该参数表示在抑制 Tick 中断的情况下，微控制器能够安全保持深度睡眠状态的时间。

11.3.2 vPortSuppressTicksAndSleep 函数

FreeRTOS 定义了 `vPortSuppressTicksAndSleep()` 函数，可用它实现无 Tick 模式。该函数在 FreeRTOS Cortex-M 移植层中以弱符号定义，因此应用程序开发者可以覆盖它。

```
void vPortSuppressTicksAndSleep( TickType_t xExpectedIdleTime );
```

代码清单 11.2 `vPortSuppressTicksAndSleep` API 函数原型

11.3.3 eTaskConfirmSleepModeStatus 函数

`eTaskConfirmSleepModeStatus()` API 函数返回睡眠模式状态，用于判断是否可以继续进入睡眠，以及是否可以无限期休眠。只有把 `configUSE_TICKLESS_IDLE` 设为 1 时，才能使用此功能。

```
eSleepModeStatus eTaskConfirmSleepModeStatus( void );
```

代码清单 11.3 `eTaskConfirmSleepModeStatus` API 函数原型

如果在 `portSUPPRESS_TICKS_AND_SLEEP()` 内部调用 `eTaskConfirmSleepModeStatus()` 时返回 `eNoTasksWaitingTimeout`，那么微控制器可以无限期保持深度睡眠状态。只有同时满足以下条件时，`eTaskConfirmSleepModeStatus()` 才会返回 `eNoTasksWaitingTimeout`：

- 没有使用软件定时器，因此调度器以后任何时候都不需要执行定时器回调函数。
- 所有应用任务都处于挂起状态，或者处于阻塞状态且超时值为 `portMAX_DELAY`，因此调度器以后不需要在任何固定时刻把任务移出阻塞状态。

为避免竞争条件，FreeRTOS 会在调用 `portSUPPRESS_TICKS_AND_SLEEP()` 之前挂起调度器，并在 `portSUPPRESS_TICKS_AND_SLEEP()` 执行完毕后恢复调度器。这样可以确保微控制器退出低功耗状态之后、`portSUPPRESS_TICKS_AND_SLEEP()` 完成执行之前，应用任务无法运行。此外，`portSUPPRESS_TICKS_AND_SLEEP()` 函数必须在停止定时器与进入睡眠模式之间建立一个短临界区，以确认此时仍可继续进入睡眠模式。应当在这个临界区内调用 `eTaskConfirmSleepModeStatus()`。

此外，FreeRTOS 还向用户提供了另外两个在 `FreeRTOSConfig.h` 中定义的接口宏。应用程序开发者可以通过它们，分别在 MCU 进入低功耗状态之前和退出低功耗状态之后添加额外处理步骤。

11.3.4 configPRE_SLEEP_PROCESSING 配置项

```
configPRE_SLEEP_PROCESSING( xExpectedIdleTime )
```

代码清单 11.4 `configPRE_SLEEP_PROCESSING` 宏的原型

在让 MCU 进入低功耗模式之前，用户必须调用 `configPRE_SLEEP_PROCESSING()`，配置用于降低系统功耗的系统参数，例如关闭其他外设时钟、降低系统频率。

11.3.5 configPOST_SLEEP_PROCESSING 配置项

```
configPOST_SLEEP_PROCESSING( xExpectedIdleTime )
```

代码清单 11.5 `configPOST_SLEEP_PROCESSING` 宏的原型

退出低功耗模式后，用户应调用 `configPOST_SLEEP_PROCESSING()`，恢复系统主频和外设功能。

11.4 实现 portSUPPRESS_TICKS_AND_SLEEP() 宏

如果所用的 FreeRTOS 移植版本没有提供 `portSUPPRESS_TICKS_AND_SLEEP()` 的默认实现，应用程序开发者可以在 `FreeRTOSConfig.h` 中定义 `portSUPPRESS_TICKS_AND_SLEEP()`，提供自己的实现。如果所用的 FreeRTOS 移植版本已经提供 `portSUPPRESS_TICKS_AND_SLEEP()` 的默认实现，应用程序开发者同样可以在 `FreeRTOSConfig.h` 中定义 `portSUPPRESS_TICKS_AND_SLEEP()`，覆盖默认实现。

下面的源代码演示应用程序开发者可以如何实现 `portSUPPRESS_TICKS_AND_SLEEP()`。这个示例比较基础，会使内核维护的时间与日历时间之间产生少量偏差。在示例调用的函数中，只有 `vTaskStepTick()` 和 `eTaskConfirmSleepModeStatus()` 属于 FreeRTOS API。其他函数取决于所用硬件提供的时钟和节能模式，必须由应用程序开发者实现。

```

/* 首先定义 portSUPPRESS_TICKS_AND_SLEEP() 宏。参数表示距内核下次需要执行
   还有多少个 Tick。 */

#define portSUPPRESS_TICKS_AND_SLEEP( xIdleTime ) vApplicationSleep( xIdleTime )

/* 定义由 portSUPPRESS_TICKS_AND_SLEEP() 调用的函数。 */
void vApplicationSleep( TickType_t xExpectedIdleTime )
{
    unsigned long ulLowPowerTimeBeforeSleep, ulLowPowerTimeAfterSleep;

    eSleepModeStatus eSleepStatus;

    /* 从一个在微控制器处于低功耗状态时仍会运行的时钟源读取当前时间。 */
    ulLowPowerTimeBeforeSleep = ulGetExternalTime();

    /* 停止生成 Tick 中断的定时器。 */
    prvStopTickInterruptTimer();

    /* 进入一个不会影响 MCU 睡眠唤醒中断的临界区。 */
    disable_interrupts();

    /* 确认此时仍可进入睡眠模式。 */
    eSleepStatus = eTaskConfirmSleepModeStatus();

    if( eSleepStatus == eAbortSleep )
    {
        /* 自本宏执行以来，已有任务移出阻塞状态，或者有上下文切换正等待执行。
           不要进入睡眠状态。重新启动 Tick，并退出临界区。 */
        prvStartTickInterruptTimer();
        enable_interrupts();
    }
    else
    {
        if( eSleepStatus == eNoTasksWaitingTimeout )
        {
            /* 无需配置一个在未来固定时刻将微控制器从低功耗状态唤醒的中断。 */
            prvSleep();
        }
        else
        {
            /* 配置一个中断，在内核下次需要执行时将微控制器从低功耗状态唤醒。
               该中断必须由微控制器处于低功耗状态时仍能运行的中断源生成。 */
            vSetWakeTimeInterrupt( xExpectedIdleTime );

            /* 进入低功耗状态。 */
            prvSleep();

            /* 确定微控制器实际处于低功耗状态的时长。如果唤醒微控制器的是
               vSetWakeTimeInterrupt() 配置之外的其他中断，该值会小于
               xExpectedIdleTime。注意，调用 portSUPPRESS_TICKS_AND_SLEEP() 之前
               调度器已经挂起，并会在 portSUPPRESS_TICKS_AND_SLEEP() 返回时恢复。
               因此，在本函数执行完毕之前，不会有其他任务运行。 */
            ulLowPowerTimeAfterSleep = ulGetExternalTime();

            /* 校正内核的 Tick 计数，以计入微控制器处于低功耗状态的时间。 */
            vTaskStepTick( ulLowPowerTimeAfterSleep - ulLowPowerTimeBeforeSleep );
        }
    }
}

```

```

    }

    /* 退出临界区。也可以考虑紧接在 prvSleep() 调用之后执行此操作。 */
    enable_interrupts();

    /* 重新启动生成 Tick 中断的定时器。 */
    prvStartTickInterruptTimer();
}
}

```

代码清单 11.6 用户自定义 `portSUPPRESS_TICKS_AND_SLEEP()` 实现示例

11.5 空闲任务钩子函数

空闲任务可以选择调用由应用程序定义的钩子函数或回调函数，即空闲钩子。空闲任务以最低优先级运行，因此只有在没有更高优先级任务能够运行时，空闲钩子函数才会执行。这使空闲钩子函数非常适合用于让处理器进入低功耗状态，在没有工作需要处理时自动节能。只有在 `FreeRTOSConfig.h` 中把 `configUSE_IDLE_HOOK` 设为 1，系统才会调用空闲钩子。

```
void vApplicationIdleHook( void );
```

代码清单 11.7 `vApplicationIdleHook` API 函数原型

只要空闲任务持续运行，就会反复调用空闲钩子。最重要的是，空闲钩子函数绝不能调用任何可能使其阻塞的 API 函数。此外，如果应用程序使用 `vTaskDelete()` API 函数，空闲任务钩子就必须能够周期性返回，因为空闲任务负责清理由 RTOS 内核为已删除任务分配的资源。

12 开发支持

12.1 简介

本章重点介绍一组用于提高开发效率的功能，它们可以：

- 帮助开发者了解应用程序的实际行为。
- 指出可以优化的环节。
- 在错误发生的位置立即捕获错误。

12.2 configASSERT()

在 C 语言中，`assert()` 宏用于验证程序作出的断言，也就是某项假设。断言以 C 表达式编写；如果表达式求值为假（0），就认为断言失败。例如，代码清单 12.1 验证指针 `pxMyPointer` 不为 NULL。

```
/* 验证 pxMyPointer 不为 NULL。 */  
assert( pxMyPointer != NULL );
```

代码清单 12.1 使用标准 C `assert()` 宏检查 `pxMyPointer` 不为 NULL

应用程序开发者通过实现 `assert()` 宏，指定断言失败时要执行的操作。

FreeRTOS 源代码不调用 `assert()`，因为并非所有用于编译 FreeRTOS 的编译器都提供 `assert()`。FreeRTOS 源代码改为大量调用 `configASSERT()` 宏。应用程序开发者可以在 `FreeRTOSConfig.h` 中定义该宏，其行为与标准 C `assert()` 完全相同。

断言失败必须被视为致命错误。不要尝试继续执行已经发生断言失败的代码行之后的代码。

使用 `configASSERT()` 可以立即捕获并定位许多最常见的错误来源，从而提高开发效率。强烈建议在开发或调试 FreeRTOS 应用程序时定义 `configASSERT()`。

定义 `configASSERT()` 对运行时调试很有帮助，但也会增大应用程序代码体积，并降低执行速度。如果没有提供 `configASSERT()` 的定义，系统会使用默认的空定义，C 预处理器会完全移除对 `configASSERT()` 的所有调用。

12.2.1 configASSERT() 定义示例

在调试器控制下运行应用程序时，可以使用代码清单 12.2 所示的 `configASSERT()` 定义。任何代码行发生断言失败时，该定义都会停止执行，因此暂停调试会话后，调试器显示的正是断言失败的代码行。

```
/* 禁用中断以停止执行 Tick 中断，然后停留在循环中，避免继续执行断言失败
   代码行之后的代码。如果硬件支持调试断点指令，可以用调试断点指令替代
   for() 循环。 */

#define configASSERT( x ) if( ( x ) == 0 ) { taskDISABLE_INTERRUPTS(); for(;;); }
```

代码清单 12.2 在调试器控制下运行时适用的简单 `configASSERT()` 定义

应用程序不在调试器控制下运行时，可以使用代码清单 12.3 所示的 `configASSERT()` 定义。该定义会输出或以其他方式记录断言失败的源代码行。它使用标准 C `__FILE__` 宏获取源文件名，并使用标准 C `__LINE__` 宏获取源文件中的行号，以确定断言失败的位置。

```
/* 必须在 C 源文件中定义此函数，而不能在 FreeRTOSConfig.h 头文件中定义。 */
void vAssertCalled( const char *pcFile, uint32_t ulLine )
{
    /* 在此函数中，pcFile 保存包含错误检测代码行的源文件名，ulLine 保存该代码行
       在源文件中的行号。进入下面的无限循环之前，可以输出或以其他方式记录
       pcFile 和 ulLine 的值。 */
    RecordErrorInformationHere( pcFile, ulLine );

    /* 禁用中断以停止执行 Tick 中断，然后停留在循环中，避免继续执行断言失败
       代码行之后的代码。 */
    taskDISABLE_INTERRUPTS();
    for( ;; );
}
/*-----*/

/* 以下两行必须放在 FreeRTOSConfig.h 中。 */
extern void vAssertCalled( const char *pcFile, unsigned long ulLine );
#define configASSERT( x ) if( ( x ) == 0 ) vAssertCalled( __FILE__, __LINE__ )
```

代码清单 12.3 记录断言失败源代码行的 `configASSERT()` 定义

12.3 Tracealyzer for FreeRTOS

Tracealyzer for FreeRTOS 是合作伙伴 Percepio 提供的运行时诊断和优化工具。

Tracealyzer for FreeRTOS 可以捕获有价值的动态行为信息，并在相互关联的图形视图中展示这些信息。该工具还可以同时显示多个同步视图。

这些捕获到的信息对于分析、排查或优化 FreeRTOS 应用程序非常有价值。

Tracealyzer for FreeRTOS 可以与传统调试器配合使用，以更高层级、基于时间的视角补充调试器提供的信息。

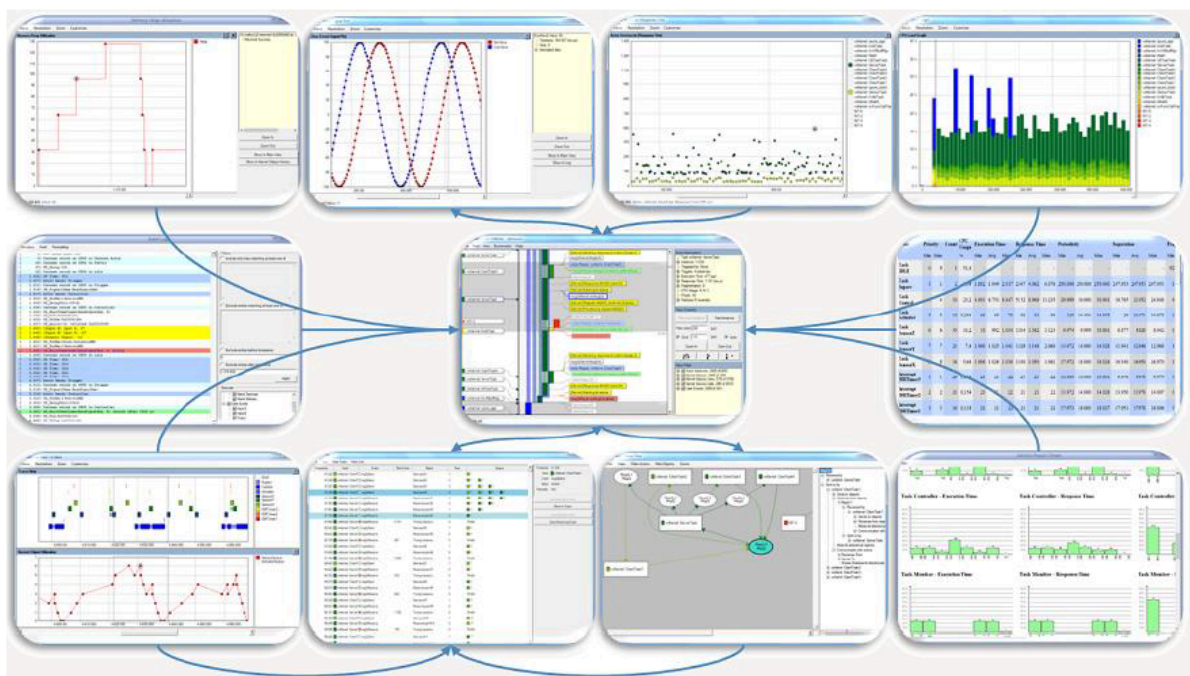


图 12.1 FreeRTOS+Trace 包含 20 多个相互关联的视图

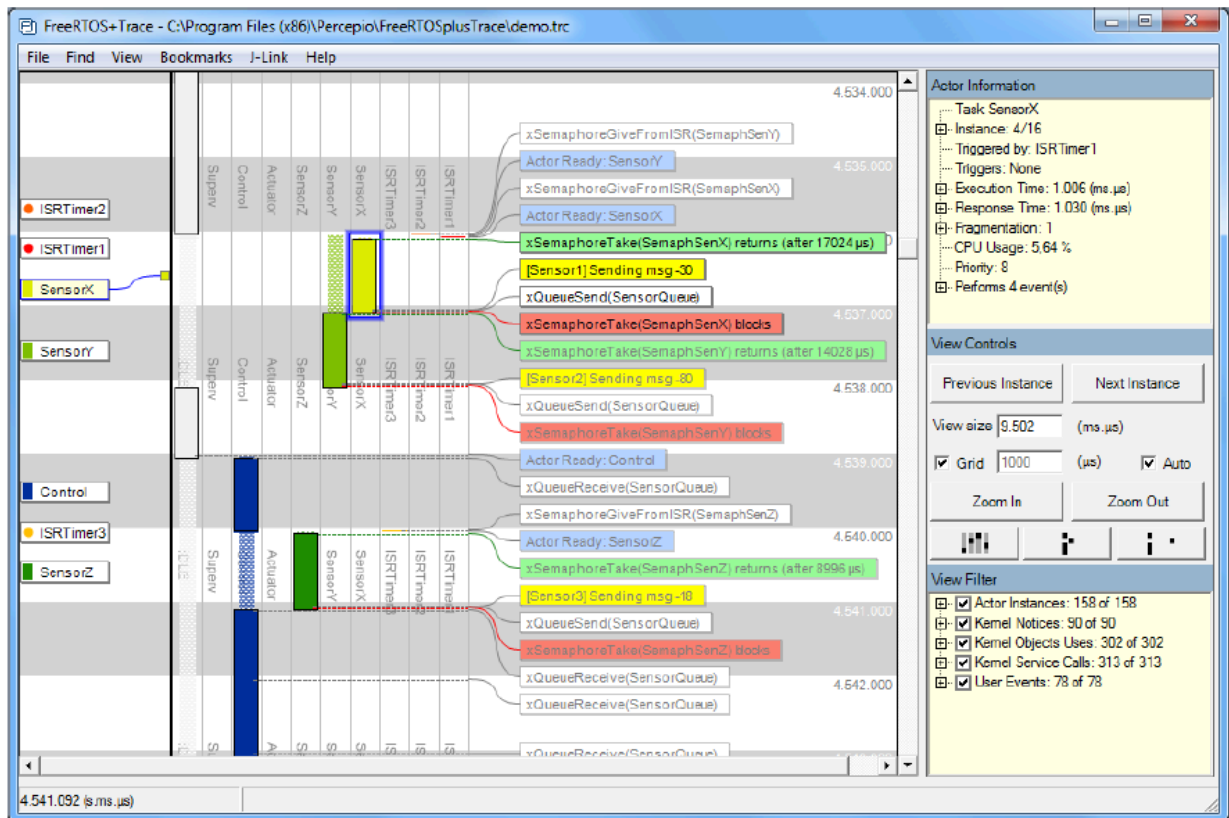


图 12.2 FreeRTOS+Trace 主跟踪视图，20 多个相互关联视图之一

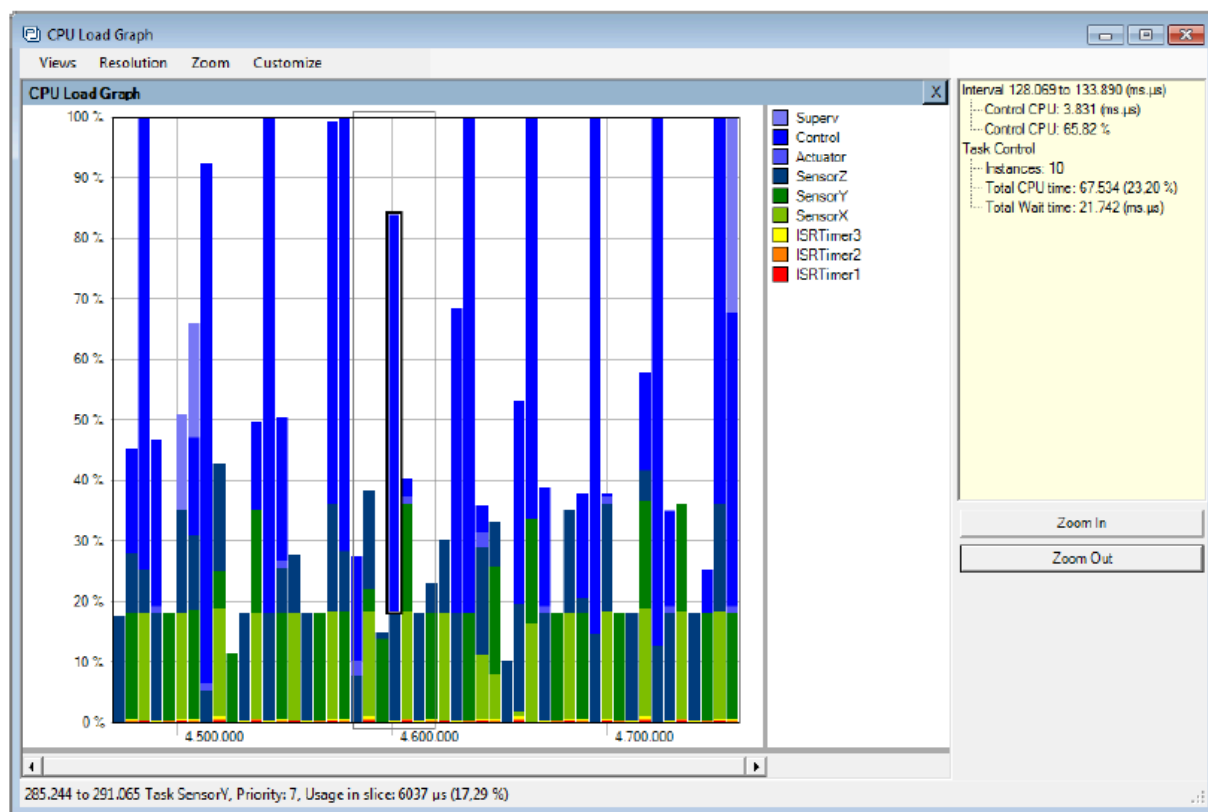


图 12.3 FreeRTOS+Trace CPU 负载视图，20 多个相互关联视图之一

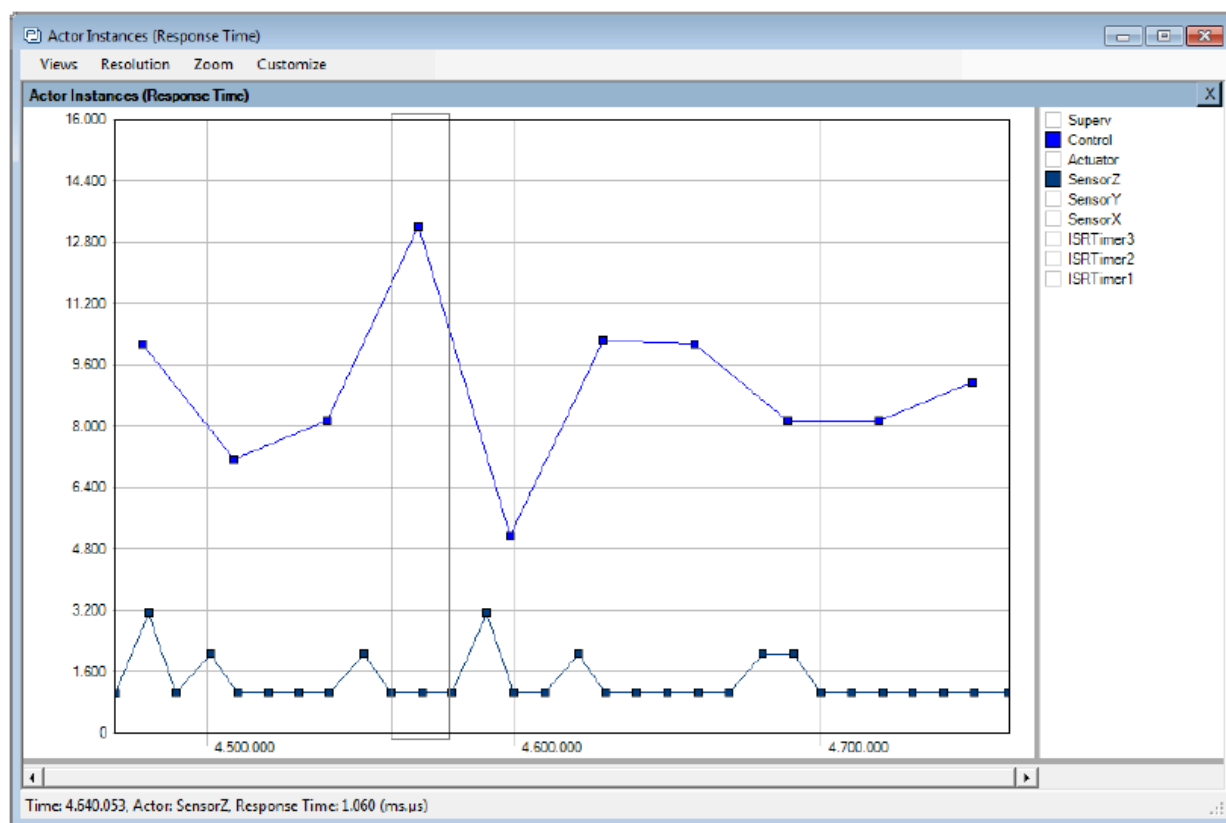


图 12.4 FreeRTOS+Trace 响应时间视图，20 多个相互关联视图之一

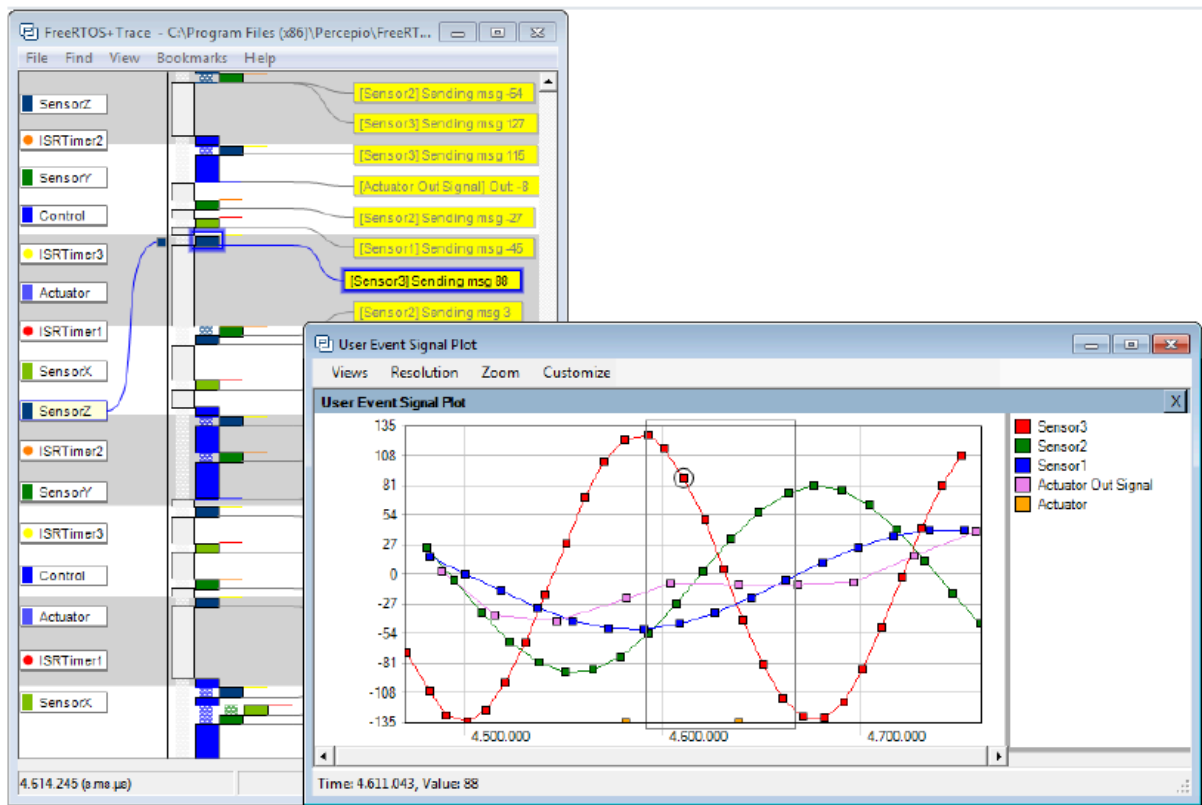


图 12.5 FreeRTOS+Trace 用户事件绘图视图，20 多个相互关联视图之一

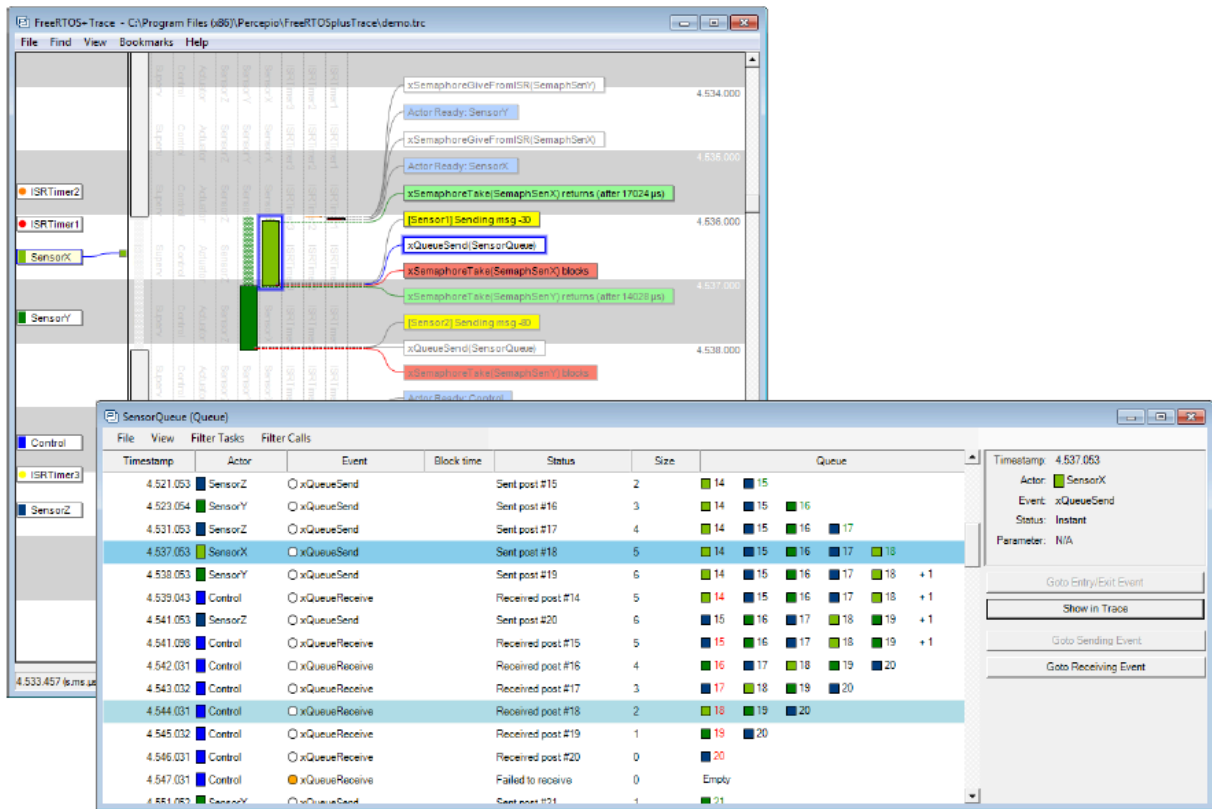


图 12.6 FreeRTOS+Trace 内核对象历史视图，20 多个相互关联视图之一

12.4 调试相关的钩子（回调）函数

12.4.1 内存分配失败钩子

第 3 章“堆内存管理”已经介绍过内存分配失败钩子或回调。

定义内存分配失败钩子，可以确保创建任务、队列、信号量或事件组失败时，应用程序开发者立即收到通知。

12.4.2 栈溢出钩子

第 13.3 节“栈溢出”详细介绍了栈溢出钩子。

定义栈溢出钩子，可以确保某个任务使用的栈空间超过为其分配的空间时，应用程序开发者收到通知。

12.5 查看运行时间与任务状态信息

12.5.1 任务运行时间统计

任务运行时间统计提供每个任务获得的处理器时间信息。任务的*运行时间*是应用程序启动以来，该任务处于运行状态的总时间。

运行时间统计旨在项目开发阶段辅助性能分析和调试。它提供的信息仅在作为运行时间统计时钟的计数器溢出之前有效。收集运行时间统计会增加任务上下文切换所需的时间。

要获得二进制形式的运行时间统计信息，请调用 `uxTaskGetSystemState()` API 函数。要获得人类可读 ASCII 表格形式的运行时间统计信息，请调用 `vTaskGetRunTimeStatistics()` 辅助函数。

12.5.2 运行时间统计时钟

运行时间统计需要测量小于一个 Tick 周期的时间。因此，系统不使用 RTOS Tick 计数作为运行时间统计时钟，而是由应用程序代码提供该时钟。建议让运行时间统计时钟的频率达到 Tick 中断频率的 10 到 100 倍。运行时间统计时钟越快，统计结果越准确，但时间值也越早溢出。

理想情况下，时间值应由一个自由运行的 32 位外设定定时器或计数器生成，并且读取该值不需要额外处理开销。如果可用外设和时钟速度无法实现这种方案，也可以使用以下效率较低的方法：

- 配置一个外设，使其以所需的运行时间统计时钟频率产生周期性中断，然后把已产生的中断次数用作运行时间统计时钟。

如果周期性中断仅用于提供运行时间统计时钟，这种方法效率很低。不过，如果应用程序已经使用频率合适的周期性中断，那么只需在现有中断服务程序中增加一个中断次数计数，就能简单而高效地实现。

- 使用自由运行的 16 位外设定定时器的当前值作为 32 位值的低 16 位，并使用定时器溢出次数作为 32 位值的高 16 位，从而生成一个 32 位值。

通过适当但较复杂的处理，也可以把 RTOS Tick 计数与 ARM Cortex-M SysTick 定时器的当前值组合起来，生成运行时间统计时钟。FreeRTOS 下载包中的一些演示项目展示了这种实现方式。

12.5.3 配置应用程序以收集运行时间统计

下面介绍收集任务运行时间统计所需的宏。这些宏最初准备放在 RTOS 移植层中，因此名称带有“port”前缀，但实践证明在 FreeRTOSConfig.h 中定义它们更合适。

用于收集运行时间统计的宏

- `configGENERATE_RUN_TIME_STATS`

必须在 `FreeRTOSConfig.h` 中把该宏设为 1。设为 1 后，调度器会在适当时机调用本节介绍的其他宏。

- `portCONFIGURE_TIMER_FOR_RUN_TIME_STATS()`

必须提供该宏，用于初始化提供运行时间统计时钟的外设。

- `portGET_RUN_TIME_COUNTER_VALUE()` 或 `portALT_GET_RUN_TIME_COUNTER_VALUE(Time)`

必须提供这两个宏中的一个，用于返回当前运行时间统计时钟值。该值以运行时间统计时钟为单位，表示应用程序自首次启动以来的总运行时间。

使用第一个宏时，必须把它定义为求值结果等于当前时钟值的表达式。使用第二个宏时，必须把它定义为将其 `Time` 参数设置为当前时钟值。

12.5.4 `uxTaskGetSystemState()` API 函数

`uxTaskGetSystemState()` 提供由 FreeRTOS 调度器管理的每个任务的状态信息快照。信息以 `TaskStatus_t` 结构体数组的形式提供，数组中的每个索引对应一个任务。代码清单 12.5 和后文介绍了 `TaskStatus_t`。

```
UBaseType_t uxTaskGetSystemState( TaskStatus_t * const pxTaskStatusArray,
                                   const UBaseType_t uxArraySize,
                                   configRUN_TIME_COUNTER_TYPE * const pulTotalRunTime );
```

代码清单 12.4 `uxTaskGetSystemState()` API 函数原型

注意：为了向后兼容，`configRUN_TIME_COUNTER_TYPE` 默认为 `uint32_t`；如果 `uint32_t` 的范围不足，可以在 `FreeRTOSConfig.h` 中覆盖该类型。

`uxTaskGetSystemState()` 参数和返回值

- `pxTaskStatusArray`

指向 `TaskStatus_t` 结构体数组的指针。

数组必须为每个任务至少包含一个 `TaskStatus_t` 结构体。可以使用

`uxTaskGetNumberOfTasks()` API 函数确定任务数量。

代码清单 12.5 给出了 `TaskStatus_t` 结构体，下一组列表介绍其成员。

- `uxArraySize`

`pxTaskStatusArray` 参数所指数组的大小。大小以数组索引数，也就是数组中包含的 `TaskStatus_t` 结构体数量表示，而不是以数组的字节数表示。

- `pulTotalRunTime`

如果在 `FreeRTOSConfig.h` 中把 `configGENERATE_RUN_TIME_STATS` 设为 1，`uxTaskGetSystemState()` 会把目标系统启动以来的总运行时间写入 `*pulTotalRunTime`。该时间由应用程序提供的运行时间统计时钟定义。

`pulTotalRunTime` 是可选参数；如果不需要总运行时间，可以将其设为 `NULL`。

- 返回值

返回 `uxTaskGetSystemState()` 实际填充的 `TaskStatus_t` 结构体数量。

返回值应当等于 `uxTaskGetNumberOfTasks()` API 函数的返回值；如果 `uxArraySize` 参数传入的值太小，则返回 0。

```
typedef struct xTASK_STATUS
{
    TaskHandle_t xHandle;
    const char *pcTaskName;
    UBaseType_t xTaskNumber;
    eTaskState eCurrentState;
    UBaseType_t uxCurrentPriority;
    UBaseType_t uxBasePriority;
    configRUN_TIME_COUNTER_TYPE ulRunTimeCounter;
    StackType_t * pxStackBase;
    #if ( ( portSTACK_GROWTH > 0 ) || ( configRECORD_STACK_HIGH_ADDRESS == 1 ) )
        StackType_t * pxTopOfStack;
        StackType_t * pxEndOfStack;
    #endif
    uint16_t usStackHighWaterMark;
    #if ( ( configUSE_CORE_AFFINITY == 1 ) && ( configNUMBER_OF_CORES > 1 ) )
        UBaseType_t uxCoreAffinityMask;
    #endif
} TaskStatus_t;
```

代码清单 12.5 `TaskStatus_t` 结构体

`TaskStatus_t` 结构体成员

- `xHandle`

该结构体中的信息所对应任务的句柄。

- `pcTaskName`

人类可读的任务文本名称。

- `xTaskNumber`

每个任务都有唯一的 `xTaskNumber` 值。

如果应用程序在运行时创建和删除任务，新任务可能获得与之前已删除任务相同的句柄。

`xTaskNumber` 使应用程序代码和支持内核感知的调试器能够区分仍然有效的任务，以及曾经具有相同句柄但已经删除的任务。

- `eCurrentState`

保存任务状态的枚举类型。 `eCurrentState` 可以是以下值之一：

- `eRunning`
- `eReady`
- `eBlocked`
- `eSuspended`
- `eDeleted`

只有从调用 `vTaskDelete()` 删除任务开始，到空闲任务释放为已删除任务的内部数据结构和栈分配的内存为止，这一小段时间内才会报告任务处于 `eDeleted` 状态。此后该任务将彻底不存在，尝试使用其句柄属于无效操作。

- `uxCurrentPriority`

调用 `uxTaskGetSystemState()` 时任务运行所采用的优先级。只有任务依据[第 8.3 节“互斥量（与二值信号量）”](#)介绍的优先级继承机制，临时获得更高优先级时， `uxCurrentPriority` 才会高于应用程序开发者为任务分配的优先级。

- `uxBasePriority`

应用程序开发者分配给任务的优先级。只有在 FreeRTOSConfig.h 中把 `configUSE_MUTEXES` 设为 1 时， `uxBasePriority` 才有效。

- `ulRunTimeCounter`

任务自创建以来使用的总运行时间。总运行时间以绝对时间值提供，并使用应用程序开发者为收集运行时间统计而提供的时钟。只有在 FreeRTOSConfig.h 中把

`configGENERATE_RUN_TIME_STATS` 设为 1 时，`ulRunTimeCounter` 才有效。

- `pxStackBase`

指向分配给该任务的栈区域基地址。

- `pxTopOfStack`

指向分配给该任务的栈区域当前栈顶地址。只有栈向上增长，也就是 `portSTACK_GROWTH` 大于 0，或者在 FreeRTOSConfig.h 中把 `configRECORD_STACK_HIGH_ADDRESS` 设为 1 时，`pxTopOfStack` 字段才有效。

- `pxEndOfStack`

指向分配给该任务的栈区域末端地址。只有栈向上增长，也就是 `portSTACK_GROWTH` 大于 0，或者在 FreeRTOSConfig.h 中把 `configRECORD_STACK_HIGH_ADDRESS` 设为 1 时，`pxEndOfStack` 字段才有效。

- `usStackHighWaterMark`

任务的栈高水位标记，也就是任务创建以来曾经剩余的最小栈空间。它反映任务离栈溢出有多近：该值越接近 0，任务就越接近栈溢出。`usStackHighWaterMark` 以字节为单位。

- `uxCoreAffinityMask`

表示任务可以在哪些核心上运行的位掩码。核心编号从 0 到 `configNUMBER_OF_CORES - 1`。例如，可以在核心 0 和核心 1 上运行的任务，其 `uxCoreAffinityMask` 会设为 0x03。只有在 FreeRTOSConfig.h 中把 `configUSE_CORE_AFFINITY` 设为 1，并且把 `configNUMBER_OF_CORES` 设为大于 1 的值时，`uxCoreAffinityMask` 字段才可用。

12.5.5 vTaskListTasks() 辅助函数

`vTaskListTasks()` 提供与 `uxTaskGetSystemState()` 类似的任务状态信息，但它把信息表示为人类可读的 ASCII 表格，而不是二进制值数组。

`vTaskListTasks()` 会占用大量处理器时间，并使调度器长时间保持挂起状态。因此，建议只将该函数用于调试，不要在生产环境的实时系统中使用。

如果在 FreeRTOSConfig.h 中把 `configUSE_TRACE_FACILITY` 设为 1，并把 `configUSE_STATS_FORMATTING_FUNCTIONS` 设为大于 0 的值，就可以使用 `vTaskListTasks()`。

```
void vTaskListTasks( char * pcWriteBuffer, size_t uxBufferLength );
```

代码清单 12.6 `vTaskListTasks()` API 函数原型

`vTaskListTasks()` 参数

- `pcWriteBuffer`

指向字符缓冲区的指针，函数会把格式化后的人类可读表格写入该缓冲区。假定该缓冲区足以容纳生成的报告。每个任务预留大约 40 字节通常已经足够。

- `uxBufferLength`

`pcWriteBuffer` 的长度。

图 12.7 展示了 `vTaskListTasks()` 生成的输出。其中：

- 每一行提供一个任务的信息。
- 第一列是任务名称。
- 第二列是任务状态：“X”表示运行，“R”表示就绪，“B”表示阻塞，“S”表示挂起，“D”表示任务已删除。只有从调用 `vTaskDelete()` 删除任务开始，到空闲任务释放为已删除任务的内部数据结构和栈分配的内存为止，这一小段时间内才会报告任务处于已删除状态。此后该任务将彻底不存在，尝试使用其句柄属于无效操作。
- 第三列是任务优先级。
- 第四列是任务的栈高水位标记。请参见 `usStackHighWaterMark` 的说明。
- 第五列是分配给任务的唯一编号。请参见 `xTaskNumber` 的说明。

tcPIP	R	3	393	0
Tmr Svc	R	3	111	48
QConsB1	R	1	143	3
QProdB5	R	0	144	7
QConsB6	R	0	143	8
PolSEM1	R	0	145	11
PolSEM2	R	0	145	12
GenQ	R	0	155	17
MuLow	R	0	147	18
Rec3	R	0	141	30
SUSP_RX	R	0	148	36
Math1	R	0	167	38
Math2	R	0	167	39

图 12.7 `vTaskListTasks()` 生成的示例输出

注意: `vTaskListTasks` 的旧版本是 `vTaskList`。 `vTaskList` 假定 `pcWriteBuffer` 的长度为 `configSTATS_BUFFER_MAX_LENGTH`。保留该函数只是为了向后兼容。建议新应用程序使用 `vTaskListTasks`，并显式提供 `pcWriteBuffer` 的长度。

```
void vTaskList( signed char *pcWriteBuffer );
```

代码清单 12.7 `vTaskList()` API 函数原型

`vTaskList()` 参数

- `pcWriteBuffer`

指向字符缓冲区的指针，函数会把格式化后的人类可读表格写入该缓冲区。由于函数不执行边界检查，缓冲区必须足以容纳整个表格。

12.5.6 `vTaskGetRunTimeStatistics()` 辅助函数

`vTaskGetRunTimeStatistics()` 把收集到的运行时间统计格式化为人类可读的 ASCII 表格。

`vTaskGetRunTimeStatistics()` 会占用大量处理器时间，并使调度器长时间保持挂起状态。因此，建议只将该函数用于调试，不要在生产环境的实时系统中使用。

当 FreeRTOSConfig.h 中的 `configGENERATE_RUN_TIME_STATS` 设为 1 、
`configUSE_STATS_FORMATTING_FUNCTIONS` 设为大于 0 的值，并且 `configUSE_TRACE_FACILITY`
设为 1 时，可以使用 `vTaskGetRunTimeStatistics()`。

```
void vTaskGetRunTimeStatistics( char * pcWriteBuffer, size_t uxBufferLength );
```

代码清单 12.8 `vTaskGetRunTimeStatistics()` API 函数原型

`vTaskGetRunTimeStatistics()` 参数

- `pcWriteBuffer`

指向字符缓冲区的指针，函数会把格式化后的人类可读表格写入该缓冲区。假定该缓冲区足以容纳生成的报告。每个任务预留大约 40 字节通常已经足够。

- `uxBufferLength`

`pcWriteBuffer` 的长度。

图 12.8 展示了 `vTaskGetRunTimeStatistics()` 生成的输出。其中：

- 每一行提供一个任务的信息。
- 第一列是任务名称。
- 第二列是任务处于运行状态的时间，以绝对值表示。请参见 `ulRunTimeCounter` 的说明。
- 第三列是任务处于运行状态的时间占目标系统启动以来总时间的百分比。显示的各项百分比之和通常小于预期的 100%，因为统计的收集和计算使用向下取整到最近整数的整数运算。

PolSEM1	994	<1%
PolSEM2	23248	1%
GenQ	194479	16%
MuLow	3690	<1%
Rec3	229450	18%
CNT1	242720	19%
PeekL	94	<1%
CNT_INC	165	<1%
CNT2	243166	20%
SUSP_RX	243192	20%
IDLE	55	<1%

图 12.8 `vTaskGetRunTimeStatistics()` 生成的示例输出

注意：`vTaskGetRunTimeStatistics` 的旧版本是 `vTaskGetRunTimeStats`。`vTaskGetRunTimeStats` 假定 `pcWriteBuffer` 的长度为 `configSTATS_BUFFER_MAX_LENGTH`。保留该函数只是为了向后兼容。建议新应用程序使用 `vTaskGetRunTimeStatistics`，并显式提供 `pcWriteBuffer` 的长度。

```
void vTaskGetRunTimeStats( signed char *pcWriteBuffer );
```

代码清单 12.9 `vTaskGetRunTimeStats()` API 函数原型

`vTaskGetRunTimeStats()` 参数

- `pcWriteBuffer`

指向字符缓冲区的指针，函数会把格式化后的人类可读表格写入该缓冲区。由于函数不执行边界检查，缓冲区必须足以容纳整个表格。

12.5.7 生成并显示运行时间统计：完整示例

本示例使用一个假想的 16 位定时器生成 32 位运行时间统计时钟。计数器配置为每当 16 位值达到最大值时产生一次中断，实际上也就是产生溢出中断。中断服务程序负责统计溢出次数。

以溢出次数作为 32 位值的最高两个字节，以当前 16 位计数器值作为 32 位值的最低两个字节，即可构成 32 位值。代码清单 12.10 给出了中断服务程序的伪代码。

```
void TimerOverflowInterruptHandler( void )
{
    /* 只统计中断次数。 */
    ulOverflowCount++;

    /* 清除中断。 */
    ClearTimerInterrupt();
}
```

代码清单 12.10 用于统计定时器溢出次数的 16 位定时器溢出中断处理程序

代码清单 12.11 给出了为启用运行时间统计收集而添加到 FreeRTOSConfig.h 中的代码。

```

/* 将 configGENERATE_RUN_TIME_STATS 设为 1 以启用运行时间统计收集。
   完成此设置后，还必须定义 portCONFIGURE_TIMER_FOR_RUN_TIME_STATS(),
   以及 portGET_RUN_TIME_COUNTER_VALUE() 或
   portALT_GET_RUN_TIME_COUNTER_VALUE(x)。 */
#define configGENERATE_RUN_TIME_STATS 1

/* 把 portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() 定义为调用
   用于配置假想 16 位定时器的函数
   (未给出该函数的实现)。 */
void vSetupTimerForRunTimeStats( void );
#define portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() vSetupTimerForRunTimeStats()

/* 把 portALT_GET_RUN_TIME_COUNTER_VALUE() 定义为将其参数设置为
   当前运行时间计数器或时间值。返回的时间值为 32 位，
   先把 16 位定时器的溢出次数左移到
   32 位数的最高两个字节，再将结果与
   当前 16 位计数器值执行按位 OR。 */
#define portALT_GET_RUN_TIME_COUNTER_VALUE( ulCountValue ) \
{ \
    extern volatile unsigned long ulOverflowCount; \
 \
    /* 断开计数器的时钟，使其数值在 \
       使用期间保持不变。 */ \
    PauseTimer(); \
 \
    /* 把溢出次数移入返回的 32 位值的 \
       最高两个字节。 */ \
    ulCountValue = ( ulOverflowCount << 16UL ); \
 \
    /* 把当前计数器值用作返回的 32 位值的 \
       最低两个字节。 */ \
    ulCountValue |= ( unsigned long ) ReadTimerCount(); \
 \
    /* 重新连接计数器的时钟。 */ \
    ResumeTimer(); \
}

```

代码清单 12.11 为启用运行时间统计收集而添加到 *FreeRTOSConfig.h* 的宏

代码清单 12.12 所示的任务每 5 秒输出一一次收集到的运行时间统计。

```

#define RUN_TIME_STATS_STRING_BUFFER_LENGTH    512

/* 为了清晰起见，本代码清单省略了对 fflush() 的调用。 */
static void prvStatsTask( void *pvParameters )
{
    TickType_t xLastExecutionTime;

    /* 用于保存格式化运行时间统计文本的缓冲区需要相当大。
       因此把它声明为 static，确保它不会
       分配到任务栈上。这也使该函数不可重入。 */
    static signed char cStringBuffer[ RUN_TIME_STATS_STRING_BUFFER_LENGTH ];

    /* 该任务每 5 秒运行一次。 */
    const TickType_t xBlockPeriod = pdMS_TO_TICKS( 5000 );

    /* 将 xLastExecutionTime 初始化为当前时间。这是唯一
       需要显式写入该变量的时刻，之后由
       vTaskDelayUntil() API 函数在内部更新它。 */
    xLastExecutionTime = xTaskGetTickCount();

    /* 与大多数任务一样，该任务通过无限循环实现。 */
    for( ;; )
    {
        /* 等待下一次运行该任务的时刻。 */
        xTaskDelayUntil( &xLastExecutionTime, xBlockPeriod );

        /* 根据运行时间统计生成文本表格。该表格必须能装入
           cStringBuffer 数组。 */
        vTaskGetRunTimeStatistics( cStringBuffer, RUN_TIME_STATS_STRING_BUFFER_LENGTH );

        /* 输出运行时间统计表的列标题。 */
        printf( "\nTask\t\tAbs\t\t\t\t%" );
        printf( "-----\n" );

        /* 输出运行时间统计本身。数据表包含
           多行，因此调用 vPrintMultipleLines(),
           而不是直接调用 printf()。vPrintMultipleLines()
           只是分别对每一行调用 printf(),
           以确保缓冲按预期工作。 */
        vPrintMultipleLines( cStringBuffer );
    }
}

```

代码清单 12.12 输出所收集运行时间统计的任务

12.6 跟踪钩子宏

跟踪宏是放置在 FreeRTOS 源代码关键位置的宏。默认情况下，这些宏为空，不生成任何代码，也不会产生运行时开销。应用程序开发者可以覆盖默认的空实现，从而：

- 在不修改 FreeRTOS 源文件的情况下，把代码插入 FreeRTOS。

- 通过目标硬件提供的任意方式输出详细的执行顺序信息。FreeRTOS 源代码中的跟踪宏分布足够广泛，可以用它们创建完整、详细的调度器活动跟踪和性能分析日志。

12.6.1 可用的跟踪钩子宏

逐一详述所有宏会占用过多篇幅。下面列出应用程序开发者最常用的一部分宏。

下列许多说明都引用名为 `pxCurrentTCB` 的变量。`pxCurrentTCB` 是 FreeRTOS 的私有变量，用于保存处于运行状态的任务句柄；从 FreeRTOS/Source/tasks.c 源文件调用的任何宏都可以访问它。

常用跟踪钩子宏选集

- `traceTASK_INCREMENT_TICK(xTickCount)`

在 Tick 中断期间、Tick 计数递增之前调用。`xTickCount` 参数把新的 Tick 计数值传入宏。

- `traceTASK_SWITCHED_OUT()`

在选择新任务运行之前调用。此时，`pxCurrentTCB` 保存即将离开运行状态的任务句柄。

- `traceTASK_SWITCHED_IN()`

在选定一个任务运行之后调用。此时，`pxCurrentTCB` 保存即将进入运行状态的任务句柄。

- `traceBLOCKING_ON_QUEUE_RECEIVE(pxQueue)`

当前执行的任务尝试从空队列读取数据，或者尝试“获取”空信号量或互斥量，因而即将进入阻塞状态时调用。`pxQueue` 参数把目标队列或信号量的句柄传入宏。

- `traceBLOCKING_ON_QUEUE_SEND(pxQueue)`

当前执行的任务尝试向已满队列写入数据，因而即将进入阻塞状态时调用。`pxQueue` 参数把目标队列的句柄传入宏。

- `traceQUEUE_SEND(pxQueue)`

在 `xQueueSend()`、`xQueueSendToFront()`、`xQueueSendToBack()` 或任意信号量“释放”函数内部，当队列发送或信号量释放成功时调用。`pxQueue` 参数把目标队列或信号量的句柄传入宏。

- `traceQUEUE_SEND_FAILED(pxQueue)`

在 `xQueueSend()`、`xQueueSendToFront()`、`xQueueSendToBack()` 或任意信号量“释放”函数内部，当队列发送或信号量释放失败时调用。如果队列已满，并且在指定的整个阻塞时间内始终保持已满，队列发送或信号量释放就会失败。`pxQueue` 参数把目标队列或信号量的句柄传入宏。

- `traceQUEUE_RECEIVE(pxQueue)`

在 `xQueueReceive()` 或任意信号量“获取”函数内部，当队列接收或信号量获取成功时调用。`pxQueue` 参数把目标队列或信号量的句柄传入宏。

- `traceQUEUE_RECEIVE_FAILED(pxQueue)`

在 `xQueueReceive()` 或任意信号量“获取”函数内部，当队列或信号量接收操作失败时调用。如果队列或信号量为空，并且在指定的整个阻塞时间内始终保持为空，队列接收或信号量获取就会失败。`pxQueue` 参数把目标队列或信号量的句柄传入宏。

- `traceQUEUE_SEND_FROM_ISR(pxQueue)`

在 `xQueueSendFromISR()` 内部，当发送操作成功时调用。`pxQueue` 参数把目标队列的句柄传入宏。

- `traceQUEUE_SEND_FROM_ISR_FAILED(pxQueue)`

在 `xQueueSendFromISR()` 内部，当发送操作失败时调用。如果队列已经满，发送操作就会失败。`pxQueue` 参数把目标队列的句柄传入宏。

- `traceQUEUE_RECEIVE_FROM_ISR(pxQueue)`

在 `xQueueReceiveFromISR()` 内部，当接收操作成功时调用。`pxQueue` 参数把目标队列的句柄传入宏。

- `traceQUEUE_RECEIVE_FROM_ISR_FAILED(pxQueue)`

在 `xQueueReceiveFromISR()` 内部，当队列已经为空而导致接收操作失败时调用。`pxQueue` 参数把目标队列的句柄传入宏。

- `traceTASK_DELAY_UNTIL( xTimeToWake )`

在 `xTaskDelayUntil()` 内部，紧接在调用任务进入阻塞状态之前调用。

- `traceTASK_DELAY()`

在 `vTaskDelay()` 内部，紧接在调用任务进入阻塞状态之前调用。

12.6.2 定义跟踪钩子宏

每个跟踪宏都有默认的空定义。可以在 `FreeRTOSConfig.h` 中提供新的宏定义，以覆盖默认定义。如果跟踪宏定义变得很长或很复杂，可以在新的头文件中实现它们，再从 `FreeRTOSConfig.h` 包含该头文件。

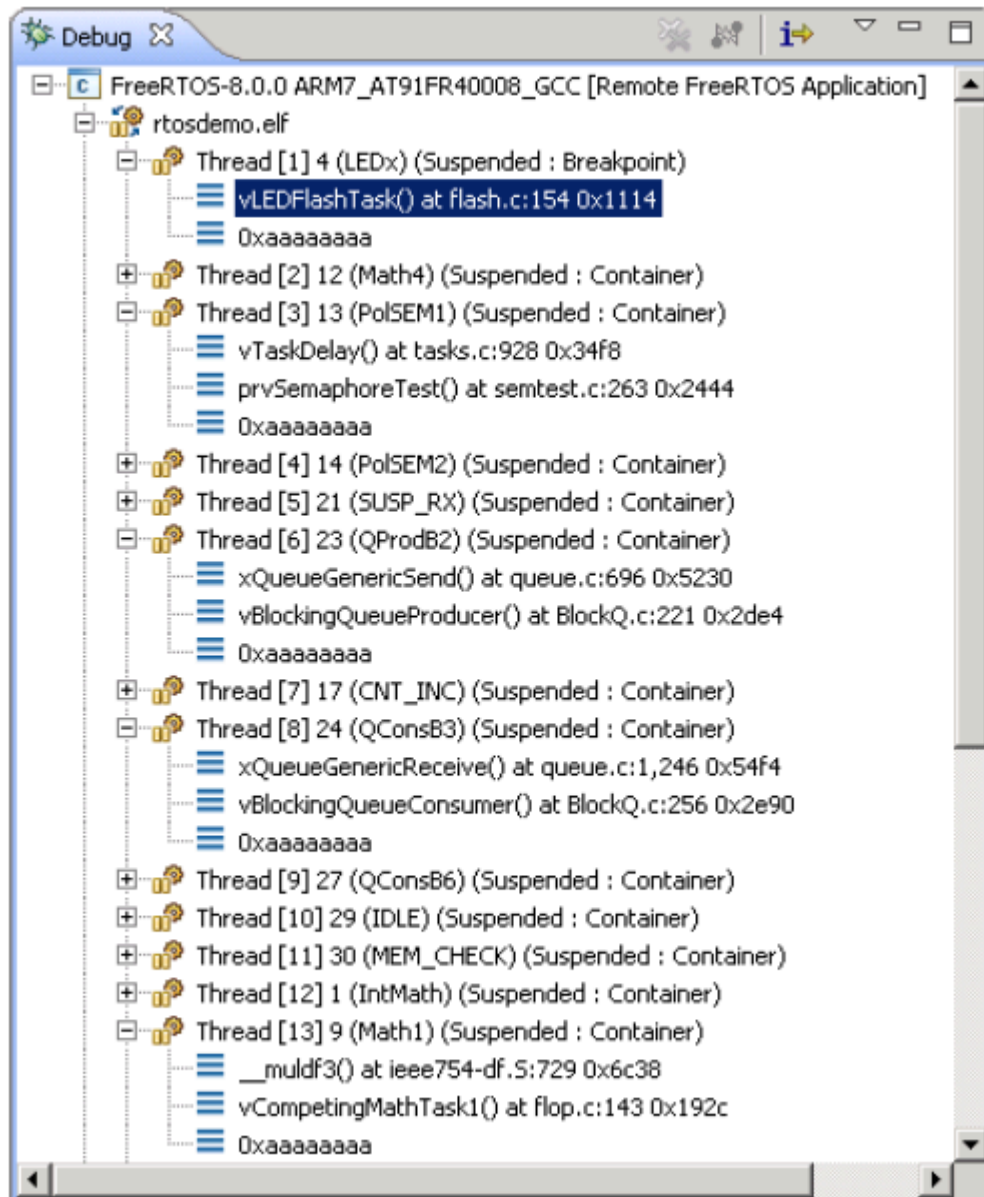
按照软件工程最佳实践，FreeRTOS 实施严格的数据隐藏策略。跟踪宏允许把用户代码加入 FreeRTOS 源文件，因此跟踪宏可见的数据类型与应用程序代码可见的数据类型不同：

- 在 `FreeRTOS/Source/tasks.c` 源文件内部，任务句柄是指向任务描述数据结构，也就是任务 *任务控制块* (Task Control Block, TCB) 的指针。在 `FreeRTOS/Source/tasks.c` 源文件外部，任务句柄是 `void` 指针。
- 在 `FreeRTOS/Source/queue.c` 源文件内部，队列句柄是指向队列描述数据结构的指针。在 `FreeRTOS/Source/queue.c` 源文件外部，队列句柄是 `void` 指针。

如果跟踪宏直接访问通常属于私有的 FreeRTOS 数据结构，必须格外谨慎，因为私有数据结构可能在不同 FreeRTOS 版本之间发生变化。

12.6.3 支持 FreeRTOS 感知的调试器插件

以下 IDE 提供一定程度的 FreeRTOS 感知插件。该列表可能并不完整：



- Eclipse (StateViewer)
- Eclipse (ThreadSpy)
- IAR
- ARM DS-5
- Atollic TrueStudio
- Microchip MPLAB
- iSYSTEM WinIDEA
- STM32CubeIDE

13 故障排查

13.1 本章简介与范围

本章重点介绍 FreeRTOS 新用户最常遇到的问题。首先讨论多年来最常引发支持请求的三类问题：中断优先级分配错误、栈溢出，以及不恰当地使用 `printf()`。随后以常见问题解答的形式，简要介绍其他常见错误、可能原因和解决方法。

使用 `configASSERT()` 可以立即捕获并定位许多最常见的错误来源，从而提高开发效率。强烈建议在开发或调试 FreeRTOS 应用程序时定义 `configASSERT()`。第 12.2 节介绍了 `configASSERT()`。

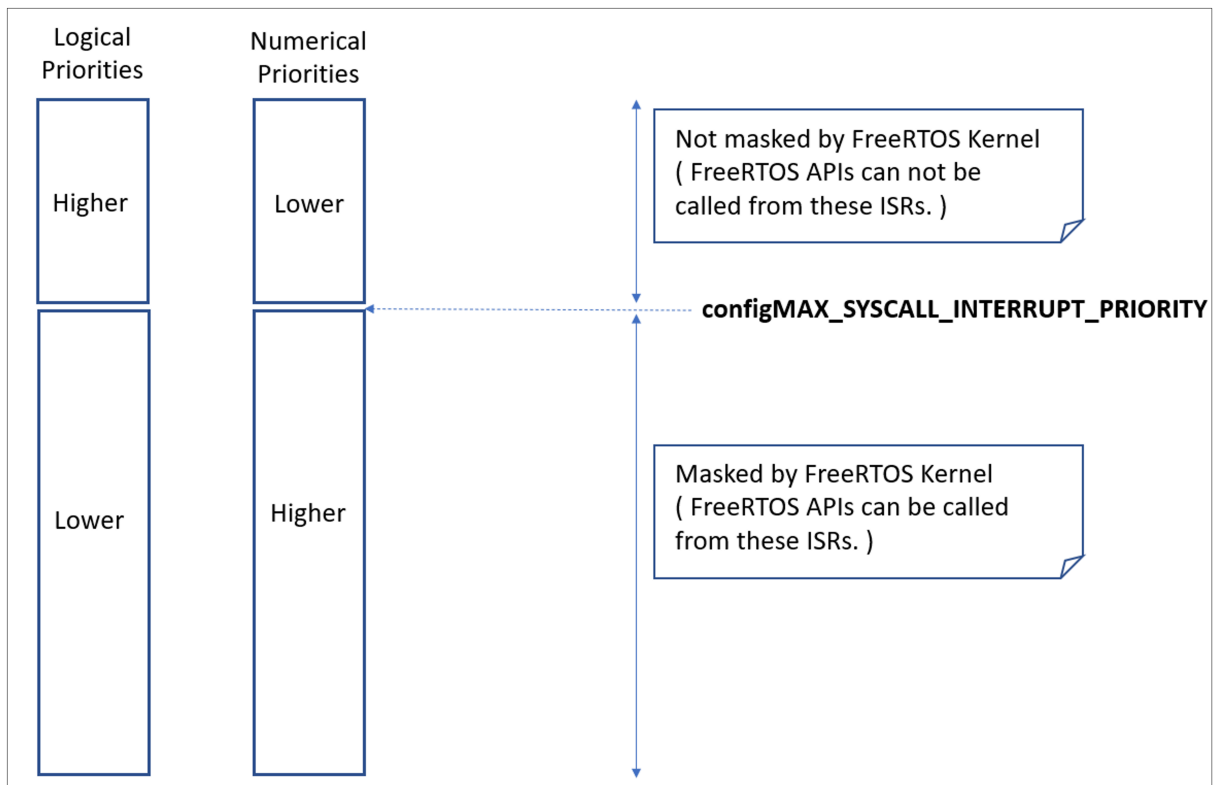
13.2 中断优先级

注意：这是引发支持请求的首要原因。在大多数移植中，定义 `configASSERT()` 可以立即捕获此错误！

如果所用 FreeRTOS 移植支持中断嵌套，并且某个中断的服务程序使用 FreeRTOS API，那么该中断的逻辑优先级必须设置为等于或低于 `configMAX_SYSCALL_INTERRUPT_PRIORITY`，具体说明参见第 7.8 节“中断嵌套”。否则临界区将无法有效工作，进而造成间歇性故障。

在具有以下特征的处理器上运行 FreeRTOS 时，应格外小心：

- 中断优先级默认处于可能的最高优先级。某些 ARM Cortex 处理器以及其他一些处理器就是如此。在这类处理器上，使用 FreeRTOS API 的中断不能保留未初始化的优先级。
- 数值较大的优先级编号表示逻辑上较低的中断优先级。这可能有违直觉，容易造成混淆。ARM Cortex 处理器以及其他一些处理器同样采用这种方式。
- 例如，在这类处理器上，以优先级 5 执行的中断可以被优先级 4 的中断抢占。因此，如果把 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 设为 5，任何使用 FreeRTOS API 的中断都只能分配数值大于或等于 5 的优先级。在这种情况下，中断优先级 5 或 6 有效，而中断优先级 3 明确无效。



- 不同的库实现要求以不同方式指定中断优先级。这一点对面向 ARM Cortex 处理器的库尤其重要，因为中断优先级在写入硬件寄存器前需要进行位移。有些库会自行执行位移，而另一些库要求在把优先级传给库函数之前完成位移。
- 同一种体系结构的不同实现可能实现不同数量的中断优先级位。例如，一家制造商的 Cortex-M 处理器可能实现 3 个优先级位，而另一家制造商的 Cortex-M 处理器可能实现 4 个优先级位。
- 定义中断优先级的位可以拆分为抢占优先级位和子优先级位。应确保所有这些位都用于指定抢占优先级，不使用子优先级。

在某些 FreeRTOS 移植中，`configMAX_SYSCALL_INTERRUPT_PRIORITY` 也称为 `configMAX_API_CALL_INTERRUPT_PRIORITY`。

13.3 栈溢出

栈溢出是引发支持请求的第二常见原因。FreeRTOS 提供了多项功能，帮助捕获并调试与栈有关的问题 [1]。

13.3.1 uxTaskGetStackHighWaterMark() API 函数

每个任务都有自己的栈，其总大小在创建任务时指定。`uxTaskGetStackHighWaterMark()` 用于查询任务距离用尽所分配栈空间还有多远。该值称为栈的“高水位标记”。

```
UBaseType_t uxTaskGetStackHighWaterMark( TaskHandle_t xTask );
```

代码清单 13.1 `uxTaskGetStackHighWaterMark()` API 函数原型

`uxTaskGetStackHighWaterMark()` 参数和返回值

- `xTask`

要查询栈高水位标记的任务，也就是目标任务句柄。有关如何获取任务句柄的信息，请参见

`xTaskCreate()` API 函数的 `pxCreatedTask` 参数。

任务可以传入 NULL 代替有效的任务句柄，以查询自身的栈高水位标记。

- 返回值

随着任务执行和中断处理，任务使用的栈空间会增加或减少。`uxTaskGetStackHighWaterMark()`

返回自任务开始执行以来曾经可用的最小剩余栈空间，也就是栈使用量达到最大值（最深位置）时仍未使用的栈空间。高水位标记越接近零，任务就越接近栈溢出。

可以使用 `uxTaskGetStackHighWaterMark2()` API 代替 `uxTaskGetStackHighWaterMark()`，两者仅返回类型不同。

```
configSTACK_DEPTH_TYPE uxTaskGetStackHighWaterMark2( TaskHandle_t xTask );
```

代码清单 13.2 `uxTaskGetStackHighWaterMark2()` API 函数原型

使用 `configSTACK_DEPTH_TYPE` 后，应用程序开发者可以控制表示栈深度所使用的类型。

13.3.2 运行时栈检查：概述

FreeRTOS 包含三种可选的运行时栈检查机制。这些机制由 FreeRTOSConfig.h 中的编译时配置常量

`configCHECK_FOR_STACK_OVERFLOW` 控制。所有方法都会增加执行上下文切换所需的时间。

栈溢出钩子（或栈溢出回调）是内核检测到栈溢出时调用的函数。要使用栈溢出钩子函数：

1. 按照以下各小节的说明，在 FreeRTOSConfig.h 中把 `configCHECK_FOR_STACK_OVERFLOW` 设为 1、2 或 3。
2. 使用代码清单 13.3 所示的准确函数名和原型，实现该钩子函数。

```
void vApplicationStackOverflowHook( TaskHandle_t *pxTask, signed char *pcTaskName );
```

代码清单 13.3 栈溢出钩子函数原型

提供栈溢出钩子是为了更容易捕获并调试栈错误，但栈溢出发生后实际上无法可靠恢复。函数参数会把发生栈溢出的任务句柄和任务名称传入钩子函数。

栈溢出钩子在中断上下文中被调用。

某些微控制器检测到错误的内存访问时会生成故障异常，因此故障可能在内核有机会调用栈溢出钩子函数之前触发。

13.3.3 运行时栈检查：方法 1

当 `configCHECK_FOR_STACK_OVERFLOW` 设为 1 时选择方法 1。

每当任务被换出时，其完整执行上下文都会保存到任务栈中，栈使用量很可能在此时达到峰值。当 `configCHECK_FOR_STACK_OVERFLOW` 设为 1 时，内核会在保存上下文后检查栈指针是否仍位于有效栈空间内。如果发现栈指针超出有效范围，就调用栈溢出钩子。

方法 1 执行速度快，但可能漏掉发生在两次上下文切换之间的栈溢出。

13.3.4 运行时栈检查：方法 2

方法 2 会在方法 1 所述检查的基础上执行额外检查。当 `configCHECK_FOR_STACK_OVERFLOW` 设为 2 时选择该方法。

创建任务时，系统会用已知模式填充任务栈。方法 2 检查任务栈空间最后 20 个有效字节，确认该模式没有被覆盖。如果这 20 个字节中有任何一个与预期值不同，就调用栈溢出钩子函数。

方法 2 的执行速度不如方法 1，但由于只检查 20 个字节，速度仍然相对较快。它很可能捕获所有栈溢出，但仍有可能漏掉某些溢出，只是这种情况极不可能发生。

13.3.4 运行时栈检查：方法 3

当 `configCHECK_FOR_STACK_OVERFLOW` 设为 3 时选择方法 3。

该方法仅在部分移植中可用。可用时，它会启用 ISR 栈检查。检测到 ISR 栈溢出时，系统会触发断言。请注意，此时不会调用栈溢出钩子函数，因为该钩子专门用于任务栈，而不是 ISR 栈。

13.4 printf() 和 sprintf() 的使用

通过 `printf()` 记录日志是一种常见错误来源。应用程序开发者如果没有意识到这一点，往往会为了辅助调试而继续增加 `printf()` 调用，反而使问题更加严重。

许多交叉编译器供应商会提供适合小型嵌入式系统使用的 `printf()` 实现。即使如此，该实现也可能不是线程安全的，很可能不适合在中断服务程序中使用，而且根据输出目标的不同，执行时间可能相当长。

如果没有专为小型嵌入式系统设计的 `printf()` 实现，而改用通用 `printf()` 实现，则必须格外谨慎，原因如下：

- 仅仅加入一次 `printf()` 或 `sprintf()` 调用，就可能显著增大应用程序可执行文件的体积。
- `printf()` 和 `sprintf()` 可能调用 `malloc()`。如果使用的内存分配方案不是 `heap_3`，这种调用可能无效。更多信息参见第 3.2 节“内存分配方案示例”。
- `printf()` 和 `sprintf()` 所需的栈空间可能达到其他情况下的许多倍。

13.4.1 printf-stdarg.c

许多 FreeRTOS 演示项目使用名为 `printf-stdarg.c` 的文件。该文件提供一种精简且节省栈空间的 `sprintf()` 实现，可替代标准库版本。大多数情况下，这样可以为每个调用 `sprintf()` 及相关函数的任务分配小得多的栈。

`printf-stdarg.c` 还提供一种把 `printf()` 输出逐字符发送到端口的机制。虽然这种方式速度较慢，但可以进一步减少栈使用量。

请注意，FreeRTOS 下载包中并非所有 `printf-stdarg.c` 副本都实现了 `snprintf()`。未实现 `snprintf()` 的版本会直接映射到 `sprintf()`，因而只是忽略缓冲区大小参数。

`printf-stdarg.c` 是开源软件，但归第三方所有，因此其许可证与 FreeRTOS 分开。许可证条款位于该源文件顶部。

13.5 其他常见错误来源

13.5.1 症状：向演示程序添加一个简单任务后，演示程序崩溃

创建任务需要从堆中获取内存。许多演示应用项目会把堆大小设为刚好足以创建演示任务。因此，创建这些任务后，剩余堆空间不足以继续添加任务、队列、事件组或信号量。

调用 `vTaskStartScheduler()` 时，系统会自动创建空闲任务，并且可能还会创建 RTOS 守护任务。只有剩余堆内存不足以创建这些任务时，`vTaskStartScheduler()` 才会返回。在 `vTaskStartScheduler()` 调用后加入一个空循环 `[ for(;;); ]`，可以让此错误更容易调试。

要添加更多任务，必须增大堆大小，或者移除一些现有演示任务。堆大小能够增加的程度始终受可用 RAM 容量限制。更多信息参见第 3.2 节“内存分配方案示例”。

13.5.2 症状：在中断中使用 API 函数导致应用程序崩溃

不要在中断服务程序中使用 API 函数，除非该 API 函数名以“...FromISR()”结尾。尤其不要在中断中创建临界区，除非使用中断安全宏。更多信息参见第 7.2 节“从 ISR 使用 FreeRTOS API”。

在支持中断嵌套的 FreeRTOS 移植中，不要在优先级高于 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 的中断内使用任何 API 函数。更多信息参见第 7.8 节“中断嵌套”。

13.5.3 症状：应用程序有时会在中断服务程序中崩溃

首先应检查该中断是否造成栈溢出。某些移植只检查任务中的栈溢出，而不检查中断中的栈溢出。

不同移植和编译器定义及使用中断的方式有所不同。因此，第二项检查是确认中断服务程序所使用的语法、宏和调用约定与当前移植的文档页面完全一致，并与该移植附带的演示应用程序完全一致。

如果应用程序运行在使用较小优先级编号表示较高逻辑优先级的处理器上，应确保为每个中断分配优先级时考虑这一点，因为这种方式可能有违直觉。如果应用程序运行在默认把每个中断设为可能的最高优先级的处理器上，应确保每个中断都没有保留默认优先级。更多信息参见第 7.8 节“中断嵌套”和第 13.2 节“中断优先级”。

13.5.4 症状：调度器尝试启动第一个任务时崩溃

确保已经安装 FreeRTOS 中断处理程序。有关说明，请参阅当前所用 FreeRTOS 移植的文档页面；有关示例，请参阅该移植附带的演示应用程序。

某些处理器必须处于特权模式才能启动调度器。最简单的实现方式是在调用 `main()` 之前，通过 C 启动代码把处理器置于特权模式。

13.5.5 症状：中断意外保持禁用，或临界区嵌套不正确

如果在调度器启动前调用 FreeRTOS API 函数，系统会有意保持中断禁用，直到第一个任务开始执行后才重新启用中断。这样可以防止系统初始化期间，在调度器尚未启动且可能处于不一致状态时，某个中断试图使用 FreeRTOS API 函数而导致系统崩溃。

除了调用 `taskENTER_CRITICAL()` 和 `taskEXIT_CRITICAL()` 之外，不要以任何其他方式修改微控制器的中断使能位或优先级标志。这些宏会记录调用嵌套深度，确保只有当调用嵌套完全退回到零时才重新启用中断。请注意，某些库函数本身也可能启用和禁用中断。

13.5.6 症状：应用程序在调度器启动前就已经崩溃

在调度器启动前，不得允许任何可能引起上下文切换的中断服务程序执行。任何试图向队列、信号量等 FreeRTOS 对象发送数据或从中接收数据的中断服务程序同样如此。调度器启动前无法进行上下文切换。

许多 API 函数必须在调度器启动后才能调用。在调用 `vTaskStartScheduler()` 之前，最好把 API 的使用限制在创建任务、队列和信号量等对象，而不要使用这些对象。

13.5.7 症状：调度器挂起或处于临界区时调用 API 函数导致应用程序崩溃

调用 `vTaskSuspendAll()` 会挂起调度器，调用 `xTaskResumeAll()` 会恢复调度器。调用 `taskENTER_CRITICAL()` 会进入临界区，调用 `taskEXIT_CRITICAL()` 会退出临界区。

不要在调度器挂起期间或临界区内部调用 API 函数。

13.6 其他调试步骤

如果遇到上述常见原因未涵盖的问题，可以尝试以下调试步骤：

- 定义 `configASSERT()`，并在应用程序的 FreeRTOSConfig 文件中启用内存分配失败检查和栈溢出检查。
- 检查 FreeRTOS API 的返回值，确认调用成功。
- 检查与调度器有关的配置，例如 `configUSE_TIME_SLICING` 和 `configUSE_PREEMPTION`，确认其设置符合应用程序要求。
- [此页面](#)详细介绍了如何调试 Cortex-M 微控制器上的硬故障。

1. FreeRTOS Windows 移植不提供这些功能。↩